EF CORE PLUGIN-ENTWICKLUNG · REFERENZ UND PRAXIS

Das EF Core Plugin-Handbuch

Von der Options-Extension bis zum NuGet-Paket: alles, was man für professionelle und auch sehr komplexe Erweiterungen von Entity Framework Core braucht. Durchgängig erklärt an einem vollständigen Plugin für Mandantenfähigkeit mit PostgreSQL und SQL Server.

Ausgabe	1.1
Versionen	EF Core 10, Npgsql.EntityFrameworkCore.PostgreSQL 10, Microsoft.EntityFrameworkCore.SqlServer 10, .NET 10
Beispiel	Vollständige Solution unter **github.com/guenni81/EF-Core-Plugin-Handbook**: Kernpaket, relationales Paket, zwei Provider-Pakete, zwei Beispiel-Apps, Unit- und Integrationstests
Geprüft	Der Beispielcode wird gegen EF Core 10 und Npgsql 10 gebaut; Unit- und Integrationstests laufen gegen echtes PostgreSQL und SQL Server.
Aufbau	11 Teile, 48 Kapitel, 6 Anhänge (A–F)
Lizenz	MIT-Lizenz, © 2026 guenni81. Frei nutzbar, kopierbar, veränderbar und weitergebbar, auch kommerziell; siehe letzte Seite.

Über dieses Buch

Viele Erweiterungen für EF Core beginnen als ein paar Extension-Methods und wachsen dann unkontrolliert. Wer verstanden hat, wie EF Core intern aufgebaut ist, kann dagegen Erweiterungen schreiben, die sich wie ein Teil von EF Core anfühlen: Sie werden über die Options konfiguriert, laufen im internen Service Provider, übersetzen eigene Funktionen nach SQL, erzeugen Migrationen, schreiben lesbare Snapshots und lassen sich mit anderen Plugins kombinieren.

Dieses Buch erklärt jeden dieser Bausteine in zwei Schichten. Zuerst kommt das **Prinzip**: Wie funktioniert der Mechanismus in EF Core, welche Varianten gibt es, worauf muss man achten? Danach folgt die **Umsetzung** im Beispiel-Plugin, mit dem Code, der auch im Repository steht. Die Kapitel bauen aufeinander auf, lassen sich aber auch einzeln als Nachschlagewerk lesen. Wer mit einem konkreten Problem kommt und den passenden Mechanismus sucht, beginnt am besten mit der Entscheidungshilfe in Kapitel 4. Die Codeauszüge dieser deutschen Fassung haben deutsche Kommentare; im Repository sind Kommentare und Meldungen englisch, der Code ist sonst identisch. Das Label jedes Auszugs nennt die Datei.

Kasten	Bedeutung
Wichtig	Fehlerquellen, die zu falschem Verhalten oder Sicherheitslücken führen.
Tipp	Bewährte Praxis, die Arbeit spart oder die Qualität erhöht.
Hinweis	Hintergrund, Einordnung und Verweise auf andere Kapitel.
Was passiert, wenn …?	Ein plausibler Fehler: was man beobachtet, warum es passiert, wie es richtig geht und welcher Test ihn absichert. Der falsche Code steht direkt davor.

> **Öffentliche und interne APIs**
>
> Einige APIs, die Plugins brauchen, liegen in Namespaces mit `.Internal`. EF Core nennt sie „pubternal“: öffentlich sichtbar, aber ohne Kompatibilitätsgarantie. Das Buch kennzeichnet diese Stellen und erklärt in Kapitel 27, wie man damit umgeht.

Inhaltsverzeichnis

I Grundlagen 5

1 Was ist ein EF-Core-Plugin? 6

2 Die Architektur von EF Core aus Plugin-Sicht 8

3 Das Beispielszenario 12

4 Den passenden Erweiterungspunkt finden 15

II Infrastruktur 24

5 Die Options-Extension 25

6 Dienste registrieren 29

7 Öffentliche API und Integration in die Anwendung 36

8 Zustand, Lebensdauern und Thread-Safety 39

III Das Modell 44

9 Annotations und Fluent API 45

10 Conventions 48

11 Modellvalidierung 55

12 Type Mappings und Value Converter 56

IV Abfragen 59

13 Die Query-Pipeline 60

14 Methoden und Member übersetzen 66

15 Clientseitige Auswertung verhindern 70

16 Weitere Erweiterungspunkte der Abfrage 72

17 Materialisierung: von Zeilen zu Objekten 73

V Laufzeit 81

18 Interceptors 82

19 Diagnose und Logging 90

VI Migrations und Design-Time 94

20 Wie Migrations funktionieren 95

21 Deklaratives SQL aus Annotations 96

22 Eigene Migrations-Operationen 99

23 Design-Time-Services 101

24 Compiled Models, Trimming und NativeAOT 103

25 Scaffolding (Reverse Engineering) 106

VII Qualität und Auslieferung 107

26 Teststrategie 108

27 Versionierung und Kompatibilität 112

28 Packaging und Dokumentation 120

29 Die Checkliste von A bis Z 121

VIII Fortgeschrittene Erweiterungspunkte 126

30 Massenoperationen und die Grenzen von Interceptors 127

31 Value Generation 130

32 Die Update-Pipeline 132

33 Complex Types und JSON 133

34 Modelle pro Laufzeitkontext 135

35 Migrations für Fortgeschrittene 136

IX Betrieb und Integration 138

36 Row Level Security in Produktion 139

37 Mandantensichere Beziehungen 143

38 Mandantenfähigkeit nachrüsten und betreiben 146

39 Performance von Plugins 151

40 Einsatz in ASP.NET Core 156

41 CI und API-Stabilität 160

X Tiefe Erweiterungen 162

42 Eigene SQL-Ausdrucksknoten 163

43 Aggregat-Translators 170

44 Fluent API auf Property-Ebene und Laufzeit-Annotations 172

45 Scaffolding in der Praxis 175

XI Mehrere Provider 177

46 Was in den Kern gehört 178

47 Ein zweiter Provider: SQL Server 180

48 Ohne SQL: Was bei Cosmos DB trägt 185

XII Anhänge 189

A Erweiterungspunkte und Lebensdauern 190

B Dateien des Beispiel-Plugins 195

C Glossar 198

D Debugging-Werkzeugkasten 199

E Übungen mit Lösungen 201

F Quellen 203

Lizenz 206

Teil I

Grundlagen

Was ein Plugin ist, wie EF Core intern aufgebaut ist, welches Szenario uns durch das Buch begleitet und welcher Erweiterungspunkt zu welchem Problem passt.

- Kapitel 1: Was ist ein EF-Core-Plugin?
- Kapitel 2: Die Architektur von EF Core aus Plugin-Sicht
- Kapitel 3: Das Beispielszenario
- Kapitel 4: Den passenden Erweiterungspunkt finden

1 Was ist ein EF-Core-Plugin?

Versionsabhängigkeit: **stabil**

Der Begriff „Plugin“ wird für sehr unterschiedliche Dinge verwendet. Für dieses Buch hilft eine Einteilung in vier Stufen, die sich darin unterscheiden, wie tief sie in EF Core eingreifen:

Stufe	Beschreibung	Beispiele
1 · Hilfsmethoden	Extension-Methods auf `DbContext` oder `IQueryable`, die nur öffentliche API aufrufen.	Paging-Helfer, `WhereIf`, Seed-Methoden
2 · Modell-Helfer	Fluent-API-Erweiterungen und Annotations, ohne eigene Dienste in EF Core.	Eigene `HasXyz()`-Methoden, ModelBuilder-Helfer
3 · Echte Plugins	Eigene Dienste im internen Service Provider: Conventions, Query-Übersetzung, Type Mappings, Migrations, Design-Time.	NetTopologySuite-Plugin, NodaTime-Plugin, EFCore.NamingConventions
4 · Provider	Komplette Datenbankanbindung inklusive Verbindung, Update-Pipeline und SQL-Generierung.	Npgsql, Pomelo (MySQL), SQLite, Cosmos (Kapitel 48 baut darauf ein Plugin der Stufe 3)

Dieses Buch behandelt **Stufe 3** vollständig. Stufen 1 und 2 brauchen kein eigenes Buch: Sie nutzen nur die öffentliche API von EF, und Kapitel 9 behandelt, was an Fluent-API-Erweiterungen speziell ist. Die Werkzeuge von Stufe 4 (etwa der Austausch des SQL-Generators) werden dort erklärt, wo Plugins sie brauchen.

Für wen dieses Buch ist

Für Entwickler, die EF Core bereits sicher einsetzen: `DbContext`, Migrationen, LINQ-Abfragen und Dependency Injection werden vorausgesetzt. Kenntnisse der Interna von EF sind nicht nötig; das Buch führt sie dort ein, wo sie gebraucht werden. Erfahrung mit PostgreSQL hilft beim Beispiel, aber jedes SQL-Konstrukt wird beim ersten Auftreten erklärt. Kapitel 2 endet mit einem vollständigen Mini-Plugin, das alle Bausteine auf einer Seite zeigt, bevor das große Beispiel beginnt.

Zwei Lesepfade

Das Buch ist Lehrbuch und Nachschlagewerk zugleich. Zwei Wege führen gezielt hindurch; beide enden mit einem Ergebnis, das sich überprüfen lässt:

	Mein erstes Plugin	Referenz für Fortgeschrittene
Vorkenntnisse	EF Core als Anwender: `DbContext`, Migrationen, LINQ, Dependency Injection	Der erste Pfad oder ein eigenes Plugin in Betrieb
Kapitel	1, 2 (mit dem Mini-Plugin), 3, 5, 6, 7, 10, 14, 18, 26, 28	4 und Anhang A als Einstieg, dann nach Bedarf 8, 13, 17, 20, 27, 36–38 und die Teile X und XI
Ergebnis	Ein eigenes Plugin mit Options-Extension, einer Convention, einem Translator und einem Interceptor, das als Paket baut	Eine begründete Entscheidung für einen Erweiterungspunkt: Lebensdauer, Kombinierbarkeit, API-Status und Upgrade-Risiko

	Mein erstes Plugin	Referenz für Fortgeschrittene
Überprüfbar durch	`MinimalPluginTests` als Vorlage für die eigenen Tests, dazu der Verbrauchertest (Seite 104) auf das eigene Paket übertragen	Die Kompatibilitätsmatrix in Kapitel 6, die Prüfung der Lebensdauern gegen EFs Service-Tabellen und die Checkliste für das Upgrade

Die Übungen in Anhang E gehören zu beiden Pfaden: Jede knüpft an einen Test des Beispiels an, mit dem sich die Lösung nachprüfen lässt.

Was ein Plugin der Stufe 3 leisten kann

Ein vollständiges Plugin kann EF Core an praktisch jeder Stelle erweitern: Es kann neue CLR-Typen als Spaltentypen verfügbar machen, eigene Funktionen in LINQ nach SQL übersetzen, das Modell automatisch per Convention anpassen, Metadaten über die Fluent API und Attribute annehmen, das Modell validieren, beim Speichern und beim Öffnen von Verbindungen eingreifen, eigene Ereignisse protokollieren, Migrationen um eigenes SQL ergänzen und die Werkzeuge von `dotnet ef` erweitern. Das Beispiel in diesem Buch nutzt jede dieser Möglichkeiten.

> **Vorbilder im Ökosystem**
>
> Bevor man selbst schreibt, lohnt der Blick in bestehende Plugins. **EFCore.NamingConventions** ist ein kleines, sauberes Convention-Plugin. Das **NetTopologySuite**-Plugin im EF-Core-Repository zeigt Type Mappings und Übersetzungen. Das **NodaTime**-Plugin von Npgsql zeigt ein provider-spezifisches Plugin, das sich über den Options-Builder des Providers aktivieren lässt.

2 Die Architektur von EF Core aus Plugin-Sicht

Versionsabhängigkeit: **mittel**

Zwei Container: der interne Service Provider

Jeder `DbContext` löst seine Infrastruktur aus einem **internen** `IServiceProvider` auf. Er ist vom DI-Container der Anwendung getrennt und enthält mehrere hundert Dienste: Type Mapping, Model Building, Query-Pipeline, Change Tracking, Update-Pipeline, Migrations. Ein Plugin klinkt sich ein, indem es in diesen Container zusätzliche Dienste registriert oder vorhandene ersetzt.



Zwei Caches sind dabei entscheidend. Der **Service Provider** wird anhand der Options gecacht: Contexts mit gleicher Konfiguration teilen sich einen Provider. Das **Modell** wird pro Provider (und Context-Typ) gecacht. Beides macht EF Core schnell, hat aber direkte Folgen für Plugins, die Kapitel 5 ausführlich behandelt.

Lebensdauern

Jeder EF-Dienst hat eine feste Lebensdauer. **Singleton** heißt hier: einmal pro internem Service Provider, nicht pro Anwendung. **Scoped** heißt: einmal pro Context-Instanz. Ein Singleton darf keinen Scoped-Dienst halten, und ein Dienst, der pro Context unterschiedliche Daten braucht, darf kein Singleton sein. Anhang A listet die Lebensdauern aller Erweiterungspunkte; Kapitel 8 fasst zusammen, welcher Zustand wohin gehört.

Das Dependencies-Muster

EF-Dienste bekommen ihre Abhängigkeiten nicht einzeln, sondern als ein Parameterobjekt, etwa `MigrationsSqlGeneratorDependencies` oder `ProviderConventionSetBuilderDependencies`. Wer von einem EF-Dienst erbt, übernimmt dieses Objekt im Konstruktor und reicht es weiter. Neue

Abhängigkeiten fügt EF Core dem Objekt hinzu, ohne Konstruktoren zu brechen. Für Plugins ist das ein Segen: Abgeleitete Dienste bleiben über Minor-Versionen kompatibel.

`Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Kopf)`

```
public class TenancyMigrationsSqlGenerator : NpgsqlMigrationsSqlGenerator
{
    public TenancyMigrationsSqlGenerator(
        MigrationsSqlGeneratorDependencies dependencies,     // Parameterobjekt von EF
        INpgsqlSingletonOptions npgsqlSingletonOptions)      // zusätzlich vom Provider
        : base(dependencies, npgsqlSingletonOptions)
    {
    }
```

Code-Stil im Beispiel

Alle Klassen im Beispiel verwenden klassische Konstruktoren mit `private readonly`-Feldern und `ArgumentNullException.ThrowIfNull`, keine Primary Constructors. Deren Parameter sind im ganzen Klassenkörper veränderbar, bieten keinen natürlichen Ort für Validierung und verbergen, welcher Zustand gespeichert wird. Klassische Konstruktoren entsprechen außerdem dem Stil von EF Core und Npgsql, von deren Klassen ein Plugin erbt. Einzige Ausnahme sind positionelle Records wie `RlsPolicy`: Dort erzeugt die Syntax echte `init`-Properties.

Erweitern oder ersetzen?

Für Erweiterungen gibt es zwei grundverschiedene Wege. **Plugin-Interfaces** wie `IMethodCallTranslatorPlugin` oder `IConventionSetPlugin` sind ausdrücklich dafür gedacht. Sie sind mehrfach registrierbar, EF fragt alle Registrierungen ab, und mehrere Plugins vertragen sich. **Ersetzen** bedeutet, einen Einzeldienst wie `IMigrationsSqlGenerator` durch eine abgeleitete Klasse auszutauschen. Das ist manchmal unvermeidlich, aber es kann pro Context immer nur einer gewinnen.

Plugin-Interfaces vor Ersetzen

Immer zuerst prüfen, ob es ein Plugin-Interface gibt. Wer einen Einzeldienst ersetzt, kollidiert mit jedem anderen Plugin, das denselben Dienst ersetzt, und muss von der Provider-Implementierung erben. Das bindet das Plugin an einen Provider und an dessen interne API. Kapitel 6 zeigt, wie man trotzdem kombinierbar bleibt.

Für jeden einzelnen Erweiterungspunkt ordnet Kapitel 4 ein, ob er ein Plugin-Interface oder ein Einzeldienst ist und welche einfachere Alternative zuerst in Frage kommt.

Zum Aufwärmen: ein vollständiges Plugin in fünf Teilen

Vor dem großen Beispiel kommt hier das kleinste Plugin, das trotzdem vollständig ist. Es macht PostgreSQLs `reverse()` in LINQ als `EF.Functions.Reverse(...)` verfügbar. Jedes spätere Kapitel vertieft einen seiner fünf Teile:

`tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (1/2, gekürzt)`

```
// 1. Die Funktion, die Anwender in LINQ aufrufen; sie wird nie ausgeführt
public static class ReverseDbFunctionsExtensions
{
```

```
    public static string Reverse(this DbFunctions _, string value)
        => throw new InvalidOperationException("Reverse can only be used in LINQ queries.");
}

// 2. Der Translator macht aus dem Methodenaufruf eine SQL-Funktion
//    (Kapitel „Methoden und Member übersetzen“)
public class ReverseTranslator : IMethodCallTranslator
{
    // Konstruktor speichert die ISqlExpressionFactory in _sql

    public SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method, IReadOnlyList<SqlExpression> arguments,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
        => method == ReverseMethod
            ? _sql.Function("reverse", [arguments[1]], nullable: true,
                argumentsPropagateNullability: [true], typeof(string),
                arguments[1].TypeMapping)
            : null;
}

// 3. Das Plugin übergibt den Translator an EF (Kapitel „Methoden und Member übersetzen“)
public class ReverseTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public ReverseTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new ReverseTranslator(sql)];
    }

    public IEnumerable<IMethodCallTranslator> Translators { get; }
}
```

tests/Acme.EntityFrameworkCore.Tenancy.Tests/MinimalPluginTests.cs (2/2, gekürzt)

```
// 4. Die Options-Extension registriert das Plugin im internen Provider
//    (Kapitel „Die Options-Extension“ und „Dienste registrieren“)
public class ReverseOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    public void ApplyServices(IServiceCollection services)
        => new EntityFrameworkRelationalServicesBuilder(services)
            .TryAdd<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();

    public void Validate(IDbContextOptions options)
    {
    }

    // ExtensionInfo: kein Provider, keine Settings -> Hash 0, immer derselbe Provider
}

// 5. Die Use-Methode aktiviert das Plugin
//    (Kapitel „Öffentliche API und Integration in die Anwendung“)
public static DbContextOptionsBuilder UseReverse(this DbContextOptionsBuilder builder)
{
    ((IDbContextOptionsBuilderInfrastructure)builder).AddOrUpdateExtension(
        builder.Options.FindExtension<ReverseOptionsExtension>()
```

```
        ?? new ReverseOptionsExtension());
    return builder;
}
```

Anwendung

```
options.UseNpgsql(connectionString).UseReverse();

db.Customers.Where(c => EF.Functions.Reverse(c.Name) == "abc");
// SQL: WHERE reverse(c."Name") = 'abc'
```

Mehr braucht ein funktionierendes Plugin nicht. EF findet den Translator, weil die Extension ihn im internen Service Provider registriert, und alle Contexts, die mit `UseReverse()` konfiguriert sind, teilen sich diesen Provider. Weil das Argument auf eine Spalte verweist, braucht die Funktion keinen Evaluatable-Expression-Filter (Kapitel 15 erklärt, wann sie einen braucht). Der Test `MinimalPluginTests` prüft die Übersetzung und dass dieselbe Abfrage ohne das Plugin fehlschlägt. Das Mandanten-Plugin besteht aus genau diesen Teilen, nur aus sehr viel mehr davon.

3 Das Beispielszenario

Versionsabhängigkeit: **stabil**

Das Beispiel-Plugin **Acme.EntityFrameworkCore.Tenancy** macht EF Core mit PostgreSQL mandantenfähig. Die Trennung der Mandanten erzwingt die Datenbank selbst über **Row Level Security** (RLS): Jede Tabelle mit Mandantendaten bekommt eine Policy, die nur Zeilen des Mandanten zeigt, dessen ID in der aktuellen Datenbank-Session gesetzt ist. Abbildungen, bei denen eine solche Tabelle keine Mandantenspalte hätte, lehnt das Plugin beim Modellbau ab (Seite 51). Selbst eine fehlerhafte LINQ-Abfrage oder rohes SQL sieht so nur die Zeilen des gesetzten Mandanten. Wer allerdings beliebiges SQL unter der Anwendungsrolle ausführen kann, kann auch das Session-Setting ändern; diese Grenze behandelt Kapitel 36.

Aus Sicht des Anwenders sieht das so aus:

Anwendersicht

```
public class Order : ITenantOwned               // Marker aus dem Kernpaket
{
    public int Id { get; set; }
    public TenantId TenantId { get; set; }      // stark typisiert, automatisch uuid
    public decimal Total { get; set; }
}

services.AddAcmeTenancy<HttpTenantProvider>();
services.AddDbContext<ShopContext>((sp, o) => o
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, tenancy => tenancy.ApplyQueryFilter()));

// Abfragen und Speichern ohne jede Mandantenlogik im Anwendungscode
db.Orders.Add(new Order { Total = 42 });         // TenantId setzt das Plugin
var mine = await db.Orders.ToListAsync();        // RLS filtert in der Datenbank
```

Damit das funktioniert, nutzt das Plugin jeden Erweiterungspunkt dieses Buchs:

Anforderung	Umsetzung	Kap.
Konfiguration über `UseAcmeTenancy()`	Options-Extension, Settings-Record, Options-Builder	5–7
`TenantId` als Spaltentyp ohne Konfiguration	Type-Mapping-Plugin, Value Converter, JSON-Reader/Writer	12
`EF.Functions.CurrentTenant()` in LINQ	Method-Translator, Evaluatable-Expression-Filter	14–15
`tenantId.IsEmpty` in LINQ	Member-Translator	14
RLS für alle Tabellen mit `ITenantOwned`-Entitäten	Convention mit Index und optionalem Query-Filter	10
Fluent API, Attribut `[NoTenantIsolation]`	Annotations, Attribut-Convention, ConfigurationSource	9–10
Fehlkonfiguration früh erkennen	Validierungs-Convention, eigene Warnungen	11, 19
TenantId automatisch setzen	SaveChanges-Interceptor	18
Mandant in der DB-Session	Connection-Interceptor	18
`ENABLE ROW LEVEL SECURITY`, `CREATE POLICY`	Annotation-Provider, Migrations-SQL-Generator	20–21

Anforderung	Umsetzung	Kap.
Rollen und Rechte in Migrationen	Eigene Migrations-Operationen	22
Lesbare Snapshots	Annotation-Code-Generator, Design-Time-Services	23
Compiled Models und AOT	`Default`-Instanzen, Source-Generatoren	24
`EF.Functions.IsNotDistinctFrom`, `EF.Functions.Every`	Eigener SQL-Knoten, Aggregat-Translator; eigenes Beispielpaket ohne Mandantenbezug	42–43
Mandanten-Property mit beliebigem Namen	`IsTenantProperty()`, Laufzeit-Annotation	44
RLS aus bestehender Datenbank scaffolden	Eigene Database-Model-Factory	45
SQL Server	Zweites Provider-Paket auf demselben Kern	46–47

Paketarchitektur

Das Plugin besteht aus vier NuGet-Paketen. Das **Kernpaket** enthält alles, was unabhängig von der Datenbank ist und was auch Assemblies mit Entitätsklassen referenzieren dürfen, ohne einen Provider mitzuziehen; es funktioniert mit jedem EF-Core-Provider. Das **relationale Paket** ergänzt, was jede SQL-Datenbank braucht: den Session-Interceptor auf der Verbindung und die Prüfung der Tabellenabbildungen. Das **PostgreSQL-Paket** ist die vollständige Implementierung, an der dieses Buch alles erklärt. Das **SQL-Server-Paket** nutzt denselben Kern und dieselbe relationale Schicht und zeigt in Kapitel 47, was ein zweiter Provider selbst umsetzen muss. Ein fünftes, kleines Paket, `Acme.EntityFrameworkCore.Functions.PostgreSQL`, enthält die Beispiele für eigene SQL-Knoten und Aggregate; sie haben nichts mit Mandantenfähigkeit zu tun (Kapitel 42). Ein sechstes, `Acme.EntityFrameworkCore.Tenancy.Cosmos`, baut allein auf dem Kern auf und zeigt, was bei einer Datenbank ohne SQL trägt (Kapitel 48).

Struktur der vier Pakete

```
Acme.EntityFrameworkCore.Tenancy              Kern (nur EF Core, jeder Provider)
├─ TenantId, TenantIdConverter, TenantIdJsonReaderWriter, TenancyException
├─ ITenantOwned, ITenantProvider, NoTenantIsolationAttribute
├─ EF.Functions.CurrentTenant(), TenancyDbFunctionsAttribute
├─ Metadata/         Laufzeit-Annotation, IsTenantProperty(), Kandidatensuche
├─ Conventions/      Value Generation für TenantId-Schlüssel
├─ Query/            Evaluatable-Expression-Filter (für alle Pakete)
├─ Interceptors/     SaveChanges-, Materialization- und Bulk-Update-Interceptor
├─ Diagnostics/      Event-IDs, Logging-Definitionen, Logger-Extensions
└─ DependencyInjection/  AddAcmeTenancy<T>()

Acme.EntityFrameworkCore.Tenancy.Relational    relationale Schicht (EF Core relational)
├─ Interceptors/     abstrakter Session-Interceptor (DbConnection)
├─ Metadata/         TenantTableMappingValidator (TPT, TPC, Entity Splitting)
└─ Database.ApplyTenantSession()

Acme.EntityFrameworkCore.Tenancy.PostgreSQL    vollständige Implementierung
├─ Infrastructure/   Options-Extension, Builder, Singleton-Options
├─ Storage/          TenantIdTypeMapping, Type-Mapping-Plugin
├─ Query/            Translators (CurrentTenant, TenantId-Member)
├─ Metadata/         RLS-Annotations, RlsPolicy, Annotation-Provider
├─ Builders/         Fluent API (HasRowLevelSecurity, HasPolicy, ...)
├─ Conventions/      Isolation, Attribut, Validierung, Convention-Set-Plugin
├─ Migrations/       SQL-Generator, eigene Operationen
```

```
├─ Design/           Code-Generatoren, Design-Time-Services
├─ Scaffolding/      RLS und Policies aus der Datenbank lesen
└─ Runtime/          TenantConnectionInterceptor

Acme.EntityFrameworkCore.Tenancy.SqlServer     zweiter Provider
                                               (Kapitel „Ein zweiter Provider: SQL Server“)
└─ Infrastructure/, Storage/, Query/, Metadata/, Conventions/, Migrations/, Runtime/
```

Tipp

Die Ordnerstruktur folgt der von EF Core selbst (Infrastructure, Storage, Query, Metadata, Migrations, Design). Wer EF Core kennt, findet sich sofort zurecht, und beim Nachschlagen im EF-Quellcode weiß man, wo das Gegenstück liegt.

4 Den passenden Erweiterungspunkt finden

Versionsabhängigkeit: **stabil**

Die Teile dieses Buchs folgen der Architektur von EF Core: Infrastruktur, Modell, Abfragen, Laufzeit, Migrations. Ein konkretes Problem folgt ihr selten. „Jede Zeile gehört einem Mandanten“ lässt sich mit einem Query-Filter, einem SaveChanges-Interceptor, einer Convention oder mit Policies aus Migrationen lösen, und jede dieser Lösungen hat einen anderen Preis und andere Lücken. Das Beispiel-Plugin aus Kapitel 3 kombiniert deshalb mehrere davon.

Dieses Kapitel geht vom Problem aus und führt zum passenden Mechanismus. Es ist eine Landkarte, keine Referenz: Wie ein Mechanismus funktioniert, steht im genannten Kapitel. Den umgekehrten Weg geht Anhang A: Er führt vom Mechanismus zu Lebensdauer, Kardinalität, API-Status, Registrierung und Kombinierbarkeit. Wer das Buch zum ersten Mal liest, überfliegt das Kapitel und kommt mit einem konkreten Problem zurück.

Die geringste Eingriffstiefe

Die wichtigste Regel ist dieselbe wie in „Erweitern oder ersetzen?“ (Kapitel 2), nur weiter gefasst: **Die geringste Eingriffstiefe wählen, die das Problem vollständig löst.** Jede Stufe der folgenden Tabelle kostet mehr Kompatibilität als die vorherige:

Eingriffstiefe	Mittel	Preis
Anwendung	`OnModelCreating`, `ConfigureConventions`, `HasDbFunction`, `migrationBuilder.Sql()`, `SaveChanges` überschreiben	Kein Plugin; jede Anwendung wiederholt den Code
Metadaten	Annotations und Fluent-API-Erweiterungen	Wirken erst, wenn ein Dienst sie liest
Plugin-Interface	Conventions, Translator-, Type-Mapping- und Filter-Plugins, Interceptors	Mehrfach registrierbar, verträgt sich mit anderen Plugins
Dekorieren	Einen Einzeldienst umhüllen und an ihn delegieren	Nur, wenn die öffentliche Schnittstelle genügt; die Aktivierungsreihenfolge zählt
Ersetzen	Einen Einzeldienst durch eine Ableitung der Provider-Klasse ersetzen	Ein Gewinner pro Context; bindet an den Provider und oft an interne API
Tiefer Eingriff	Eigene SQL-Knoten, eigener Differ, Postprocessor	Mehrere Einzeldienste zugleich; mit ähnlichen Plugins kaum kombinierbar

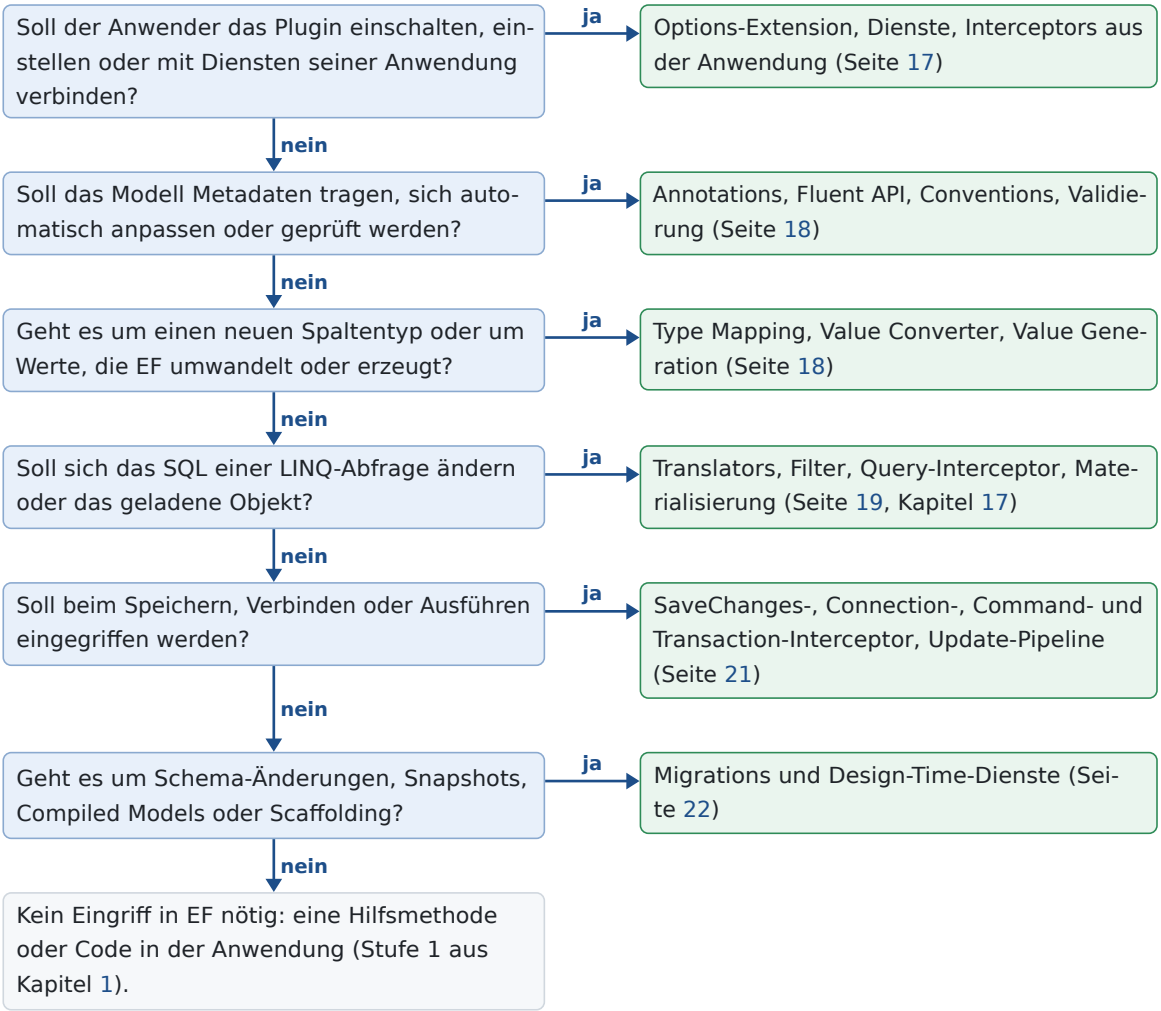
„Vollständig“ ist dabei wörtlich gemeint. Ein Query-Filter ist einfacher als Row Level Security, schützt aber nicht vor rohem SQL (Kapitel 30). Die einfachere Lösung gewinnt nur, wenn sie dieselbe Garantie gibt. Ob sie das tut, klären fünf Fragen:

Frage	Warum sie zählt
Wann läuft der Mechanismus?	Beim Modellbau einmal pro internem Provider, bei der Übersetzung einmal pro Abfrageform, bei der Ausführung jedes Mal, zur Design-Zeit nur in `dotnet ef`. Daraus folgt, welchen Zustand er verwenden darf (Kapitel 39).
Gibt es eine einfachere Alternative?	Die geringere Eingriffstiefe gewinnt, wenn sie dieselbe Garantie gibt.

Frage	Warum sie zählt
Öffentlich oder intern?	Interne API (EF1001) kann sich in jeder Minor-Version ändern (Kapitel 27).
Mehrfach oder einzeln?	Mehrfach registrierbare Dienste vertragen sich mit anderen Plugins; bei einem Einzeldienst gewinnt genau einer.
Welche Lebensdauer?	Ein Singleton darf weder Scoped-Dienste halten noch Daten pro Context; ein Scoped-Dienst darf beides (Kapitel 8).

Der zuständige Bereich

Der erste Schritt ist fast immer die Frage, *wo* das Problem liegt. Der folgende Entscheidungsbaum führt zum zuständigen Bereich; jeder Bereich hat danach einen eigenen Abschnitt mit einer Tabelle.



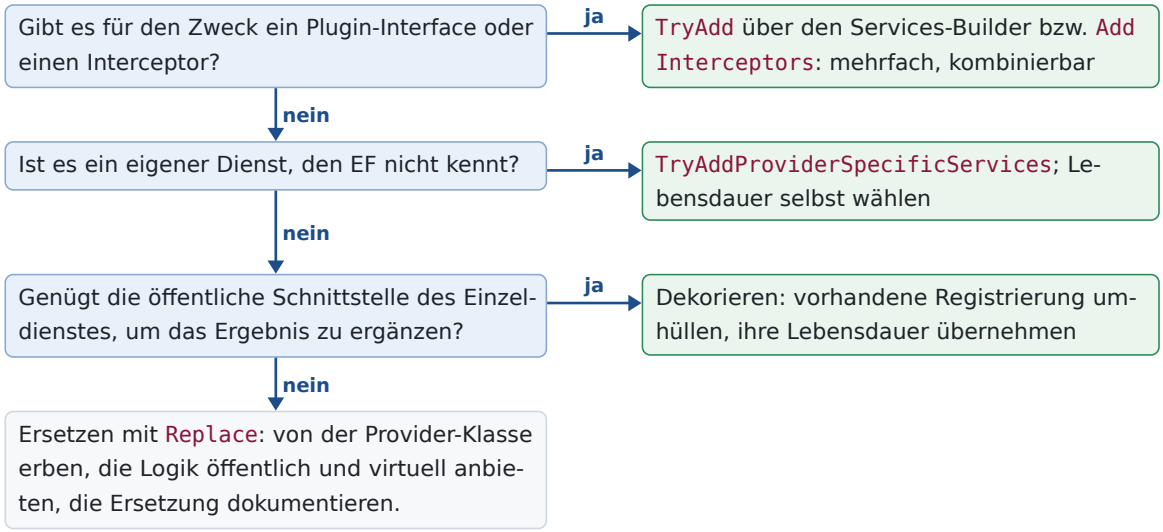
Jede Zeile der Bereichstabellen beschreibt ein Problem, den Mechanismus dafür, die einfachere Alternative, die zuerst geprüft werden sollte, und die typische Fehlentscheidung. Unter dem Mechanismus stehen in grauer Schrift seine Eigenschaften:

Angabe	Bedeutung
öffentlich	Kommt mit der öffentlichen API von EF Core aus.

Angabe	Bedeutung
Provider-Basis	Erbt von der Implementierung des Providers. Bindet an diesen Provider; bei Npgsql liegen die Basisklassen oder ihre Abhängigkeiten meist in `.Internal`-Namespaces (EF1001).
pubternal	Schon EFs eigener Typ liegt in einem `.Internal`-Namespace.
mehrfach, einzeln	Kardinalität laut den Service-Tabellen von EF Core 10 (Anhang A).
kein Dienst	Wirkt über Modell oder Options, nicht über den internen Service Provider.
Singleton, Scoped	Lebensdauer im internen Provider; mit „Design-Time“ im Service Provider von `dotnet ef` (Kapitel 23).

Konfiguration und Dienste

Jedes Plugin der Stufe 3 beginnt mit einer Options-Extension. Die eigentliche Entscheidung fällt bei jedem einzelnen Dienst: Wie kommt er in den internen Provider? Der Baum folgt der Rangfolge aus „Erweitern oder ersetzen?“ (Kapitel 2):



Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Das Plugin soll sich einschalten und einstellen lassen. `IDbContextOptionsExtension`, `Use`-Methode, Options-Builder öffentlich · kein Dienst, Teil der Options	Braucht das Plugin eigene Dienste? Reine Modellkonfiguration kommt mit einer Fluent-API-Erweiterung aus (Stufe 2).	Werte, die pro Context wechseln (Mandant, Benutzer), in Options oder Hash legen; die Extension nach dem Hinzufügen verändern.	5, 7
Ein Singleton-Dienst braucht die Einstellungen. `ISingletonOptions` öffentlich · mehrfach · Singleton	Scoped-Dienste können `IDbContextOptions` direkt injizieren.	Nur unter dem eigenen Interface registrieren: Dann initialisiert EF die Instanz nie.	6
Das Plugin bringt eigene Dienste mit. `ApplyServices` mit dem Services-Builder öffentlich · Lebensdauer aus EFs Tabelle oder selbst gewählt	Der Entscheidungsbaum oben: Plugin-Interface vor Ersetzen.	EF-Dienste mit `AddSingleton` und falscher Lebensdauer registrieren; Plugin-Dienste in den App-Container statt in den internen Provider.	6

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Ein Einzeldienst soll zusätzlich etwas tun. Decorator um die registrierte Implementierung öffentlich · einzeln · Lebensdauer des Originals	Gibt es ein Plugin-Interface für denselben Zweck?	`Replace`, obwohl die öffentliche Schnittstelle genügt; den Decorator vor dem Provider aktivieren.	6
Das Plugin braucht Dienste der Anwendung (Mandant, Benutzer, Uhr). Interceptor-Instanzen aus dem App-Container über `AddInterceptors` öffentlich · mehrfach · Scoped bzw. Singleton	Steht der Wert beim Konfigurieren fest? Dann genügt eine Option.	App-Dienste im internen Provider auflösen wollen; Scoped-Interceptors mit DbContext-Pooling, die den Mandanten der ersten Anfrage behalten.	7, 40

Modell, Typen und Werte

Alles in diesem Bereich läuft beim Modellbau, also einmal pro internem Provider, und landet im gecachten Modell. Laufzeitwerte haben hier deshalb nichts zu suchen.

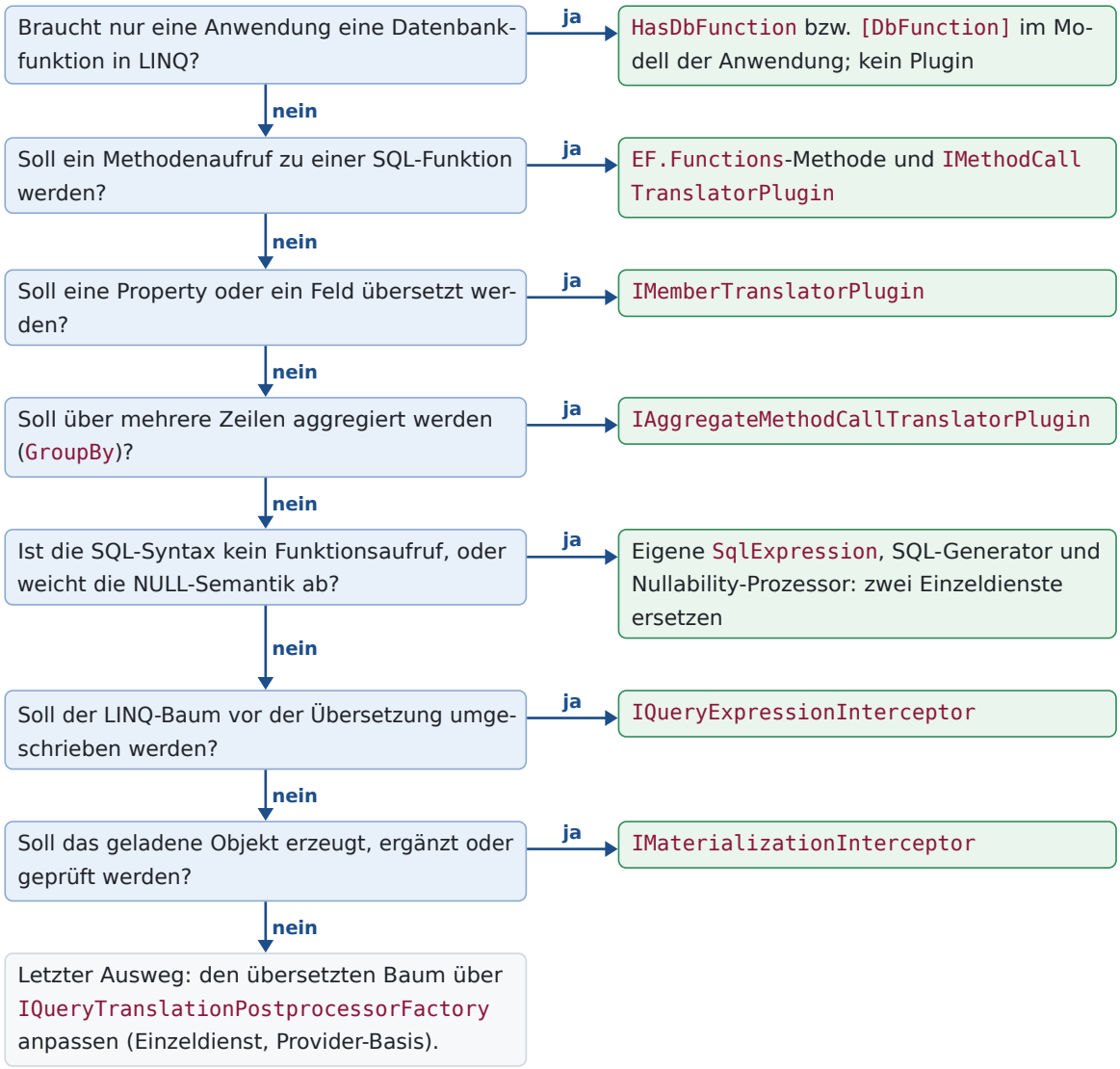
Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Das Plugin muss sich Konfiguration am Modell merken. Annotations mit eigenem Präfix öffentlich · kein Dienst	Deckt eine vorhandene Konfiguration von EF oder Provider den Bedarf (Kommentar, Check-Constraint, Index-Methode)?	Objekte statt einfacher Werte speichern; Default-Werte speichern, was leere Migrationen erzeugt.	9
Anwender sollen das Plugin pro Entität oder Property konfigurieren. Fluent-API-Erweiterung, Convention-Builder-Überladung, Attribut mit Attribut-Convention öffentlich · kein Dienst	Genügt eine globale Einstellung im Options-Builder?	Die Logik in die generische statt in die nicht-generische Variante legen (der Snapshot nutzt diese); in Conventions an den Metadaten statt über den Convention-Builder schreiben, der als einziger stärkere Konfigurationsquellen respektiert.	9, 10, 44
Das Modell soll sich automatisch anpassen. Convention über `IConvention` `SetPlugin` öffentlich · mehrfach · Scoped	Betrifft es nur eine Anwendung? Dann genügen `OnModelCreating` oder `ConfigureConventions`.	Eine reaktive Convention, die von Namen oder Basistypen abhängt, statt `ModelFinalizing`; `IModel` `Customizer` ersetzen (einzeln, Singleton).	10
Fehlkonfiguration soll früh auffallen. `IModelFinalized` `Convention`, Warnungen mit eigenen Event-IDs öffentlich · über das Convention-Set-Plugin	Kann schon die Fluent API die Eingabe prüfen (wie der Policy-Builder)?	`IModelValidator` ersetzen (einzeln, Singleton, Provider-Basis); Fragwürdiges als Exception statt als Warnung melden.	11, 19

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Das Modell hängt von Laufzeitdaten ab (Schema pro Mandant). `IModelCacheKeyFactory` öffentlich · einzeln · Singleton	Lässt sich der Wert zur Laufzeit in SQL auswerten (Parameter, Session-Setting)? Dann bleibt es bei einem Modell.	Laufzeitdaten in den Options-Hash legen (ein interner Provider pro Wert); ungeprüfte Eingaben als Schlüssel.	34
Laufzeitcode braucht Informationen aus dem Modell. Annotation mit einfachem Wert, die im Laufzeitmodell bleibt (Laufzeit-Annotation); für abgeleitete, zwischengespeicherte Werte EFs Runtime-Annotation-API (`GetOrAddRuntimeAnnotationValue`) öffentlich · kein Dienst	Nichts weiter: Einfache Werte übernimmt EF selbst ins Laufzeit- und Compiled Model.	Einen eigenen Code-Generator schreiben, „damit die Annotation erhalten bleibt“; abgeleitete Werte bei jedem Zugriff neu berechnen.	44
Ein eigener CLR-Typ soll ohne Konfiguration als Spalte funktionieren. `IRelationalTypeMappingSourcePlugin` mit eigenem `RelationalTypeMapping` öffentlich · mehrfach · Singleton	Reicht ein Value Converter, den Anwender per `HasConversion` oder `ConfigureConventions` setzen?	`IRelationalTypeMappingSource` ersetzen (einzeln, Provider-Basis); `Clone` oder die statische `Default`-Property vergessen.	12
Werte sollen beim Lesen und Schreiben umgewandelt werden. `ValueConverter` öffentlich · kein Dienst	Soll der Typ überall gelten, eigene Literale oder einen eigenen Spaltentyp haben? Dann gehört der Konverter in ein Type Mapping (Zeile oben).	Erwarten, dass Member des Modelltyps in Abfragen übersetzt werden, obwohl die Datenbank nur den Providerwert kennt; pro Mapping eine neue Konverter-Instanz statt einer gemeinsamen.	12, 24
EF soll Werte für neue Entitäten erzeugen. `ValueGenerator` mit Factory, gesetzt von einer Convention per `HasValueGeneratorFactory(Type)` öffentlich · kein Dienst	Kann die Datenbank den Wert erzeugen (Identity, Sequenz, `HasDefaultValueSql`)?	`HasValueGenerator` statt der Factory-Variante (Compiled Models brechen ab); `IValueGeneratorSelector` ersetzen (einzeln, Scoped, Provider-Basis).	31

Abfragen

An welcher Stufe der Pipeline eine gewünschte Änderung sitzt, zeigt die Landkarte in Kapitel 13; dieser Abschnitt wählt dann den Mechanismus. Die meisten Abfrage-Erweiterungen laufen einmal pro Abfrageform, und das Ergebnis wird gecacht. Werte, die sich pro Ausführung ändern, gehören deshalb als Parameter oder Datenbankfunktion ins SQL, nie als Konstante. Der folgende Baum ist nach steigender Eingriffstiefe geordnet; eine Frage betrifft jede eigene Funktion und steht deshalb vor ihm:

> **Methoden ohne Datenbezug**
>
> Hat eine eigene Methode kein Argument, das auf eine Spalte zeigt (`CurrentTenant()`, Session- oder Zeitfunktionen), braucht sie zusätzlich zum Translator ein `IEvaluatableExpressionFilterPlugin`. Sonst wertet EF sie vorab in .NET aus.



Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Eine Datenbankfunktion soll in LINQ aufrufbar sein. `EF.Functions`-Methode und `IMethodCallTranslatorPlugin` öffentlich · mehrfach · Scoped	`HasDbFunction` in der Anwendung, bei Bedarf mit `HasTranslation`.	Für fremde Methoden eine Exception werfen statt `null` zurückgeben; Laufzeitwerte als Konstante einsetzen; Nullability falsch angeben.	14
Eine Property oder ein Feld eines eigenen Typs soll übersetzt werden. `IMemberTranslatorPlugin` öffentlich · mehrfach · Scoped	Lässt sich der Wert als gemappte oder berechnete Spalte abbilden?	Konstanten ohne das Type Mapping der Spalte erzeugen; dann entsteht ein falsches Literal.	14
Die Funktion hat kein Argument mit Datenbezug. `IEvaluatableExpressionFilterPlugin` zusätzlich zum Translator öffentlich · mehrfach · Singleton	Hat die Methode ein Spaltenargument? Dann ist das Plugin nicht nötig.	Das Plugin vergessen: EF führt die Methode in .NET aus, und die Exception kommt aus ihrem Körper.	15

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Eine Aggregatfunktion soll nach `GroupBy` verfügbar sein. `IAggregateMethodCallTranslatorPlugin` öffentlich · mehrfach · Scoped	Lässt sich das Ergebnis aus vorhandenen Aggregaten bilden (`Count` mit Bedingung, `Min`, `Max`)?	Leere Mengen und NULL-Werte anders behandeln als die .NET-Seite.	43
Die SQL-Syntax ist kein Funktionsaufruf, oder die NULL-Semantik weicht ab. Eigene `SqlExpression` mit `IQuerySqlGeneratorFactory` und `IRelationalParameterBasedSqlProcessorFactory` Provider-Basis · einzeln · Singleton bzw. Scoped; `Quote` experimentell (EF9100)	Lässt sich der Ausdruck mit der `ISqlExpressionFactory` aus Funktionen, Operatoren und `CASE` zusammensetzen?	Den Knoten nur im SQL-Generator behandeln: Ohne Nullability-Prozessor wirft EF, ohne Generator fehlt der Operator stillschweigend.	42
Der LINQ-Baum soll vor der Übersetzung umgeschrieben werden (Query-Rewriting). `IQueryExpressionInterceptor` öffentlich · mehrfach · Singleton	Genügt ein benannter Query-Filter oder ein Translator?	Zustand pro Anfrage verwenden, obwohl das Ergebnis pro Abfrageform gecacht wird; pro Context eine neue Instanz übergeben, was jedes Mal einen neuen internen Provider erzeugt.	16, 30
Der übersetzte Ausdrucksbaum soll angepasst werden. `IQueryTranslationPostprocessorFactory` Provider-Basis · einzeln · Scoped	Lässt es sich vorher erledigen, mit Query-Interceptor oder Translator?	Den Dienst ersetzen, ohne von der Provider-Implementierung zu erben; dann fehlen deren Nachbearbeitungen.	16
Geladene Objekte sollen erzeugt, initialisiert oder geprüft werden (Materialisierung). `IMaterializationInterceptor` öffentlich · mehrfach · Singleton	Konstruktorbindung (EF übergibt `DbContext`, `ILazyLoader`, `IEntityType`) oder das Ereignis `ChangeTracker.Tracked` in der Anwendung.	Zustand pro Context im Interceptor halten; teure Arbeit pro Zeile; pro Context eine neue Instanz übergeben.	17

Ausführung und Speichern

Diese Mechanismen laufen bei jeder Ausführung und liegen damit auf dem heißen Pfad. Sie sehen außerdem nur, was durch EF läuft: Eine Garantie, die auch für rohes SQL oder fremde Werkzeuge gelten soll, gehört in die Datenbank, nicht in einen Interceptor.

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Beim Speichern sollen Werte gesetzt oder Änderungen geprüft werden. `ISaveChangesInterceptor` öffentlich · mehrfach · Scoped	Betrifft es nur eine Anwendung? Dann genügt es, `SaveChanges` zu überschreiben oder das Ereignis `DbContext.SavingChanges` zu abonnieren.	Nur die asynchrone Variante implementieren; sich darauf verlassen, dass `ExecuteUpdate` und rohes SQL vorbeikommen.	18, 30

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Beim Öffnen einer Verbindung soll Session-Zustand gesetzt werden. `IDbConnectionInterceptor` öffentlich · mehrfach · Scoped	Ist der Wert für alle Verbindungen gleich? Dann gehört er in den Connection-String.	Sich auf den Interceptor verlassen, wenn die Anwendung bereits geöffnete Verbindungen übergibt; nur die synchrone oder nur die asynchrone Variante implementieren.	18
Befehle sollen beobachtet oder ergänzt oder ihre Fehler übersetzt werden. `IDbCommandInterceptor` öffentlich · mehrfach · Scoped	`TagWith` für Kommentare im SQL; ein Translator, wenn sich die Bedeutung der Abfrage ändern soll.	`CommandText` per Textersetzung umbauen; das bricht bei jeder Änderung der SQL-Form.	18, 32
Zustand soll an eine Transaktion gebunden sein. `IDbTransactionInterceptor` öffentlich · mehrfach · Scoped	Genügt der Connection-Interceptor? Er genügt, solange kein Transaction-Pooling im Spiel ist.	Transaktionslokale Settings verwenden, aber Abfragen außerhalb von Transaktionen zulassen: Diese laufen ohne Zustand. Im Beispiel: `UseTransactionPooling()`.	36
INSERT, UPDATE oder DELETE sollen eine andere Form bekommen (Upsert). Update-Pipeline, etwa `IUpdateSqlGenerator` Provider-Basis · einzeln · Singleton bzw. Scoped	SaveChanges-Interceptor (Zustände umschreiben), Command-Interceptor oder `ExecuteUpdate`.	Die Pipeline für Soft Delete ersetzen; das leistet ein SaveChanges-Interceptor.	32

Schema und Werkzeuge

Migrations laufen gegen das Design-Time-Modell, die übrigen Werkzeuge von `dotnet ef` in einem eigenen Service Provider. Dienste für diesen Provider registriert ein Plugin über `IDesignTimeServices`, nicht in `ApplyServices`.

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Schema-Objekte sollen aus dem Modell entstehen und seinen Änderungen folgen. Annotations, `IRelationalAnnotationProvider` und `IMigrationsSqlGenerator` Provider-Basis · einzeln · Singleton bzw. Scoped	`migrationBuilder.Sql()` in den Migrationen der Anwendung, vorhandene Konfiguration wie Check-Constraints.	Die Annotations auch ins Laufzeitmodell kopieren (`designTime` ignorieren); Bezeichner per String-Verkettung statt über den SQL-Helfer.	20, 21
Migrationen brauchen Schritte, die nicht ins Modell gehören (Rollen, Rechte). Eigene `MigrationOperation` mit Erweiterung für `MigrationBuilder`, SQL im Generator öffentlich · Generator: Provider-Basis · einzeln · Scoped	`migrationBuilder.Sql()` genügt, wenn das Plugin den Schritt nicht für viele Anwendungen anbieten muss.	Operationen nicht idempotent formulieren; die Reihenfolge zu den generierten Operationen nicht beachten.	22, 35

Problem und Mechanismus	Zuerst prüfen	Typische Fehlentscheidung	Kap.
Der Differ soll eigene Operationen selbst erzeugen. `IMigrationsModelDiffer` und `ICSharpMigrationOperationGenerator` pubternal · einzeln · Scoped bzw. Singleton (Design-Time)	Der deklarative Weg über Annotations, fast immer die bessere Wahl.	Den Differ ersetzen, nur weil eine geänderte Annotation SQL braucht; das leistet der SQL-Generator.	22
Snapshots sollen Fluent API statt `HasAnnotation` zeigen. `IAnnotationCodeGenerator`, geladen über `IDesignTimeServices` und `DesignTimeServicesReference` Provider-Basis · einzeln · Singleton (Design-Time)	Nichts tun: `HasAnnotation` im Snapshot funktioniert, ist nur schwer lesbar.	Design-Time-Dienste in `ApplyServices` registrieren, also im Laufzeit-Container; übersetzte Annotations nicht aus dem Dictionary entfernen.	23
Das Compiled Model soll Design-Time-Annotations weglassen oder komplexe Werte enthalten. `ICSharpRuntimeAnnotationCodeGenerator` pubternal · einzeln · Singleton (Design-Time)	Einfache Annotationswerte übernimmt EF ohne eigenen Generator.	Laufzeit-Annotations mitfiltern, die ein Interceptor braucht; das relationale Modell vergessen, das EF für NativeAOT zusätzlich schreibt.	24, 44
Scaffolding soll die Schema-Objekte des Plugins erkennen. `IDatabaseModelFactory` und `IAnnotationCodeGenerator` Provider-Basis · einzeln · Scoped bzw. Singleton (Design-Time)	T4-Vorlagen (`dotnet new ef-templates`), wenn nur der erzeugte Code anders aussehen soll.	Ein eigenes Datenformat statt der Annotations der Fluent API verwenden; dann passen Snapshot und Migrations nicht dazu.	25, 45

Vier Fehlentscheidungen, die in jedem Bereich vorkommen

Ersetzen, wo ein Plugin-Interface oder ein Decorator genügt. Jede Ersetzung schließt ein anderes Plugin aus (Kapitel 6).

Laufzeitwerte dort ablegen, wo EF cacht. Options-Hash, Modell und übersetzte Abfragen werden wiederverwendet. Werte pro Anfrage gehören in Parameter, in die Datenbank-Session oder in Interceptors aus der Anwendung.

Einen Laufzeit-Haken für eine Garantie halten. Interceptors sehen nur, was durch EF läuft (Kapitel 30).

Design-Time und Laufzeit verwechseln. Design-Time-Dienste gehören nicht in den internen Provider, Design-Time-Annotations nicht ins Laufzeit- oder Compiled Model.

Teil II

Infrastruktur

Wie ein Plugin konfiguriert wird, seine Dienste registriert, sich mit dem DI-Container der Anwendung verbindet und wo welcher Zustand hingehört.

- Kapitel 5: Die Options-Extension
- Kapitel 6: Dienste registrieren
- Kapitel 7: Öffentliche API und Integration in die Anwendung
- Kapitel 8: Zustand, Lebensdauern und Thread-Safety

5 Die Options-Extension

Versionsabhängigkeit: **mittel**

Der Einstiegspunkt jedes echten Plugins ist eine Klasse, die `IDbContextOptionsExtension` implementiert. Sie hat drei Aufgaben: Sie trägt die Konfiguration, sie registriert in `ApplyServices` die Dienste im internen Provider, und sie beschreibt sich über `Info` gegenüber dem Cache.

Konfiguration als Record

Alle modellrelevanten Einstellungen des Beispiels liegen in einem Record. Records liefern `Equals` und `GetHashCode` über alle Felder, was gleich für den Cache-Schlüssel gebraucht wird. Neue Einstellungen fügt man dem Record hinzu, ohne Hash und Vergleich anfassen zu müssen.

Infrastructure/TenancySettings.cs

```
public sealed record TenancySettings
{
    public static TenancySettings Default { get; } = new();

    public string SettingName { get; init; } = "app.tenant_id";
    public string PolicyName { get; init; } = "tenant_isolation";
    public bool ForceRowLevelSecurity { get; init; } = true;
    public bool ApplyQueryFilter { get; init; }
    public bool CreateTenantIndex { get; init; } = true;
    public bool VerifyMaterializedTenants { get; init; }
}
```

Immutabilität

EF vergleicht und cached Options. Eine Extension darf sich nach dem Hinzufügen deshalb nie mehr ändern; jede Änderung erzeugt eine Kopie. Was sonst passiert, zeigt Fehlerfall 4 in Kapitel 8: Neue Contexts arbeiten still mit dem alten Wert weiter. Das Muster aus EF Core selbst: ein geschützter Kopierkonstruktor und eine virtuelle `Clone`-Methode, damit abgeleitete Extensions ihren eigenen Typ klonen.

Infrastructure/TenancyOptionsExtension.cs (Auszug, vereinfacht)

```
public class TenancyOptionsExtension : IDbContextOptionsExtension
{
    private DbContextOptionsExtensionInfo? _info;

    public TenancyOptionsExtension()
    {
    }

    protected TenancyOptionsExtension(TenancyOptionsExtension copyFrom)
    {
        ArgumentNullException.ThrowIfNull(copyFrom);

        Settings = copyFrom.Settings;
    }
```

```
    public virtual TenancySettings Settings { get; private set; } = TenancySettings.Default;

    public virtual DbContextOptionsExtensionInfo Info => _info ??= new ExtensionInfo(this);

    protected virtual TenancyOptionsExtension Clone() => new(this);

    public virtual TenancyOptionsExtension WithSettings(TenancySettings settings)
    {
        var clone = Clone();
        clone.Settings = settings;
        return clone;
    }
```

Die Info-Klasse und der Service-Provider-Cache

Die Info-Klasse entscheidet, ob EF einen gecachten internen Service Provider wiederverwenden darf. Dafür gibt es zwei Methoden, die zusammenpassen müssen: `GetServiceProviderHashCode` als schneller Vorfilter und `ShouldUseSameServiceProvider` als exakter Vergleich.

Infrastructure/TenancyOptionsExtension.cs › ExtensionInfo (vereinfacht)

```
private sealed class ExtensionInfo : DbContextOptionsExtensionInfo
{
    public ExtensionInfo(TenancyOptionsExtension extension)
        : base(extension)
    {
    }

    private new TenancyOptionsExtension Extension => (TenancyOptionsExtension)base.Extension;

    public override bool IsDatabaseProvider => false;

    public override string LogFragment
        => $"AcmeTenancy(Setting={Extension.Settings.SettingName}) ";

    public override int GetServiceProviderHashCode() => Extension.Settings.GetHashCode();

    public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
        => other is ExtensionInfo o && o.Extension.Settings == Extension.Settings;

    public override void PopulateDebugInfo(IDictionary<string, string> debugInfo)
    {
        debugInfo["Acme:Tenancy:Force"] =
            Extension.Settings.ForceRowLevelSecurity.ToString();
        // ... weitere Einstellungen
    }
}
```

Bei diesen beiden Methoden gibt es zwei klassische Fehler mit gegensätzlichen Folgen:

Fehler	Folge
Modellrelevante Option fehlt im Hash	Zwei Contexts mit unterschiedlicher Konfiguration teilen sich Provider und Modell. Der zweite bekommt das gecachte, **falsche** Modell. Fällt im Test oft nicht auf, weil dort nur eine Konfiguration läuft.

Fehler	Folge
Pro Context variierender Wert im Hash (Mandant, Connection-String, Instanz)	Für jeden Context entsteht ein neuer interner Provider samt Modell. Speicher und Startzeit explodieren; ab dem 21. Provider bricht EF standardmäßig mit `ManyServiceProvidersCreatedWarning` ab (Fehlerfall 2 in Kapitel 8).

Der erste Fehler entsteht typischerweise, wenn ein Plugin ohne Einstellungen später eine bekommt und die Info-Klasse aus der ersten Version stehen bleibt:

`Info-Klasse (FALSCH)`

```
// Aus der ersten Version ohne Einstellungen; SettingName kam später dazu
public override int GetServiceProviderHashCode() => 0;

public override bool ShouldUseSameServiceProvider(DbContextOptionsExtensionInfo other)
    => other is ExtensionInfo;
```

Was passiert, wenn der Hash eine Einstellung vergisst?

Beobachtbar. Ein Context mit `app.b` läuft auf dem Provider, den EF für `app.a` gebaut hat: Singletons und Modell tragen den Wert der ersten Konfiguration. Es gibt weder Fehler noch Warnung, und Tests mit nur einer Konfiguration sehen nichts.

Ursache. Hash und Vergleich sind der Schlüssel des Provider-Caches. Sagen beide „gleich“, verwendet EF den gecachten Provider; `ISingletonOptions` wird nicht neu initialisiert, und `Validate` läuft im Normalfall nicht (Kapitel 8).

Lösung. Alle modellrelevanten Einstellungen in Hash und Vergleich aufnehmen, am einfachsten als Record wie `TenancySettings`.

Test: `FailureCaseTests.Setting_missing_from_the_hash_…`; die Gegenprobe ist `OptionsTests.Different_settings_produce_different_models_…`.

Die Faustregel für den Hash

In den Hash gehört genau das, was **beeinflusst, welche Dienste registriert werden oder wie das Modell aussieht**. Alles andere gehört nicht hinein. Der Mandant selbst ändert sich pro Request und darf deshalb nie in den Options landen; er kommt zur Laufzeit über einen Interceptor (Kapitel 18).

`LogFragment` erscheint in EFs Startmeldung („initialized using provider … with options: …“) und hilft bei der Fehlersuche. `PopulateDebugInfo` liefert Werte für EFs Diagnose, wenn zu viele Service Provider entstehen. `IsDatabaseProvider` ist nur für echte Provider `true`.

Validierung

`Validate` wird beim Erstellen des Service Providers aufgerufen. Hier prüft das Plugin Voraussetzungen, die sich aus der Kombination von Extensions ergeben, und meldet sie mit einer verständlichen Fehlermeldung statt eines späteren Folgefehlers:

`Infrastructure/TenancyOptionsExtension.cs › Validate`

```
public virtual void Validate(IDbContextOptions options)
{
    if (options.FindExtension<NpgsqlOptionsExtension>() is null)
```

```
        throw new InvalidOperationException(
            "UseAcmeTenancy() erfordert UseNpgsql(). Für andere Datenbanken wird ein "
            + "eigenes Provider-Paket benötigt.");
}
```

6 Dienste registrieren

Versionsabhängigkeit: **mittel**

`ApplyServices` füllt den internen Service Provider. EF stellt dafür einen Builder bereit, der die Lebensdauer und Kardinalität jedes EF-Dienstes kennt: `EntityFrameworkServicesBuilder` für Kerndienste und `EntityFrameworkRelationalServicesBuilder` zusätzlich für relationale Dienste. Er registriert jeden Dienst mit der richtigen Lebensdauer und nutzt für mehrfach registrierbare Dienste automatisch `TryAddEnumerable`.

`Infrastructure/TenancyOptionsExtension.cs › ApplyServices`

```
public virtual void ApplyServices(IServiceCollection services)
{
    // 1) Plugin-Interfaces: mehrfach registrierbar, kombinierbar mit anderen Plugins
    new EntityFrameworkRelationalServicesBuilder(services)
        .TryAdd<IRelationalTypeMappingSourcePlugin, TenancyTypeMappingSourcePlugin>()
        .TryAdd<IMethodCallTranslatorPlugin, TenancyMethodCallTranslatorPlugin>()
        .TryAdd<IMemberTranslatorPlugin, TenancyMemberTranslatorPlugin>()
        .TryAdd<IEvaluatableExpressionFilterPlugin,
                TenancyEvaluatableExpressionFilterPlugin>()
        .TryAdd<IConventionSetPlugin, TenancyConventionSetPlugin>()
        .TryAdd<ISingletonOptions, ITenancySingletonOptions>(
            p => p.GetRequiredService<ITenancySingletonOptions>())
        // 2) Eigene Dienste, die EF nicht kennt
        .TryAddProviderSpecificServices(m => m
            .TryAddSingleton<ITenancySingletonOptions, TenancySingletonOptions>()
            .TryAddSingleton<TenancyLoggingDefinitions, TenancyLoggingDefinitions>());

    // 3) Einzeldienste des Providers ersetzen
    services.Replace(ServiceDescriptor
        .Scoped<IMigrationsSqlGenerator, TenancyMigrationsSqlGenerator>());
    services.Replace(ServiceDescriptor
        .Singleton<IRelationalAnnotationProvider, TenancyAnnotationProvider>());
}
```

Methode	Wann	Warum
`TryAdd<TService, TImpl>()`	EF-Dienste und Plugin-Interfaces	Lebensdauer und Mehrfachregistrierung kommen aus EFs Tabelle. Falsche Lebensdauern führen sonst zu Exceptions oder subtil falschem Verhalten.
`TryAddProviderSpecificServices`	Eigene Dienste, die EF nicht kennt	Der Builder lehnt unbekannte Typen ab; hier legt man die Lebensdauer selbst fest.
`services.Replace(...)`	Einzeldienst des Providers ersetzen	Der Provider registriert per `TryAdd`. `Replace` gewinnt unabhängig davon, ob das Plugin vor oder nach dem Provider läuft.

> **Warum Replace in beiden Reihenfolgen funktioniert**
>
> EF ruft `ApplyServices` der Extensions in der Reihenfolge auf, in der sie hinzugefügt wurden. Läuft das Plugin zuerst, fügt `Replace` den Dienst hinzu und das spätere `TryAdd` des Providers greift nicht mehr. Läuft es danach, ersetzt `Replace` den Provider-Dienst. Beide Reihenfolgen führen zum selben Ergebnis.

Wer den Builder umgeht, wählt die Lebensdauer selbst, und die naheliegende Wahl ist falsch:

ApplyServices (FALSCH)

```
public void ApplyServices(IServiceCollection services)
    => services.AddSingleton<IMethodCallTranslatorPlugin, ReverseTranslatorPlugin>();
```

Was passiert, wenn ein Plugin-Interface die falsche Lebensdauer bekommt?

Beobachtbar. Registrierung und Modellbau laufen durch. Erst die erste Abfrage, die die Funktion verwendet, bricht ab, mit `ArgumentNullException`: „Value cannot be null. (Parameter 'provider')“. Die Meldung nennt weder das Plugin noch die Lebensdauer.

Ursache. EF erwartet Translator-Plugins als Scoped. Als Singleton entsteht das Plugin im Root-Scope des internen Providers, wo sich die Scoped-Abhängigkeit `ISqlExpressionFactory` nicht auflösen lässt. Der interne Provider prüft Lebensdauern nicht (Kapitel 8).

Lösung. EF-Dienste und Plugin-Interfaces immer über den Services-Builder registrieren; `TryAdd` übernimmt die Lebensdauer aus EFs Tabelle.

Test: `FailureCaseTests.Plugin_interface_registered_as_singleton_…`.

Singleton-Options

Singleton-Dienste wie Type-Mapping-Plugins können `IDbContextOptions` nicht pro Context auflösen. Für sie gibt es `ISingletonOptions`: ein Singleton, das EF einmal beim Erstellen des Providers mit den Options initialisiert. Wann EF `Initialize` und `Validate` aufruft, erklärt Kapitel 8.

Infrastructure/ITenancySingletonOptions.cs (vereinfacht)

```
public class TenancySingletonOptions : ITenancySingletonOptions
{
    public TenancySettings Settings { get; private set; } = TenancySettings.Default;

    // Einmal pro internem Service Provider
    public virtual void Initialize(IDbContextOptions options)
        => Settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                      ?? TenancySettings.Default;

    // Optionen dürfen nicht vom Stand des Providers abweichen (geteilter Provider)
    public virtual void Validate(IDbContextOptions options)
    {
        var settings = options.FindExtension<TenancyOptionsExtension>()?.Settings
                       ?? TenancySettings.Default;
        if (settings != Settings)
            throw new InvalidOperationException("Tenancy-Optionen weichen ab ...");
    }
}
```

Die doppelte Registrierung in `ApplyServices` ist nötig: einmal unter dem eigenen Interface, damit Dienste es injizieren können, und einmal als `ISingletonOptions`, damit EF es findet und initialisiert. Die Factory-Variante sorgt dafür, dass beide Registrierungen *dieselbe* Instanz liefern.

Kombinierbarkeit trotz Ersetzen

Ersetzt ein Plugin einen Einzeldienst, kann ein zweites Plugin, das dasselbe tut, nicht gleichzeitig aktiv sein. Professionelle Plugins entschärfen das, indem sie die eigentliche Logik zusätzlich öffentlich und wiederverwendbar anbieten. Wie eine Anwendung damit zwei Plugins zusammenführt, zeigt der Abschnitt „Integration als Ausweg“ (Seite 34); diese Möglichkeit gehört in die Dokumentation.

Noch besser vermeidet ein Plugin den Konflikt ganz, indem es den Dienst **dekoriert** statt ersetzt: Es nimmt die Implementierung, die zu diesem Zeitpunkt registriert ist (die des Providers oder eines anderen Plugins), umhüllt sie und delegiert an sie. Ein Beispiel ist ein (fiktives) Audit-Plugin, das an jede neue Tabelle einen Trigger hängt. In seinem `ApplyServices` sucht es die vorhandene Registrierung und ersetzt nur den Deskriptor, nicht die Logik:

tests/Acme.EntityFrameworkCore.Tenancy.Tests/CompositionTests.cs › AuditOptionsExtension

```
public void ApplyServices(IServiceCollection services)
{
    // Bisher registriert: der Generator des Providers oder eines anderen Plugins
    var inner = services.LastOrDefault(d => d.ServiceType == typeof(IMigrationsSqlGenerator))
        ?? throw new InvalidOperationException("Call UseContosoAudit() after UseNpgsql().");

    services.Replace(ServiceDescriptor.Describe(
        typeof(IMigrationsSqlGenerator),
        sp => new AuditMigrationsSqlGenerator(
            (IMigrationsSqlGenerator)CreateInstance(sp, inner),
            sp.GetRequiredService<MigrationsSqlGeneratorDependencies>()),
        inner.Lifetime));     // Lebensdauer des Originals
}

// Erzeugt die Implementierung so, wie es der ursprüngliche Deskriptor getan hätte
private static object CreateInstance(IServiceProvider sp, ServiceDescriptor descriptor)
    => descriptor.ImplementationInstance
       ?? descriptor.ImplementationFactory?.Invoke(sp)
       ?? ActivatorUtilities.CreateInstance(sp, descriptor.ImplementationType!);
```

Der Trick funktioniert, weil EF `ApplyServices` in der Reihenfolge aufruft, in der die Extensions hinzugefügt wurden; wenn das Audit-Plugin läuft, hat der Provider seinen Generator bereits registriert. `CreateInstance` bildet die ursprüngliche Registrierung exakt nach: eine Instanz, eine Factory oder einen Typ, dessen Abhängigkeiten (etwa Npgsqls Singleton-Options) `ActivatorUtilities` aus dem Container auflöst. Der Decorator selbst nutzt nur die öffentliche Schnittstelle:

tests/.../CompositionTests.cs › AuditMigrationsSqlGenerator (gekürzt)

```
public class AuditMigrationsSqlGenerator : IMigrationsSqlGenerator
{
    // Konstruktor: (IMigrationsSqlGenerator inner, MigrationsSqlGeneratorDependencies deps)

    public IReadOnlyList<MigrationCommand> Generate(
        IReadOnlyList<MigrationOperation> operations,
        IModel? model = null,
        MigrationsSqlGenerationOptions options = MigrationsSqlGenerationOptions.Default)
    {
        var commands = _inner.Generate(operations, model, options).ToList();   // unverändert
        var builder = new MigrationCommandListBuilder(_dependencies);
        var sql = _dependencies.SqlGenerationHelper;
```

```
        // Am Ende angehängt: dann existieren alle Tabellen der Migration
        foreach (var table in operations.OfType<CreateTableOperation>())
        {
            builder.Append("CREATE TRIGGER audit_changes ")
                .Append("AFTER INSERT OR UPDATE OR DELETE ON ")
                .Append(sql.DelimitIdentifier(table.Name, table.Schema))
                .Append(" FOR EACH ROW EXECUTE FUNCTION audit.log_change()")
                .AppendLine(sql.StatementTerminator)
                .EndCommand();
        }

        commands.AddRange(builder.GetCommandList());
        return commands;
    }
}
```

In der Anwendung lassen sich beide Plugins ohne eine Zeile Verbindungscode kombinieren:

`Program.cs (vereinfacht)`

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp)       // ersetzt IMigrationsSqlGenerator
    .UseContosoAudit());      // umhüllt, was registriert ist: hier Acmes
```

Dekorieren, wo möglich; ersetzen, wo nötig

Decorators lassen sich in beliebiger Reihenfolge kombinieren, weil jeder umhüllt, was er vorfindet. Ein Plugin, das `Replace` verwendet, verwirft dagegen alles, was vor ihm registriert wurde, auch Decorators. Ein ersetzendes Plugin muss deshalb zuerst aktiviert werden, und ein Decorator muss prüfen, ob es etwas zu dekorieren gibt, und sonst mit einer klaren Meldung abbrechen (Provider fehlt oder wurde später hinzugefügt).

Dekorieren setzt voraus, dass die öffentliche Schnittstelle genügt. Das Tenancy-Plugin braucht geschützte Einstiegspunkte innerhalb von `CREATE TABLE` und erbt deshalb vom Npgsql-Generator. Für solche Fälle bleibt der oben beschriebene Ausweg: öffentliche Logik wie `CreatePolicySql`, virtuelle Methoden und `ReplaceService` in der Anwendung.

Tipp

Der Test `CompositionTests` im Beispiel enthält das vollständige Audit-Plugin. Er prüft, dass der Decorator Acmes Generator umhüllt, dass die Migration das SQL beider Plugins enthält und dass das Aktivieren vor dem Provider fehlschlägt.

Zwei Architekturen: Mehrfachregistrierung und Einzeldienst

Ob zwei Plugins nebeneinander funktionieren, entscheidet sich an einer Stelle: Fragt EF *alle* Registrierungen eines Dienstes oder genau *eine*? Bei einem Plugin-Interface ist die Zusammenarbeit Teil von EFs Entwurf. EF fragt alle Translator-Plugins, bis eines antwortet, lässt jedes Convention-Set-Plugin seine Conventions ergänzen und ruft jeden Interceptor auf. Zwei Plugins müssen nichts voneinander wissen.

Bei einem Einzeldienst gibt es diese Zusammenarbeit nicht. EF fragt genau eine Implementierung, und die muss alles können, was alle aktiven Plugins brauchen. Ersetzen zwei Plugins denselben Dienst, gewinnt eines, und das andere verliert seine Funktion, ohne dass EF warnt. Komposition ist

dann nicht mehr EFs Aufgabe, sondern die der Plugin-Autoren: über Decorators, die delegieren, oder über eine Integration in der Anwendung.

Die Spielregeln in EF Core 10

Die Methoden aus der Tabelle am Anfang dieses Kapitels verhalten sich bei zwei Plugins so. Jede Zeile ist in `CompositionMatrixTests` nachgestellt:

Mechanismus	Verhalten	Folge
Options-Extensions	Eine Extension pro Typ: Ein zweites `AddOrUpdateExtension` desselben Typs ersetzt die erste. `ApplyServices` läuft in Aktivierungsreihenfolge; jede Extension trägt zum Provider-Schlüssel bei	Verschiedene Plugins koexistieren; die Use-Methode holt die vorhandene Extension und ändert eine Kopie
`TryAdd` für ein Plugin-Interface	Fügt hinzu, außer dieselbe Implementierung ist schon registriert (`TryAddEnumerable`)	Idempotent, kombinierbar
`TryAdd` für einen Einzeldienst	Die erste Registrierung gewinnt. Der Provider registriert per `TryAdd`: Ein Plugin vor dem Provider gewinnt, danach verliert es still	Für Einzeldienste ungeeignet
`Replace`	Die letzte Registrierung gewinnt, unabhängig vom Provider	Verwirft frühere Ersetzungen und Decorators still
Decorator	Umhüllt, was zu seinem Zeitpunkt registriert ist	Muss nach Provider und Ersetzungen laufen
`ReplaceService` (Anwendung)	Wird nach allen Extensions angewendet und gewinnt immer	Das Werkzeug der Anwendung für die Integration
Translator-Plugins	Plugins kommen vor den Translatoren des Providers; zwischen Plugins antwortet das zuerst aktivierte	Gleiche Methode: Reihenfolge entscheidet
Convention-Set-Plugins	Laufen in Aktivierungsreihenfolge; spätere Conventions sehen die Ergebnisse früherer	Gleiche Metadaten: Reihenfolge und Konfigurationsquelle entscheiden
Interceptors	Injizierte zuerst, dann die aus `AddInterceptors` in deren Reihenfolge	Wer ein Ergebnis unterdrückt oder ersetzt, beeinflusst die nachfolgenden

Kompatibilitätsmatrix

Die folgende Matrix bewertet typische Kombinationen zweier Plugins. Die Bewertungen bedeuten: **kombinierbar** (funktioniert ohne Absprache), **reihenfolgeabhängig** (funktioniert, wenn die Aktivierungsreihenfolge stimmt), **potenzieller Konflikt** (funktioniert, solange sich die Plugins nicht überschneiden) und **nur mit Integration** (ohne Code in der Anwendung geht eine Funktion verloren).

Kombination	Bewertung	Begründung
Zwei `IMethodCallTranslatorPlugin`	kombinierbar; potenzieller Konflikt	Verschiedene Methoden: beide aktiv. Dieselbe Methode: Das zuerst aktivierte gewinnt, das andere wird nie gefragt
Zwei `IMemberTranslatorPlugin`	kombinierbar; potenzieller Konflikt	Derselbe Mechanismus wie bei Method-Translatoren
Zwei `IConventionSetPlugin`	kombinierbar; reihenfolgeabhängig	Beide laufen. Setzen beide dieselben Metadaten mit Convention-Quelle, gewinnt das spätere; explizite Konfiguration schlägt beide
Zwei Type-Mapping-Plugins für denselben CLR-Typ	potenzieller Konflikt	EF nimmt das erste Mapping, das nicht `null` ist; das andere Plugin ist für diesen Typ wirkungslos
Zwei Interceptors desselben Typs	kombinierbar; reihenfolgeabhängig	Beide laufen. Wer ein Ergebnis unterdrückt oder einen Befehl umschreibt, wirkt auf die nachfolgenden
Zwei Plugins ersetzen `IMigrationsSqlGenerator`	nur mit Integration	Das zuletzt aktivierte gewinnt; das erste verliert sein SQL ohne Warnung (im Test: keine Policies mehr)
Zwei Plugins ersetzen `IQuerySqlGeneratorFactory`	nur mit Integration	Der Verlierer behält seine Translatoren, aber seine Knoten erreichen einen Generator, der sie nicht kennt: Das SQL fehlt still. Ersetzt der Gewinner auch den parameterabhängigen Prozessor, wirft die Abfrage (Kapitel 42)
Decorator und Ersetzung	reihenfolgeabhängig	Ersetzung zuerst, dann Decorator: Er umhüllt die Ersetzung. Umgekehrt verwirft `Replace` den Decorator still
Zwei Decorators	kombinierbar	Jeder umhüllt, was er vorfindet
Zwei Options-Extensions verschiedener Typen	kombinierbar	Beide tragen zum Schlüssel bei, beide registrieren ihre Dienste; Konflikte entstehen erst bei den Diensten
Zwei Plugins mit gleichnamiger Use-Methode, Annotation-Präfix oder Event-ID-Bereich	potenzieller Konflikt	Mehrdeutige Aufrufe, vermischte Metadaten oder Meldungen (Kapitel 46, 9, 19)
Plugin-Ersetzung und `ReplaceService` der Anwendung	kombinierbar, wenn gewollt	Die Anwendung gewinnt immer; das ist der vorgesehene Weg zur Integration

Integration als Ausweg

Was „nur mit Integration“ geht, löst die Anwendung mit einer eigenen Klasse, die beide Plugins zusammenführt, und registriert sie mit `ReplaceService`. Möglich ist das nur, wenn mindestens eines der Plugins seine Logik öffentlich anbietet: Die Anwendung erbt von dessen Klasse und ruft die Logik des anderen auf. Genau deshalb sind im Beispiel alle Generator-Methoden virtuell und `CreatePolicySql` öffentlich:

Anwendung (Skizze)

```
// Beide Plugins ersetzen IMigrationsSqlGenerator; die Anwendung führt sie zusammen
public class ShopMigrationsSqlGenerator : TenancyMigrationsSqlGenerator
{
    // Konstruktor wie TenancyMigrationsSqlGenerator
```

```
    protected override void Generate(CreateTableOperation operation, IModel? model,
        MigrationCommandListBuilder builder, bool terminate = true)
    {
        base.Generate(operation, model, builder, terminate);   // Acme: Tabelle und RLS
        OtherPluginSql.AppendTableSql(operation, builder);     // Logik des anderen Plugins
    }
}

options.UseNpgsql(connectionString)
       .UseAcmeTenancy(sp)
       .UseOtherPlugin()
       .ReplaceService<IMigrationsSqlGenerator, ShopMigrationsSqlGenerator>();
```

Ein Plugin, das einen Einzeldienst ersetzt und seine Logik nur in privaten oder internen Methoden hat, lässt diesen Ausweg nicht offen. Dann bleibt der Anwendung nur, sich für eines der Plugins zu entscheiden.

Vor der Veröffentlichung: Koexistenz prüfen

Ein Plugin-Autor kann mit wenig Aufwand prüfen, wie gut sein Plugin mit anderen auskommt. `CompositionMatrixTests` zeigt die Technik: ein fiktives zweites Plugin mit eigener Options-Extension, das dieselben Dienste registriert, und Assertions für beide Aktivierungsreihenfolgen.

- ☐ Jeden ersetzten Einzeldienst auflisten und in der Dokumentation nennen.
- ☐ Einzeldienste mit `Replace` ersetzen, nie mit `TryAdd`; wo die öffentliche Schnittstelle genügt, dekorieren.
- ☐ Decorators brechen mit klarer Meldung ab, wenn es nichts zu umhüllen gibt.
- ☐ Logik ersetzter Dienste öffentlich und virtuell anbieten, damit Anwendungen integrieren können.
- ☐ Plugin-Interfaces geben für Fremdes `null` zurück und beanspruchen keine Methoden, Typen oder Metadaten, die ihnen nicht gehören.
- ☐ Eigenes Annotation-Präfix, eigener Event-ID-Bereich, eindeutige Use-Methode.
- ☐ Mit einem zweiten Plugin in beiden Aktivierungsreihenfolgen testen, dazu mit verbreiteten Plugins wie EFCore.NamingConventions (Kapitel 10).
- ☐ Die nötige Aktivierungsreihenfolge dokumentieren, falls es eine gibt.

7 Öffentliche API und Integration in die Anwendung

Versionsabhängigkeit: **stabil**

Die Use-Methode

Nutzer aktivieren das Plugin über eine Extension-Method auf `DbContextOptionsBuilder`. Sie holt die vorhandene Extension oder legt eine neue an und registriert sie über `IDbContextOptionsBuilderInfrastructure`:

`Infrastructure/TenancyDbContextOptionsBuilderExtensions.cs (Auszug)`

```
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null)
{
    ArgumentNullException.ThrowIfNull(optionsBuilder);

    var extension = optionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                    ?? new TenancyOptionsExtension();

    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder).AddOrUpdateExtension(extension);
    configure?.Invoke(new TenancyOptionsBuilder(optionsBuilder));
    return optionsBuilder;
}

// Generische Überladung: Method Chaining bleibt typisiert
public static DbContextOptionsBuilder<TContext> UseAcmeTenancy<TContext>(
    this DbContextOptionsBuilder<TContext> optionsBuilder,
    Action<TenancyOptionsBuilder>? configure = null) where TContext : DbContext
    => (DbContextOptionsBuilder<TContext>)UseAcmeTenancy(
        (DbContextOptionsBuilder)optionsBuilder, configure);
```

Die Datei liegt im Namespace `Microsoft.EntityFrameworkCore`. So ist die Methode ohne zusätzliches `using` auffindbar, genau wie `UseNpgsql`. Das ist die verbreitete Konvention im EF-Ökosystem.

Der Options-Builder

Einstellungen werden über einen eigenen Builder gesetzt. Er validiert Eingaben sofort und schreibt jede Änderung als neue Extension zurück:

`Infrastructure/TenancyOptionsBuilder.cs (Auszug, vereinfacht)`

```
public class TenancyOptionsBuilder
{
    public TenancyOptionsBuilder(DbContextOptionsBuilder optionsBuilder)
    {
        ArgumentNullException.ThrowIfNull(optionsBuilder);

        OptionsBuilder = optionsBuilder;
    }
```

```
    protected virtual DbContextOptionsBuilder OptionsBuilder { get; }

    public virtual TenancyOptionsBuilder UseSettingName(string settingName)
        => With(s => s with
        {
            SettingName = TenancyIdentifiers.ValidateSettingName(settingName),
        });

    public virtual TenancyOptionsBuilder ApplyQueryFilter(bool apply = true)
        => With(s => s with { ApplyQueryFilter = apply });

    protected virtual TenancyOptionsBuilder With(
        Func<TenancySettings, TenancySettings> change)
    {
        var extension = OptionsBuilder.Options.FindExtension<TenancyOptionsExtension>()
                        ?? new TenancyOptionsExtension();

        ((IDbContextOptionsBuilderInfrastructure)OptionsBuilder)
            .AddOrUpdateExtension(extension.WithSettings(change(extension.Settings)));
        return this;
    }
}
```

Alternative: Aktivierung über den Provider-Builder

Provider-spezifische Plugins werden manchmal auch über den Options-Builder des Providers aktiviert, etwa `UseNpgsql(cs, o => o.UseNodaTime())`. Dafür schreibt man eine Extension-Method auf `NpgsqlDbContextOptionsBuilder` und kommt über `IRelationalDbContextOptionsBuilderInfrastructure.OptionsBuilder` an den eigentlichen Builder. Das Beispiel nutzt den einfacheren Weg über `DbContextOptionsBuilder`, weil das Plugin mehr ist als eine Provider-Option.

Zwei Container verbinden

Manche Teile eines Plugins brauchen Dienste der *Anwendung*: Der Connection-Interceptor braucht den aktuellen Mandanten, den nur die Anwendung kennt. Der interne EF-Provider kann App-Dienste aber nicht auflösen. Die saubere Lösung: Solche Komponenten leben im App-Container und werden per `AddInterceptors` an EF übergeben.

`DependencyInjection/… (Kern), Infrastructure/… (PostgreSQL), Auszüge`

```
// Kernpaket: nur den Mandanten-Provider im App-Container registrieren
public static IServiceCollection AddAcmeTenancy<TTenantProvider>(
    this IServiceCollection services,
    ServiceLifetime lifetime = ServiceLifetime.Scoped)
    where TTenantProvider : class, ITenantProvider
{
    services.TryAdd(ServiceDescriptor.Describe(
        typeof(ITenantProvider), typeof(TTenantProvider), lifetime));
    return services;
}

// Provider-Paket: Use-Überladung erzeugt die Interceptors aus diesem Provider
public static DbContextOptionsBuilder UseAcmeTenancy(
    this DbContextOptionsBuilder optionsBuilder,
    IServiceProvider applicationServices,
```

```
    Action<TenancyOptionsBuilder>? configure = null)
{
    var tenants = applicationServices.GetRequiredService<ITenantProvider>();

    return optionsBuilder
        .UseAcmeTenancy(configure)
        .AddInterceptors(
            new TenantConnectionInterceptor(tenants),
            new TenantSaveChangesInterceptor(tenants));
}
```

Bei `AddDbContext((sp, o) => ...)` ist `sp` der Scope der aktuellen Anfrage, weil EF die Options standardmäßig pro Scope baut. Der Mandanten-Provider und damit die Interceptors sehen den richtigen Mandanten. Die Registrierung liegt im Kernpaket, weil sie für jeden Provider gleich ist (Kapitel 46). Für die Brücke in die andere Richtung kann ein Interceptor über `eventData.Context.GetService<T>()` Dienste aus dem *internen* Provider lesen; der Connection-Interceptor holt sich so den Setting-Namen aus den Singleton-Options (Kapitel 18).

DbContext-Pooling

Bei `AddDbContextPool` werden Contexts wiederverwendet und bekommen ihre Options nur einmal, gebaut mit dem Root-Provider. Die Interceptor-Instanzen entstehen dabei einmal und dienen allen späteren Anfragen. Ein Tenant-Provider, der seinen Mandanten pro Scope hält, liefert dann immer den der ersten Anfrage. Für Pooling muss er deshalb ein Singleton sein, der den Mandanten bei jedem Aufruf neu ermittelt, etwa über `IHttpContextAccessor`. Kapitel 40 zeigt diese Variante, Kapitel 8 die Regel dahinter. Dasselbe gilt für `AddDbContextFactory`, dessen Factory standardmäßig ein Singleton ist.

8 Zustand, Lebensdauern und Thread-Safety

Versionsabhängigkeit: **stabil**

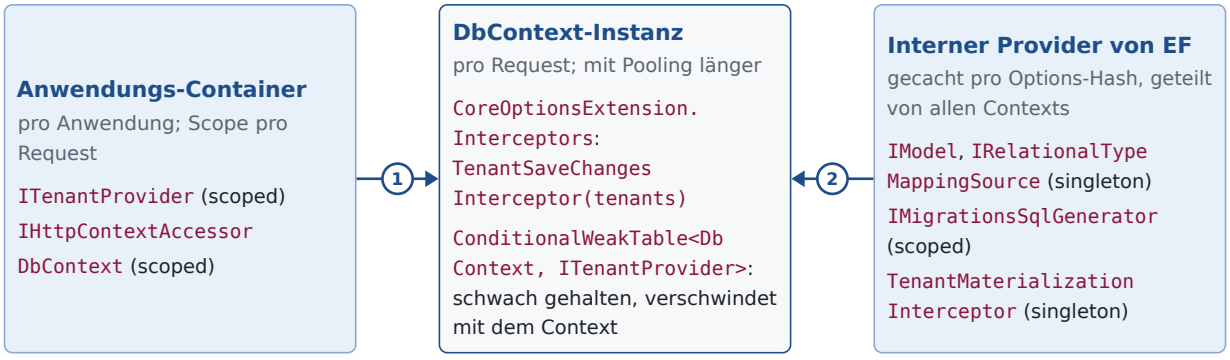
Fast jeder schwer zu findende Plugin-Fehler hat dieselbe Ursache: Zustand liegt an der falschen Stelle. Ein Mandant in einem Singleton, ein Wert im Hash, der dort nicht hingehört, ein Scoped-Dienst, den ein Singleton festhält. Die vorigen Kapitel enthalten die einzelnen Regeln; dieses Kapitel führt sie zu einer zusammen und beantwortet eine Frage: **Welche Art von Zustand darf in einem EF-Core-Plugin wo gespeichert werden?**

Zwei Container, zwei Bedeutungen von Lebensdauer

Ein Plugin lebt in zwei Containern (Kapitel 2 und 7). Dieselben Wörter bedeuten dort Verschiedenes:

Lebensdauer	App-Container	Interner Provider von EF
Singleton	Einmal pro Anwendung	Einmal pro internem Provider, also pro Konfiguration. Mehrere Konfigurationen in einem Prozess ergeben mehrere Instanzen.
Scoped	Einmal pro Scope, in ASP.NET Core pro Anfrage	Einmal pro Context-Instanz. Mit Pooling überlebt die Instanz die Anfrage und dient der nächsten.
Transient	Bei jeder Auflösung neu	Ebenso; bei EF-Diensten selten (`ILazyLoader`)
`static`	Einmal pro Prozess, über alle Container hinweg	Ebenso

Wer welchen Container baut, entscheidet über die Lebensdauer. Den App-Container baut die Anwendung; seine Dienste kann der interne Provider nicht auflösen. Den internen Provider baut EF aus den Options, cacht ihn anhand des Hashs der Options-Extensions (Kapitel 5) und teilt ihn zwischen allen Contexts mit derselben Konfiguration. Die Lebensdauer eines EF-Dienstes legt EF fest (Anhang A); `TryAdd` übernimmt sie (Kapitel 6).



(1) `UseAcmeTenancy(sp)` holt den Tenant-Provider aus dem Anwendungs-Container und gibt ihn dem SaveChanges-Interceptor in den Options dieses Contexts.

(2) Ein Singleton im internen Provider bekommt mit jedem Ereignis den Context und sucht den Tenant-Provider genau dieses Contexts. Im internen Provider und im Hash liegt nichts Mandantenspezifisches, ein interner Provider bedient also alle Mandanten, ohne Leck und ohne Warnung vor vielen Service Providern.

Welcher Zustand wohin gehört

Zustand	Richtiger Ort	Gültig
Statische Plugin-Konfiguration (Annotation-Namen, Event-IDs, Standardwerte)	`const` oder `static readonly`, unveränderlich	Prozess
Gecachte Reflection-Metadaten (`MethodInfo`, `PropertyInfo`)	`static readonly`-Felder; sie sind unveränderlich und damit threadsicher	Prozess
Modellrelevante Konfiguration (Setting-Name, Policy-Name)	Unveränderliche Options-Extension, im Hash; Singleton-Dienste lesen sie über `ISingletonOptions`	Interner Provider
Provider- und modellspezifische Daten (abgeleitet aus dem Modell oder den Options)	Annotation oder Laufzeit-Annotation am Modell; Zwischenergebnisse in einem threadsicheren Singleton oder über `GetOrAddRuntimeAnnotationValue`	Interner Provider, Modell
DbContext-spezifische Daten	Scoped-Dienst im internen Provider oder der Context aus den Event-Daten; mit Pooling zurücksetzen (`IResettableService`)	Context-Instanz
Connection-spezifische Daten (Session-Settings)	Die Datenbank-Session selbst, gesetzt bei jedem Öffnen durch einen Connection-Interceptor (Kapitel 18)	Solange die Verbindung offen ist
Aktueller Mandant	`ITenantProvider` im App-Container, bei jedem Zugriff gelesen; ins SQL nur als Parameter oder über die Session	Anfrage oder Job
Aktueller Benutzer	Wie der Mandant: im App-Container, pro Zugriff (etwa über `IHttpContextAccessor`)	Anfrage
Sonstige anfragespezifische Daten (Korrelations-ID, Sprache)	App-Scope oder `AsyncLocal`; im SQL höchstens als Parameter oder Tag	Anfrage oder Job

Genauso wichtig ist, wo Zustand *nie* hingehört: Werte einer Anfrage nie in die Options, den Hash, das Modell, ein Singleton-Feld oder eine übersetzte Abfrage. Alle diese Orte werden über Anfragen hinweg wiederverwendet.

Singleton heißt: einer pro internem Provider

Ein Singleton im internen Provider ist nicht „global für den Prozess“. Zwei Contexts mit unterschiedlichem Setting-Namen bekommen zwei Provider und damit zwei Instanzen von `TenancySingletonOptions`. Umgekehrt teilen sich alle Contexts einer Konfiguration *eine* Instanz, auf allen Threads gleichzeitig: Parallele Anfragen arbeiten mit verschiedenen Contexts, aber mit denselben Singletons.

Ein Singleton muss deshalb nach seiner Initialisierung unveränderlich sein oder threadsicher cachen. Das Beispiel zeigt drei Formen: Das Type-Mapping-Plugin und der Materialization-Interceptor sind zustandslos, `TenancySingletonOptions` wird einmal initialisiert und dann nur gelesen, und `TenancyLoggingDefinitions` legt seine Definitionen mit `LazyInitializer.EnsureInitialized` an (Kapitel 19). Ein einzelner `DbContext` dagegen ist nicht threadsicher und wird nie gleichzeitig benutzt; ein Scoped-Dienst sieht deshalb immer nur einen Aufrufer zur selben Zeit.

`ISingletonOptions` beantwortet die Frage, wie ein Singleton an die Konfiguration kommt. `IDbContextOptions` ist Scoped und im Singleton nicht verfügbar (siehe Fehlerfall 3). EF ruft `Initialize` einmal auf, wenn es den Provider baut, mit den Options des ersten Contexts. Das genügt, weil der Hash dafür sorgt, dass nur Contexts mit denselben modellrelevanten Options diesen Provider bekommen. `Validate` ruft EF auf, wenn ein Context einen Provider verwendet, den EF nicht für genau diese Options gebaut hat, vor allem bei `UseInternalServiceProvider`: Dann teilen sich womöglich Contexts mit verschiedenen Einstellungen einen Provider, und `Validate` muss das ablehnen.

Service-Provider- und Modell-Cache

Beide Caches verstärken jede Entscheidung über Zustand. Der Provider-Cache verwendet die Options-Extensions als Schlüssel; das Modell wird pro Provider und Context-Typ gebaut (Kapitel 34). Daraus folgen zwei Fehlerrichtungen, die Kapitel 5 ausführlich behandelt: Was im Schlüssel fehlt, obwohl es das Modell beeinflusst, führt zu einem falschen, geteilten Modell. Was im Schlüssel steht, obwohl es pro Anfrage wechselt, erzeugt pro Wert einen Provider samt Modell; ab dem 21. bricht EF standardmäßig mit `ManyServiceProvidersCreatedWarning` ab. Instanzen von Singleton-Interceptors zählen ebenfalls zum Schlüssel (Kapitel 17).

Pooling, Factories, Jobs und offene Verbindungen

Die Lebensdauern oben gelten für `AddDbContext`. Andere Arten, Contexts zu erzeugen, verschieben sie:

Situation	Was sich ändert	Regel
`AddDbContextPool`	Options und Interceptors entstehen einmal aus dem Root-Provider; Context-Instanz und ihre Scoped-Dienste dienen nacheinander vielen Anfragen	Tenant-Provider liest pro Zugriff; Scoped-Dienste mit Zustand implementieren `IResettableService` (Kapitel 40)
`AddDbContextFactory`	Factory und Options sind standardmäßig Singletons; der Options-Callback bekommt den Root-Provider	Factory Scoped registrieren oder einen Tenant-Provider verwenden, der pro Zugriff liest
Parallele Anfragen	Verschiedene Contexts, dieselben Singletons	Singletons zustandslos oder threadsicher
Hintergrunddienste	Kein `HttpContext`; der Mandant kommt aus dem Job	Mandant vor Scope und Context setzen, etwa über `AsyncLocal`; nie auf „alle Mandanten“ zurückfallen
Lange offene Verbindungen	Der Session-Zustand gehört zur Verbindung, nicht zum Context	EF die Verbindung öffnen lassen; bei Mandantenwechsel `ApplyTenantSession()` (Kapitel 18); Transaction-Pooling beachten (Kapitel 36)

Sechs Fehlerfälle

Jeder der folgenden Fälle ist im Beispiel als Test nachgestellt, die falsche Variante neben der richtigen (`StateAndLifetimeTests`, Fall 5 in `MaterializationTests`, Fall 6 in `FailureCaseTests`).

Fall	Ursache	Beobachtbar	Lösung
1. Aktueller Mandant in einem Singleton	Ein Tenant-Provider merkt sich den ersten Mandanten, etwa als „Optimierung“	Mit Pooling oder Singleton-Factory arbeitet Anfrage B vollständig als Mandant A: Session, Lesen und Schreiben. RLS hilft nicht, weil alle Werte zusammenpassen	Den Mandanten bei jedem Zugriff lesen (`HttpContext`, `AsyncLocal`)
2. Aktueller Mandant im Hash	Die Options-Extension trägt den Mandanten in den Provider-Schlüssel	Ein Provider samt Modell pro Mandant: Speicher und Startzeit wachsen, ab dem 21. eine Exception	Mandant über Interceptors und Session; im Hash nur Modellrelevantes
3. Scoped-Dienst in einem Singleton	Ein Singleton-Dienst injiziert `ICurrentDbContext` oder `IDbContextOptions`	Kein Fehler bei der Registrierung, denn EF validiert Scopes im internen Provider nicht; der Singleton bekommt `null` und scheitert später	Dienst Scoped registrieren, den Context aus den Event-Daten nehmen oder Konfiguration über `ISingletonOptions` lesen
4. Options-Extension nach der Registrierung geändert	Eine veränderliche Property der Extension wird später gesetzt	Der Provider ist mit dem alten Wert gebaut; neue Contexts arbeiten still mit ihm weiter	Unveränderliche Extension, Änderungen als Kopie (`WithSettings`, Kapitel 5)
5. Context-Zustand in einem Singleton-Interceptor	Der Interceptor merkt sich Mandant oder Context des ersten Aufrufs	Fehlalarme für andere Mandanten, verdeckte Lecks, Race Conditions; eine Instanz pro Context erzeugt einen Provider pro Context	Daten pro Aufruf über den Context holen (Kapitel 17)
6. Scoped-Zustand bei Pooling	Ein Scoped-Plugin-Dienst sammelt Daten „pro Anfrage“, etwa Zähler oder Puffer	Mit Pooling sieht die nächste Anfrage die Daten der vorigen, weil Context und Scope wiederverwendet werden	Dienst zusätzlich als `IResettableService` registrieren und in `ResetState` zurücksetzen

Fall 3 ist der tückischste, weil er ohne jede Warnung beginnt. Im App-Container meldet ASP.NET Core in der Entwicklungsumgebung eine solche „Captive Dependency“ beim Auflösen; der interne Provider von EF prüft das nicht:

`tests/.../StateAndLifetimeTests.cs (gekürzt)`

```
// FALSCH: ein Singleton, der den aktuellen Context braucht
m.TryAddSingleton<ContextHolder, ContextHolder>();     // ContextHolder(ICurrentDbContext)

using var ctx = ContextWithHolder(ServiceLifetime.Singleton);
Assert.Null(ctx.GetService<ContextHolder>().Current);  // Root-Scope: kein Context

// Richtig: Scoped, eine Instanz pro Context
```

```
Assert.Same(ctx, ctx.GetService<ContextHolder>().Current!.Context);
```

Fall 4 lässt sich mit `ISingletonOptions.Validate` nur dort erkennen, wo EF es aufruft, also bei einem geteilten Provider über `UseInternalServiceProvider`. Der Test `Shared_provider_rejects_contexts_with_different_tenancy_settings` zeigt, dass `TenancySingletonOptions` genau das tut. Im Normalfall schützt nur die Unveränderlichkeit selbst.

> **Die Regel in einem Satz**
>
> Was für alle Contexts einer Konfiguration gleich ist, gehört in Options, Modell oder Singletons; was sich pro Context ändert, in Scoped-Dienste; was sich pro Anfrage ändert, in den App-Container, und von dort pro Aufruf in Parameter oder Session, nie in einen Cache.

Teil III

Das Modell

Metadaten, Fluent API, Conventions, Validierung und neue Spaltentypen.

› Kapitel 9: Annotations und Fluent API

› Kapitel 10: Conventions

› Kapitel 11: Modellvalidierung

› Kapitel 12: Type Mappings und Value Converter

9 Annotations und Fluent API

Versionsabhängigkeit: **stabil**

Annotations sind EFs Mechanismus für erweiterbare Metadaten: Schlüssel-Wert-Paare an jedem Modellelement (Modell, Entity-Typ, Property, Index, ...). Ein Plugin speichert seine Konfiguration als Annotations, und die gesamte restliche Infrastruktur (Conventions, Migrations, Snapshots, Compiled Models) kann damit arbeiten.

Regeln für Annotations

Regel	Begründung
Eigenes Präfix (`Acme:Rls:`)	Vermeidet Kollisionen mit EF, Providern und anderen Plugins.
Nur einfache Werte: string, bool, Zahlen, Enums	Snapshots und Migrationen schreiben Werte als C#-Literale. Eigene Klassen lassen sich nicht serialisieren.
Komplexe Werte als String (z. B. JSON)	Strings vergleicht der Differ zuverlässig per `Equals`. Ein Record mit `string[]` würde per Referenz verglichen und ständig „geändert“ wirken.
Default-Werte nicht speichern	„Annotation fehlt“ und „Annotation hat Default“ sind für den Differ verschieden und erzeugen leere Migrationen.
Je Teilkonfiguration eine Annotation	Jede Policy hat eine eigene Annotation. Dann zeigt der Diff genau, welche sich geändert hat.

`Metadata/RlsAnnotationNames.cs, Metadata/RlsPolicy.cs (gekürzt, vereinfacht)`

```
public static class RlsAnnotationNames
{
    public const string Prefix = "Acme:Rls:";
    public const string Enabled = Prefix + "Enabled";
    public const string Forced = Prefix + "Forced";
    public const string PolicyPrefix = Prefix + "Policy:";     // + Policy-Name
    public const string QueryFilterKey = "Acme.Tenancy";
}

public sealed record RlsPolicy(string Name, PolicyCommand Command, bool Permissive,
    string[]? Roles, string? Using, string? WithCheck)
{
    public string Serialize()
        => JsonSerializer.Serialize(this, RlsJsonContext.Default.RlsPolicy);

    public static RlsPolicy Deserialize(string json) => /* ... */;
}
```

Die Fluent API

Fluent-API-Methoden sind Extension-Methods auf den Buildern von EF. Drei Gestaltungsregeln machen sie professionell: Die **nicht-generische** Variante ist die eigentliche Implementierung (der Model-Snapshot nutzt nicht-generische Builder, Kapitel 23). Die **generische** Variante delegiert nur und erhält den Typ für weiteres Chaining. Und die Klasse liegt im Namespace `Microsoft.EntityFrameworkCore`, damit sie überall ohne `using` verfügbar ist.

Builders/RlsEntityTypeBuilderExtensions.cs (gekürzt)

```
namespace Microsoft.EntityFrameworkCore;

public static class RlsEntityTypeBuilderExtensions
{
    public static EntityTypeBuilder HasRowLevelSecurity(
        this EntityTypeBuilder builder, bool force = false)
    {
        ArgumentNullException.ThrowIfNull(builder);
        builder.Metadata.SetAnnotation(RlsAnnotationNames.Enabled, true);

        if (force)
            builder.Metadata.SetAnnotation(RlsAnnotationNames.Forced, true);
        else
            builder.Metadata.RemoveAnnotation(RlsAnnotationNames.Forced);   // kein Default!

        return builder;
    }

    public static EntityTypeBuilder HasPolicy(
        this EntityTypeBuilder builder, string name, Action<PolicyBuilder> configure)
    {
        var policyBuilder = new PolicyBuilder(name);
        configure(policyBuilder);
        builder.Metadata.SetAnnotation(
            RlsAnnotationNames.PolicyPrefix + name, policyBuilder.Build().Serialize());
        return builder;
    }

    public static EntityTypeBuilder<TEntity> HasRowLevelSecurity<TEntity>(
        this EntityTypeBuilder<TEntity> builder, bool force = false) where TEntity : class
        => (EntityTypeBuilder<TEntity>)HasRowLevelSecurity(
            (EntityTypeBuilder)builder, force);

    // ... HasNoRowLevelSecurity, HasPolicy<TEntity> analog
}
```

Verschachtelte Konfiguration wie bei `HasPolicy` nimmt ein `Action<PolicyBuilder>` entgegen. Der Builder prüft die Regeln der Datenbank schon beim Modellbau. Ein Fehler wie „INSERT-Policy mit USING“ fällt so beim ersten Start der Anwendung auf und nicht erst beim Ausführen der Migration in Produktion:

Builders/PolicyBuilder.cs › Build

```
internal RlsPolicy Build()
{
    if (_using is null && _withCheck is null)
        throw new InvalidOperationException(
            $"Policy '{_name}' braucht USING oder WITH CHECK.");

    if (_command == PolicyCommand.Insert && _using is not null)
        throw new InvalidOperationException(
            $"INSERT-Policy '{_name}' erlaubt nur WITH CHECK.");

    if (_command is PolicyCommand.Select or PolicyCommand.Delete && _withCheck is not null)
        throw new InvalidOperationException(
```

```
            $"{_command}-Policy '{_name}' erlaubt nur USING.");

    return new RlsPolicy(_name, _command, _permissive,
        _roles.Count > 0 ? [.. _roles] : null, _using, _withCheck);
}
```

Konfigurationsquellen

EF merkt sich zu jedem Metadatum, *woher* es stammt. Eine Quelle kann eine gleich starke oder schwächere überschreiben, aber nie eine stärkere. Das ist der Mechanismus, mit dem Plugins sinnvolle Defaults setzen, die Nutzer jederzeit übersteuern können:

ConfigurationSource	Gesetzt durch	Stärke
`Explicit`	Fluent API im `OnModelCreating` (`Metadata.SetAnnotation`)	stärkste
`DataAnnotation`	Attribut-Conventions (`fromDataAnnotation: true`)	mittel
`Convention`	Conventions über `IConvention*Builder`	schwächste

Für eigene Conventions bietet das Plugin deshalb zusätzlich Überladungen auf dem Convention-Builder an. Sie geben `null` zurück, wenn eine stärkere Quelle anders entschieden hat:

Builders/RlsEntityTypeBuilderExtensions.cs › Convention-Variante

```
public static IConventionEntityTypeBuilder? HasRowLevelSecurity(
    this IConventionEntityTypeBuilder builder,
    bool enabled,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(RlsAnnotationNames.Enabled, enabled, fromDataAnnotation)
           is null
        ? null
        : builder;
```

Die Prüfung der Quelle steckt im Builder, nicht in den Metadaten. Wer in einer Convention direkt an die Metadaten schreibt, umgeht sie:

Convention (FALSCH)

```
// In ProcessModelFinalizing: direkt an den Metadaten statt über entityType.Builder
entityType.SetAnnotation(RlsAnnotationNames.Enabled, true);
```

Was passiert, wenn eine Convention die Konfigurationsquelle umgeht?

Beobachtbar. Der Anwender schaltet RLS für eine Entität mit `HasNoRowLevelSecurity()` ab, und nach dem Modellbau ist es wieder an. Die nächste Migration aktiviert RLS für eine Tabelle, die ausdrücklich ausgenommen war.

Ursache. `SetAnnotation` am Convention-Metadatum überschreibt jeden vorhandenen Wert. Nur der Convention-Builder vergleicht die Quellen und gibt `null` zurück, wenn eine stärkere entschieden hat.

Lösung. In Conventions immer über `entityType.Builder` schreiben oder die Convention-Überladungen des Plugins verwenden.

Test: `FailureCaseTests.Convention_writing_to_the_metadata_…` und die Gegenprobe über den Builder.

10 Conventions

Versionsabhängigkeit: **stabil**

Conventions verändern das Modell automatisch. EF selbst besteht zum größten Teil aus Conventions: Schlüssel erkennen, Beziehungen finden, Tabellennamen ableiten. Ein Plugin fügt eigene hinzu, um Konfiguration überflüssig zu machen.

Convention-Typen und Zeitpunkte

Eine Convention implementiert ein oder mehrere Interfaces. Jedes steht für ein Ereignis während des Modellbaus. Die wichtigsten:

Interface	Ereignis	Typische Verwendung
`IEntityTypeAddedConvention`	Entity-Typ wurde hinzugefügt	Attribute auswerten, Grundkonfiguration
`IPropertyAddedConvention`	Property wurde hinzugefügt	Spaltentypen, Namenskonventionen
`IForeignKeyAddedConvention`	Beziehung entstanden	Indizes, Löschverhalten
`IEntityTypeBaseTypeChangedConvention`	Vererbung geändert	Konfiguration verschieben
`IModelFinalizingConvention`	Modell wird abgeschlossen (noch änderbar)	Alles, was vom fertigen Modell abhängt
`IModelFinalizedConvention`	Modell ist fertig (read-only)	Validierung

Der häufigste Convention-Fehler ist der falsche Zeitpunkt:

Convention (FALSCH)

```
public void ProcessPropertyAdded(IConventionPropertyBuilder propertyBuilder,
    IConventionContext<IConventionPropertyBuilder> context)
{
    // Spaltennamen gleich merken, etwa für den Policy-Ausdruck
    propertyBuilder.HasAnnotation("Acme:Column", propertyBuilder.Metadata.GetColumnName());
}
```

Was passiert, wenn eine Convention zu früh läuft?

Beobachtbar. Konfiguriert der Anwender danach `HasColumnName("tenant")` oder benennt ein Naming-Plugin die Spalte um, bleibt der gemerkte Name `TenantId`. Die Policy verweist auf eine Spalte, die es nicht gibt, und die Migration scheitert erst in der Datenbank.

Ursache. Reaktive Conventions (`*Added`, `*Changed`) laufen, während der Anwender noch konfiguriert. Ein Spaltenname kann sich danach noch ändern, ein Entity-Typ noch einen Basistyp bekommen.

Lösung. Was vom fertigen Modell abhängt, in einer `IModelFinalizingConvention` lesen, und zwar über `GetColumnName(table)`. Wer reaktiv bleiben muss, reagiert auf *alle* relevanten Ereignisse.

Test: `FailureCaseTests.Convention_reading_the_column_name_too_early_…` und die Gegenprobe bei `ModelFinalizing`.

Conventions einhängen

Das Plugin-Interface `IConventionSetPlugin` bekommt das Convention-Set und fügt eigene Conventions hinzu. `ConventionSet.Add` trägt eine Convention in alle Listen ein, deren Interfaces sie implementiert, jeweils am Ende:

Conventions/TenancyConventionSetPlugin.cs

```
public class TenancyConventionSetPlugin : IConventionSetPlugin
{
    private readonly ProviderConventionSetBuilderDependencies _dependencies;
    private readonly IDbContextOptions _options;
    private readonly ISqlGenerationHelper _sqlGenerationHelper;
    private readonly TenancyLoggingDefinitions _loggingDefinitions;

    public TenancyConventionSetPlugin(
        ProviderConventionSetBuilderDependencies dependencies,
        IDbContextOptions options,
        ISqlGenerationHelper sqlGenerationHelper,
        TenancyLoggingDefinitions loggingDefinitions)
    {
        // ... ArgumentNullException.ThrowIfNull für alle Parameter ...
        _dependencies = dependencies;
        _options = options;
        _sqlGenerationHelper = sqlGenerationHelper;
        _loggingDefinitions = loggingDefinitions;
    }

    public virtual ConventionSet ModifyConventions(ConventionSet conventionSet)
    {
        var settings = _options.FindExtension<TenancyOptionsExtension>()?.Settings;
        if (settings is null)
            return conventionSet;

        // Kapitel „Value Generation“
        conventionSet.Add(new TenantIdKeyValueGenerationConvention());
        conventionSet.Add(new NoTenantIsolationAttributeConvention(_dependencies));
        conventionSet.Add(new TenantIsolationConvention(
            settings, _sqlGenerationHelper, _dependencies.Logger, _loggingDefinitions));
        conventionSet.Add(new TenancyModelValidationConvention(
            _dependencies.Logger, _loggingDefinitions));

        return conventionSet;
    }
}
```

`ProviderConventionSetBuilderDependencies` ist das Parameterobjekt, das auch EFs eigene Conventions bekommen. Es enthält unter anderem den Model-Logger, den Validation-Logger und die Type-Mapping-Source. Andere Dienste wie `ISqlGenerationHelper` lassen sich zusätzlich injizieren, weil das Plugin selbst im internen Provider aufgelöst wird.

Die Isolations-Convention

Die zentrale Convention des Beispiels läuft bei `ModelFinalizing`, weil Spaltennamen erst dann feststehen. Sie findet die Mandanten-Property (per `IsTenantProperty()` markiert oder

`TenantId` bei `ITenantOwned`, Kapitel 44), aktiviert RLS mit Policy, legt einen Index an, setzt eine Laufzeit-Annotation für den SaveChanges-Interceptor und optional einen Query-Filter:

Conventions/TenantIsolationConvention.cs › ProcessModelFinalizing (vereinfacht)

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    // TPT, TPC, Entity Splitting, Owned Types in eigener Tabelle und implizite
    // Join-Tabellen ließen Mandantendaten ohne Mandantenspalte zurück: zuerst ablehnen
    TenantTableMappingValidator.Validate(modelBuilder.Metadata, e => e.IsRlsOptedOut());

    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null       // TPH: Tabelle des Root-Typs
            || entityType.IsOwned())              // geprüft: Eigentümertabelle oder JSON
            continue;

        // Markierte Property oder "TenantId" bei ITenantOwned (Kernpaket,
        // Kapitel „Fluent API auf Property-Ebene und Laufzeit-Annotations“)
        if (entityType.FindTenantPropertyCandidate() is not IConventionProperty property)
            continue;                             // (+ Typprüfung, gekürzt)

        if (entityType.IsRlsOptedOut())
        {
            _logger.TenantIsolationOptedOut(_loggingDefinitions, entityType);  // Warnung
            continue;
        }

        // Keine Tabelle (View, ToSqlQuery, Keyless) -> kein RLS
        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table)
            is not { } table)
            continue;

        // Laufzeit-Annotation: der SaveChanges-Interceptor findet so die Property
        entityType.Builder.HasAnnotation(
            TenancyAnnotationNames.TenantProperty, property.Name);

        ApplyRowLevelSecurity(entityType, property, table);
        if (_settings.CreateTenantIndex) EnsureTenantIndex(entityType, property);
        if (_settings.ApplyQueryFilter) ApplyQueryFilter(entityType, property);

        _logger.TenantIsolationApplied(_loggingDefinitions, entityType);       // Debug
    }
}
```

Conventions/TenantIsolationConvention.cs › ApplyRowLevelSecurity

```
protected virtual void ApplyRowLevelSecurity(
    IConventionEntityType entityType,
    IConventionProperty property,
    StoreObjectIdentifier table)
{
    // Spaltenname für genau diese Tabelle (bei Table Splitting kann er abweichen)
    var column = _sqlGenerationHelper.DelimitIdentifier(property.GetColumnName(table)!);
    var condition =
        $"{column} = nullif(current_setting('{_settings.SettingName}', true), '')::uuid";
```

```
        var policy = new RlsPolicy(_settings.PolicyName, PolicyCommand.All, Permissive: true,
            Roles: null, Using: condition, WithCheck: condition);

        // Convention-Builder: überschreibt explizite Konfiguration NICHT
        var builder = entityType.Builder;
        builder.HasAnnotation(RlsAnnotationNames.Enabled, true);
        if (_settings.ForceRowLevelSecurity)
            builder.HasAnnotation(RlsAnnotationNames.Forced, true);
        builder.HasAnnotation(RlsAnnotationNames.PolicyPrefix + _settings.PolicyName,
            policy.Serialize());
    }
```

Selbst erzeugtes SQL validieren

Der Setting-Name wird unmaskiert in den Policy-Ausdruck geschrieben. Anders als bei SQL, das der Entwickler selbst in `Using(...)` schreibt, setzt hier das *Plugin* SQL zusammen und ist dafür verantwortlich, dass nichts Ungültiges hineinkommt. Deshalb validiert der Options-Builder den Namen mit einem strengen regulären Ausdruck (`[a-z_][a-z0-9_]*\.[a-z_][a-z0-9_]*`).

Welche Tabellenabbildungen geschützt sind

RLS schützt Tabellen, keine Entitäten. Die Policy vergleicht die Mandantenspalte *derselben* Tabelle mit dem Session-Setting. Eine Tabelle mit Mandantendaten, aber ohne diese Spalte, lässt sich mit dieser Policy nicht schützen. Eine Entität kann mehrere Tabellen belegen, und mehrere Entitäten können sich eine Tabelle teilen. Deshalb prüft `TenantTableMappingValidator` aus dem relationalen Paket vor jeder Annotation, ob jede Tabelle mit Mandantendaten die Mandantenspalte enthält. Beide Provider-Pakete rufen ihn auf, der Vertrag ist also für PostgreSQL und SQL Server gleich:

Abbildung	Tabellen mit Mandantendaten	Verhalten
Eine Tabelle pro Entität	die Tabelle, mit Mandantenspalte	geschützt
TPH (Standard)	eine Tabelle für die ganze Hierarchie	geschützt, Policy am Root-Typ
Mandantenbezogener abgeleiteter Typ unter einem Root ohne Mandant	dieselbe Tabelle, Zeilen anderer Typen ohne Mandant	abgelehnt
TPT	je abgeleitetem Typ eine Tabelle *ohne* Mandantenspalte	abgelehnt
TPC	je konkretem Typ eine Tabelle, die abstrakte Basis hat keine	abgelehnt (nicht implementiert)
Owned Type in der Eigentümertabelle (`OwnsOne`)	Spalten der Eigentümertabelle	geschützt
Owned Type als JSON (`ToJson`)	eine Spalte der Eigentümertabelle	geschützt
Owned Type in eigener Tabelle (`OwnsMany`, `ToTable`)	eigene Tabelle ohne Mandantenspalte	abgelehnt
Table Splitting	dieselben Zeilen für alle Typen	geschützt, wenn alle mandantenbezogenen Typen dieselbe Spalte nutzen
Entity Splitting (`SplitToTable`)	weitere Tabelle ohne Mandantenspalte	abgelehnt

Abbildung	Tabellen mit Mandantendaten	Verhalten
Implizite Many-to-many-Tabelle mit mandantenbezogener Seite	Join-Tabelle ohne Mandantenspalte	abgelehnt
Join-Entität mit Mandanten-Property (`UsingEntity`)	Join-Tabelle mit Mandantenspalte	geschützt
View, `ToSqlQuery`, keyless ohne Tabelle	keine eigene Tabelle	keine Policy; die Basistabellen tragen sie

Abgelehnt heißt: Das Modell wirft beim ersten Zugriff auf `Model` eine `InvalidOperationException`, die Tabelle, Typ und Ausweg nennt. Das passiert vor `EnsureCreated`, vor jeder Migration und vor der ersten Abfrage. Eine scheinbare Unterstützung, die eine Policy auf eine Tabelle ohne passende Spalte schreibt, wäre schlimmer als die Ablehnung. TPC wäre grundsätzlich schützbar, weil jede konkrete Tabelle die Mandantenspalte erbt. Das Plugin unterstützt aber genau eine Tabelle pro Hierarchie, und eine halbe Umsetzung soll keine Sicherheit vortäuschen. Als Ausweg bleiben TPH oder eigenständige Entitäten.

Bewusste Ausnahmen bleiben möglich: Eine mit `[NoTenantIsolation]` oder `HasNoRowLevelSecurity()` ausgenommene Entität wird nicht geprüft, und die Convention protokolliert die Ausnahme als Warnung. So unterscheidet das Modell eine Entscheidung von einem Versehen. Für die implizite Join-Tabelle gibt es im PostgreSQL-Paket `UsingEntity(j => j.HasNoRowLevelSecurity())`; das SQL-Server-Paket kennt nur das Attribut und braucht dafür eine Join-Klasse. Besser ist fast immer eine Mandanten-Property an der Join-Entität:

Anwendung

```
modelBuilder.Entity<Basket>()
    .HasMany(b => b.Labels).WithMany(l => l.Baskets)
    .UsingEntity("BasketLabel",
        j => j.Property<TenantId>("TenantId").IsTenantProperty());   // Shadow-Property
```

Die Shadow-Property füllt der SaveChanges-Interceptor wie jede andere Mandanten-Property. `MappingIsolationTests` prüft beides gegen ein echtes PostgreSQL mit eingeschränkter Rolle: Ein TPC-Modell scheitert, bevor eine einzige Tabelle existiert. In einem Modell mit TPH, Owned Type, JSON-Spalte und dieser Join-Entität hat jede Tabelle RLS, und ohne Mandant ist in keiner Tabelle eine Zeile sichtbar. `TableMappingTests` und `SqlServerTableMappingTests` decken jede Zeile der Matrix ohne Datenbank ab.

Strukturelle Änderungen: Index und Query-Filter

Conventions dürfen nicht nur Annotations setzen, sondern das Modell strukturell verändern. Der Index wird nur angelegt, wenn nicht schon ein Index oder Schlüssel mit der Mandantenspalte beginnt. Der Query-Filter wird als Expression Tree gebaut und als **benannter Filter** registriert, eine Neuerung in EF Core 10. Benannte Filter koexistieren mit eigenen Filtern des Nutzers und lassen sich gezielt per `IgnoreQueryFilters(["Acme.Tenancy"])` abschalten.

Conventions/TenantIsolationConvention.cs (Auszug)

```
protected virtual void EnsureTenantIndex(
    IConventionEntityType entityType, IConventionProperty property)
{
    var covered = entityType.GetIndexes().Any(i => i.Properties[0] == property)
```

```
                    || entityType.GetKeys().Any(k => k.Properties[0] == property);
    if (!covered)
        entityType.Builder.HasIndex(new[] { property });
}

protected virtual void ApplyQueryFilter(
    IConventionEntityType entityType, IConventionProperty property)
{
    // e => EF.Property<TenantId>(e, "...") == EF.Functions.CurrentTenant()
    // EF.Property funktioniert für jeden Property-Namen und auch für Shadow-Properties
    var parameter = Expression.Parameter(entityType.ClrType, "e");
    var body = Expression.Equal(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        Expression.Call(CurrentTenantMethod, Expression.Property(null, FunctionsProperty)));

    entityType.Builder.HasQueryFilter(
        RlsAnnotationNames.QueryFilterKey, Expression.Lambda(body, parameter));
}
```

Bemerkenswert ist, dass der Filter die eigene Funktion `EF.Functions.CurrentTenant()` verwendet. Er wird damit über den Translator aus Kapitel 14 nach SQL übersetzt. Die Bausteine des Plugins greifen ineinander.

Zusammen mit Soft Delete. Vor EF Core 10 hatte ein Entitätstyp genau einen Query-Filter: Ein zweites `HasQueryFilter` ersetzte das erste, und `IgnoreQueryFilters()` schaltete alle auf einmal ab, den Mandantenfilter eingeschlossen. Mit benannten Filtern bleiben die eigenen Filter der Anwendung unabhängig von denen des Plugins:

`tests/.../QueryTranslationTests.cs (Auszug)`

```
m.Entity<Order>().HasQueryFilter("SoftDelete", o => !EF.Property<bool>(o, "IsDeleted"));

// WHERE NOT (o."IsDeleted") AND o."TenantId" = nullif(current_setting(...), '')::uuid
var both = ctx.Orders.ToQueryString();

// Gelöschte Zeilen werden sichtbar, der Mandantenfilter bleibt
var withDeleted = ctx.Orders.IgnoreQueryFilters(["SoftDelete"]).ToQueryString();

// Nur der Mandantenfilter ist aus (mit RLS filtert die Datenbank weiter)
var allTenants = ctx.Orders.IgnoreQueryFilters(["Acme.Tenancy"]).ToQueryString();
```

> **Filter nur mit Namen abschalten**
>
> Ein `IgnoreQueryFilters()` ohne Namen schaltet auch den Mandantenfilter ab. Wo eine Anwendung gelöschte Datensätze zeigt, etwa in einem Papierkorb, nennt sie ihren eigenen Filter: `IgnoreQueryFilters(["SoftDelete"])`. `Acme.Tenancy` abzuschalten ist Support- oder Reporting-Rollen vorbehalten: Mit RLS filtert die Datenbank weiter, ohne RLS (Cosmos DB, Kapitel 48) sieht die Abfrage dann alle Mandanten.

Endgültige Namen, egal wer sie setzt

Die Isolations-Convention liest den Spaltennamen erst bei `ModelFinalizing` und nur über `GetColumnName(table)`. Namen, die andere Conventions oder Plugins ändern, stehen zu diesem

Zeitpunkt fest. Das Beispiel prüft das gegen EFCore.NamingConventions, das Tabellen und Spalten in snake_case umbenennt, und zwar in beiden Aktivierungsreihenfolgen:

Anwendung

```
options.UseNpgsql(connectionString).UseAcmeTenancy().UseSnakeCaseNamingConvention();
```

Erzeugtes Migrations-SQL (Auszug, tests/.../NamingConventionTests.cs)

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders AS PERMISSIVE FOR ALL
    USING (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid)
    WITH CHECK (tenant_id = nullif(current_setting('app.tenant_id', true), '')::uuid);
CREATE INDEX ix_orders_tenant_id ON orders (tenant_id);
```

Namen nie selbst zusammensetzen

Ein Plugin, das Namen selbst zusammensetzt (aus `Name`-Properties oder per String-Verkettung), bricht, sobald ein Naming-Plugin aktiv ist. Immer das Modell nach dem endgültigen Namen fragen, und zwar so spät wie möglich.

Attribut-Conventions

Data Annotations wertet man mit einer Attribut-Convention aus. EF bietet dafür die Basisklasse `TypeAttributeConventionBase<TAttribute>`, die das Attribut findet und nur noch die Reaktion verlangt. Der Wert wird mit `fromDataAnnotation: true` gesetzt, sodass die Fluent API ihn überstimmen kann:

Conventions/NoTenantIsolationAttributeConvention.cs

```
public class NoTenantIsolationAttributeConvention
    : TypeAttributeConventionBase<NoTenantIsolationAttribute>
{
    public NoTenantIsolationAttributeConvention(
        ProviderConventionSetBuilderDependencies dependencies)
        : base(dependencies)
    {
    }

    protected override void ProcessEntityTypeAdded(
        IConventionEntityTypeBuilder entityTypeBuilder,
        NoTenantIsolationAttribute attribute,
        IConventionContext<IConventionEntityTypeBuilder> context)
        => entityTypeBuilder.HasRowLevelSecurity(enabled: false, fromDataAnnotation: true);
}
```

Tipp

Das Attribut selbst liegt im *Kernpaket*, die Convention im Provider-Paket. Entitätsklassen können das Attribut verwenden, ohne von PostgreSQL abzuhängen. Das ist dieselbe Aufteilung wie bei EFs eigenen Attributen in `System.ComponentModel.DataAnnotations`.

11 Modellvalidierung

Versionsabhängigkeit: **stabil**

Ein Plugin sollte Fehlkonfiguration erkennen, bevor sie zu falschem SQL oder Datenlecks führt. EF validiert Modelle über `IModelValidator`, den es aber nur einmal gibt und der dem Provider gehört. Für Plugins ist eine `IModelFinalizedConvention` der bessere Ort: Sie läuft auf dem fertigen Modell und kollidiert mit niemandem.

Conventions/TenancyModelValidationConvention.cs (vereinfacht)

```
public virtual IModel ProcessModelFinalized(IModel model)
{
    foreach (var entityType in model.GetEntityTypes())
    {
        if (!entityType.HasRlsConfiguration())
            continue;

        if (entityType.BaseType is not null)
            throw new InvalidOperationException(
                $"RLS an '{entityType.DisplayName()}' ist ungültig: nur am Root-Typ "
                + $"('{entityType.GetRootType().DisplayName()}') erlaubt.");

        if (StoreObjectIdentifier.Create(entityType, StoreObjectType.Table) is null)
            throw new InvalidOperationException("... nicht auf eine Tabelle gemappt ...");

        if (!entityType.IsRlsEnabled() && entityType.GetRlsPolicies().Count > 0)
            _logger.PoliciesWithoutRowLevelSecurity(_loggingDefinitions, entityType);
    }
    return model;
}
```

Die Validierung unterscheidet zwei Arten von Problemen. **Strukturelle Fehler**, die garantiert zu falschem Verhalten führen, werfen eine Exception. **Fragwürdige Konfiguration**, die gewollt sein kann, erzeugt eine Warnung über das Logging-System. Nutzer können solche Warnungen gezielt in Fehler umwandeln (Kapitel 19).

12 Type Mappings und Value Converter

Versionsabhängigkeit: **mittel**

Ein Type Mapping verbindet einen CLR-Typ mit einem Datenbanktyp. Es bestimmt den Spaltentyp in Migrationen, wie Parameter gesetzt werden, wie Literale im SQL aussehen, wie Werte verglichen und kopiert werden und wie sie in JSON-Spalten gespeichert werden. Für eigene Typen hat ein Plugin zwei Möglichkeiten:

Weg	Vorteile	Nachteile
Value Converter (Nutzer konfiguriert `HasConversion`)	Kein Plugin nötig	Muss für jede Property oder per `ConfigureConventions` konfiguriert werden
`IRelationalTypeMapping` `SourcePlugin`	Typ funktioniert überall ohne Konfiguration, eigene Literale, volle Kontrolle	Provider-spezifisch (Store-Typ)

Der Typ und sein Konverter

Der stark typisierte `TenantId` und sein Konverter liegen im Kernpaket. Der Konverter hat eine gemeinsame Instanz; warum, zeigt der Abschnitt über Compiled Models.

TenantId.cs, TenantIdConverter.cs (gekürzt, vereinfacht)

```
public readonly record struct TenantId(Guid Value)
{
    public static TenantId Empty => default;
    public bool IsEmpty => Value == Guid.Empty;              // wird nach SQL übersetzt
    public static TenantId New() => new(Guid.CreateVersion7());
}

public sealed class TenantIdConverter : ValueConverter<TenantId, Guid>
{
    public static TenantIdConverter Instance { get; } = new();

    public TenantIdConverter()
        : base(id => id.Value, value => new TenantId(value))
    {
    }
}
```

Das Type Mapping

```
public class TenantIdTypeMapping : RelationalTypeMapping
{
    // Pflicht für Compiled Models: öffentliche statische Default-Property
    public static TenantIdTypeMapping Default { get; } = new();

    public TenantIdTypeMapping()
        : base(new RelationalTypeMappingParameters(
            new CoreTypeMappingParameters(
                typeof(TenantId),
```

```
                converter: TenantIdConverter.Instance,
                jsonValueReaderWriter: TenantIdJsonReaderWriter.Instance),
            storeType: "uuid",
            dbType: System.Data.DbType.Guid))
    {
    }

    protected TenantIdTypeMapping(RelationalTypeMappingParameters parameters)
        : base(parameters)
    {
    }

    // EF klont Mappings, z. B. für Facetten oder geänderte Comparer
    protected override RelationalTypeMapping Clone(
        RelationalTypeMappingParameters parameters)
        => new TenantIdTypeMapping(parameters);

    protected override string GenerateNonNullSqlLiteral(object value)
    {
        var guid = value switch
        {
            TenantId id => id.Value,
            Guid g => g,
            _ => throw new ArgumentException($"Unerwarteter Typ {value.GetType()}."),
        };
        return $"'{guid}'::uuid";
    }
}
```

Jedes eigene Mapping braucht den geschützten Konstruktor mit `RelationalTypeMappingParameters` und eine `Clone`-Überschreibung, die den eigenen Typ erzeugt. Fehlt sie, verliert ein geklontes Mapping sein Verhalten, zum Beispiel die Literal-Erzeugung.

Das Type-Mapping-Plugin

`RelationalTypeMappingSource` befragt alle Plugins, *bevor* es auf die Mappings des Providers zurückgreift. Ein Plugin gibt `null` zurück, wenn es nicht zuständig ist:

`Storage/TenancyTypeMappingSourcePlugin.cs (vereinfacht)`

```
public class TenancyTypeMappingSourcePlugin : IRelationalTypeMappingSourcePlugin
{
    public virtual RelationalTypeMapping? FindMapping(
        in RelationalTypeMappingInfo mappingInfo)
    {
        if (mappingInfo.ClrType != typeof(TenantId))
            return null;

        // Explizit anderer Spaltentyp (HasColumnType("text")): nicht zuständig
        var storeTypeBase = mappingInfo.StoreTypeNameBase;
        if (storeTypeBase is not null
            && !string.Equals(storeTypeBase, "uuid", StringComparison.OrdinalIgnoreCase))
            return null;

        return TenantIdTypeMapping.Default;
    }
```

```
}
```

> **Hinweis**
>
> Ohne Mapping würde EF `TenantId` beim Modellbau nicht als Skalartyp erkennen und mit einer Fehlermeldung abbrechen. Mit dem Plugin funktioniert der Typ als Property, als Schlüssel, in Abfragen, als Parameter und in JSON-Spalten, ohne eine Zeile Konfiguration.

JSON-Unterstützung

Für JSON-Spalten braucht ein Mapping einen `JsonValueReaderWriter`. Er hat drei Pflichten: lesen, schreiben und beschreiben, wie er im generierten Code erzeugt wird. Das Muster folgt EFs eigenem `JsonGuidReaderWriter`:

TenantIdJsonReaderWriter.cs

```
public sealed class TenantIdJsonReaderWriter : JsonValueReaderWriter<TenantId>
{
    private static readonly PropertyInfo InstanceProperty =
        typeof(TenantIdJsonReaderWriter).GetProperty(nameof(Instance))!;

    public static TenantIdJsonReaderWriter Instance { get; } = new();

    private TenantIdJsonReaderWriter()
    {
    }

    public override TenantId FromJsonTyped(
        ref Utf8JsonReaderManager manager, object? existingObject = null)
        => new(manager.CurrentReader.GetGuid());

    public override void ToJsonTyped(Utf8JsonWriter writer, TenantId value)
        => writer.WriteStringValue(value.Value);

    // Für Compiled Models und vorkompilierte Abfragen
    public override Expression ConstructorExpression
        => Expression.Property(null, InstanceProperty);
}
```

> **Compiled Models brauchen Default und Instance**
>
> Beim Generieren für NativeAOT (`dotnet ef dbcontext optimize --nativeaot`) erzeugt der Generator für Compiled Models (Kapitel 24) für jedes Type Mapping Code der Form `TenantIdTypeMapping.Default.Clone(...)`. Er verlangt eine **öffentliche, statische Property** namens `Default` auf einem **öffentlichen** Typ und bricht sonst ab. Konverter und Reader/Writer übernimmt er nur dann ohne zusätzlichen Code, wenn sie *dieselbe Instanz* wie im Default-Mapping sind. Deshalb nutzt das Mapping die statischen Instanzen statt `new ...()`. Ohne NativeAOT löst EF 10 Type Mappings stattdessen beim Start auf; ein Plugin sollte die Anforderungen trotzdem erfüllen, denn über NativeAOT entscheiden die Anwender, nicht das Plugin.

Teil IV

Abfragen

Wie LINQ zu SQL wird, wie ein Plugin eigene Funktionen und Member übersetzt und wie aus Zeilen wieder Objekte werden.

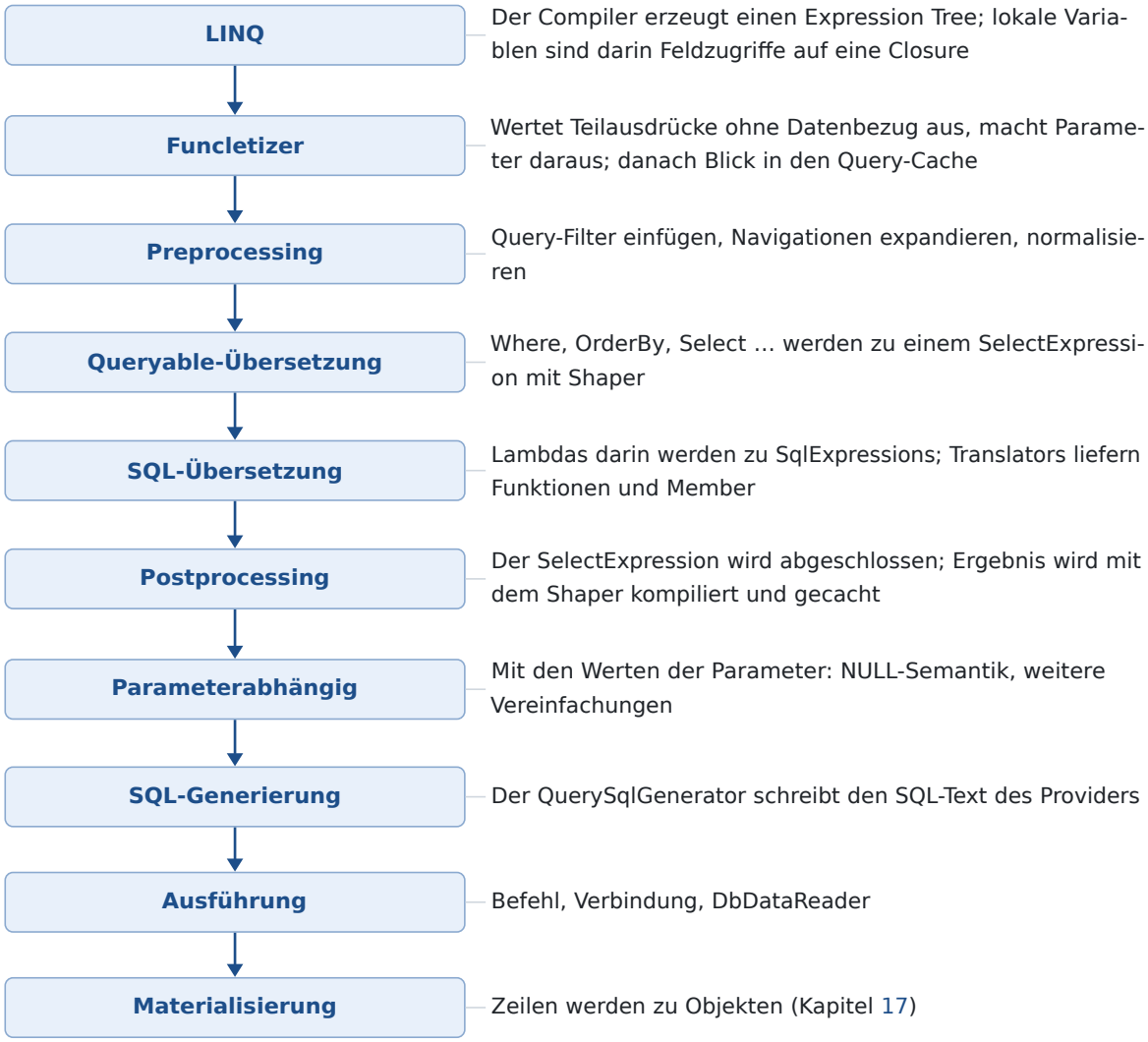
- Kapitel 13: Die Query-Pipeline
- Kapitel 14: Methoden und Member übersetzen
- Kapitel 15: Clientseitige Auswertung verhindern
- Kapitel 16: Weitere Erweiterungspunkte der Abfrage
- Kapitel 17: Materialisierung: von Zeilen zu Objekten

13 Die Query-Pipeline

Versionsabhängigkeit: **mittel**

Eine LINQ-Abfrage durchläuft in EF Core eine Reihe von Stufen, bevor SQL entsteht, und eine weitere, bevor Objekte zurückkommen. Jede Query-Erweiterung dieses Buchs sitzt an genau einer dieser Stufen. Dieses Kapitel ist die Landkarte dazu: Es verfolgt eine konkrete Abfrage durch die Pipeline, zeigt, wer an jeder Stufe verantwortlich ist und wo ein Plugin eingreifen kann, und hilft, eine gewünschte Änderung zuerst zu *lokalisieren*. Welcher Mechanismus dann der richtige ist, entscheidet der Abschnitt „Abfragen“ der Entscheidungshilfe (Seite 19); wie er funktioniert, steht im jeweiligen Kapitel.

Die Stufen im Überblick:



Eine Abfrage auf ihrem Weg

Die Beispielabfrage läuft mit aktiviertem Mandanten-Query-Filter (`ApplyQueryFilter()`, Kapitel 10). Der Name kommt aus einer Anfrage und kann fehlen:

Anwendung

```
string? name = filter.Name;                        // kann null sein

var customers = await db.Customers
    .Where(c => c.Name == name)
    .OrderBy(c => c.Id)
    .ToListAsync();
```

Was mit der Beispielabfrage an den einzelnen Stufen passiert, zeigen die Debug-Logs von EF (`CoreEventId.QueryCompilationStarting` und `QueryExecutionPlanned`) und `ToQueryString()`.

Funcletizer. Der lokale Wert `name` hat keinen Datenbezug. EF wertet ihn nicht als Konstante ein, sondern macht einen Parameter daraus. Erst dieser Baum ist der Schlüssel für den Query-Cache: Dieselbe Abfrage mit einem anderen Namen trifft denselben Eintrag.

Log: Compiling query expression

```
DbSet<Customer>()
    .Where(c => c.Name == @name)
    .OrderBy(c => c.Id)
```

Preprocessing. Der benannte Query-Filter des Plugins kommt hinzu, mit `EF.Functions.CurrentTenant()`. Dass dieser Aufruf hier noch ein Aufruf ist und nicht vorab in .NET ausgeführt wurde, verdankt er dem Evaluatable-Expression-Filter des Plugins (Kapitel 15).

Queryable- und SQL-Übersetzung. Beide Stufen greifen ineinander: Für jeden Queryable-Operator übersetzt EF dessen Lambda in SQL-Ausdrücke. Aus `c.Name == @name` wird ein binärer SQL-Vergleich, aus `CurrentTenant()` liefert der Method-Translator des Plugins einen Funktionsausdruck. Das Ergebnis ist ein `SelectExpression`, die relationale Darstellung der Abfrage, noch unabhängig von der SQL-Syntax des Providers:

Log: Generated query execution expression (Auszug)

```
SELECT c.Id, c.Name, c.TenantId
FROM Customers AS c
WHERE (c.TenantId == CAST(nullif(current_setting('app.tenant_id', TRUE), '') AS uuid))
    && (c.Name == @name)
ORDER BY c.Id ASC
```

Postprocessing und Kompilierung. Der `SelectExpression` wird abgeschlossen und zusammen mit dem Shaper, der später die Objekte baut, zu einem Delegaten kompiliert. Bis hierher läuft alles einmal pro Abfrageform.

Parameterabhängige Verarbeitung. Erst bei der Ausführung kennt EF die Werte der Parameter. Der `RelationalParameterBasedSqlProcessor` wertet die NULL-Semantik aus. Ist `name` null, kann `c.Name = NULL` bei einer Pflichtspalte nie wahr sein, und der ganze Filter fällt zu `FALSE` zusammen. Mit einem Wert bleibt der Vergleich stehen.

SQL-Generierung. Der Generator des Providers schreibt den Text: Npgsql macht aus dem `CAST` die PostgreSQL-Schreibweise `::uuid` und setzt Bezeichner in Anführungszeichen.

ToQueryString() mit name = "Contoso" und mit name = null

```
-- @name='Contoso'
SELECT c."Id", c."Name", c."TenantId"
FROM "Customers" AS c
```

```
WHERE c."TenantId" = nullif(current_setting('app.tenant_id', TRUE), '')::uuid
    AND c."Name" = @name
ORDER BY c."Id"

SELECT c."Id", c."Name", c."TenantId"
FROM "Customers" AS c
WHERE FALSE
ORDER BY c."Id"
```

Ausführung und Materialisierung. Der Befehl läuft, und der Shaper baut aus jeder Zeile einen `Customer`; die Mandanten-ID kommt dabei über den Value Converter zurück. Kapitel 17 beschreibt diesen Rückweg.

Wie oft welche Stufe läuft

Die Pipeline hat drei Zeitpunkte, und jeder Mechanismus gehört zu genau einem davon. Für die Beispielabfrage, viermal ausgeführt (zweimal mit `"Contoso"`, einmal mit null, einmal mit `"Fabrikam"`), ergibt das:

Zeitpunkt	Stufen	Im Beispiel
Einmal pro Abfrageform	Preprocessing bis Postprocessing; Query-Interceptor, Translators, Kompilierung des Shapers	1-mal
Einmal pro Parameterkonstellation	Parameterabhängige Verarbeitung und SQL-Generierung, gecacht nach dem, was die Entscheidungen beeinflusst hat (hier: null oder nicht null)	2-mal
Bei jeder Ausführung	Funcletizer (liest die Parameterwerte aus), Befehl, Command-Interceptor, Materialisierung	4-mal

Daraus folgt die wichtigste Regel für Query-Erweiterungen: Werte, die sich pro Ausführung ändern (Parameter, Mandant, Uhrzeit), müssen im SQL als Parameter oder Datenbankfunktion stehen, nie als zur Übersetzungszeit eingesetzte Konstante. Genau deshalb übersetzt das Plugin `CurrentTenant()` in `current_setting(...)` und setzt nicht den aktuellen Mandanten ein. Mit vorkompilierten Abfragen (Kapitel 16) rückt der erste Zeitpunkt sogar in den Build.

Die Landkarte

Die folgende Tabelle ordnet jeder Stufe die verantwortliche EF-Komponente zu, den Eingriff, den ein Plugin dort hat, und das Kapitel, das ihn behandelt. Die Angaben in grauer Schrift folgen der Legende der Entscheidungshilfe in Kapitel 4.

Stufe	Verantwortlich	Eingriff	Kap.
Funcletizer	`ExpressionTreeFuncletizer` (intern), fragt `IEvaluatable` `ExpressionFilter`	`IEvaluatableExpression` `FilterPlugin` öffentlich · mehrfach · Singleton	15
Query-Cache	`ICompiledQueryCache` (intern, Singleton)	Kein Eingriff	–

Stufe	Verantwortlich	Eingriff	Kap.
Vor dem Preprocessing	`QueryCompilationContext` zu Beginn der Kompilierung	`IQueryExpressionInterceptor` öffentlich · mehrfach · Singleton	16, 30
Preprocessing	`IQueryTranslationPreprocessorFactory`; Navigation Expansion intern	Query-Filter im Modell; sonst Einzeldienst ersetzen öffentlich · einzeln · Scoped	10
Queryable-Übersetzung	`IQueryableMethodTranslatingExpressionVisitorFactory`; Npgsql-Implementierung intern	Einzeldienst ersetzen Provider-Basis · einzeln · Scoped	–
SQL-Übersetzung	`IRelationalSqlTranslatingExpressionVisitorFactory`, `ISqlExpressionFactory`, Translator-Provider	Method-, Member-, Aggregate-Translator-Plugins öffentlich · mehrfach · Scoped	14, 43
Postprocessing	`IQueryTranslationPostprocessorFactory`; Npgsql-Implementierung intern	Einzeldienst ersetzen Provider-Basis · einzeln · Scoped	16
Shaper	`IShapedQueryCompilingExpressionVisitorFactory`	Über Type Mappings, Constructor Binding und `IMaterializationInterceptor`	17
Parameterabhängig	`IRelationalParameterBasedSqlProcessorFactory`, darin `SqlNullabilityProcessor`; Npgsql intern	Einzeldienst ersetzen, nötig für eigene Knoten Provider-Basis · einzeln · Scoped	42
SQL-Generierung	`IQuerySqlGeneratorFactory`; `NpgsqlQuerySqlGenerator` intern	Einzeldienst ersetzen, nötig für eigene Knoten Provider-Basis · einzeln · Singleton	42
Ausführung	Relationale Befehle und Verbindung	`IDbCommandInterceptor` öffentlich · mehrfach · Scoped	18
Materialisierung	Kompilierter Shaper, Materializer	`IMaterializationInterceptor` öffentlich · mehrfach · Singleton	17

Wo providerinterne API beginnt

Bis einschließlich SQL-Übersetzung kommt ein Plugin mit öffentlichen Plugin-Interfaces aus. Die Basisklassen der späteren Stufen sind in EF zwar öffentlich (`RelationalQueryTranslationPostprocessor`, `RelationalParameterBasedSqlProcessor`, `QuerySqlGenerator`), aber jeder Provider leitet eigene Klassen davon ab, bei Npgsql alle in `Npgsql.EntityFrameworkCore.PostgreSQL.Query.Internal`. Wer einen dieser Einzeldienste ersetzt, muss von der Provider-Klasse erben, um deren Verhalten zu behalten, und arbeitet damit gegen interne API (Kapitel 27).

Sechs Arten, in eine Abfrage einzugreifen

Die Mechanismen der folgenden Kapitel werden leicht verwechselt, weil sie alle „die Abfrage ändern“. Sie unterscheiden sich darin, was sie sehen und wann sie laufen:

Mechanismus	Arbeitet auf	Richtig für
Expression-Rewriting (`IQueryExpressionInterceptor`)	Dem LINQ-Baum nach dem Funcletizer: Parameter statt Werte, noch kein SQL	Bedingungen ergänzen, Aufrufe ersetzen, Operationen ablehnen; einmal pro Abfrageform
Method-, Member-, Aggregate-Translation	Einem einzelnen Aufruf, Member oder Aggregat; baut aus vorhandenen SQL-Bausteinen	Neue Funktionen in LINQ; `null` für alles, was nicht passt
Eigene `SqlExpression`	Einem neuen Knotentyp im `SelectExpression`	Syntax, die kein Funktionsaufruf ist, oder eigene NULL-Semantik; muss an drei Stellen bekannt sein: Translator, parameterabhängige Verarbeitung, Generator
`RelationalParameterBasedSqlProcessor`	Dem fertigen `SelectExpression` *mit* Parameterwerten	Entscheidungen, die vom Wert eines Parameters abhängen, vor allem NULL-Semantik; gecacht pro Konstellation
`QuerySqlGenerator`	Dem endgültigen Baum, schreibt nur Text	Die Schreibweise eines Knotens; trifft keine inhaltlichen Entscheidungen mehr
Materialisierung (`IMaterializationInterceptor`)	Den Zeilen nach der Ausführung	Objekte prüfen, ergänzen, ersetzen; ändert kein SQL

Zwei Verwechslungen sind besonders häufig. Wer im Generator Bedingungen umbaut, umgeht die NULL-Optimierung, die vorher lief, und sieht keine Parameterwerte. Und wer mit einem Query-Interceptor etwas „übersetzen“ will, kann nur LINQ durch anderes LINQ ersetzen; einen SQL-Knoten, den es nicht gibt, erzeugt nur ein Translator.

Erst lokalisieren, dann wählen

Eine gewünschte Änderung an einer Abfrage lässt sich fast immer durch eine Frage einer Stufe zuordnen. Erst danach lohnt die Wahl des Mechanismus in der Entscheidungshilfe (Seite 19).

Was soll sich ändern?	Stufe	Weiter in
Ein Wert ohne Datenbezug wird zu früh in .NET ausgewertet	Funcletizer	Kapitel 15
Welche Abfrage gestellt wird: zusätzliche Bedingung, ersetzter Aufruf	Vor dem Preprocessing	Query-Filter (Kapitel 10) oder Kapitel 16
Wie ein .NET-Aufruf, Member oder Aggregat als SQL aussieht	SQL-Übersetzung	Kapitel 14, 43
SQL braucht eine Form, die es als Knoten nicht gibt	SQL-Übersetzung bis SQL-Generierung	Kapitel 42
Die Form hängt vom Wert eines Parameters ab	Parameterabhängig	Kapitel 42
Nur der Text: Kommentar, Hinweis, Schreibweise	SQL-Generierung oder Ausführung	`TagWith`, Kapitel 18
Die zurückgegebenen Objekte	Materialisierung	Kapitel 17

> **Tipp**
>
> Zum Lokalisieren genügen drei Werkzeuge: das Debug-Log `QueryCompilationStarting` zeigt den Baum nach dem Funcletizer, `QueryExecutionPlanned` den `SelectExpression` samt Shaper, und `ToQueryString()` das fertige SQL (Anhang D). Wo sich der Unterschied zwischen zweien davon zeigt, liegt die Stufe.

14 Methoden und Member übersetzen

Versionsabhängigkeit: **mittel**

Das EF.Functions-Muster

Datenbankfunktionen, die keine Entsprechung in .NET haben, werden als Extension-Methods auf `DbFunctions` deklariert. Sie werden nie ausgeführt; der Körper wirft nur eine Exception für den Fall, dass jemand sie außerhalb von LINQ aufruft. Die Deklaration liegt im Kernpaket, die Übersetzung im Provider-Paket:

TenancyDbFunctionsExtensions.cs (Kernpaket)

```
namespace Microsoft.EntityFrameworkCore;

public static class TenancyDbFunctionsExtensions
{
    public static TenantId CurrentTenant(this DbFunctions _)
        => throw new InvalidOperationException(
            $"{nameof(CurrentTenant)} ist nur in LINQ-to-Entities-Abfragen verwendbar.");
}
```

Ein Method-Call-Translator

Ein `IMethodCallTranslator` bekommt jeden Methodenaufruf, den EF nicht selbst kennt. Er prüft, ob er zuständig ist, und baut dann aus `ISqlExpressionFactory` einen SQL-Ausdrucksbaum:

Query/CurrentTenantTranslator.cs (vereinfacht)

```
public class CurrentTenantTranslator : IMethodCallTranslator
{
    private static readonly MethodInfo CurrentTenantMethod =
        typeof(TenancyDbFunctionsExtensions).GetRuntimeMethod(
            nameof(TenancyDbFunctionsExtensions.CurrentTenant), [typeof(DbFunctions)])!;

    private readonly ISqlExpressionFactory _sql;
    private readonly IRelationalTypeMappingSource _typeMappingSource;
    private readonly ITenancySingletonOptions _options;

    public CurrentTenantTranslator(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        // ... ArgumentNullException.ThrowIfNull für alle Parameter ...
        _sql = sql;
        _typeMappingSource = typeMappingSource;
        _options = options;
    }

    public virtual SqlExpression? Translate(
        SqlExpression? instance, MethodInfo method,
        IReadOnlyList<SqlExpression> arguments,
```

```
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (method != CurrentTenantMethod)
            return null;                          // nicht zuständig -> nächster Translator

        var text = _typeMappingSource.FindMapping(typeof(string))!;
        var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

        // Liefert NULL, ohne dass ein Argument NULL ist: kein Argument propagiert
        var currentSetting = _sql.Function(
            "current_setting",
            [
                _sql.Constant(_options.Settings.SettingName, text),
                _sql.Constant(true, boolean),
            ],
            nullable: true,
            argumentsPropagateNullability: [false, false],
            typeof(string), text);

        // Für EF bewusst nie NULL, passend zum CLR-Typ TenantId (siehe unten)
        var nullIfEmpty = _sql.Function(
            "nullif",
            [currentSetting, _sql.Constant(string.Empty, text)],
            nullable: false,
            argumentsPropagateNullability: [false, false],
            typeof(string), text);

        return _sql.Convert(nullIfEmpty, typeof(TenantId), TenantIdTypeMapping.Default);
    }
}
```

Aus `o.TenantId == EF.Functions.CurrentTenant()` wird damit:

Erzeugtes SQL (Ausschnitt)

```
WHERE o."TenantId" = nullif(current_setting('app.tenant_id', TRUE), '')::uuid
```

Detail	Bedeutung
`return null`	Vertrag für „nicht zuständig“. Niemals eine Exception für fremde Methoden werfen; dann ist der nächste Translator an der Reihe.
`arguments`	Bei Extension-Methods ist `arguments[0]` der `this`-Parameter (hier `DbFunctions`).
`nullable`	Kann das Ergebnis NULL sein? EF optimiert damit `IS NULL`-Prüfungen und erzeugt die NULL-Semantik von C#. `CurrentTenant` sagt bewusst `false`: Der CLR-Typ `TenantId` ist nicht nullbar, und ohne Mandanten soll jeder Vergleich NULL ergeben, also keine Zeile treffen.
`argumentsPropagateNullability`	„Das Ergebnis ist genau dann NULL, wenn dieses Argument NULL ist.“ Gibt es solche Argumente, leitet EF die Nullbarkeit allein aus ihnen ab. Für Funktionen, die selbst NULL erzeugen (`nullif`, `current_setting(…, true)`), deshalb überall `false`; für `reverse(x)` oder `upper(x)` `true`.
Type Mappings	Jeder SQL-Ausdruck braucht ein Mapping. Es bestimmt Literale, Parameter und Casts. `Convert` mit dem TenantId-Mapping erzeugt einen Cast, den Npgsql als `::uuid` schreibt.

Detail	Bedeutung
`Constant(...)`	Konstanten werden als Literal über das Mapping erzeugt und korrekt maskiert, auch der Setting-Name.

Die Angaben zur Nullability sind die tückischste Stelle eines Translators, weil ein Fehler nur bei bestimmten Abfragen sichtbar wird. Ein Beispiel ist eine Funktion `EF.Functions.EmptyAsNull(c.Name)`, die einen leeren String in NULL verwandelt und `string?` zurückgibt:

Translator (FALSCH)

```
_sql.Function("nullif", [arguments[1], _sql.Constant("", arguments[1].TypeMapping)],
    nullable: true,                                  // richtig: kann NULL sein ...
    argumentsPropagateNullability: [true, false],    // ... aber hier hebelt es EF aus
    typeof(string), arguments[1].TypeMapping);
```

Was passiert, wenn ein Translator die Nullability falsch angibt?

Beobachtbar. `Where(c => EF.Functions.EmptyAsNull(c.Name) == null)` wird zu `WHERE FALSE`: Kunden mit leerem Namen findet die Abfrage nie, ohne Fehler und ohne Warnung. Mit einer nullbaren Spalte als Argument wäre das Ergebnis richtig; der Fehler zeigt sich nur bei Pflichtspalten.

Ursache. Hat eine Funktion propagierende Argumente, bestimmt EFs Nullability-Prozessor ihre Nullbarkeit allein aus diesen. Ist das Argument nicht nullbar (eine Pflichtspalte, eine Konstante), gilt die Funktion als nie NULL, egal was `nullable` sagt.

Lösung. Propagierend ist ein Argument nur, wenn „Argument ist NULL“ das *einzige* ist, was das Ergebnis NULL macht. Funktionen, die selbst NULL erzeugen, markieren kein Argument. Und `nullable` folgt dem CLR-Typ: Ist er nullbar, darf EF NULL erwarten; ist er es nicht, wie bei `TenantId`, ist `false` die ehrliche Angabe.

Test: `FailureCaseTests.Nullable_function_…` mit beiden Varianten; `QueryTranslationTests.Inequality_with_CurrentTenant_…` hält die sichere Richtung fest.

Ein Member-Translator

Für Properties und Felder gibt es `IMemberTranslator`. Das Beispiel übersetzt `tenantId.IsEmpty`. Die Konstante bekommt das Type Mapping der Spalte, damit das Literal im richtigen Format entsteht:

Query/TenantIdMemberTranslator.cs

```
public class TenantIdMemberTranslator : IMemberTranslator
{
    private static readonly PropertyInfo IsEmptyProperty =
        typeof(TenantId).GetProperty(nameof(TenantId.IsEmpty))!;

    private readonly ISqlExpressionFactory _sql;

    public TenantIdMemberTranslator(ISqlExpressionFactory sql)
    {
        ArgumentNullException.ThrowIfNull(sql);

        _sql = sql;
    }

    public virtual SqlExpression? Translate(
```

```
        SqlExpression? instance, MemberInfo member, Type returnType,
        IDiagnosticsLogger<DbLoggerCategory.Query> logger)
    {
        if (instance is null || !Equals(member, IsEmptyProperty))
            return null;

        return _sql.Equal(instance, _sql.Constant(TenantId.Empty, instance.TypeMapping));
    }
}
```

Translator-Plugins

Translators werden nicht direkt registriert, sondern über Plugin-Klassen, die eine Liste liefern. Die Plugins sind **Scoped** (Anhang A) und können daher Scoped-Dienste wie `ISqlExpressionFactory` injizieren:

`Query/TenancyTranslatorPlugins.cs (vereinfacht)`

```
public class TenancyMethodCallTranslatorPlugin : IMethodCallTranslatorPlugin
{
    public TenancyMethodCallTranslatorPlugin(
        ISqlExpressionFactory sql,
        IRelationalTypeMappingSource typeMappingSource,
        ITenancySingletonOptions options)
    {
        Translators = [new CurrentTenantTranslator(sql, typeMappingSource, options)];
    }

    public virtual IEnumerable<IMethodCallTranslator> Translators { get; }
}

public class TenancyMemberTranslatorPlugin : IMemberTranslatorPlugin
{
    public TenancyMemberTranslatorPlugin(ISqlExpressionFactory sql)
    {
        Translators = [new TenantIdMemberTranslator(sql)];
    }

    public virtual IEnumerable<IMemberTranslator> Translators { get; }
}
```

> **Tipp**
>
> `query.ToQueryString()` erzeugt das SQL einer Abfrage, ohne sie auszuführen und ohne Datenbankverbindung. Es ist das wichtigste Werkzeug, um Translators zu entwickeln und zu testen (Kapitel 26).

15 Clientseitige Auswertung verhindern

Versionsabhängigkeit: **stabil**

Der erste Schritt der Pipeline, der Funcletizer, wertet jeden Teilausdruck vorab in .NET aus, der keinen Bezug zu Abfragedaten hat. Aus `o.Total > limit * 2` wird so ein Parameter. Für `EF.Functions.CurrentTenant()` wäre das fatal: Der Ausdruck hat keinen Datenbezug, also würde EF die Methode ausführen, und die wirft ihre Exception.

EF kennt nur wenige Ausnahmen. Im Quellcode von `EvaluatableExpressionFilter` (EF Core 10) sind das `DateTime.Now`, `DateTime.UtcNow`, `DateTime.Today`, die `DateTimeOffset`-Gegenstücke, `Guid.NewGuid()`, `Random.Next(...)` und alle Methoden, die in EFs eigener Klasse `DbFunctionsExtensions` deklariert sind. Eigene EF.Functions-Methoden liegen in einer anderen Klasse und fallen **nicht** darunter. Danach fragt EF alle registrierten Plugins:

Kernpaket: TenancyDbFunctionsAttribute.cs, Query/TenancyEvaluatable…Plugin.cs

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]
public sealed class TenancyDbFunctionsAttribute : Attribute;

public class TenancyEvaluatableExpressionFilterPlugin : IEvaluatableExpressionFilterPlugin
{
    public virtual bool IsEvaluatableExpression(Expression expression)
        => expression is not MethodCallExpression { Method.DeclaringType: { } type }
           || !type.IsDefined(typeof(TenancyDbFunctionsAttribute), inherit: false);
}

[TenancyDbFunctions]
public static class TenancyDbFunctionsExtensions { /* CurrentTenant() */ }
```

Das Plugin liegt im **Kernpaket** und erkennt Funktionsklassen an einem Attribut. So deckt ein einziges Plugin auch die PostgreSQL-spezifischen Funktionen aus Kapitel 42 und 43 und jedes künftige Provider-Paket ab, ohne dass der Kern diese Klassen kennen muss.

Ob eine eigene EF.Functions-Methode dieses Plugin braucht, hängt davon ab, ob sie Argumente mit Datenbezug hat. `EF.Functions.Soundex(c.Name)` ist unproblematisch, weil `c.Name` auf eine Spalte zeigt. Methoden ohne Argumente oder nur mit Konstanten (`CurrentTenant()`, `Random()`, `ServerTime()`) brauchen es immer:

Eigene Funktionsklasse (FALSCH)

```
public static class ShopDbFunctionsExtensions          // ohne [TenancyDbFunctions]
{
    public static DateTime ServerTime(this DbFunctions _)
        => throw new InvalidOperationException("Only in LINQ queries.");
}
```

Was passiert, wenn der Evaluatable-Filter fehlt?

Beobachtbar. Jede Abfrage mit der Funktion wirft `InvalidOperationException`: „An exception was thrown while attempting to evaluate a LINQ query parameter expression“, innen die Meldung aus dem Methodenkörper. Es sieht aus, als würde die Methode außerhalb von LINQ aufgerufen, obwohl der Translator registriert und richtig ist.

Ursache. Der Funcletizer läuft vor jedem Translator und wertet Ausdrücke ohne Datenbezug in .NET aus. Ausnahmen kennt EF nur für seine eigene Klasse `DbFunctionsExtensions`.

Lösung. Ein `IEvaluatableExpressionFilterPlugin`, im Beispiel über das Attribut an der Funktionsklasse.

Test: `FailureCaseTests.Function_without_data_reference_…`, mit und ohne Attribut.

16 Weitere Erweiterungspunkte der Abfrage

Versionsabhängigkeit: **mittel**

Das Beispiel braucht Method- und Member-Translators. Komplexe Plugins greifen auf weitere Punkte zurück; hier ein Überblick mit Einordnung:

Erweiterungspunkt	Zweck	Hinweise
`IAggregateMethodCallTranslatorPlugin`	Eigene Aggregatfunktionen in `GroupBy` (z. B. `bool_and`, Statistikfunktionen)	Scoped, mehrfach registrierbar; umgesetzt in Kapitel 43
`IQueryExpressionInterceptor`	LINQ-Ausdrucksbaum vor der Übersetzung umschreiben	Singleton-Interceptor; ideal für Query-Rewrites, Tags, Hints
`IMaterializationInterceptor`	Objekte bei der Materialisierung erzeugen, initialisieren oder prüfen	Singleton-Interceptor; muss zustandslos und thread-sicher sein; umgesetzt in Kapitel 17
`IMethodCallTranslatorProvider` u. a.	Translator-Infrastruktur ersetzen	Einzeldienste des Providers; nur in Ausnahmefällen
`IQuerySqlGeneratorFactory`	SQL-Generierung für eigene SqlExpression-Typen	Singleton, Provider-Dienst; umgesetzt in Kapitel 42
`IRelationalParameterBasedSqlProcessorFactory`	Nullability-Verarbeitung eigener Ausdrucksknoten	Scoped, Provider-Dienst; umgesetzt in Kapitel 42
`IQueryTranslationPostprocessorFactory`	Ausdrucksbaum nach der Übersetzung anpassen	Provider-Dienst; tiefer Eingriff, eng an EF-Interna gebunden

> **Vorkompilierte Abfragen**
>
> Mit NativeAOT und vorkompilierten Abfragen (seit EF Core 9) übersetzt EF Abfragen bereits beim Build. Translators laufen dann im Build-Prozess, nicht zur Laufzeit. Sie dürfen deshalb nur von Options und Modell abhängen, nie von Laufzeitzustand. Das Beispiel erfüllt das: Der Setting-Name ist eine Option, der Mandant selbst wird erst in der Datenbank-Session ausgewertet.

17 Materialisierung: von Zeilen zu Objekten

Versionsabhängigkeit: **stabil**

Die bisherigen Kapitel dieses Teils beschreiben den Hinweg: aus LINQ wird SQL. Kapitel 32 beschreibt den Weg von Entitäten zu INSERT, UPDATE und DELETE. Dieses Kapitel schließt den Kreis und beschreibt den Rückweg: Wie werden aus den Zeilen, die die Datenbank liefert, Objekte, und wo kann ein Plugin dabei eingreifen?

Der Rückweg im Überblick



Die Identity-Prüfung steht *vor* dem Materializer: Eine Entität, die der Context bereits trackt, wird nicht ein zweites Mal erzeugt. Wo ein Plugin eingreifen kann und was EF für sich behält, zeigt die folgende Übersicht:

Stufe	Eingriff für Plugins	EF-intern
Ausführung	`IDbCommandInterceptor` kann den Befehl ändern oder einen eigenen `DbDataReader` liefern (Kapitel 18)	Verbindung, Ausführungsstrategie, Wiederholungen

Stufe	Eingriff für Plugins	EF-intern
Shaping	Indirekt: Type Mapping und Value Converter bestimmen, wie Werte gelesen und umgewandelt werden (Kapitel 12)	Aufbau und Kompilierung des Shapers
Materializer	`IMaterializationInterceptor` mit vier Haken; Constructor Binding über das Modell	Wie EF Konstruktor und Werte zusammensetzt
Tracking	Ereignisse `Tracked` und `StateChanged` des Change Trackers (Anwendung)	Identity Map, Zustände, Schnappschüsse für die Änderungserkennung
Fixup	Kein Haken	Navigation Fixer

Shaping: der kompilierte Rückweg

Beim Übersetzen einer Abfrage entsteht nicht nur SQL, sondern auch ein **Shaper**: ein Ausdrucksbaum, der beschreibt, wie aus einer Zeile des `DbDataReader` das Ergebnis wird. EF kompiliert ihn zu einem Delegaten und cacht ihn zusammen mit dem SQL pro Abfrageform (Kapitel 13). Pro Zeile läuft nur noch dieser Delegat, ohne Nachschlagen im Modell.

Das hat eine wichtige Folge: Alles, was den Shaper prägt, wird beim Übersetzen entschieden, nicht beim Ausführen. Dazu gehören die Lesemethode des Type Mappings, der Value Converter, der gewählte Konstruktor und die registrierten Materialization-Interceptors. Für `TenantId` liest der Shaper den Providerwert mit `GetGuid` und ruft den kompilierten Konvertierungsdelegaten des `TenantIdConverter` auf (`ConvertFromProviderTyped`). Werte in JSON-Spalten liest er über den `JsonValueReaderWriter` des Mappings. Außer dem Konverter selbst läuft dafür kein Plugin-Code; ein Value Converter ist für reine Wertumwandlung deshalb billiger als ein Interceptor, der für jede Entität aufgerufen wird.

Eine Projektion wie `Select(o => o.Total)` erzeugt einen Shaper, der nur diese Spalte liest. Es entsteht keine Entität, und kein Materializer läuft.

Der Materializer: Instanz, Konstruktor, Felder

Für eine Entität erzeugt der Shaper zuerst eine Instanz und setzt dann die übrigen Werte. Welchen Konstruktor er verwendet, legt das Modell fest (Constructor Binding):

Regel	Verhalten in EF Core 10
Kandidaten	Alle deklarierten Instanzkonstruktoren, unabhängig von ihrer Sichtbarkeit.
Bindbar	Jeder Parameter passt nach Name und Typ zu einer gemappten Property oder zu einem Dienst. Über Konstruktor gesetzte Properties setzt EF danach nicht noch einmal.
Dienste als Parameter	`DbContext` (auch der abgeleitete Typ), `ILazyLoader`, der Lazy-Loading-Delegat `Action<object, string>` und `IEntityType`; weitere über eigene `IParameterBindingFactory`-Plugins.

Regel	Verhalten in EF Core 10
Auswahl	EF geht die bindbaren Konstruktoren der Reihe nach durch. Ein Konstruktor ersetzt den bisherigen Favoriten, wenn er mehr Dienst-Parameter hat oder weniger Property-Parameter; bei Gleichstand in beidem bricht der Modellbau ab. Ein parameterloser Konstruktor gewinnt also gegen Property-Konstruktoren, aber nicht gegen einen mit `DbContext`-Parameter. Mischen Konstruktoren Dienste und Properties, kann die Deklarationsreihenfolge entscheiden.
Übrige Properties	EF schreibt sie über das Backing-Field, wenn es eines gibt. Logik im Setter läuft beim Laden deshalb *nicht*.
Shadow-Properties	Haben kein Feld; beim Tracking liegen ihre Werte im Change Tracker.
Owned Types	Sind Entity-Typen und werden einzeln materialisiert.
Complex Types	Gehören zur Entität und werden mit ihr materialisiert, ohne eigenen Durchlauf.

Die Auswahl steht in `ConstructorBindingFactory` (EF Core 10.0.0). `ConstructorBindingTests` hält beide Fälle fest: Ohne Dienste gewinnt der parameterlose Konstruktor, neben einem Konstruktor mit `DbContext` verliert er.

Dass Setter beim Laden nicht laufen, ist gewollt: Laden ist kein Erzeugen. Prüfungen, die beim Anlegen oder Ändern greifen sollen, gehören in Konstruktor und Methoden der Domäne. Was beim Laden gilt, garantiert die Datenbank.

Tracking, Identity Resolution und Fixup

Was nach dem Materializer passiert, hängt vom Tracking-Verhalten der Abfrage ab:

Modus	Identität	Fixup
Tracking (Standard)	Vor dem Materializer prüft EF den Schlüssel im Change Tracker. Bei einem Treffer gibt es die vorhandene Instanz zurück, unverändert: Lokale Änderungen bleiben, neue Werte aus der Datenbank werden verworfen.	Neue Instanzen werden Unchanged; EF verbindet Navigationen zu allen getrackten Entitäten, auch aus früheren Abfragen.
`AsNoTracking`	Jede Zeile ergibt eine neue Instanz, auch dieselbe Entität mehrfach.	Nur innerhalb des Ergebnisses: Der Shaper verbindet, was über `Include` geladen wurde.
`AsNoTrackingWithIdentityResolution`	Eine Identity Map nur für diese Abfrage: Dieselbe Entität wird einmal materialisiert.	Wie ohne Tracking; danach trackt niemand die Instanzen.

Relationship Fixup ist EF-intern. Ein Plugin kann es nicht beeinflussen, eine Anwendung kann es über `ChangeTracker.Tracked` und `StateChanged` beobachten.

> **IIdentityResolutionInterceptor ist kein Materialisierungs-Haken**
>
> Der Name legt es nahe, aber `IIdentityResolutionInterceptor` (Scoped, mehrfach) läuft bei Abfragen nie. Er greift, wenn `Attach`, `Update` oder `Add` eine Instanz mit einem Schlüssel anhängen wollen, den der Context schon trackt, und entscheidet, was mit der getrackten Instanz geschieht. EF liefert dafür `IgnoringIdentityResolutionInterceptor` und `UpdatingIdentityResolutionInterceptor` mit.

> Abfragen lösen Identität ohne ihn: beim Tracking über den Change Tracker, sonst wie in der Tabelle oben.

IMaterializationInterceptor im Detail

`IMaterializationInterceptor` hat vier Haken, die EF für jede Entität in dieser Reihenfolge aufruft. Alle haben eine Standardimplementierung; ein Plugin überschreibt nur, was es braucht.

Methode	Zeitpunkt	Möglichkeiten
`CreatingInstance`	Vor dem Konstruktor	Werte der Zeile lesen, auch Shadow-Properties; mit `SuppressWithResult` eine eigene Instanz liefern
`CreatedInstance`	Nach dem Konstruktor	Über den Konstruktor gesetzte Werte sind da; eine andere Instanz zurückgeben
`InitializingInstance`	Vor dem Setzen der übrigen Werte	Mit `Suppress` das Setzen selbst übernehmen
`InitializedInstance`	Alle Werte gesetzt	Prüfen, ergänzen oder eine andere Instanz zurückgeben

Jede Methode bekommt `MaterializationInterceptionData`: den Context, den `IEntityType`, das Tracking-Verhalten der Abfrage und `GetPropertyValue` für die Werte der Zeile. Weil der Typ ein `IEntityType` ist, laufen Complex Types nicht einzeln durch den Interceptor, Owned Types dagegen schon. Der Konstruktor der Struktur ist EF-intern; Tests rufen deshalb nicht die Haken auf, sondern die Logik dahinter (siehe unten).

> **Lebensdauer: ein Singleton, einkompiliert in den Shaper**
>
> Der Interceptor ist ein `ISingletonInterceptor`. EF baut ihn beim Übersetzen in den Shaper ein, und der Shaper wird pro internem Service Provider gecacht. Eine Instanz bedient deshalb alle Contexts und alle Threads gleichzeitig; sie muss zustandslos und threadsicher sein. Daten einer Anfrage holt sie bei jedem Aufruf über `materializationData.Context`.
>
> Singleton-Interceptor-Instanzen gehören außerdem zum Cache-Schlüssel des internen Providers. Wer pro Context eine neue Instanz über `AddInterceptors` übergibt, bekommt pro Context einen neuen internen Provider samt Modell; ab dem 21. bricht EF standardmäßig mit `ManyServiceProvidersCreatedWarning` ab. Ein Plugin registriert ihn in `ApplyServices`; eine Anwendung übergibt stets dieselbe Instanz.

Beispiel: Mandanten beim Laden prüfen

Kapitel 18 zeigt eine echte Lücke: Übergibt die Anwendung eine bereits geöffnete Verbindung, läuft der Connection-Interceptor nicht, und eine Abfrage liefert still die Zeilen des Mandanten, dessen Wert noch in der Session steht. Dasselbe passiert, wenn die Anwendung versehentlich mit einer Rolle verbindet, die RLS umgeht. Das Plugin bietet dafür eine optionale Prüfung an, die genau an der richtigen Stelle sitzt: Jede mandantenfähige Entität, die eine Abfrage materialisiert, muss zum Mandanten des Contexts gehören.

Anwendung

```
options.UseNpgsql(connectionString)
       .UseAcmeTenancy(sp, tenancy => tenancy.VerifyMaterializedTenants());
```

Interceptors/TenantMaterializationInterceptor.cs (Kernpaket, gekürzt, vereinfacht)

```
public class TenantMaterializationInterceptor : IMaterializationInterceptor
{
    // Provider pro Context-Instanz, schwach gehalten: entfällt mit dem Context
    private static readonly ConditionalWeakTable<DbContext, ITenantProvider> Providers =
        new();

    public virtual InterceptionResult<object> CreatingInstance(
        MaterializationInterceptionData materializationData,
        InterceptionResult<object> result)
    {
        Verify(materializationData.Context, materializationData.EntityType,
            materializationData.GetPropertyValue);

        return result;
    }

    internal static void Verify(
        DbContext context, IEntityType entityType, Func<string, object?> readValue)
    {
        // Laufzeit-Annotation der Isolations-Convention; ohne sie keine Isolation
        if (entityType.GetTenantPropertyName() is not { } propertyName)
            return;

        // Der Mandant wird bei jeder Zeile gelesen, nie gemerkt: Alle Contexts teilen sich
        // diese Instanz, und ein Context aus dem Pool wechselt ihn zwischen den Ausleihen
        var current = FindTenantProvider(context).CurrentTenantId;
        var value = (TenantId)readValue(propertyName)!;

        if (value != current)
            throw new TenancyException("... isolation did not take effect ...");
    }

    // Einmal pro Context über seine Interceptors (wie ApplyTenantSession) zum
    // ITenantProvider des SaveChanges-Interceptors, danach aus der Tabelle
    internal static ITenantProvider FindTenantProvider(DbContext context)
        => Providers.GetValue(context, LookUpTenantProvider);
}
```

Infrastructure/TenancyOptionsExtension.cs › ApplyServices (Ergänzung)

```
// Optionale Prüfung beim Laden: ein Singleton-Interceptor im internen Provider, eine
// Instanz pro Provider. Die Einstellung steht im Settings-Record und damit im Hash.
if (Settings.VerifyMaterializedTenants)
    new EntityFrameworkServicesBuilder(services)
        .TryAdd<ISingletonInterceptor, TenantMaterializationInterceptor>();
```

Entscheidung	Begründung
`CreatingInstance` statt `InitializedInstance`	Die Prüfung liest den Wert aus der Zeile. Eine fremde Zeile wird nie zum Objekt, und die Mandanten-Property darf eine Shadow-Property sein.

Entscheidung	Begründung
Laufzeit-Annotation	Welche Entität isoliert ist und wie ihre Mandanten-Property heißt, steht seit dem Modellbau am Entity-Typ (Kapitel 44). Kein Raten zur Laufzeit.
Mandant pro Aufruf	Über den Context zum `ITenantProvider` dieses Contexts; der Provider ist pro Context gecacht, der Mandant wird pro Entität gelesen und verglichen. Das kostet trotzdem pro Zeile, deshalb ist die Prüfung optional.
Kein Mandant	Ohne Mandanten liefert RLS keine Zeilen. Kommt trotzdem eine, wirkt RLS nicht, und die Prüfung schlägt an.
Im Kernpaket	Die Prüfung kennt weder SQL noch Provider-Typen (Kapitel 46); beide Provider-Pakete aktivieren sie mit einer Zeile.

Die Integrationstests stellen die Lücke aus Kapitel 18 nach: Ein Context für Mandant A über eine Verbindung, deren Session noch Mandant B trägt. Ohne Prüfung liefert die Abfrage die Zeilen von B, mit Prüfung wirft sie eine `TenancyException`. Eigene Zeilen passieren mit und ohne Tracking und über `FromSql`.

Der Fehlerfall: Anfragedaten im Singleton

Der naheliegende erste Entwurf übergibt dem Interceptor den Mandanten im Konstruktor und erzeugt ihn wie die anderen Interceptors pro Context. Das scheitert am Cache-Schlüssel: Jeder Context bekommt einen eigenen internen Provider. Der naheliegende zweite Entwurf teilt eine Instanz und merkt sich den Mandanten beim ersten Aufruf, um das Nachschlagen pro Zeile zu sparen:

tests/...IntegrationTests/MaterializationTests.cs (FALSCH, gekürzt)

```
private sealed class RememberingTenantInterceptor : IMaterializationInterceptor
{
    private TenantId? _tenant;                      // Anfragedaten in einem Singleton

    public InterceptionResult<object> CreatingInstance(
        MaterializationInterceptionData materializationData,
        InterceptionResult<object> result)
    {
        // "Optimierung": den Mandanten nur einmal ermitteln
        _tenant ??= _tenantOf(materializationData.Context);

        if (materializationData.GetPropertyValue<TenantId>(nameof(Order.TenantId)) != _tenant)
            throw new TenancyException("Row of another tenant.");

        return result;
    }
}
```

Der Test zeigt beide Folgen. Die zweite Anfrage für Mandant B bekommt einen Fehler für ihre eigenen Zeilen, weil der Interceptor sich Mandant A gemerkt hat. Schlimmer: Ein Context für B über eine Verbindung, deren Session noch A trägt, liefert die Zeilen von A, und der gemerkte Mandant A lässt sie passieren. Die Prüfung verhindert das Leck nicht, sie verdeckt es. Unter Last kommt eine Race Condition hinzu, denn alle Threads schreiben dasselbe Feld.

> **Tipp**
>
> Ein Singleton-Interceptor hat nur zwei Quellen für Daten: was zur Konfiguration gehört (über Konstruktor oder Singleton-Options, für alle Contexts gleich) und was der Aufruf mitbringt (`Context`, Entity-Typ, Werte der Zeile). Alles andere gehört in die Anwendung oder in einen Scoped-Dienst, den der Interceptor über den Context erreicht (Kapitel 8).

Die richtige Optimierung merkt sich nicht den Mandanten, sondern den Weg zu ihm. Den `ITenantProvider` pro Zeile über `GetService<IDbContextOptions>`, die Extension und eine LINQ-Suche in den Interceptors zu finden, kostet bei tausend Zeilen tausendmal dasselbe. Das Beispiel legt den Provider deshalb in einer statischen `ConditionalWeakTable<DbContext, ITenantProvider>` ab. Sie hält ihre Schlüssel schwach: Ein Context, der freigegeben wird, verschwindet samt Eintrag, ohne dass der Interceptor davon erfährt. Mit Pooling bleibt der Eintrag über die Ausleihen hinweg richtig, weil ein Context seine Options und damit seine Interceptors sein Leben lang behält; der Mandant selbst wird weiterhin bei jeder Zeile gelesen. Scheitert das Nachschlagen, speichert die Tabelle nichts, und der Konfigurationsfehler bleibt bei jedem Aufruf sichtbar. `MaterializationTests` prüft wiederholte Zeilen in einem Context, Contexts mit verschiedenen Mandanten und einen gepoolten Context über zwei Ausleihen.

Grenzen

Grenze	Folge
Nur Entitäten	Projektionen einzelner Spalten oder in DTOs laufen nicht durch den Materializer. Ein Integrationstest belegt, dass `Select(o => o.Total)` auch Werte eines fremden Mandanten ungeprüft liefert; schützen kann hier nur die Datenbank.
Keine getrackten Entitäten	Ist eine Entität schon getrackt, wird sie nicht erneut materialisiert; der Interceptor sieht nur die erste Abfrage.
Kein Filter	Zeilen auslassen kann der Interceptor nicht, nur werfen oder die Instanz ersetzen. Filtern ist Aufgabe von Query-Filter oder Datenbank.
Kein SQL	Er sieht das Ergebnis, nicht die Abfrage; `ExecuteUpdate`, `ExecuteDelete` und Skalarabfragen materialisieren nichts.
Heißer Pfad	Er läuft für jede Entität jeder Abfrage (Kapitel 39); teure Arbeit vervielfacht sich mit der Ergebnisgröße.
Exception bricht die Abfrage ab	Auch alle übrigen Zeilen sind verloren. Das ist für eine Sicherheitsprüfung richtig, für eine Bequemlichkeitsfunktion nicht.

Wann Materialisierung, wann etwas anderes?

Der Interceptor ist der richtige Ort für Querschnittsaufgaben, die *jede* geladene Entität betreffen, über alle Abfragen, mit und ohne Tracking, auch bei rohem SQL. Für fast alles andere gibt es einen einfacheren und günstigeren Weg:

Ziel	Besser	Warum
Einen Wert anders speichern als darstellen (Format, Typ, Verschlüsselung)	Value Converter	Wirkt in beide Richtungen und in Parametern, direkt im Shaper statt über einen Interceptor

Ziel	Besser	Warum
Unveränderliche Objekte, Invarianten	Constructor Binding	EF ruft den Konstruktor selbst auf
Nur einige Spalten oder ein DTO	Projektion mit `Select`	Keine Entität, kein Tracking, weniger Daten
Dieselbe Einstellung für viele Typen (Konverter, Feldzugriff)	Convention	Einmal beim Modellbau statt bei jeder Zeile
Abgeleitete Werte, Geschäftsregeln	Domänenlogik (Methode, berechnete Property)	Laden ist kein Geschäftsvorfall
Reagieren, wenn eine Anwendung Entitäten lädt	`ChangeTracker.Tracked`	Öffentliche API ohne Plugin; nur bei Tracking
Prüfung oder Ergänzung jeder geladenen Entität, auch ohne Tracking	`IMaterializationInterceptor`	Der einzige Haken, der alle Wege zur Entität sieht

Teil V

Laufzeit

Eingreifen beim Speichern, Verbinden und Ausführen, und Ereignisse sauber melden.

› Kapitel 18: Interceptors

› Kapitel 19: Diagnose und Logging

18 Interceptors

Versionsabhängigkeit: **stabil**

Interceptors sind Haken in EFs Laufzeit: vor und nach Befehlen, Verbindungen, Transaktionen, SaveChanges und der Materialisierung. Sie können beobachten, Werte ändern, Operationen unterdrücken oder Ergebnisse ersetzen.

Interface	Ereignisse	Lebensdauer
`ISaveChangesInterceptor`	SavingChanges, SavedChanges, SaveChangesFailed, ThrowingConcurrencyException	Scoped
`IDbConnectionInterceptor`	Öffnen, Schließen, Erzeugen der Verbindung	Scoped
`IDbCommandInterceptor`	Erzeugen und Ausführen von Befehlen, DataReader	Scoped
`IDbTransactionInterceptor`	Beginnen, Commit, Rollback, Savepoints	Scoped
`IIdentityResolutionInterceptor`	Konflikte beim Anhängen bereits getrackter Entitäten; nie bei Abfragen (Kapitel 17)	Scoped
`IMaterializationInterceptor`	Erzeugen und Initialisieren von Entitäten (Kapitel 17)	**Singleton**
`IQueryExpressionInterceptor`	LINQ-Ausdruck vor der Kompilierung	**Singleton**

Die beiden Singleton-Interceptors implementieren `ISingletonInterceptor`. Sie werden einmal pro internem Provider verwendet, nicht pro Context, und müssen deshalb zustandslos und thread-sicher sein. Im internen Provider registriert, findet EF aber nur den Materialisierungs-Interceptor unter `ISingletonInterceptor`; einen Query-Expression-Interceptor ruft es nur auf, wenn er als `IInterceptor` registriert ist (Kapitel 30).

Zwei Wege der Registrierung

Weg	Eigenschaften	Wann
`optionsBuilder.AddInterceptors(...)`	Instanzen aus der Anwendung; können App-Dienste nutzen; pro Context verschieden	Interceptors mit App-Abhängigkeiten (Mandant, Benutzer, Uhr)
`services.AddScoped<IInterceptor, X>()` in `ApplyServices`	Im internen Provider aufgelöst („injected interceptors“); haben Zugriff auf EF-Dienste, nicht auf App-Dienste	Rein technische Interceptors ohne App-Abhängigkeit

Beide Wege kombiniert EF zu einer Liste. Das Beispiel nutzt den ersten, weil beide Interceptors den Mandanten der Anwendung brauchen (Kapitel 7).

Der SaveChanges-Interceptor

Er setzt die `TenantId` neuer Entitäten und verhindert, dass Daten eines anderen Mandanten geschrieben oder umgehängt werden. Das ergänzt RLS (Defense in Depth) und liefert verständlichere Fehler als die Datenbank. Kapitel 30 zeigt, welche Wege an ihm vorbeiführen:

Interceptors/TenantSaveChangesInterceptor.cs (1/2)

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    private readonly ITenantProvider _tenants;

    public TenantSaveChangesInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        _tenants = tenants;
    }

    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData, InterceptionResult<int> result)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return result;
    }

    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default)
    {
        if (eventData.Context is { } context) ApplyTenant(context);
        return ValueTask.FromResult(result);
    }
```

Interceptors/TenantSaveChangesInterceptor.cs (2/2, vereinfacht)

```
    internal void ApplyTenant(DbContext context)
    {
        // Welche Entitäten betroffen sind, steht in der Laufzeit-Annotation
        // (Kapitel „Fluent API auf Property-Ebene und Laufzeit-Annotations“)
        var changes = context.ChangeTracker.Entries()
            .Where(e => e.State is EntityState.Added or EntityState.Modified
                                or EntityState.Deleted)
            .Select(e => (Entry: e, Name: e.Metadata.GetTenantPropertyName()))
            .Where(x => x.Name is not null)
            .Select(x => (x.Entry, Property: x.Entry.Property(x.Name!)))
            .ToList();
        if (changes.Count == 0) return;

        var current = _tenants.CurrentTenantId
            ?? throw new TenancyException("Kein aktiver Mandant ...");

        foreach (var (entry, property) in changes)
        {
            var currentValue = (TenantId)property.CurrentValue!;
            var originalValue = (TenantId)property.OriginalValue!;
            switch (entry.State)
            {
                case EntityState.Added when currentValue.IsEmpty:
                    property.CurrentValue = current;
                    break;
                case EntityState.Added when currentValue != current:
                    throw Mismatch(entry, currentValue, current);
                // Nur ein tatsächlicher Wertwechsel zählt, nicht die Markierung
```

```
                case EntityState.Modified
                    when property.IsModified && currentValue != originalValue:
                    throw new TenancyException("Mandanten-Property ist unveränderlich.");
                case EntityState.Modified or EntityState.Deleted:
                    if (entry.State == EntityState.Modified)
                        property.IsModified = false;    // Spalte nicht schreiben
                    // Leer = unbekannt (Shadow-Property): die Datenbank entscheidet
                    if (!originalValue.IsEmpty && originalValue != current)
                        throw Mismatch(entry, originalValue, current);
                    break;
            }
        }
    }
}
```

Wer nur mit asynchronem Code arbeitet, überschreibt leicht nur eine Variante:

Interceptor (FALSCH)

```
public class TenantSaveChangesInterceptor : SaveChangesInterceptor
{
    public override ValueTask<InterceptionResult<int>> SavingChangesAsync(
        DbContextEventData eventData, InterceptionResult<int> result,
        CancellationToken cancellationToken = default) { /* Mandant setzen */ }

    // SavingChanges fehlt
}
```

Was passiert, wenn nur die asynchrone Variante implementiert ist?

Beobachtbar. Mit `SaveChangesAsync` funktioniert alles. Ein synchrones `SaveChanges`, etwa in einem Konsolenwerkzeug oder einem älteren Codepfad, speichert ohne Mandant; RLS lehnt die Zeile mit `WITH CHECK` ab, ein Plugin ohne Datenbankschutz schreibt sie still falsch.

Ursache. EF ruft je nach Aufruf genau eine Variante auf. Die Basisklasse `SaveChangesInterceptor` implementiert die andere als leere Methode, also ohne Fehler. Dasselbe gilt für Connection- und Command-Interceptors.

Lösung. Immer beide Varianten überschreiben und dieselbe Logik aufrufen, wie `ApplyTenant` im Beispiel.

Test: `FailureCaseTests.Interceptor_with_only_the_async_variant_….`

Markierung, Wertwechsel und Datenbankschutz

Drei Dinge sehen beim Speichern ähnlich aus und bedeuten Verschiedenes:

- **Die Tracking-Markierung.** `DbContext.Update` und `Attach` einer nicht getrackten Entität setzen `IsModified` für *jede* Property, auch für die Mandanten-Property. Original- und aktueller Wert sind dabei gleich, nämlich der Wert, den die Entität mitbringt. Bis 1.0 lehnte der Interceptor schon diese Markierung als Mandantenwechsel ab, und ein gewöhnliches Update aus einer API scheiterte.
- **Der tatsächliche Wertwechsel.** Nur wenn sich der Wert gegenüber dem Originalwert ändert, wird eine Zeile umgehängt. Das lehnt der Interceptor ab, auch wenn der neue Wert der aktive Mandant ist. Bei einem unveränderten Wert nimmt er die Markierung zurück, sodass das `UPDATE` die Mandantenspalte nicht schreibt.

- **Der Datenbankzustand.** Bei einer getrackten Entität stammt der Originalwert aus der Datenbank. Bei einer abgekoppelten Entität ist er nur eine Behauptung des Aufrufers. Eine manipulierte Entität mit dem eigenen Mandanten und der ID einer fremden Zeile besteht deshalb jede Prüfung im Interceptor. Erst die Policy entscheidet: Das `UPDATE` sieht die fremde Zeile nicht, ändert null Zeilen, und EF meldet eine `DbUpdateConcurrencyException`. Ein leerer Originalwert (eine Shadow-Property, die die Entität nicht tragen kann) gilt als unbekannt, nicht als fremd; auch hier entscheidet die Datenbank.

Der Interceptor verspricht also nur, was er ohne Datenbankzugriff wissen kann: keinen Mandantenwechsel im Change Tracker, keine bekannt fremden Werte, keine Änderung ohne aktiven Mandanten. Ob eine Zeile wirklich dem Mandanten gehört, prüft allein RLS. `SaveChangesInterceptorTests` deckt die Fälle ohne Datenbank ab, `DetachedUpdateTests` gegen PostgreSQL mit eingeschränkter Rolle: `Update` mit unverändertem Mandanten synchron und asynchron, die manipulierte fremde Zeilen-ID und das Umhängen einer getrackten Zeile.

Die Logik liegt in einer `internal`-Methode `ApplyTenant`. Tests können sie über `InternalsVisibleTo` direkt aufrufen, ohne eine Datenbank zu brauchen (Kapitel 26).

Der Connection-Interceptor

Er setzt bei jedem Öffnen der Verbindung das Session-Setting, das die Policies und `CurrentTenant()` auswerten. Der Ablauf (Öffnen abfangen, sync und async, Mandant holen) ist für jede Datenbank gleich und liegt deshalb in einer abstrakten Basis im relationalen Paket (Kapitel 46). Das PostgreSQL-Paket liefert nur den Befehl. Den Setting-Namen holt es über den Context aus den Event-Daten (`context.GetService<T>()`) aus dem *internen* Provider, den Mandanten aus dem *App*-Container:

`Interceptors/TenantSessionInterceptor.cs (relational), Runtime/TenantConnectionInterceptor.cs (vereinfacht)`

```
// Relationales Paket
public abstract class TenantSessionInterceptor : DbConnectionInterceptor
{
    protected TenantSessionInterceptor(ITenantProvider tenants)
    {
        ArgumentNullException.ThrowIfNull(tenants);

        Tenants = tenants;
    }

    protected ITenantProvider Tenants { get; }

    public override async Task ConnectionOpenedAsync(
        DbConnection connection, ConnectionEndEventData eventData,
        CancellationToken cancellationToken = default)
    {
        await using var command =
            CreateSessionCommand(connection, eventData.Context, Tenants.CurrentTenantId);
        await command.ExecuteNonQueryAsync(cancellationToken);
    }

    // ConnectionOpened (synchron) analog

    protected abstract DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant);
```

```
}

// PostgreSQL-Paket
public class TenantConnectionInterceptor : TenantSessionInterceptor
{
    public TenantConnectionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var settingName = context?.GetService<ITenancySingletonOptions>()
                              .Settings.SettingName
                          ?? TenancySettings.Default.SettingName;

        var command = connection.CreateCommand();
        command.CommandText = "SELECT set_config(@name, @value, false)";
        AddParameter(command, "name", settingName);
        // Immer setzen, auch leer: kein alter Wert bleibt stehen
        AddParameter(command, "value", tenant?.ToString() ?? string.Empty);
        return command;
    }
}
```

Leerer String statt NULL

Npgsql setzt den Session-Zustand zurück, wenn eine Verbindung in den Pool zurückkehrt. Danach liefert `current_setting('app.tenant_id', true)` einen leeren String statt `NULL`. Ein `''::uuid` würde jede Abfrage abbrechen. Policy und Translator verwenden deshalb `nullif(..., '')`: Ohne Mandant ist der Wert `NULL`, der Vergleich nie wahr, und es sind keine Zeilen sichtbar. Das ist die sichere Richtung.

EF öffnet und schließt die Verbindung standardmäßig für jede Abfrage. Jedes Öffnen kostet hier einen zusätzlichen Roundtrip. Bei vielen kleinen Abfragen lohnt es sich, die Verbindung pro Anfrage einmal explizit zu öffnen (Kapitel 39).

Wo der Connection-Interceptor nicht läuft

Der Interceptor läuft, wenn EF eine Verbindung öffnet, und nur dann. Zwei Situationen umgehen ihn, und beide entstehen leicht aus Versehen:

Situation	Folge	Abhilfe
Eine bereits geöffnete Verbindung wird übergeben (`UseNpgsql(connection)`, `SetDbConnection`)	EF öffnet sie nie, also läuft der Interceptor nie. Die Session behält ihren bisherigen Wert: keinen oder den eines früheren Mandanten	Verbindungen geschlossen übergeben und EF öffnen lassen; sonst `Database.ApplyTenantSession()` aufrufen
Die Verbindung bleibt offen (`OpenConnection`, eine offene Transaktion), während der Mandant wechselt	Der neue Mandant wirkt erst, wenn die Verbindung erneut geöffnet wird	Ein Context, ein Mandant; ist das nicht möglich, nach dem Wechsel `Database.ApplyTenantSession()` aufrufen

tests/...IntegrationTests/ConnectionLifetimeTests.cs (gekürzt, vereinfacht)

```
// Bereits offen, mit einem Session-Wert aus früherer Arbeit (Mandant B)
await using var connection = new NpgsqlConnection(appConnectionString);
await connection.OpenAsync();
// ... SELECT set_config('app.tenant_id', <tenant B>, false) ...

// Gedacht für Mandant A, aber EF öffnet die Verbindung nicht: kein Interceptor
await using var db = CreateAppContext(connection, tenantA);
var orders = await db.Orders.ToListAsync();       // die Zeilen von Mandant B!
```

Für diese Fälle bietet das Plugin das explizite Gegenstück zum Interceptor. `Database.ApplyTenantSession()` schreibt den aktuellen Mandanten in die Session, wenn die Verbindung offen ist, und tut nichts, wenn sie geschlossen ist, denn dann öffnet EF sie und der Interceptor übernimmt. Die Methode verwendet den Befehl des Interceptors selbst und funktioniert deshalb mit jedem Provider-Paket:

Verwendung

```
await using var db = CreateAppContext(connection, tenantA);   // Verbindung bereits offen
await db.Database.ApplyTenantSessionAsync();                   // Session jetzt: Mandant A
var orders = await db.Orders.ToListAsync();                    // die Zeilen von Mandant A
```

Interceptors/TenantSessionInterceptor.cs (Auszug)

```
public virtual async Task<bool> ApplyToOpenConnectionAsync(
    DbContext context, CancellationToken cancellationToken = default)
{
    var connection = context.Database.GetDbConnection();
    if (connection.State != ConnectionState.Open)
        return false;                     // EF öffnet sie später; der Interceptor übernimmt

    await using var command =
        CreateSessionCommand(connection, context, Tenants.CurrentTenantId);
    // In einer expliziten Transaktion muss der Befehl ihr zugeordnet sein
    command.Transaction = context.Database.CurrentTransaction?.GetDbTransaction();
    await command.ExecuteNonQueryAsync(cancellationToken);
    return true;
}
```

Die Erweiterungsmethode `ApplyTenantSession` im relationalen Paket sucht den Interceptor unter den Interceptoren des Contexts und bricht mit einer klaren Meldung ab, wenn keiner registriert ist. Zwei Details sind wichtig: Die Methode braucht den Context, nicht nur die Verbindung, weil der Provider seine Einstellungen aus dem internen Service Provider liest; deshalb erhält `CreateSessionCommand` den `DbContext` statt der Event-Daten. Und der Befehl muss sich einer laufenden Transaktion anschließen: PostgreSQL ist hier nachsichtig, SQL Server lehnt einen Befehl ab, der ihr nicht zugeordnet ist. Die SQL-Server-Tests prüfen genau diesen Fall; ohne die Zeile schlagen sie fehl.

In `ConnectionOpened` fehlt diese Zuordnung mit Absicht. Eine lokale Transaktion setzt eine offene Verbindung voraus, und EF öffnet die Verbindung, bevor es `BeginTransaction` aufruft; beim Öffnen gibt es also noch keine Transaktion. Eine umgebende `TransactionScope` wiederum übernehmen Npgsql und SqlClient beim Öffnen selbst, und jeder Befehl auf der Verbindung läuft darin, ohne dass ihm eine `DbTransaction` zugewiesen wird. `TransactionContextTests` und die SQL-Server-Tests prüfen alle drei Wege: die Verbindung, die `BeginTransactionAsync` öffnet, `ApplyTenantSession` in einer expliziten Transaktion und Lesen und Schreiben unter einer `TransactionScope`.

> **Ein Context, ein Mandant**
>
> Die Isolation folgt der Verbindung, nicht dem Context. Ein Context, der für mehrere Mandanten verwendet wird, oder eine Verbindung, die eine Anfrage überdauert, trägt den Mandanten des Moments, in dem sie geöffnet wurde. `ConnectionLifetimeTests` zeigt beide Fälle gegen eine echte Datenbank, zusammen mit den beiden Abhilfen: einer geschlossenen Verbindung, die EF selbst öffnet, und `ApplyTenantSession()`. Ein Plugin, dessen Hooks nicht jeden Weg abdecken, sollte eine solche explizite API anbieten, statt die Lücke nur zu dokumentieren.

Erkennen lässt sich die Lücke zusätzlich beim Laden: Mit `VerifyMaterializedTenants()` prüft das Plugin jede geladene Entität gegen den Mandanten des Contexts und wirft, statt fremde Zeilen zurückzugeben (Kapitel 17).

Dapper und rohes ADO.NET

Reports und Massenexporte laufen oft über Dapper oder rohes ADO.NET auf der Verbindung des Contexts (`Database.GetDbConnection()`). Row Level Security gilt auch für diese Befehle, weil sie in der Datenbank liegt und nicht in EF. Es kommt nur darauf an, ob der Session-Wert gesetzt ist, und das hängt davon ab, wer die Verbindung öffnet:

Wer die Verbindung öffnet	Ergebnis
EF, etwa `OpenConnectionAsync()` oder eine offene Transaktion	Der Interceptor hat die Session gesetzt; Dapper sieht die Zeilen des aktuellen Mandanten
Dapper oder ADO.NET selbst, auch nachdem EF die Verbindung nach einer Abfrage wieder geschlossen hat	Kein Session-Wert: Die Policies treffen keine Zeile (fail closed), bis `ApplyTenantSessionAsync()` ihn setzt
Mit Transaction Pooling (`UseTransactionPooling`)	Der Mandant existiert nur in der Transaktion von EF; Dapper muss darin laufen

Die zweite Zeile ist die Falle: EF öffnet und schließt die Verbindung für jede Abfrage, eine Verbindung, die eben noch den Mandanten trug, ist danach also wieder geschlossen. Dapper öffnet eine geschlossene Verbindung selbst, und der Interceptor läuft nicht. Das sichere Muster öffnet die Verbindung für den ganzen Block über EF:

Aufruf (Dapper)

```
await db.Database.OpenConnectionAsync();                // EF öffnet: Session gesetzt
var connection = db.Database.GetDbConnection();
var orders = await connection.QueryAsync<OrderDto>(
    "SELECT \"Id\", \"Total\" FROM \"Orders\"");             // RLS: nur der Mandant
await db.Database.CloseConnectionAsync();

// PgBouncer im Transaktionsmodus: Mandant nur in der Transaktion von EF
await using var tx = await db.Database.BeginTransactionAsync();
var rows = await connection.QueryAsync<OrderDto>(sql, transaction: tx.GetDbTransaction());
```

tests/.../RawAdoNetTests.cs (Auszug)

```
var tenant = await CreateTenantWithOrdersAsync(2);
await using var db = _fixture.CreateAppContext(
    new MutableTenantProvider { CurrentTenantId = tenant });
await db.Orders.CountAsync();                // EF hat geöffnet und wieder geschlossen
```

```
// Direkt geöffnet, wie Dapper es bei geschlossener Verbindung tut: ohne EF
var connection = db.Database.GetDbConnection();
await connection.OpenAsync();
Assert.Equal(0, await CountOrdersAsync(connection));      // fail closed

await db.Database.ApplyTenantSessionAsync();
Assert.Equal(2, await CountOrdersAsync(connection));
```

`RawAdoNetTests` prüft alle drei Zeilen gegen PostgreSQL mit einem einfachen Befehl, mehr schickt Dapper auch nicht. Das Ergebnis ist nie ein Leck: Ohne Session-Wert sieht die Abfrage nichts. Es ist ein falsches leeres Ergebnis, und deshalb gehört das Muster in die Richtlinien der Anwendung.

19 Diagnose und Logging

Versionsabhängigkeit: **stabil**

Ein professionelles Plugin meldet Ereignisse über EFs Logging-System. Dann landen die Meldungen in denselben Kanälen, die Nutzer ohnehin konfigurieren (`ILogger`, `LogTo`, `DiagnosticSource`), und Warnungen lassen sich per `ConfigureWarnings` ignorieren, protokollieren oder zu Fehlern machen.

Event-IDs

Diagnostics/TenancyEventId.cs (gekürzt)

```
public static class TenancyEventId
{
    public const int TenancyBaseId = 1_150_000;     // eigener, dokumentierter Bereich

    private enum Id
    {
        TenantIsolationApplied = TenancyBaseId,
        TenantIsolationOptedOut,
        PoliciesWithoutRowLevelSecurity,
    }

    private static readonly string ModelPrefix = "Acme.EntityFrameworkCore.Tenancy.Model.";

    public static readonly EventId TenantIsolationApplied =
        Make(Id.TenantIsolationApplied);
    public static readonly EventId TenantIsolationOptedOut =
        Make(Id.TenantIsolationOptedOut);
    // ...

    private static EventId Make(Id id) => new((int)id, ModelPrefix + id);
}
```

EF und Provider belegen niedrige Bereiche. Ein Plugin wählt einen eigenen, hohen Bereich und dokumentiert ihn. Die Namen folgen EFs Schema „Assembly.Kategorie.Ereignis“, damit Filter nach Präfix funktionieren.

Event-Definitionen

Eine `EventDefinition<T>` bündelt Event-ID, Log-Level, Meldungsvorlage und das Warnverhalten aus den Options. EF cached seine Definitionen pro internem Provider; das Plugin macht es genauso mit einer eigenen Klasse, die als Singleton registriert ist (Kapitel 6):

Diagnostics/TenancyLoggingDefinitions.cs (Auszug)

```
public class TenancyLoggingDefinitions
{
    private EventDefinition<string>? _tenantIsolationOptedOut;

    public virtual EventDefinition<string> LogTenantIsolationOptedOut(
        IDiagnosticsLogger logger)
        => LazyInitializer.EnsureInitialized(ref _tenantIsolationOptedOut, () =>
```

```
            Create(logger, TenancyEventId.TenantIsolationOptedOut, LogLevel.Warning,
                nameof(TenancyEventId.TenantIsolationOptedOut),
                "Entität '{entityType}' implementiert ITenantOwned, ist aber von der "
                + "Mandantenisolation ausgenommen."));

    private static EventDefinition<string> Create(
        IDiagnosticsLogger logger, EventId eventId, LogLevel level,
        string name, string message)
        => new(
            logger.Options,                      // enthält das Warnverhalten
            eventId,
            level,
            "TenancyEventId." + name,            // steht in Warning-as-Error-Meldungen
            l => LoggerMessage.Define<string>(l, eventId, message));
}
```

Die Logger-Extension

Die eigentliche Log-Methode folgt exakt dem Muster aus EFs `CoreLoggerExtensions`: erst klassisches Logging, dann die Ereignisdaten für DiagnosticSource und `LogTo`:

Diagnostics/TenancyLoggerExtensions.cs (Auszug, vereinfacht)

```
private static void LogEntityType(
    IDiagnosticsLogger<DbLoggerCategory.Model> diagnostics,
    EventDefinition<string> definition,
    IReadOnlyEntityType entityType)
{
    // 1. ILogger; wirft bei ConfigureWarnings(w => w.Throw(...))
    if (diagnostics.ShouldLog(definition))
        definition.Log(diagnostics, entityType.DisplayName());

    // 2. DiagnosticSource und LogTo (simple logging)
    if (diagnostics.NeedsEventData(definition, out var diagnosticSource, out var simpleLog))
    {
        var eventData = new EntityTypeEventData(definition, GenerateMessage, entityType);
        diagnostics.DispatchEventData(definition, eventData, diagnosticSource, simpleLog);
    }
}
```

Nutzer können die Warnung damit in einen Fehler verwandeln:

Anwendung

```
options.UseNpgsql(cs)
       .UseAcmeTenancy()
       .ConfigureWarnings(w => w.Throw(TenancyEventId.TenantIsolationOptedOut));

// -> InvalidOperationException beim Modellbau:
//    "An error was generated for warning '...TenantIsolationOptedOut': Entität
//     'AuditEntry' ... This exception can be suppressed or logged by passing event ID
//     'TenancyEventId.TenantIsolationOptedOut' to the 'ConfigureWarnings' method ..."
```

> **Tipp**
>
> Sicherheitsrelevante Abweichungen gehören als **Warnung** ins Log und nicht als Debug-Meldung.

> Teams mit strengen Anforderungen eskalieren sie per `ConfigureWarnings` zu Fehlern; andere sehen sie zumindest beim Start. Das Plugin muss diese Entscheidung nicht für alle treffen.

OpenTelemetry: der Mandant im Trace

Verteiltes Tracing mit OpenTelemetry baut auf `System.Diagnostics.Activity` auf. EF Core startet selbst keine Activities. Die Spans der Datenbankbefehle kommen entweder von der OpenTelemetry-Instrumentierung für EF Core, die auf die `DiagnosticSource`-Ereignisse von EF hört, oder vom Datenbanktreiber: Npgsql und SqlClient haben eigene Activity-Quellen. Beiden fehlt meist der Mandant, die erste Frage, wenn eine Abfrage im Betrieb langsam ist oder scheitert. Mit `EnrichOpenTelemetry()` ergänzt das Plugin ihn als Tag `tenant.id` (Standard: aus):

Aufruf

```
options.UseNpgsql(cs).UseAcmeTenancy(sp, t => t.EnrichOpenTelemetry());
// SQL Server: UseSqlServer(cs, sql => sql.UseAcmeTenancy(sp).EnrichOpenTelemetry())
```

Ein Command-Interceptor taggt vor jedem Befehl `Activity.Current`. Er ist im internen Service Provider registriert, eine Instanz für alle Contexts, und liest den Mandanten aus dem Context des Befehls:

Interceptors/TenantActivityEnrichmentInterceptor.cs (Auszug)

```
protected virtual void Enrich(CommandEventData eventData)
{
    // Billige Prüfungen zuerst: Ohne aufgezeichnete Activity gibt es nichts zu tun
    if (Activity.Current is not { IsAllDataRequested: true } activity
        || eventData.Context is not { } context)
        return;

    // Der Tenant-Provider dieses Contexts reist mit seinem SaveChanges-Interceptor
    var tenant = context.GetService<IDbContextOptions>()
        .FindExtension<CoreOptionsExtension>()?.Interceptors?
        .OfType<TenantSaveChangesInterceptor>().FirstOrDefault()?.Tenants.CurrentTenantId;

    // Ohne Mandant kein Tag: Ein leerer oder Platzhalterwert wäre irreführend
    if (tenant is { } id)
        activity.SetTag(TagName, id.Value.ToString("D"));
}
```

Welche Activity das Tag erhält, hängt von der Instrumentierung ab:

Instrumentierung	Getaggte Activity
OpenTelemetry-Instrumentierung für EF Core	Der Span des Befehls selbst: EF 10 löst sein `CommandExecuting`-Ereignis, das den Span startet, vor den Interceptors aus
Nur der Treiber (Npgsql, SqlClient)	Der umgebende Span, meist der HTTP-Request: Der Treiber startet seinen Span erst in `ExecuteReader`, nach dem Interceptor
Kein Listener	Keine; der Interceptor kehrt nach einer Prüfung zurück, ohne Allokationen

`ActivityEnrichmentTests` prüft das Tag gegen PostgreSQL mit einem `ActivityListener`: gesetzt, wenn eingeschaltet, fehlend, wenn ausgeschaltet, und fehlend ohne Mandant.

> **Mandanten-IDs in der Telemetrie**
>
> Eine Mandanten-ID ist ein Bezeichner und kein Geheimnis, aber Traces verlassen oft die Anwendung: zu einem Monitoring-Dienst, manchmal zu einem anderen Unternehmen. Ob Mandanten-IDs dorthin dürfen, entscheidet der Betreiber nach Datenschutzgesichtspunkten, deshalb ist die Option standardmäßig aus.

Teil VI

Migrations und Design-Time

Schema-Änderungen erzeugen, die Werkzeuge von dotnet ef erweitern und Compiled Models unterstützen.

- Kapitel 20: Wie Migrations funktionieren
- Kapitel 21: Deklaratives SQL aus Annotations
- Kapitel 22: Eigene Migrations-Operationen
- Kapitel 23: Design-Time-Services
- Kapitel 24: Compiled Models, Trimming und NativeAOT
- Kapitel 25: Scaffolding (Reverse Engineering)

20 Wie Migrations funktionieren

Versionsabhängigkeit: **stabil**

Migrations vergleichen zwei Modelle und erzeugen daraus Operationen und SQL. Ein Plugin kann sich an mehreren Stellen einklinken; um die richtige zu wählen, muss man den Ablauf kennen:



Für Plugins gibt es zwei grundsätzliche Strategien. **Deklarativ**: Das Plugin beschreibt den gewünschten Zustand als Annotations; EF erkennt Änderungen von selbst, und das Plugin muss nur SQL erzeugen. **Imperativ**: Das Plugin bietet eigene Operationen an, die Nutzer von Hand in Migrationen schreiben. Gute Plugins nutzen beide: deklarativ für alles, was zum Modell gehört, imperativ für alles andere.

> **Hinweis**
>
> `Database.EnsureCreated()` nutzt dieselbe Pipeline: Es berechnet den Diff vom leeren Zustand zum Design-Time-Modell und führt das SQL des Migrations-Generators aus. Eigene Migrations-SQL eines Plugins wirkt also auch dort, was Integrationstests erheblich vereinfacht (Kapitel 26).

21 Deklaratives SQL aus Annotations

Versionsabhängigkeit: **volatil**

Der Annotation-Provider

Der Differ vergleicht das *relationale* Modell (Tabellen, Spalten), nicht die Entity-Typen. Welche Annotations eine Tabelle trägt, entscheidet der `IRelationalAnnotationProvider`. Den gibt es pro Context einmal, und er gehört dem Provider; das Plugin erbt deshalb von der Npgsql-Implementierung:

`Metadata/Internal/TenancyAnnotationProvider.cs`

```
public class TenancyAnnotationProvider : NpgsqlAnnotationProvider
{
    public TenancyAnnotationProvider(RelationalAnnotationProviderDependencies dependencies)
        : base(dependencies)
    {
    }

    public override IEnumerable<IAnnotation> For(ITable table, bool designTime)
    {
        foreach (var annotation in base.For(table, designTime))
            yield return annotation;

        if (!designTime)          // nur für Migrations: Laufzeitmodell bleibt schlank
            yield break;

        // TPH/Table Splitting: der Typ mit RLS-Konfiguration, nicht einfach der erste
        var entityType = table.EntityTypeMappings
            .Select(m => (IEntityType)m.TypeBase)
            .FirstOrDefault(e => e.HasRlsConfiguration())
            ?? (IEntityType)table.EntityTypeMappings.First().TypeBase;
        foreach (var annotation in entityType.GetAnnotations())
        {
            if (!annotation.Name.StartsWith(RlsAnnotationNames.Prefix,
                                            StringComparison.Ordinal))
                continue;
            // Opt-out-Marker nicht weitergeben, sonst entstehen leere Migrationen
            if (annotation.Name == RlsAnnotationNames.Enabled && annotation.Value is false)
                continue;
            yield return annotation;
        }
    }
}
```

Ab hier übernimmt EF: Ändert sich eine Policy, erzeugt der Differ eine `AlterTableOperation` mit neuen und alten Annotations. In der generierten Migration sieht das so aus, ganz ohne eigenen Code-Generator:

Generierte Migration (Ausschnitt)

```
migrationBuilder.AlterTable(name: "Orders")
    .Annotation("Acme:Rls:Policy:tenant_isolation", "{...neu...}")
    .OldAnnotation("Acme:Rls:Policy:tenant_isolation", "{...alt...}");
```

Weil der Annotation-Provider die Annotations nur für `designTime` weitergibt, fehlen sie in einem Modell: dem Laufzeitmodell, das `context.Model` liefert.

Startprüfung eines Plugins (FALSCH)

```
// Hat jede isolierte Tabelle RLS? Gelesen aus dem Laufzeitmodell
var table = context.Model.GetRelationalModel().FindTable("Orders", null);
var enabled = table?[RlsAnnotationNames.Enabled];          // immer null
```

Was passiert, wenn Plugin-Code das Laufzeitmodell statt des Design-Time-Modells liest?

Beobachtbar. Code, der Schema-Annotations des Plugins aus dem relationalen Laufzeitmodell liest, findet keine: Eine Prüfung meldet nichts, ein selbst erzeugtes Skript enthält keine Policies. Den Differ schützt EF selbst: `GetDifferences` mit dem Laufzeitmodell bricht mit dem Hinweis auf `IDesignTimeModel` ab.

Ursache. Das Laufzeitmodell ist schlank. Der Annotation-Provider gibt Annotations für Tabellen nur zur Design-Zeit weiter, damit sie die Laufzeit nicht belasten.

Lösung. Für alles, was mit dem Schema zu tun hat, `context.GetService<IDesignTimeModel>().Model` verwenden.

Test: `FailureCaseTests.Table_annotations_of_the_runtime_model_…` und `Differ_rejects_the_runtime_model_…`.

Der SQL-Generator

Auch den `IMigrationsSqlGenerator` gibt es nur einmal. Das Plugin erbt vom Npgsql-Generator und ergänzt `CreateTable` und `AlterTable`:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Auszug)

```
protected override void Generate(
    CreateTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // Unterminiertes SQL kann keine weiteren Statements aufnehmen (wie bei Npgsql)
    if (!terminate && operation.HasRlsConfiguration())
        throw new ArgumentException(
            "RLS kann nicht in unterminiertem CREATE TABLE entstehen.");

    base.Generate(operation, model, builder, terminate);
    if (terminate)
        GenerateRowLevelSecurity(operation.Name, operation.Schema, previous: null,
            operation, builder);
}

protected override void Generate(
    AlterTableOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    base.Generate(operation, model, builder);
```

```
        GenerateRowLevelSecurity(operation.Name, operation.Schema, operation.OldTable,
            operation, builder);
    }
```

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (gekürzt, vereinfacht)

```
    protected virtual void GenerateRowLevelSecurity(string tableName, string? schema,
        IReadOnlyAnnotatable? previous, IReadOnlyAnnotatable current,
        MigrationCommandListBuilder builder)
    {
        var table = Delimit(tableName, schema);
        var oldPolicies = previous?.GetRlsPolicies() ?? new Dictionary<string, string>();
        var newPolicies = current.GetRlsPolicies();

        // 1. Entfernte oder geänderte Policies löschen
        foreach (var (name, json) in oldPolicies)
            if (!newPolicies.TryGetValue(name, out var n) || n != json)
                Statement(builder, $"DROP POLICY {Delimit(name)} ON {table}");

        // 2. RLS-Status (ENABLE/DISABLE, FORCE/NO FORCE)
        if ((previous?.IsRlsEnabled() ?? false) != current.IsRlsEnabled())
            Statement(builder, $"ALTER TABLE {table} "
                + (current.IsRlsEnabled() ? "ENABLE" : "DISABLE") + " ROW LEVEL SECURITY");
        // ... FORCE analog

        // 3. Neue oder geänderte Policies anlegen
        foreach (var (name, json) in newPolicies)
            if (!oldPolicies.TryGetValue(name, out var o) || o != json)
                Statement(builder, CreatePolicySql(table, RlsPolicy.Deserialize(json)));
    }

    protected virtual void Statement(MigrationCommandListBuilder builder, string sql)
    {
        builder.Append(sql).AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
        EndStatement(builder);
    }
```

Detail	Begründung
DROP + CREATE statt `ALTER POLICY`	`ALTER POLICY` kann weder `FOR` noch PERMISSIVE/RESTRICTIVE ändern. DDL ist in PostgreSQL transaktional; es entsteht keine sichtbare Lücke.
`EndStatement(builder)`	Beendet den Befehl wie die Basisklasse; berücksichtigt dabei Transaktions-unterdrückung.
Eigene `Delimit`-Helfer	Npgsqls `DelimitIdentifier`-Helfer sind privat. Bezeichner immer über `Dependencies.SqlGenerationHelper` maskieren, nie per String-Verkettung.
DropTable, RenameTable	Nichts zu tun: PostgreSQL entfernt bzw. verschiebt Policies mit der Tabelle.

22 Eigene Migrations-Operationen

Versionsabhängigkeit: **mittel**

Manches gehört nicht ins Modell, aber in Migrationen: Rollen, Rechte, Extensions, Daten-Backfills. Dafür bietet das Plugin eigene Operationen an. Sie bestehen aus drei Teilen: der Operationsklasse, einer Extension-Method auf `MigrationBuilder` und der SQL-Erzeugung im Generator.

Migrations/Operations/TenancyOperations.cs (Auszug)

```
public class CreateApplicationRoleOperation : MigrationOperation
{
    public required string Name { get; init; }
    public bool CanLogin { get; init; }       // Passwörter gehören NICHT in Migrationen
}
```

Migrations/Operations/TenancyMigrationBuilderExtensions.cs (Auszug)

```
namespace Microsoft.EntityFrameworkCore.Migrations;      // in Migrationen immer importiert

public static class TenancyMigrationBuilderExtensions
{
    public static OperationBuilder<CreateApplicationRoleOperation> CreateApplicationRole(
        this MigrationBuilder migrationBuilder, string name, bool canLogin = false)
    {
        var operation = new CreateApplicationRoleOperation
        {
            Name = TenancyIdentifiers.ValidateIdentifier(name, "Rollenname"),
            CanLogin = canLogin,
        };
        migrationBuilder.Operations.Add(operation);
        return new OperationBuilder<CreateApplicationRoleOperation>(operation);
    }
}
```

Der Generator verteilt Operationen über die zentrale `Generate(MigrationOperation, ...)`-Methode. Unbekannte Typen gehen an die Basisklasse:

Migrations/Internal/TenancyMigrationsSqlGenerator.cs (Auszug, vereinfacht)

```
protected override void Generate(
    MigrationOperation operation, IModel? model, MigrationCommandListBuilder builder)
{
    switch (operation)
    {
        case CreateApplicationRoleOperation createRole: Generate(createRole, builder); break;
        case DropApplicationRoleOperation dropRole:     Generate(dropRole, builder); break;
        case GrantTablePrivilegesOperation grant:       Generate(grant, builder); break;
        default: base.Generate(operation, model, builder); break;
    }
}

protected virtual void Generate(CreateApplicationRoleOperation operation,
    MigrationCommandListBuilder builder)
{
    var literal = Literal(operation.Name);           // 'shop_app'  (für den Vergleich)
```

```
    var identifier = Delimit(operation.Name);    // shop_app (CREATE ROLE, ggf. in "...")
    var login = operation.CanLogin ? "LOGIN" : "NOLOGIN";

    builder
        .AppendLine("DO $$")
        .AppendLine("BEGIN")
        .AppendLine("    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles")
        .AppendLine($"                   WHERE rolname = {literal}) THEN")
        .AppendLine($"        CREATE ROLE {identifier} {login} NOSUPERUSER NOBYPASSRLS;")
        .AppendLine("    END IF;")
        .AppendLine("END")
        .Append("$$");
    builder.AppendLine(Dependencies.SqlGenerationHelper.StatementTerminator);
    EndStatement(builder);
}
```

Literal oder Bezeichner

Der Rollenname kommt zweimal vor, aber in zwei Formen: als **String-Literal** im Vergleich (`'shop_app'`, erzeugt über das String-Type-Mapping) und als **Bezeichner** in `CREATE ROLE` (`shop_app`, bei Bedarf in doppelten Anführungszeichen). Wer beides verwechselt oder per String-Verkettung baut, erzeugt ungültiges SQL oder eine Injection-Lücke. Zusätzlich validiert die Extension-Method den Namen.

Tipp

Operationen idempotent formulieren (`IF NOT EXISTS`, `DROP ... IF EXISTS`). Rollen existieren clusterweit und überleben Datenbanken; eine Migration, die beim zweiten Ausführen in einer neuen Datenbank scheitert, ist ein häufiger Ärger.

In einer Migration nutzt man die Operationen wie EFs eigene:

`Migrations/20261021_Initial.cs (handergänzt)`

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // VOR den Tabellen (Kapitel „Migrations für Fortgeschrittene“)
    migrationBuilder.CreateApplicationRole("shop_support");
    // ... von EF generierte Operationen (Tabellen, RLS über Annotations) ...
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");
}

protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropApplicationRole("shop_app");
    // ...
}
```

Wenn der Differ eigene Operationen erzeugen soll

Sollen eigene Operationen vom *Differ* automatisch erzeugt werden, braucht es einen abgeleiteten `IMigrationsModelDiffer` und einen `ICSharpMigrationOperationGenerator`, der die Operation als C# schreibt. Beides sind Einzeldienste mit viel interner API. Fast immer ist der deklarative Weg über Annotations (Kapitel 21) die bessere Wahl.

23 Design-Time-Services

Versionsabhängigkeit: **volatil**

Die Werkzeuge von `dotnet ef` (Migrationen anlegen, Snapshots schreiben, Scaffolding, Compiled Models) verwenden einen eigenen Service Provider mit Design-Time-Diensten. Ein Plugin erweitert ihn über `IDesignTimeServices`.

Automatisch geladen

Damit Nutzer nichts konfigurieren müssen, verweist ein Assembly-Attribut auf die Klasse. Das Tooling findet es in jeder Assembly, die das Startprojekt referenziert. Das zweite Argument beschränkt das Laden auf Contexts mit dem passenden Provider:

Properties/AssemblyInfo.cs

```
[assembly: DesignTimeServicesReference(
    "Acme.EntityFrameworkCore.Tenancy.PostgreSQL.Design.TenancyDesignTimeServices, "
        + "Acme.EntityFrameworkCore.Tenancy.PostgreSQL",
    "Npgsql.EntityFrameworkCore.PostgreSQL")]
```

Design/TenancyDesignTimeServices.cs

```
public class TenancyDesignTimeServices : IDesignTimeServices
{
    public void ConfigureDesignTimeServices(IServiceCollection services)
    {
        // Alle Registrierungen entfernen (EF-Default, Npgsql), dann unsere setzen:
        // unabhängig von der Reihenfolge, in der das Tooling Services lädt
        services.RemoveAll<IAnnotationCodeGenerator>();
        services.AddSingleton<IAnnotationCodeGenerator, TenancyAnnotationCodeGenerator>();
    }
}
```

Lesbare Snapshots: der Annotation-Code-Generator

Ohne Zutun schreibt EF unbekannte Annotations als `HasAnnotation(...)` in den Snapshot, bei Policies mit JSON-Werten. Das funktioniert, ist aber kaum lesbar. Ein eigener `IAnnotationCodeGenerator` übersetzt sie in die Fluent API:

ShopContextModelSnapshot.cs

```
// Vorher
b.HasAnnotation("Acme:Rls:Enabled", true);
b.HasAnnotation("Acme:Rls:Policy:tenant_isolation", "{\"Name\":\"tenant_isolation\",...}");

// Nachher
RlsEntityTypeBuilderExtensions.HasRowLevelSecurity(b, true);
RlsEntityTypeBuilderExtensions.HasPolicy(b, "tenant_isolation", p => p
    .Using("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid")
    .WithCheck("\"TenantId\" = nullif(current_setting('app.tenant_id', true), '')::uuid"));
```

EF Core 10 schreibt Erweiterungsmethoden im Snapshot als statische Aufrufe (`Extensions.Methode(b, ...)`) statt als `b.Methode(...)`. Das vermeidet Mehrdeutigkeiten, wenn mehrere

Provider oder Plugins gleichnamige Methoden definieren. Tests, die den erzeugten Code prüfen, sollten diese Form erwarten.

Design/Internal/TenancyAnnotationCodeGenerator.cs (Auszug, vereinfacht)

```
public override IReadOnlyList<MethodCallCodeFragment> GenerateFluentApiCalls(
    IEntityType entityType, IDictionary<string, IAnnotation> annotations)
{
    var calls = new List<MethodCallCodeFragment>();

    // VERTRAG: Übersetzte Annotations aus dem Dictionary ENTFERNEN
    if (annotations.TryGetValue(RlsAnnotationNames.Enabled, out var enabled)
        && enabled.Value is true)
    {
        annotations.Remove(RlsAnnotationNames.Enabled);
        var forced = annotations.Remove(RlsAnnotationNames.Forced, out var f)
                     && f.Value is true;
        calls.Add(new MethodCallCodeFragment(HasRowLevelSecurityMethod, forced));
    }

    foreach (var key in annotations.Keys
                 .Where(k => k.StartsWith(RlsAnnotationNames.PolicyPrefix,
                                          StringComparison.Ordinal))
                 .ToList())                          // Kopie: Dictionary ändert sich
    {
        var policy = RlsPolicy.Deserialize((string)annotations[key].Value!);
        annotations.Remove(key);
        calls.Add(new MethodCallCodeFragment(HasPolicyMethod, policy.Name,
            new NestedClosureCodeFragment("p", BuildPolicyChain(policy))));
    }

    calls.AddRange(base.GenerateFluentApiCalls(entityType, annotations));
    return calls;
}
```

Stolperstelle	Lösung
Übersetzte Annotation bleibt im Dictionary	Erscheint doppelt im Snapshot. Immer entfernen; was übrig bleibt, schreibt EF als `HasAnnotation`.
Generische Methode im `MethodInfo`	Der Snapshot nutzt den nicht-generischen `EntityTypeBuilder`. Nicht-generische Überladung per `GetRuntimeMethod` mit exakten Parametertypen wählen.
`params`-Argument als Array	`MethodCallCodeFragment` nimmt `params object?[]`. Ein `string[]` würde wegen Array-Kovarianz zur Argumentliste: in `new object?[] { roles }` verpacken.
Enums im Snapshot	Werden als `PolicyCommand.Select` gerendert. Das Enum liegt deshalb im immer importierten Namespace `Microsoft.EntityFrameworkCore`.
Lambda-Argumente	`NestedClosureCodeFragment("p", kette)` erzeugt `p => p.Using(...)`; Ketten entstehen mit `fragment.Chain(...)`.

Der Generator betrifft nur Snapshot und Designer-Dateien. Die eigentlichen Migrationen behalten `.Annotation(...)` an `AlterTable`; das ist der vorgesehene Mechanismus für Änderungen.

24 Compiled Models, Trimming und NativeAOT

Versionsabhängigkeit: **mittel**

`dotnet ef dbcontext optimize` erzeugt ein **Compiled Model**: C#-Code, der das Laufzeitmodell direkt aufbaut, statt es beim Start über Conventions zu berechnen. Das verkürzt die Startzeit großer Modelle deutlich und ist Voraussetzung für NativeAOT. Ein Plugin muss dafür an vier Stellen vorbereitet sein:

Baustein	Anforderung	Im Beispiel
Type Mappings	Öffentlicher Typ mit öffentlicher statischer Property `Default` (für NativeAOT genutzt); Konverter/Comparer als gemeinsame Instanzen	`TenantIdTypeMapping.Default`, `TenantIdConverter.Instance`, `TenantIdValueComparer`
JSON-Reader/Writer	Statische `Instance` und `ConstructorExpression`	`TenantIdJsonReaderWriter`
Value Generators	Konfiguriert mit `HasValueGeneratorFactory(Type)`; Factory mit öffentlichem parameterlosem Konstruktor	`TenantIdValueGeneratorFactory` (Kapitel 31)
Annotations	Einfache Werte übernimmt EF automatisch; ein `ICSharpRuntimeAnnotationCodeGenerator` filtert Design-Time-Annotations und übersetzt komplexe Werte	Filtert RLS-Policies, behält die Laufzeit-Annotation (Kapitel 44)

Der dritte Punkt wird oft missverstanden. EF übernimmt Annotations mit einfachen Werten (Strings, Zahlen, bool, Enums) **automatisch** ins Compiled Model; das zeigt der Quellcode von `CSharpRuntimeAnnotationCodeGenerator`, der sie mit `GenerateSimpleAnnotations` schreibt. Ein eigener Generator ist deshalb nicht nötig, damit Annotations *erhalten* bleiben, sondern um reine Design-Time-Annotations *herauszufiltern*, die zur Laufzeit niemand liest, und um Annotations mit komplexen Werten in Code zu übersetzen. Kapitel 44 zeigt beides im Beispiel.

Trimming und AOT

Alle vier Pakete setzen `<IsAotCompatible>true</IsAotCompatible>`. Der Compiler meldet dann Code, der Reflection ohne Annotationen nutzt. Die JSON-Serialisierung der Policies verwendet einen Source-Generator statt Reflection:

Metadata/RlsPolicy.cs (Auszug)

```
[JsonSourceGenerationOptions(UseStringEnumConverter = true)]
[JsonSerializable(typeof(RlsPolicy))]
internal partial class RlsJsonContext : JsonSerializerContext;
```

Wo Dynamik unvermeidbar ist, wird die Warnung mit Begründung unterdrückt statt übergangen. Die Isolations-Convention baut den Query-Filter als Expression Tree, was dynamischen Code braucht. Conventions laufen aber nur beim Modellbau, und NativeAOT setzt ein Compiled Model voraus, in dem sie nicht mehr laufen; die Unterdrückung nennt genau diesen Grund. Die Warnungen erscheinen nur bei einem vollständigen Build zuverlässig, inkrementelle Builds verschweigen sie (Upgrade-Playbook in Kapitel 27).

> **Hinweis**
>
> Conventions, Snapshots und Migrations laufen nur zur Design-Zeit oder beim Modellbau; mit Compiled Model und vorkompilierten Abfragen fallen sie zur Laufzeit weg. Für AOT kritisch sind deshalb vor allem Type Mappings, Konverter, Translators (laufen beim Build) und Interceptors.

Was als Verbraucher tatsächlich geprüft ist

`IsAotCompatible` lässt Analyzer laufen, beweist aber nichts zur Laufzeit. Auch ein Unit-Test, der Compiled-Model-Code erzeugt und Textstellen prüft (`CompiledModelTests`), kompiliert und lädt das Modell nicht. Deshalb prüft `eng/consumer-test/run.sh` das Plugin so, wie eine Anwendung es bekommt, in einem temporären Verzeichnis, das am Ende gelöscht wird:

Schritt	Prüfung
Pakete	`dotnet pack` der drei Bibliotheken in eine lokale Quelle; ein frisches Projekt referenziert die Pakete, nicht die Projekte. Eigener NuGet-Cache, Paketquellen-Zuordnung `Acme.*` nur aus der lokalen Quelle und ein SHA-512-Vergleich belegen, dass genau die eben gebauten Pakete aufgelöst werden.
Tooling	`dotnet-ef` als lokales Tool. Die Design-Time-Dienste werden ohne Zutun gefunden. Migration und SQL-Skript enthalten RLS, `FORCE`, Policy und Index (PostgreSQL) sowie Security Policy und Session-Schlüssel (SQL Server). `has-pending-model-changes` meldet danach keine Änderung.
Laufzeitmodell	`TenantId` wird `uuid`, die Laufzeit-Annotation nennt die Property, `CurrentTenant()` und der Query-Filter werden übersetzt.
Compiled Model	`dotnet ef dbcontext optimize`, kompilieren, mit `UseModel` laden, dieselben Prüfungen.
NativeAOT (`--nativeaot`)	Veröffentlichen mit `PublishAot` und vorkompilierten Abfragen, dann gegen PostgreSQL in Docker starten: zwei Mandanten schreiben über die Interceptors, einer liest, RLS liefert nur seine Zeilen. Keine Trim- oder AOT-Warnung darf aus einer Acme-Assembly stammen.

Die CI führt den Ablauf in einem eigenen Job mit dem neuesten stabilen 10.0.x-Patch für das Tooling aus und gibt die Versionen aus. Der Test hat dabei Dinge gefunden, die die Unit-Tests nicht zeigen konnten:

- **Query-Filter und Compiled Models schließen sich aus.** EF Core 10 lehnt `optimize` ab: „Compiled model can't be generated, because query filters are not supported." Wer den optionalen Filter des Plugins einschaltet, verzichtet auf Compiled Model und NativeAOT; der Schutz kommt dann allein von RLS. Der Test erwartet diesen Fehler ausdrücklich, damit eine spätere EF-Version, die Filter kann, auffällt.
- **Value Comparer per Reflection.** Das Type Mapping für `TenantId` brachte keine eigenen Comparer mit. Das Compiled Model verweist dann nur auf `TenantIdTypeMapping.Default`, und EF erzeugt den Comparer beim ersten Tracking mit `MakeGenericMethod`, was die NativeAOT-Anwendung abbrechen ließ. Seitdem tragen beide Mappings gemeinsame Instanzen aus `TenantIdValueComparer`, so wie Konverter und JSON-Reader.
- **Grenzen von EF selbst.** Vorkompilierte Abfragen und NativeAOT sind in EF Core 10 experimentell (Warnung EF9100). `PublishAot` darf nicht im Projekt stehen, weil EF sonst schon zur Design-Zeit den Modellbau verweigert. Das Paket `Microsoft.EntityFrameworkCore.`

> `Tasks` erzeugt Modell und Abfragen beim Veröffentlichen, ein zusätzlich manuell erzeugtes Modell stört es. Die Vorkompilierung lehnt `o.Total > 0` bei einer `decimal`-Property ab, `0m` geht. EF, EF Relational und Npgsql melden selbst Trim- und AOT-Warnungen (IL2104, IL3053).

Nicht geprüft ist SQL Server unter NativeAOT; das Paket ist dafür markiert, ein Lauf mit einer echten Datenbank fehlt.

25 Scaffolding (Reverse Engineering)

Versionsabhängigkeit: **mittel**

`dotnet ef dbcontext scaffold` liest ein bestehendes Datenbankschema und erzeugt Entitäten und einen Context. Ein Plugin, das eigene Schema-Elemente verwaltet, kann sie dabei ebenfalls erkennen. Der Weg führt über zwei Design-Time-Dienste:

Dienst	Aufgabe
`IDatabaseModelFactory`	Liest das Schema in ein `DatabaseModel`. Ein abgeleiteter Npgsql-Factory könnte `pg_policies` und `pg_class.relrowsecurity` abfragen und als Annotations an `DatabaseTable` hängen.
`IAnnotationCodeGenerator`	Schreibt diese Annotations als Fluent API in den generierten `OnModelCreating`, mit derselben Logik wie beim Snapshot.

Das Beispiel implementiert beides: Eine eigene Database-Model-Factory liest RLS-Status und Policies aus der Datenbank, und der Annotation-Code-Generator aus Kapitel 23 schreibt sie in den gescaffoldeten Context. Kapitel 45 zeigt die Umsetzung im Detail.

Teil VII

Qualität und Auslieferung

Testen, versionieren, paketieren, und die Checkliste für den Weg von A bis Z.

- Kapitel 26: Teststrategie
- Kapitel 27: Versionierung und Kompatibilität
- Kapitel 28: Packaging und Dokumentation
- Kapitel 29: Die Checkliste von A bis Z

26 Teststrategie

Versionsabhängigkeit: **mittel**

Plugins lassen sich erstaunlich weit ohne Datenbank testen. Modellbau, Type Mappings, SQL-Übersetzung, Migrations-SQL und Snapshots brauchen nur einen Connection-String, keine Verbindung. Die Teststrategie des Beispiels hat vier Ebenen:

Ebene	Prüft	Datenbank
Modelltests	Annotations, Conventions, Validierung, Optionen, Caching	nein
SQL-Tests	Übersetzungen per `ToQueryString`, Migrations-SQL per Differ und Generator	nein
Snapshot-Tests	Code-Generator als String und als Roundtrip (Roslyn)	nein
Integrationstests	Echtes Verhalten: RLS, Rollen, Interceptors (Testcontainers)	ja (Docker)

Testhilfen

Tests bauen viele Contexts mit unterschiedlichem `OnModelCreating`. Weil EF das Modell pro internem Provider cached, muss jeder Test einen eigenen Provider bekommen; dafür gibt es `EnableServiceProviderCaching(false)`:

tests/.../TestModel.cs (gekürzt, vereinfacht)

```
public static TestContext Create(
    Action<TenancyOptionsBuilder>? tenancy = null,
    Action<ModelBuilder>? model = null,
    bool cacheServiceProvider = false,
    Action<DbContextOptionsBuilder>? configure = null)
{
    var builder = new DbContextOptionsBuilder<TestContext>()
        .UseNpgsql("Host=localhost;Database=unused")       // keine Verbindung nötig
        .UseAcmeTenancy(tenancy)
        .EnableServiceProviderCaching(cacheServiceProvider);
    configure?.Invoke(builder);
    return new TestContext(builder.Options, model);
}

public static IReadOnlyList<string> MigrationSql(DbContext? source, DbContext target)
{
    var operations = target.GetService<IMigrationsModelDiffer>().GetDifferences(
        source?.GetService<IDesignTimeModel>().Model.GetRelationalModel(),
        target.GetService<IDesignTimeModel>().Model.GetRelationalModel());

    return target.GetService<IMigrationsSqlGenerator>()
        .Generate(operations, target.GetService<IDesignTimeModel>().Model)
        .Select(c => c.CommandText.Trim()).ToList();
}
```

> **Design-Time-Modell verwenden**
>
> Für Migrations-Tests immer das **Design-Time-Modell** verwenden (`IDesignTimeModel`). `context.Model` ist das schlanke Laufzeitmodell; Annotations, die der Annotation-Provider nur zur Design-Zeit weitergibt, fehlen darin (Fehlerfall in Kapitel 21).

> **Warum nicht SQLite oder InMemory?**
>
> Tests gegen SQLite oder den InMemory-Provider sind verlockend, weil sie kein Docker brauchen. Für ein Plugin beweisen sie wenig: InMemory ist nicht relational, Type Mappings, Translators und Migrationen laufen also nie; SQLite hat einen anderen SQL-Dialekt, andere Type Mappings und keine Row Level Security. Ein provider-spezifisches Plugin wird mit seinem eigenen Provider getestet: ohne Datenbank (nur Connection-String) für alles bis zum SQL-Text, gegen die echte Datenbank für das Verhalten. Für Anwendungen, die das Plugin nutzen, sieht es anders aus (nächster Abschnitt).

Testmodus für Anwendungen

Anwendungen haben oft hunderte Tests ihrer Geschäftslogik auf SQLite oder InMemory. Sie prüfen nicht die Isolation, sondern die eigene Logik, und sie dürfen nicht schon deshalb scheitern, weil die Entitäten `ITenantOwned` implementieren. Das PostgreSQL-Paket lehnt SQLite bewusst ab, und diese Prüfung abzuschalten hilft nicht: Das Paket ersetzt Provider-Dienste durch Npgsql-Ableitungen, bildet `TenantId` auf `uuid` ab und filtert mit `current_setting()`. Der Kern hat deshalb einen eigenen Testmodus, der nur die anwendungsseitige Hälfte des Plugins enthält:

`tests/.../TestingModeTests.cs (Auszug)`

```
private sealed class AppContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasTenantQueryFilters(this);
}

new DbContextOptionsBuilder<AppContext>()
    .UseSqlite(connection)
    .UseAcmeTenancyForTesting(Services(tenants))
    .Options

ctx.Orders.Add(new Order { Total = 1 });      // TenantId bewusst leer
await ctx.SaveChangesAsync();
```

`UseAcmeTenancyForTesting(sp)` bildet `TenantId` für jeden Provider als `Guid` ab (über einen Value-Converter-Selector, wie EF es für Enums macht), ergänzt den SaveChanges-Interceptor des Kerns und hinterlegt dafür die Mandanten-Property. `HasTenantQueryFilters(this)` filtert mit dem Mandanten als Abfrageparameter, derselbe Filter wie für Cosmos DB (Kapitel 48). Row Level Security, Migrations-SQL, Session und Translators fehlen. `TestingModeTests` betreibt den Modus auf SQLite und InMemory.

> **Ein Testmodus, kein Produktivmodus**
>
> `UseAcmeTenancyForTesting` lehnt PostgreSQL, SQL Server und Cosmos DB ab, damit er die echte Isolation nicht versehentlich ersetzt. Ob die Isolation hält, zeigen die Integrationstests des Plugins und die Integrationstests der Anwendung gegen die echte Datenbank (Testcontainers); der Testmodus hält nur die Tests der Geschäftslogik am Laufen.

Was getestet wird

Test	Sichert ab
Caching mit verschiedenen Optionen	Zwei Contexts mit verschiedenen Settings bekommen verschiedene Modelle (Hash-Fehler aus Kapitel 5).
Ungültige Setting-Namen	Validierung inklusive Injection-Versuch.
Type Mapping ohne Konfiguration	`TenantId` wird `uuid`; Literal-Format; anderer Spaltentyp wird nicht beansprucht.
`CurrentTenant()` per `ToQueryString`	Übersetzung und dass nichts clientseitig ausgewertet wird (Kapitel 15).
Explizit schlägt Convention	Eigene Policy gleichen Namens bleibt erhalten; Fluent API überstimmt das Attribut.
Warning-as-Error	Opt-out-Warnung wird mit `ConfigureWarnings` zum Fehler.
Validierung	RLS am abgeleiteten Typ und ungültige Policies werfen beim Modellbau.
Keine Phantom-Migration	Unverändertes Modell erzeugt keine einzige Operation.
Migrations-SQL	Initial, Änderung (DROP + CREATE), Opt-out, eigene Operationen.
SaveChanges-Logik	Setzen, fremder Mandant, Umhängen, kein Mandant.
Snapshot	Fluent API statt Annotations; Roundtrip ohne Unterschiede.
RLS in PostgreSQL	Isolation, keine Zeilen ohne Mandant, WITH CHECK greift.
Grenzfälle (Kapitel 30–35)	ExecuteUpdate und Raw SQL gegen RLS, Value Generation, JSON-Spalten, Compiled Model, idempotentes Skript.
Kombinierbarkeit	Decorator um den Migrations-Generator und die Kompositionsmatrix zweier Plugins (Kapitel 6), EFCore.NamingConventions in beiden Reihenfolgen (Kapitel 10).
Annotation-Format	Serialisierte Policies bleiben identisch; tolerantes Lesen (Kapitel 27).
Context-Lebensdauern	DbContext-Factory, Hintergrundjobs, offene Verbindungen (Kapitel 18, 40).
Mandantenübergreifender Zugriff	Reporting-Rolle liest alle Mandanten, kann aber nicht schreiben; die App-Rolle kann RLS nicht abschalten (Kapitel 36).

Der Snapshot-Roundtrip

Der wertvollste Test für Design-Time-Code: Snapshot generieren, mit Roslyn kompilieren, Modell daraus bauen und gegen das Original diffen. Jede Abweichung zwischen Code-Generator und Fluent API würde sonst bei jedem Nutzer eine Phantom-Migration erzeugen:

tests/.../SnapshotTests.cs (Auszug)

```
[Fact]
public void Snapshot_roundtrip_produces_no_differences()
{
    using var ctx = CreateContext();
    var snapshot = Compile(GenerateSnapshot(ctx));

    // Wie im EF-Migrator: finalisieren und initialisieren
    var model = snapshot.Model;
    if (model is IMutableModel mutable) model = mutable.FinalizeModel();
```

```
    model = ctx.GetService<IModelRuntimeInitializer>()
        .Initialize(model, designTime: true, validationLogger: null);

    var differences = ctx.GetService<IMigrationsModelDiffer>().GetDifferences(
        model.GetRelationalModel(), ctx.DesignModel().GetRelationalModel());

    Assert.Empty(differences);
}
```

Integrationstests mit Testcontainers

Das echte Sicherheitsverhalten lässt sich nur gegen PostgreSQL prüfen. Die Fixture startet einen Container, legt das Schema mit `EnsureCreated` an (inklusive RLS, Kapitel 20), führt die eigenen Migrations-Operationen real aus und verbindet die Tests mit einer eingeschränkten Rolle:

tests/...IntegrationTests/PostgresFixture.cs (Auszug)

```
await owner.Database.EnsureCreatedAsync();          // RLS, Policies, Index

var commands = owner.GetService<IMigrationsSqlGenerator>().Generate(
[
    new CreateApplicationRoleOperation { Name = AppRole, CanLogin = true },
    new GrantTablePrivilegesOperation { Role = AppRole },
]);
await owner.GetService<IMigrationCommandExecutor>().ExecuteNonQueryAsync(
    commands, owner.GetService<IRelationalConnection>(),
    new MigrationExecutionState(), commitTransaction: true);
```

tests/...IntegrationTests/RowLevelSecurityTests.cs (Auszug, vereinfacht)

```
[Fact]
public async Task Database_rejects_rows_of_other_tenant()
{
    // Interceptor umgehen: direktes SQL. WITH CHECK der Policy muss greifen.
    var tenants = new MutableTenantProvider { CurrentTenantId = TenantId.New() };
    await using var db = _fixture.CreateAppContext(tenants);

    var ex = await Assert.ThrowsAsync<PostgresException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO \"Orders\" (\"TenantId\", \"Total\") VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

> **Tipp**
>
> Für Provider und sehr tiefe Plugins stellt EF Core die **Specification Tests** bereit (`Microsoft.EntityFrameworkCore.Relational.Specification.Tests`). Das ist dieselbe Testsuite, gegen die EF und die Provider selbst getestet werden. Das NetTopologySuite-Plugin im EF-Repository zeigt, wie ein Plugin sie nutzt.

27 Versionierung und Kompatibilität

Versionsabhängigkeit: **mittel**

Paketversion und unterstützte EF-Version

Das Beispiel versioniert Plugin und Handbuch gemeinsam: Die Paketversion entspricht der Buchausgabe, Pakete 1.1.0 gehören zu Ausgabe 1.1. So gehört jeder Codeauszug zu genau einem Paketstand. Welche EF-Version ein Stand unterstützt, steht deshalb nicht in der Versionsnummer, sondern im Abhängigkeitsbereich der Pakete und in der Kompatibilitätstabelle der README: Plugin 1.x unterstützt EF Core 10. Der Wechsel auf eine neue EF-Major-Version ist für Anwender ein Breaking Change und bekommt eine neue Major-Version, 2.0 für EF Core 11, zusammen mit einer neuen Buchausgabe. Viele Plugins koppeln die Major-Version stattdessen an EF Core (10.x für EF Core 10); das macht die passende Kombination an der Nummer sichtbar. Beide Wege funktionieren, solange die Abhängigkeiten auf eine EF-Major-Version begrenzt sind. Eine Eigenschaft hält den Bereich an einer Stelle, damit die CI ihn für den Patch-Kanal überschreiben kann (Abschnitt „Patches und Previews in der CI“):

`Directory.Packages.props (Auszug)`

```
<PropertyGroup>
  <EFCoreVersion Condition="'$(EFCoreVersion)' == ''">[10.0.0,11.0.0)</EFCoreVersion>
  <CentralPackageFloatingVersionsEnabled>true</CentralPackageFloatingVersionsEnabled>
</PropertyGroup>
<PackageVersion Include="Microsoft.EntityFrameworkCore" Version="$(EFCoreVersion)" />
<PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="$(EFCoreVersion)" />
```

Umgang mit interner API

Plugins kommen um interne APIs selten herum: Das Beispiel erbt von `NpgsqlMigrationsSqlGenerator`, `NpgsqlAnnotationProvider` und `NpgsqlAnnotationCodeGenerator` und nutzt `NpgsqlOptionsExtension`; insgesamt berührt das PostgreSQL-Paket 13 interne Typen, das SQL-Server-Paket 2 und der Kern keinen (Inventar im Upgrade-Playbook). Solche Typen können sich auch in Minor-Versionen ändern. EF meldet ihre Verwendung mit der Warnung **EF1001**.

Maßnahme	Wirkung
Öffentliche Alternative suchen	Jede vermiedene interne API ist ein Risiko weniger.
Interne Zugriffe bündeln	Wenige Klassen kapseln sie; ein Bruch betrifft eine Stelle.
`<NoWarn>EF1001</NoWarn>` bewusst setzen	Projektweit und dokumentiert, nicht pro Zeile verstreut.
Versionsbereich begrenzen	Verhindert, dass Nutzer ungetestete Kombinationen bekommen.
CI gegen neueste Patches und Previews	Zeigt Brüche, bevor Anwender sie melden; die Kanäle beschreibt der Abschnitt „Patches und Previews in der CI“.
Roundtrip- und SQL-Tests	Bemerken Verhaltensänderungen, die der Compiler nicht sieht.

> **Multi-Targeting**
>
> Mehrere EF-Major-Versionen gleichzeitig zu unterstützen ist per Multi-Targeting möglich, verdoppelt aber Testaufwand und bedingte Kompilierung. Für die meisten Plugins ist ein Release-Zweig pro EF-Major-Version einfacher.

Annotation-Formate weiterentwickeln

Alles, was ein Plugin in Annotations schreibt, landet in den Model-Snapshots seiner Anwender, oft lange vor dem nächsten Upgrade. Serialisiert eine neue Plugin-Version dieselbe Konfiguration anders (eine zusätzliche JSON-Property, eine andere Reihenfolge, eine andere Enum-Schreibweise), vergleicht der Differ zwei verschiedene Strings. Jeder Anwender bekommt dann eine Migration, die alle Policies ohne echte Änderung löscht und neu anlegt. Das serialisierte Format gehört deshalb genauso zur öffentlichen API wie die Methoden.

Regel	Wie
Format per Test festschreiben	Ein Golden-Value-Test vergleicht die serialisierte Form Zeichen für Zeichen.
Tolerant lesen	Fehlende Properties werden zu Defaults, unbekannte werden ignoriert (das Standardverhalten von System.Text.Json).
Nur ergänzen, Defaults nicht schreiben	Neue optionale Properties mit `[JsonIgnore(Condition = WhenWritingDefault)]` lassen bestehende Annotations unverändert.
Echte Formatänderungen nur mit Major-Version	Dokumentieren, am besten zusammen mit dem EF-Major-Upgrade, wenn Anwender ohnehin mit Migrationen rechnen.

tests/.../AnnotationFormatTests.cs (Auszug)

```
[Fact]
public void Serialized_policy_format_is_stable()
{
    var policy = new RlsPolicy("support_read", PolicyCommand.Select, Permissive: false,
        Roles: ["shop_support"], Using: "true", WithCheck: null);

    // Golden Value: eine Änderung bricht bestehende Snapshots
    Assert.Equal(
        """{"Name":"support_read","Command":"Select","Permissive":false"""
        + ""","Roles":["shop_support"],"Using":"true","WithCheck":null}""",
        policy.Serialize());
}
```

Zwei weitere Tests prüfen das tolerante Lesen in beide Richtungen. Der Snapshot-Roundtrip aus Kapitel 26 ergänzt sie: Der Roundtrip zeigt, dass der aktuelle Code-Generator und die Fluent API übereinstimmen, der Golden Value zeigt, dass die nächste Version noch zu den Snapshots passt, die Anwender bereits haben.

Fehlerklassen beim Upgrade

Ein Upgrade auf eine neue EF-Major-Version bricht ein Plugin selten nur an einer Stelle, und nicht jeder Bruch meldet sich. Wer die Fehlerklassen kennt, weiß, welcher Test sie findet, und vor allem, welche er nicht findet:

Fehlerklasse	Typisches Beispiel	Im Beispiel erkannt durch
Compile-Time Break	Konstruktor einer Provider-Klasse hat einen neuen Parameter, ein interner Typ ist umgezogen	Build; `PublicAPI`-Dateien für die eigene API (Kapitel 41)
Runtime Break	Ein Dienst wechselt die Lebensdauer, eine Registrierung schlägt fehl	`ServiceRegistrationTests`, `DesignTimeServicesTests`, `OptionsTests`, `CompositionTests`
Silent Behavioral Change	Eine Reihenfolge, ein Vorrang oder eine Nullability-Regel von EF ändert sich	`FailureCaseTests`, `CompositionMatrixTests`, `StateAndLifetimeTests`, `SaveChangesInterceptorTests`; mit Datenbank `MaterializationTests`, `ConnectionLifetimeTests`
SQL Change	Der Provider schreibt einen Ausdruck anders oder optimiert eine Bedingung weg	`QueryTranslationTests`, `DeepQueryTests`, `DebuggingTests`
Model Change	Eine Convention läuft früher, eine Standardkonfiguration ändert sich	`ConventionTests`, `TenantPropertyTests`, `ValidationTests`, `TypeMappingTests`, `JsonTests`, `ValueGenerationTests`, `NamingConventionTests`
Snapshot/Migration Change	Der Code-Generator oder der Differ verhält sich anders; jeder Anwender bekäme eine Phantom-Migration	`SnapshotTests` (Roundtrip), `MigrationSqlTests`, `MigrationScriptTests`, `AnnotationFormatTests`, `CompiledModelTests`, Verbrauchertest
Performance Regression	Ein heißer Pfad allokiert mehr oder wird langsamer	Das Benchmark-Projekt (Kapitel 39)

Die dritte Zeile ist die gefährlichste. Viele Tests des Beispiels prüfen nicht das Plugin, sondern das Verhalten von EF, auf das sich das Plugin und dieses Buch verlassen. Sie sind beim Upgrade wertvoller als jeder andere Test, weil sie genau das melden, was kein Compiler sieht.

Das Upgrade-Playbook

Der Ablauf ist am Wechsel von EF Core 10 auf EF Core 11 beschrieben, gilt aber für jede Major-Version: Die Nummern sind austauschbar, die Reihenfolge nicht. Vorher wird die bisherige Linie in einem Release-Zweig eingefroren; sie bekommt weiter Patches für ihre EF-Version, während `main` auf die neue wechselt.

Schritt 1: Zweig und Versionen

```
git switch -c release/1.x main && git push -u origin release/1.x
git switch main

global.json               "version": "11.0.100"
Directory.Build.props     <TargetFramework>net11.0</TargetFramework>
                          <VersionPrefix>2.0.0</VersionPrefix>
Directory.Packages.props  <EFCoreVersion ...>[11.0.0,12.0.0)</EFCoreVersion>
```

Welches Target-Framework eine EF-Version voraussetzt, folgt keiner festen Regel: EF Core 9 zielt auf `net8.0`, EF Core 10 auf `net10.0`, die Release Candidates von EF Core 11 auf `net11.0` (EF Core releases and planning). Vor dem Upgrade gehört deshalb ein Blick in die Pakete von EF *und* Provider: Welche Frameworks bieten sie an, welches SDK brauchen sie? Im Beispiel verlangt EF Core 11 `net11.`

0, also wechseln Target-Framework, SDK und Pakete gemeinsam; ein neues EF-Paket auf dem alten Target-Framework ließe sich nicht einmal wiederherstellen.

Nr.	Schritt	Vorgehen	Im Beispiel
1	Pakete aktualisieren	Wie oben, dazu Begleitpakete, die mit EF erscheinen (`Microsoft.Extensions.*`); fest gepinnte Begleitpakete blockieren schon den Restore	`EFCoreVersion`
2	Compilerfehler klassifizieren	Nach Fehlercode zählen: verschobene Typen (CS0234, CS0246), geänderte Konstruktoren und Dependencies (CS1729, CS7036), entfallene Überschreibungen (CS0115)	Befehl unten
3	Interne API untersuchen	Das EF1001-Inventar (unten) gegen die Quellen beider Versionen vergleichen; jede Stelle einzeln, auch die, die noch kompilieren; mit den volatilen Kapiteln beginnen (Seite 116)	Kern: 0, PostgreSQL: 10, SQL Server: 2 interne Typen (Funktionen-Beispiel: 5)
4	Registrierungen und Lebensdauern vergleichen	EFs Service-Tabellen beider Versionen vergleichen; die eigenen `Replace`-Aufrufe legen Lebensdauern fest	`Service Registration Tests`
5	Abgeleitete Provider-Klassen prüfen	Der Compiler meldet entfallene Methoden, aber nicht neue virtuelle Überladungen, die die Basis jetzt stattdessen aufruft: Basisklasse im Diff lesen	`MigrationSql Tests`, `DeepQuery Tests`
6	Modell und Conventions prüfen	Modell-Tests; zusätzlich `ToDebugString` des Design-Time-Modells beider Versionen vergleichen (Anhang D)	`ConventionTests` u. a.
7	Annotation-Formate prüfen	Das eigene Format darf sich nicht ändern; EFs eigene Annotations, die das Plugin liest, können es	`Annotation FormatTests`
8	Query-SQL testen	Alle `ToQueryString`-Tests; jede Abweichung erklären, nicht nur die Erwartung anpassen	`Query Translation Tests` u. a.
9	Migrations-SQL vergleichen	Generiertes SQL der Tests; bei Beispiel-anwendungen das Skript von `dotnet ef migrations script` vor und nach dem Upgrade vergleichen	`MigrationSql Tests`, `Migration ScriptTests`
10	Snapshot-Roundtrip	Snapshot erzeugen, kompilieren, vergleichen; eine neue Migration auf einem bestehenden Projekt muss leer sein	`SnapshotTests`
11	Compiled Models	Generieren, kompilieren, mit `UseModel` laden, auch die Laufzeit-Annotations	`CompiledModel Tests`, `run.sh` (Seite 104)
12	Trimming und NativeAOT	Neue IL-Warnungen (IL2xxx, IL3xxx) im Build lesen und eine NativeAOT-Anwendung starten; die experimentelle API für vorkompilierte Abfragen (EF9100) ändert sich besonders oft	`IsAot Compatible`, `run.sh --nativeaot`

Nr.	Schritt	Vorgehen	Im Beispiel
13	Integrationstests	Gegen echte Datenbanken, mit den Provider-Versionen der neuen Linie	Integrations- und SQL-Server-Tests
14	Eigene öffentliche API vergleichen	Der Public-API-Analyzer bricht den Build bei jeder Abweichung; gewollte Brüche bewusst eintragen	`PublicAPI.*.txt`
15	Benchmarks vergleichen	Dasselbe Benchmark auf beiden Linien; Verhältnis und Allokationen vergleichen, nicht absolute Zeiten	Benchmark-Projekt
16	Breaking Changes dokumentieren	CHANGELOG, Kompatibilitätstabelle (Plugin 2.x zu EF Core 11), Hinweise für Anwender, Supportdauer der alten Linie	`CHANGELOG.md`

Für die Schritte 2 und 3 genügen zwei Befehle. Der zweite hebt die projektweite Unterdrückung von EF1001 nur für diesen Build auf und zeigt so jede Stelle, an der das Plugin interne API berührt:

Schritte 2 und 3

```
# Compilerfehler nach Code
dotnet build -c Release 2>&1 | grep -oE "error [A-Z]+[0-9]+" | sort | uniq -c | sort -rn

# Inventar interner API: NoWarn ohne EF1001 (EF9100 bleibt unterdrückt)
dotnet build -c Release --no-incremental -p:NoWarn=EF9100 2>&1 \
  | grep -oE "EF1001: [A-Za-z.]+" | sort | uniq -c | sort -rn
```

Wie stark ein Kapitel von der EF-Version abhängt, steht an seinem Anfang. Bei einem Major-Upgrade prüft man die **volatilen** Kapitel zuerst: Ihr Code leitet von Klassen aus `.Internal`-Namespaces ab oder nutzt sie direkt (EF1001), und Brüche dort zeigen sich oft erst zur Laufzeit. Danach folgen die mittleren, die stabilen zuletzt.

Einstufung	Bedeutung	Kapitel
volatil	Stark von `.Internal`-Namespaces abhängig (EF1001); bei jedem Major-Upgrade zuerst prüfen	21, 23, 42, 44, 45
mittel	Provider-Basisklassen oder öffentliche, aber oft geänderte API	2, 5, 6, 12, 13, 14, 16, 22, 24, 25, 26, 27, 30, 32, 33, 43, 47, 48
stabil	Konzepte und öffentliche API	alle übrigen

> **Abweichungen erklären, nicht wegtesten**
>
> Schlägt nach dem Upgrade ein SQL- oder Verhaltenstest fehl, ist die Versuchung groß, nur die Erwartung anzupassen. Jede Abweichung gehört aber erst erklärt: Ist sie eine Verbesserung von EF, ein Bruch des Plugins oder eine geänderte Semantik, die Anwender betrifft? Erst die Antwort entscheidet, ob der Test, der Code oder die Dokumentation geändert wird.

Patches und Previews in der CI

Die Versionsbereiche lösen nur einen Teil des Problems. NuGet nimmt aus einem Bereich die *niedrigste* Version, die alle Abhängigkeiten erfüllt; die reguläre CI testet also gegen den Anfang der

Linie, während Anwender neuere Patches bekommen. Die CI des Beispiels arbeitet deshalb mit drei Kanälen:

Kanal	Auslöser und Pakete	Zweck	Bei Fehler
Regulär (`ci.yml`)	Jeder Push und Pull Request; niedrigste Version des Bereichs	Die Linie, gegen die das Plugin gebaut wird	Pflicht-Check
Patches (`compatibility.yml`)	Wöchentlich und manuell; `-p:EFCoreVersion=10.0.*`, nur stabile Pakete	Neueste Patches der eigenen Linie	Untersuchen, Patch-Release
Nächste Major-Version	Eigener Job, zeitgesteuert; Previews und RCs	Brüche Monate vor dem Release sehen	Kein Pflicht-Check

Der Patch-Kanal ist im Beispiel aktiv. Beim ersten Lauf zeigte er bereits einen echten Befund: Das Plugin selbst lief mit EF Core 10.0.14, aber Test- und Beispielprojekte ließen sich nicht wiederherstellen, weil sie Begleitpakete fest auf 10.0.0 und eine eigene Roslyn-Version gepinnt hatten. Seitdem folgen diese Pakete `EFCoreVersion`.

Ein Kanal für die nächste Major-Version ist im Beispiel bewusst nicht aktiv: Er arbeitet mit Vorabversionen. Wer ihn einrichtet, hält vier Regeln ein, damit normale Releases nie an Preview-Pakete gekoppelt werden:

Regel	Warum
Alles über globale Eigenschaften dieses Jobs umstellen: SDK, Target-Framework und `EFCoreVersion`	Die Standardwerte in den Projektdateien bleiben stabil; ein Preview-Paket kann so nie in einen regulären Build gelangen
SDK, Target-Framework und Pakete gemeinsam wechseln	Mit dem alten Target-Framework scheitert schon der Restore; ein solcher Job kann nie grün werden
Nie packen oder veröffentlichen	Releases entstehen nur aus regulären Builds der Release-Zweige
Kein Pflicht-Check	Previews brechen regelmäßig; der Job ist eine Frühwarnung, die ein Issue erzeugt, kein Tor für Pull Requests

Job für die nächste EF-Version (Skizze, nicht im Repository)

```
ef-next:
  runs-on: ubuntu-latest
  continue-on-error: true                # Frühwarnung, kein Pflicht-Check
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-dotnet@v4
      with: { dotnet-version: 11.0.x, dotnet-quality: preview }
    - run: dotnet new globaljson --sdk-version "$(dotnet --version)" --force
    - name: Build against the next EF Core version (never packed or published)
      run: dotnet build -c Release -p:TargetFramework=net11.0 -p:EFCoreVersion=11.0.0-*
    - run: >
        dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
        -p:TargetFramework=net11.0
```

Test-Abhängigkeiten von Drittanbietern wie EFCore.NamingConventions erscheinen oft erst später für die neue Version und können den Restore dieses Jobs blockieren. Solche Tests nimmt man im Preview-Kanal gezielt aus, statt den Kanal abzuschalten.

NuGet-Advisories und transitive Abhängigkeiten

Ein Plugin liefert nicht nur eigenen Code aus, sondern auch seinen Abhängigkeitsgraphen. Das NuGet-Audit meldet bekannte Schwachstellen als Warnungen NU1901 bis NU1904 (NuGet: Auditing package dependencies). Entscheidend ist, *wen* ein Befund trifft. Ermittelt wird das mit `dotnet nuget why`, das den Pfad von der direkten Abhängigkeit bis zum betroffenen Paket zeigt. Im Beispiel gab es drei Fälle:

Paket	Herkunft	Betrifft	Entscheidung
`System.Security.Cryptography.Pkcs` 9.0.4	EF Core SqlServer → `Microsoft.Data.SqlClient` 6.1	Verbraucher des SQL-Server-Pakets	Kein Patch und keine SqlClient-Version (auch nicht 7.x) bringt bisher 9.0.21. Deshalb eine direkte Abhängigkeit auf 9.0.22 im SQL-Server-Paket; sie landet im nuspec und korrigiert den Graphen der Anwendung.
`Cryptography.Xml`/`Pkcs` 9.0.0	EF Core Design 10.0.0 → MSBuild 17.14, Workspaces.MSBuild 4.14	Nur Werkzeuge: Tests, Beispiel	Neuere Design-Patches brauchen andere MSBuild-Pakete, würden aber EF selbst mitziehen und den Test der Mindestversion aufheben. Deshalb `Xml` 9.0.22 nur in Test- und Beispielprojekten.
`SSH.NET` 2024.2.0	Testcontainers 4.6	Nur Tests	Testcontainers 4.16 innerhalb derselben Major-Version; bringt 2026.0.0.

Jede direkte Abhängigkeit auf ein eigentlich transitives Paket steht in `Directory.Packages.props` mit Herkunft, Advisory und dem Kriterium, wann sie wieder entfallen kann. Unterdrückt wird nichts: Weder `NoWarn` noch ein abgeschaltetes Audit machen eine Abhängigkeit sicher, sie verstecken nur den Befund. Beide CI-Kanäle führen zusätzlich `dotnet list package --vulnerable --include-transitive` aus und scheitern an jedem Fund. Der reguläre Lauf baut weiterhin gegen EF Core 10.0.0, der Kompatibilitätslauf gegen den neuesten 10.0.x-Patch; die Korrekturen sind Mindestversionen und gelten in beiden. Für Werkzeuge gibt es eine Wechselwirkung: `Microsoft.EntityFrameworkCore.Tasks` wertet die Audit-Warnungen beim Veröffentlichen als Fehler, sodass NativeAOT mit EF Design 10.0.0 scheitert (Seite 104).

Checkliste für das Upgrade

- ☐ Release-Zweig für die alte Linie anlegen; `main` auf die neue Version heben.
- ☐ SDK, Target-Framework, `EFCoreVersion`, Begleitpakete und `VersionPrefix` gemeinsam ändern.
- ☐ Compilerfehler nach Code zählen und klassenweise abarbeiten.
- ☐ EF1001-Inventar erstellen und jede Stelle gegen die neuen Quellen prüfen, auch die kompilierenden.
- ☐ Lebensdauern gegen EFs Service-Tabelle prüfen; abgeleitete Provider-Klassen auf neue virtuelle Überladungen lesen.
- ☐ Modell-, SQL-, Migrations-, Snapshot- und Compiled-Model-Tests grün; jede Abweichung erklärt.

☐ Annotation-Formate unverändert (Golden-Value-Tests).

☐ Neue IL-Warnungen für Trimming und AOT gelesen.

☐ Integrationstests gegen echte Datenbanken mit den neuen Providern.

☐ Public-API-Diff geprüft; gewollte Brüche eingetragen.

☐ Benchmarks beider Linien verglichen.

☐ CHANGELOG, Kompatibilitätstabelle und Supportdauer der alten Linie dokumentiert.

28 Packaging und Dokumentation

Versionsabhängigkeit: **stabil**

Gemeinsame Einstellungen liegen in `Directory.Build.props`, Paketversionen zentral in `Directory.Packages.props` (Central Package Management):

`Directory.Build.props (gekürzt)`

```
<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  <NoWarn>$(NoWarn);EF1001</NoWarn>
  <Deterministic>true</Deterministic>
  <WarningsAsErrors>$(WarningsAsErrors);RS0016;RS0017</WarningsAsErrors>
</PropertyGroup>
<PropertyGroup>
  <PackageLicenseExpression>MIT</PackageLicenseExpression>
  <PackageReadmeFile>README.md</PackageReadmeFile>
  <VersionPrefix>1.1.0</VersionPrefix>       <!-- = Buchausgabe -->
  <IncludeSymbols>true</IncludeSymbols>
  <SymbolPackageFormat>snupkg</SymbolPackageFormat>
  <PublishRepositoryUrl>true</PublishRepositoryUrl>      <!-- SourceLink -->
  <EmbedUntrackedSources>true</EmbedUntrackedSources>
</PropertyGroup>
```

Element	Warum
`GenerateDocumentationFile`	XML-Doku im Paket: Nutzer sehen Erklärungen in der IDE.
`WarningsAsErrors` RS0016, RS0017	Eine geänderte öffentliche API bricht den Build (Kapitel 41).
`IsAotCompatible`	Analyzer melden AOT- und Trimming-Probleme beim Build.
SourceLink, `.snupkg`	Nutzer können in den Plugin-Code debuggen. Seit dem .NET-8-SDK eingebaut.
README im Paket	Erscheint auf nuget.org; Minimalbeispiel und Kompatibilität.
CHANGELOG	Breaking Changes klar markieren, besonders bei EF-Major-Wechseln.
`InternalsVisibleTo`	Tests erreichen interne Logik ohne öffentliche API.
Paketnamen	Kern ohne Provider, Provider-Paket mit Suffix (`.PostgreSQL`, `.SqlServer`).

Zur Dokumentation gehören neben der API-Doku vor allem die Dinge, die Nutzer nicht im Code sehen: welche Einzeldienste ersetzt werden (Kombinierbarkeit), welche Event-IDs es gibt, welche Einschränkungen bestehen (Pooling) und welche Sicherheitsannahmen gelten (Datenbankrollen).

29 Die Checkliste von A bis Z

Versionsabhängigkeit: **stabil**

Für jedes neue Plugin, in der Reihenfolge der Entwicklung. Jeder Punkt fasst eine Regel zusammen; die Begründung steht in den Kapiteln, die unter jeder Überschrift genannt sind.

Architektur

Details: Kapitel 3, 4.

- ☐ Stufe festlegen: Hilfsmethoden, Modell-Helfer, echtes Plugin oder Provider?
- ☐ Kern- und Provider-Paket trennen; Typen für Entitäts-Assemblies in den Kern.
- ☐ Für jeden Erweiterungspunkt zuerst ein Plugin-Interface suchen, erst dann ersetzen.
- ☐ Jeden Mechanismus mit der Entscheidungshilfe (Kapitel 4) gegen die einfachere Alternative prüfen.

Options

Details: Kapitel 5.

- ☐ Extension immutable mit Kopierkonstruktor und virtuellem Clone.
- ☐ Alle modellrelevanten Optionen im Hash, nichts, was pro Context variiert.
- ☐ Validate für Voraussetzungen (Provider, Kombinationen).
- ☐ LogFragment und PopulateDebugInfo pflegen.
- ☐ Use-Methode im Namespace Microsoft.EntityFrameworkCore, generisch und nicht-generisch.

Dienste

Details: Kapitel 6, 8.

- ☐ Registrierung über den ServicesBuilder (Lebensdauern aus EFs Tabelle).
- ☐ Einzeldienste dekorieren, wo das öffentliche Interface genügt (Kapitel 6).
- ☐ Sonst Replace, und die Logik öffentlich und virtuell anbieten.
- ☐ ISingletonOptions für Singleton-Dienste, doppelt registriert.
- ☐ Jeden Zustand nach Kapitel 8 einordnen: Anfragedaten nie in Options, Hash, Modell, Singleton oder übersetzte Abfrage.
- ☐ Scoped-Dienste mit Zustand für Pooling zurücksetzen (IResettableService).
- ☐ App-Abhängigkeiten über AddInterceptors, Pooling-Verhalten dokumentieren.

Modell

Details: Kapitel 9, 10, 11.

- ☐ Annotations mit Präfix, nur einfache Werte, keine Defaults speichern.
- ☐ Fluent API nicht-generisch implementieren, generisch delegieren.
- ☐ Convention-Builder-Überladungen für ConfigurationSource.
- ☐ Conventions bei ModelFinalizing, wenn sie vom fertigen Modell abhängen.
- ☐ Attribute im Kernpaket, Attribut-Conventions im Provider-Paket.
- ☐ Validierung als IModelFinalizedConvention: Fehler werfen, Fragwürdiges warnen.
- ☐ Tabellenabbildungen gegen die Unterstützungsmatrix prüfen und nicht unterstützte beim Modellbau ablehnen (Seite 51).

Type Mappings

Details: Kapitel 12.

- ☐ Geschützter Parameter-Konstruktor und Clone-Überschreibung.
- ☐ Literal-Erzeugung für Modell- und Provider-Typ.
- ☐ Öffentliche statische Default-Property, gemeinsame Konverter-Instanz.
- ☐ Gemeinsame Value-Comparer-Instanzen, sonst erzeugt EF sie per Reflection (NativeAOT).
- ☐ JsonValueReaderWriter mit Instance und ConstructorExpression.

Abfragen

Details: Kapitel 13, 14.

- ☐ EF.Functions-Methoden im Kern, Übersetzung im Provider-Paket.
- ☐ Translators: null für fremde Methoden, Nullability korrekt, Type Mappings setzen.
- ☐ IEvaluatableExpressionFilterPlugin für Methoden ohne Datenbezug.
- ☐ Keine Laufzeitwerte zur Übersetzungszeit einsetzen (Query-Cache, AOT).

Laufzeit

Details: Kapitel 18, 19.

- ☐ Interceptors synchron und asynchron implementieren.
- ☐ Tracking-Markierung von tatsächlicher Wertänderung unterscheiden (Seite 84).
- ☐ Singleton-Interceptors zustandslos und thread-sicher.
- ☐ Singleton-Interceptors in ApplyServices registrieren oder als eine geteilte Instanz übergeben, nie pro Context neu; Anfragedaten pro Aufruf über den Context holen.
- ☐ Eigene Event-IDs in eigenem Bereich, EventDefinitions gecacht.
- ☐ Sicherheitsrelevantes als Warnung, eskalierbar per ConfigureWarnings.
- ☐ Wo Hooks nicht laufen, eine explizite API anbieten (z. B. ApplyTenantSession).

Migrations & Design-Time

Details: Kapitel 20, 23.

- ☐ Deklarativ über Annotation-Provider (nur Design-Time) und SQL-Generator.
- ☐ Bezeichner maskieren, Literale über Type Mappings, selbst erzeugtes SQL validieren.
- ☐ Eigene Operationen idempotent, Extensions im Migrations-Namespace.
- ☐ Schemagebundene Objekte bei Umbenennungen vor der Änderung entfernen und danach mit den endgültigen Namen neu anlegen (Seite 182).
- ☐ DesignTimeServicesReference mit forProvider; Code-Generator entfernt Annotations.
- ☐ Compiled Models prüfen: Default, Instance, Laufzeit-Annotations.

Qualität

Details: Kapitel 26, 27, 28.

- ☐ Modell-, SQL-, Snapshot-Roundtrip- und Integrationstests.
- ☐ Caching-Test mit verschiedenen Optionen; Test auf keine Phantom-Migration.
- ☐ Versionsbereich je EF-Major, CI gegen neueste Patches und Previews, EF1001 bewusst; Upgrades nach dem Playbook (Kapitel 27).
- ☐ XML-Doku, SourceLink, Symbole, README, CHANGELOG, Einschränkungen dokumentiert.
- ☐ Die Pakete als Verbraucher prüfen: Tooling, Compiled Model, NativeAOT (Seite 104).
- ☐ Serialisierte Annotation-Formate mit Golden-Value-Tests festschreiben.
- ☐ Zusammen mit verbreiteten Plugins testen (Naming Conventions, Decorators) und die Koexistenz mit einem zweiten Plugin in beiden Reihenfolgen prüfen (Kapitel 6).

Fortgeschrittene Erweiterungspunkte (Teil VIII)

Details: Kapitel 30, 31, 35.

- ☐ Wege dokumentieren, die Interceptors umgehen (ExecuteUpdate, rohes SQL).
- ☐ Value Generators mit HasValueGeneratorFactory(Type) konfigurieren (Compiled Models).
- ☐ Conventions für Complex Types und JSON-Spalten prüfen.
- ☐ Reihenfolge eigener Migrations-Operationen dokumentieren und testen.
- ☐ Idempotente Skripte und suppressTransaction berücksichtigen.

Betrieb und Integration (Teil IX)

Details: Kapitel 36, 37, 38.

- ☐ Betriebsannahmen dokumentieren (Rollen, Pooling, Transaction-Pooling).
- ☐ Context-Lebensdauern dokumentieren: Factories, Hintergrundjobs, offene Verbindungen.
- ☐ Mandantenübergreifender Zugriff nur über eigene Rollen, nie über App-Schalter.
- ☐ Bedrohungsmodell dokumentieren: was geschützt ist und was nicht (Seite 139).
- ☐ Beziehungen mandantensicher: zusammengesetzte Fremdschlüssel, eindeutig pro Mandant.
- ☐ Bestehende Tabellen in zwei Migrationen mit Wächter nachrüsten.
- ☐ Heiße Pfade (Interceptors) messen, nicht vermuten.
- ☐ Second-Level-Caches: der Mandant in jedem Cache-Schlüssel, ohne Mandant kein Cache.
- ☐ CI mit Patch-Kanal, Preview-Kanal nur als Frühwarnung, Public-API-Analyzer als Fehler.

Tiefe Erweiterungen (Teil X)

Details: Kapitel 42, 45.

- ☐ Eigene SQL-Knoten: VisitChildren, Update, Quote, Print, Equals/GetHashCode.
- ☐ Eigene Knoten im SQL-Generator UND im Nullability-Prozessor behandeln.
- ☐ Aggregate: Prädikat, DISTINCT und leere Mengen bewusst behandeln.
- ☐ Laufzeit-Annotations bewusst wählen, Design-Time-Annotations filtern.
- ☐ Scaffolding: Schema-Objekte des Plugins aus der Datenbank lesen.

Mehrere Provider (Teil XI)

Details: Kapitel 46, 47, 48.

- ☐ Provider-neutrale Bausteine in den Kern, Unterschiede ins Provider-Paket.
- ☐ Relationale Bausteine in eine eigene Schicht, damit der Kern auch ohne SQL nutzbar bleibt.
- ☐ Ohne Row Level Security: dokumentieren, dass der Query-Filter die einzige Isolation ist und wie roher Zugriff ihn umgeht.

Teil VIII

Fortgeschrittene Erweiterungspunkte

Grenzen der Laufzeit-Haken und weitere Erweiterungspunkte: Massenoperationen, Value Generation, Update-Pipeline, Complex Types, Modelle pro Kontext und Migrations für Fortgeschrittene.

› Kapitel 30: Massenoperationen und die Grenzen von Interceptors

› Kapitel 31: Value Generation

› Kapitel 32: Die Update-Pipeline

› Kapitel 33: Complex Types und JSON

› Kapitel 34: Modelle pro Laufzeitkontext

› Kapitel 35: Migrations für Fortgeschrittene

30 Massenoperationen und die Grenzen von Interceptors

Versionsabhängigkeit: **mittel**

Ein SaveChanges-Interceptor sieht nur, was über `SaveChanges` läuft. `ExecuteUpdate` und `ExecuteDelete` schreiben direkt in die Datenbank, ohne Change Tracker und ohne SaveChanges. Dasselbe gilt für rohes SQL. Ein Plugin, das seine Garantien nur über SaveChanges durchsetzt, hat hier eine Lücke:

Weg	SaveChanges-Interceptor	Connection-Interceptor	RLS
`SaveChanges`	ja	ja	ja
`ExecuteUpdate`, `ExecuteDelete`	**nein**	ja	ja
`FromSql`, `SqlQuery`, `ExecuteSql`	**nein**	ja	ja
ADO.NET oder psql als App-Rolle	**nein**	**nein**	ja (ohne Mandant: keine Zeilen)

Im Beispiel fängt RLS jeden dieser Wege ab, weil die Datenbank die Regeln selbst durchsetzt. Ein Integrationstest beweist das, indem er den Interceptor gezielt umgeht:

`tests/...IntegrationTests/AdvancedTests.cs (Auszug)`

```
[Fact]
public async Task ExecuteUpdate_bypasses_interceptor_but_not_rls()
{
    // ... Bestellung für "tenant" anlegen ...
    await using var attacker = ContextFor(tenant);

    // Kein SaveChanges -> der SaveChanges-Interceptor läuft NICHT.
    // WITH CHECK der Policy verhindert das Umhängen trotzdem.
    var ex = await Assert.ThrowsAsync<PostgresException>(() =>
        attacker.Orders.ExecuteUpdateAsync(
            s => s.SetProperty(o => o.TenantId, TenantId.New())));

    Assert.Equal(PostgresErrorCodes.InsufficientPrivilege, ex.SqlState);
}
```

Was ein Plugin dokumentieren muss

Plugins ohne Datenbank-Absicherung (Soft Delete, Audit-Felder, Mandantenfilter nur per Query-Filter) müssen dokumentieren, dass Massenoperationen und rohes SQL ihre Regeln umgehen. Wer das verhindern will, kann einen `IQueryExpressionInterceptor` einsetzen: `ExecuteUpdate` und `ExecuteDelete` laufen durch die Query-Pipeline, der Interceptor sieht ihren Ausdrucksbaum und kann etwa ein `SetProperty` auf geschützte Spalten ablehnen. Rohes SQL bleibt trotzdem offen.

Beispiel: die Mandantenspalte bei Massen-Updates schützen

Das Beispiel setzt genau diesen Interceptor um, als Option: `UseAcmeTenancy(t => t.ProtectBulkOperations())` für PostgreSQL, `sql.ProtectBulkOperations()` für SQL Server. EF Core 10 nimmt die Setter von `ExecuteUpdate` nicht mehr als Ausdrucksbaum, sondern als `Action<UpdateSettersBuilder<T>>` entgegen. Im Baum, den der Interceptor sieht, stehen sie danach als Array von Tupeln: `ExecuteUpdate(source, new[] { new Tuple(e => e.TenantId, wert), ... })`. Das erste Element jedes Tupels ist das Property-Lambda, und genau das prüft der Visitor:

Interceptors/TenantQueryExpressionInterceptor.cs (Kernpaket, gekürzt)

```
protected override Expression VisitMethodCall(MethodCallExpression node)
{
    if (node.Method.DeclaringType == typeof(EntityFrameworkQueryableExtensions)
        && node.Method.Name is nameof(EntityFrameworkQueryableExtensions.ExecuteUpdate)
            or nameof(EntityFrameworkQueryableExtensions.ExecuteUpdateAsync)
        && node.Arguments.Count > 1
        && node.Arguments[1] is NewArrayExpression setters)
    {
        foreach (var setter in setters.Expressions)
        {
            if (setter is NewExpression { Arguments: [var selector, ..] }
                && Unquote(selector) is LambdaExpression lambda)
                Check(lambda);
        }
    }

    return base.VisitMethodCall(node);
}

private void Check(LambdaExpression selector)
{
    var parameter = selector.Parameters[0];
    var name = selector.Body switch
    {
        // e => e.TenantId
        MemberExpression member when member.Expression == parameter => member.Member.Name,
        // e => EF.Property<TenantId>(e, "TenantId"), auch für Shadow-Properties
        MethodCallExpression
        {
            Method.Name: nameof(EF.Property),
            Arguments: [var target, ConstantExpression { Value: string property }]
        } when target == parameter => property,
        _ => null,
    };
    // ...
}
```

Ist die gesetzte Property die Mandanten-Property laut Laufzeit-Annotation, wirft der Interceptor eine `TenancyException`, noch bevor eine Verbindung geöffnet wird. Er läuft einmal pro Abfrageform, vor Übersetzung und Query-Cache; spätere Ausführungen kosten nichts. Andere Spalten und gewöhnliche Abfragen passieren unverändert. RLS bleibt die eigentliche Grenze: Der Interceptor verhindert nur das Umhängen von Zeilen früh und mit klarer Meldung, `ExecuteDelete` und rohes SQL prüft er nicht.

Eine Falle steckt in der Registrierung. `IQueryExpressionInterceptor` erbt wie `IMaterializationInterceptor` von `ISingletonInterceptor`. Im internen Provider unter diesem Typ registriert, ruft

EF ihn aber nie auf: Query-Expression-Interceptoren holt EF aus den `IInterceptor`-Registrierungen und den Options, Materialisierungs-Interceptoren auch aus den `ISingletonInterceptor`-Registrierungen. Das Plugin registriert ihn deshalb als `IInterceptor`; zustandslos, wie er ist, schadet die Instanz pro Context nicht. `BulkOperationTests` hält beide Registrierungen fest, die Integrationstests zeigen ein erlaubtes und ein abgelehntes Update gegen PostgreSQL.

31 Value Generation

Versionsabhängigkeit: **stabil**

EF erzeugt Schlüsselwerte entweder in der Datenbank (Identity, Sequenzen) oder clientseitig über einen `ValueGenerator`. Für `Guid`-Schlüssel macht EF das von selbst, für eigene Typen wie `TenantId` nicht. Ein Plugin schließt diese Lücke mit einem Generator, einer Factory und einer Convention:

`ValueGeneration/TenantIdValueGenerator.cs (Kernpaket)`

```
public class TenantIdValueGenerator : ValueGenerator<TenantId>
{
    // Endgültige Werte, keine temporären Platzhalter bis zum Speichern
    public override bool GeneratesTemporaryValues => false;

    public override TenantId Next(EntityEntry entry) => TenantId.New();   // UUIDv7
}
```

`ValueGeneration/TenantIdValueGeneratorFactory.cs (Kernpaket)`

```
public class TenantIdValueGeneratorFactory : ValueGeneratorFactory
{
    // Zustandsloser Generator: EF darf ihn cachen und zwischen Contexts teilen
    public override ValueGenerator Create(IProperty property, ITypeBase typeBase)
        => new TenantIdValueGenerator();
}
```

`Conventions/TenantIdKeyValueGenerationConvention.cs (Kernpaket)`

```
public virtual void ProcessModelFinalizing(
    IConventionModelBuilder modelBuilder,
    IConventionContext<IConventionModelBuilder> context)
{
    foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
    {
        if (entityType.BaseType is not null
            || entityType.FindPrimaryKey() is not { Properties: [var property] }
            || property.ClrType != typeof(TenantId))
            continue;

        // Convention-Quelle: ein explizites ValueGeneratedNever() gewinnt (-> null)
        if (property.Builder.ValueGenerated(ValueGenerated.OnAdd) is null)
            continue;

        property.Builder.HasValueGeneratorFactory(typeof(TenantIdValueGeneratorFactory));
    }
}
```

> **Factory-Typ statt Generator**
>
> Einen Value Generator kann man auf mehreren Wegen konfigurieren: `HasValueGenerator` mit einem Typ oder einer Factory-Funktion, oder `HasValueGeneratorFactory` mit einem Factory-Typ. Der Generator für Compiled Models in EF Core 10 akzeptiert nur die letzte Variante und bricht sonst mit „The property … has a value generator configured. Use 'HasValueGeneratorFactory' to configure the value generator factory type.“ ab. Plugins sollten deshalb `HasValueGeneratorFactory(Type)` verwenden und der

> Factory einen öffentlichen parameterlosen Konstruktor geben.

Die Convention liegt im Kernpaket, weil sie nichts mit PostgreSQL zu tun hat; das Provider-Paket hängt sie über sein Convention-Set-Plugin ein. Der Compiled-Model-Test prüft, dass die Factory im erzeugten Code auftaucht.

32 Die Update-Pipeline

Versionsabhängigkeit: **mittel**

Kapitel 13 beschreibt den Weg von LINQ zu SQL. Für das Schreiben gibt es eine eigene Pipeline, die Plugins für Bulk-Operationen, datenbankseitige Soft-Deletes oder zurückgelieferte Werte erweitern:



Dienst	Lebensdauer	Art
`IModificationCommandFactory`	Singleton	einzeln
`ICommandBatchPreparer`	Scoped	einzeln
`IModificationCommandBatchFactory`	Scoped	einzeln
`IUpdateSqlGenerator`	Singleton	einzeln

Alle Dienste sind Einzeldienste des Providers, mit den Konsequenzen aus Kapitel 6. Bevor man sie ersetzt, lohnt die Prüfung der Alternativen: Ein SaveChanges-Interceptor kann Zustände umschreiben (aus Löschen wird Ändern beim Soft Delete), ein Command-Interceptor kann SQL ergänzen, und für Massenänderungen gibt es `ExecuteUpdate`. Ein eigener `IUpdateSqlGenerator` ist erst dann gerechtfertigt, wenn das SQL selbst eine andere Form braucht, etwa `INSERT ... ON CONFLICT` für Upserts.

33 Complex Types und JSON

Versionsabhängigkeit: **mittel**

Complex Types

Complex Types (seit EF Core 8) sind Werttypen ohne eigene Identität, deren Properties in der Tabelle des Besitzers landen. Für Plugins heißt das: Conventions, die über `entityType.GetProperties()` laufen, sehen deren Properties **nicht**. Sie hängen an `IComplexProperty` und müssen eigens durchlaufen werden:

Muster für Conventions (Skizze)

```
foreach (var complexProperty in entityType.GetComplexProperties())
{
    foreach (var property in complexProperty.ComplexType.GetProperties())
    {
        // Spaltennamen sind präfixiert (z. B. "Address_City"); immer über
        // property.GetColumnName(table) ermitteln, nie selbst zusammensetzen
    }
}
```

Fluent-API-Methoden für Properties sollten zusätzlich auf `ComplexTypePropertyBuilder` angeboten werden, wenn sie auch dort sinnvoll sind. Das Beispiel-Plugin braucht das nicht: Isolation und Wertgenerierung betreffen nur Entitäten.

JSON-Spalten

Owned Types und (seit EF Core 10) Complex Types lassen sich mit `ToJson()` als eine JSON-Spalte speichern. Werte darin liest und schreibt EF nicht über Konverter, sondern über den `JsonValueReaderWriter` des Type Mappings (Kapitel 12). Fehlt er, scheitert der Modellbau. Unit- und Integrationstest des Beispiels prüfen beide Seiten:

tests/.../JsonTests.cs

```
using var ctx = Contexts.Create(model: m =>
    m.Entity<Shipment>().OwnsOne(s => s.Details, d => d.ToJson()));

var property = ctx.DesignModel()
    .FindEntityType(typeof(ShipmentDetails))!
    .FindProperty(nameof(ShipmentDetails.ApprovedBy))!;

Assert.Same(TenantIdJsonReaderWriter.Instance,
    property.GetTypeMapping().JsonValueReaderWriter);
```

Davon getrennt ist JSON außerhalb der Datenbank. System.Text.Json kennt den `JsonValueReaderWriter` von EF nicht und würde `TenantId` als Objekt mit `Value` und `IsEmpty` schreiben. Ein `[JsonConverter(typeof(TenantIdJsonConverter))]` am Typ im Kernpaket macht daraus eine schlichte GUID, auch als Dictionary-Schlüssel, ohne dass eine Web-API etwas registrieren muss; ungültige Werte führen zu einer `JsonException`. Beide Wege brauchen ihren eigenen Baustein, weil EF und System.Text.Json unabhängig voneinander serialisieren (`TenantIdJsonConverterTests`).

> **Hinweis**
>
> PostgreSQL-Arrays (`List<TenantId>` als `uuid[]`) sind kein JSON: Npgsql bildet primitive Collections nativ als Arrays ab. Ob ein Provider Collections mit konvertierten Elementtypen unterstützt, ist providerspezifisch und gehört in einen eigenen Test, bevor man es dokumentiert.

34 Modelle pro Laufzeitkontext

Versionsabhängigkeit: **stabil**

EF baut das Modell einmal pro Context-Typ und cached es (Kapitel 2). Soll das Modell von Laufzeitdaten abhängen, etwa einem Schema pro Mandant, muss der Cache-Schlüssel diese Daten enthalten. Dafür gibt es `IModelCacheKeyFactory`:

`tests/.../SchemaPerTenantTests.cs (gekürzt, vereinfacht)`

```
public sealed class SchemaModelCacheKeyFactory : IModelCacheKeyFactory
{
    public object Create(DbContext context, bool designTime)
        => context is ISchemaScoped scoped
            ? (context.GetType(), scoped.Schema, designTime)
            : (object)(context.GetType(), designTime);
}

public sealed class SchemaContext : DbContext, ISchemaScoped
{
    // Konstruktor: (DbContextOptions<SchemaContext> options, string schema)
    public string Schema { get; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
        => modelBuilder.HasDefaultSchema(Schema);
}

// Registrierung
options.UseNpgsql(connectionString)
       .ReplaceService<IModelCacheKeyFactory, SchemaModelCacheKeyFactory>();
```

Der Test `SchemaPerTenantTests` prüft beide Seiten des Cache-Schlüssels: Zwei Schemas ergeben zwei Modelle mit jeweils eigenem Standardschema, und dasselbe Schema verwendet das gecachte Modell wieder, statt es neu zu bauen.

Das ist die Grundlage für „Schema pro Mandant“, eine Alternative zu RLS. Die drei üblichen Ansätze im Vergleich:

Ansatz	Stärken	Schwächen
Gemeinsame Tabellen + RLS (Beispiel)	Ein Modell, ein Schema, einfache Migrationen; Trennung erzwingt die Datenbank	Alle Mandanten teilen Tabellen und Indizes; Fehlkonfiguration der Rollen hebelt die Trennung aus
Schema pro Mandant	Starke Trennung, getrennte Backups pro Schema möglich	Ein Modell pro Mandant im Cache (Speicher!), Migrationen für jedes Schema
Datenbank pro Mandant	Maximale Trennung, unabhängige Skalierung	Ein Modell genügt, aber Verbindungen, Migrationen und Betrieb pro Datenbank

> **Modell-Cache und Speicher**
>
> Jeder Schlüsselwert erzeugt ein eigenes Modell im Speicher. Bei Hunderten Mandanten mit großen Modellen wird das teuer. Außerdem dürfen Werte, die das Modell verändern, nie aus ungeprüften Eingaben stammen.

35 Migrations für Fortgeschrittene

Versionsabhängigkeit: **stabil**

Reihenfolge von Operationen

Generierte und handgeschriebene Operationen laufen in der Reihenfolge, in der sie in `Up()` stehen. Das ist eine echte Fehlerquelle. Die Policy `support_read` des Beispiels gilt `TO shop_support`. EF erzeugt sie direkt nach `CREATE TABLE`. Wird die Rolle erst am Ende von `Up()` angelegt, scheitert die Migration.

Migrations/20261021_Initial.cs

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // 1. Was Policies referenzieren: VOR den generierten Operationen
    migrationBuilder.CreateApplicationRole("shop_support");

    // 2. Von EF generiert: Tabellen, RLS, Policies (TO shop_support)
    migrationBuilder.CreateTable(name: "Customers", /* ... */);

    // 3. Was Tabellen voraussetzt: NACH den generierten Operationen
    migrationBuilder.CreateApplicationRole("shop_app", canLogin: true);
    migrationBuilder.GrantTablePrivileges("shop_app");     // wirkt auf bestehende Tabellen
}
```

Idempotente Skripte

`dotnet ef migrations script --idempotent` erzeugt ein Skript, das gegen jeden Datenbankstand laufen kann. Npgsql verpackt dazu jede Operation in einen Block `DO $EF$ ... $EF$`, der die Historientabelle prüft. Eigene Operationen, die selbst einen `DO $$`-Block erzeugen, stecken dann darin. Das funktioniert, weil die Dollar-Quotes verschiedene Tags haben. Solche Annahmen gehören in einen Test:

tests/...IntegrationTests/AdvancedTests.cs (Auszug)

```
var script = db.GetService<IMigrator>()
    .GenerateScript(options: MigrationsSqlGenerationOptions.Idempotent);

await db.Database.ExecuteSqlRawAsync(script);   // verschachteltes DO gültig?
await db.Database.ExecuteSqlRawAsync(script);   // zweiter Lauf wird übersprungen
```

Operationen außerhalb von Transaktionen

Manche Befehle dürfen nicht in einer Transaktion laufen, in PostgreSQL etwa `CREATE INDEX CONCURRENTLY`. Eigene Operationen mit solchem SQL beenden ihren Befehl mit `EndStatement( builder, suppressTransaction: true)`. EF führt ihn dann außerhalb der Migrationstransaktion aus. Der Preis: Schlägt die Migration danach fehl, wird dieser Befehl nicht zurückgerollt; er sollte deshalb idempotent sein.

Migration Bundles

`dotnet ef migrations bundle` erzeugt eine ausführbare Datei, die Migrationen ohne SDK anwendet. Sie enthält das Startprojekt samt Plugin-Assemblies; eigene Operationen und der SQL-Generator des Plugins wirken also auch dort. Wichtig ist, dass die Design-Time-Factory oder der Host dieselben Plugin-Optionen verwendet wie die Anwendung, sonst entsteht SQL für ein anderes Modell. Wie das Bundle mit getrennten Rollen in eine Pipeline gehört, zeigt Seite 148.

Teil IX

Betrieb und Integration

Betrieb in Produktion: Row Level Security, mandantensichere Beziehungen, Nachrüsten, Performance und der Weg vom Plugin in die Anwendung und in die CI.

- Kapitel 36: Row Level Security in Produktion
- Kapitel 37: Mandantensichere Beziehungen
- Kapitel 38: Mandantenfähigkeit nachrüsten und betreiben
- Kapitel 39: Performance von Plugins
- Kapitel 40: Einsatz in ASP.NET Core
- Kapitel 41: CI und API-Stabilität

36 Row Level Security in Produktion

Versionsabhängigkeit: **stabil**

RLS ist nur so stark wie seine Betriebsbedingungen. Diese Fallen machen die Policies stillschweigend wirkungslos oder unbrauchbar:

Falle	Folge	Abhilfe
App läuft als Tabelleneigentümer	Policies werden ignoriert	`FORCE ROW LEVEL SECURITY` (Default im Plugin) und getrennte Rollen
Superuser oder `BYPASSRLS`	RLS wird immer umgangen, auch mit FORCE	App-Rolle mit `NOSUPERUSER NOBYPASSRLS` (eigene Operation, Kap. 22)
Setting nicht gesetzt	Ohne `nullif`: Fehler; mit: leere Ergebnisse	Interceptor setzt den Wert bei jedem Öffnen
Kein Index auf der Mandantenspalte	Jede Abfrage scannt die ganze Tabelle	Index-Convention (Kapitel 10)
Transaction-Pooling (PgBouncer)	Session-Settings landen bei anderen Clients oder gehen verloren	`UseTransactionPooling()`: Setting transaktionslokal (siehe unten)
Verbindung außerhalb von EF geöffnet	Die Session behält einen alten Mandanten oder keinen	EF die Verbindungen öffnen lassen oder `ApplyTenantSession()` (Kapitel 18)
„Bypass“-Schalter für die App-Rolle	Wer SQL als App-Rolle ausführen kann, kann ihn umlegen	Eigene Rollen für mandantenübergreifenden Zugriff (siehe unten)

Das Bedrohungsmodell

RLS ist eine Schutzschicht mit klaren Grenzen. Wer sie einsetzt, sollte wissen, wovor sie schützt, worauf sie sich verlässt und wo sie aufhört:

Aspekt	Im Beispiel
Geschützt	Die Zeilen mandantenbezogener Tabellen in den unterstützten Abbildungen (Seite 51). Nicht geschützt sind gemeinsame Stammdaten, das Schema und, ohne zusammengesetzte Schlüssel, die Existenz fremder Werte (Kapitel 37).
Vertrauenswürdig	Der Datenbankserver und seine Rollen; die Migrationsrolle als Tabelleneigentümer; der Code von Anwendung und Plugin, also auch die Interceptors; der `ITenantProvider` und die Quelle, aus der er den Mandanten liest.
Annahmen	Die App-Rolle ist weder Superuser noch Eigentümer und hat kein `BYPASSRLS`. Der Mandant stammt aus einem authentifizierten Claim, nicht aus einem Header oder Parameter, den der Client frei wählt. Verbindungen werden von EF geöffnet, und es gibt kein Transaction-Pooling, außer mit `UseTransactionPooling()`.
Grenze	Die App-Rolle darf das Session-Setting selbst setzen; das muss sie, damit der Interceptor es setzen kann. Wer beliebiges SQL unter dieser Rolle ausführt, wechselt damit den Mandanten.

Die Grenze ist kein Fehler des Plugins, sondern eine Eigenschaft des Session-Modells. `ThreatModelTests` zeigt sie gegen PostgreSQL: Ein `SELECT set_config('app.tenant_id', ...)` unter der App-Rolle genügt, und die Abfrage danach liefert die Zeilen des anderen Mandanten. `SqlServerThreatModelTests` zeigt dasselbe mit `sp_set_session_context`. Daraus folgt, wovor RLS hier schützt und wovor nicht:

- **Geschützt:** vergessene Filter, fehlerhafte LINQ-Abfragen, rohes SQL der Anwendung mit falschem `WHERE`, Massenoperationen, Berichte, die versehentlich über die App-Rolle laufen. In all diesen Fällen setzt das Plugin den Mandanten, und der Code ändert ihn nicht.
- **Nicht geschützt:** SQL-Injection und kompromittierter Anwendungscode. Beides kann das Setting ändern, auch innerhalb eines einzigen Statements, weil `set_config` eine Funktion ist. Dagegen helfen Parameter statt zusammengesetzter Strings (EF parametrisiert LINQ und `FromSql`/`SqlQuery` mit interpolierten Strings; `FromSqlRaw` mit verketteten Werten nicht) und Rechte, die der App-Rolle nur die Tabellen geben, die sie braucht.
- **Außerhalb der Datenbank:** Ein gefälschter oder falsch zugeordneter Claim gibt Zugriff auf genau diesen Mandanten. Das ist Sache der Authentifizierung; das Plugin übernimmt den Mandanten aus dem `ITenantProvider`.

Mögliche Härtungen setzt das Beispiel bewusst nicht um. SQL Server kann den Schlüssel mit `sp_set_session_context ..., @read_only = 1` bis zum Ende der Verbindung einfrieren (SQL Server: Row-Level Security); der Test oben belegt, dass ein späterer Wechsel dann scheitert. Der Preis: Auch `ApplyTenantSession` kann den Mandanten einer offenen Verbindung nicht mehr ändern. PostgreSQL kennt kein schreibgeschütztes Setting; dort bleiben eine Rolle pro Mandant mit `SET ROLE` oder ein signiertes Mandanten-Token, das eine `SECURITY DEFINER`-Funktion prüft. Beides verlagert Komplexität in die Datenbank und ist hier nur als Richtung genannt. Mandantenübergreifender Zugriff gehört weiterhin in eigene Rollen (Abschnitt unten).

Transaction-Pooling

PgBouncer im Modus „transaction“ teilt eine Serververbindung zwischen Clients pro Transaktion. Ein mit `set_config(..., false)` gesetzter Session-Wert kann dann bei einem anderen Client ankommen, eine Sicherheitslücke. Die sichere Variante setzt den Wert mit `set_config(..., true)` *transaktionslokal*: Er endet mit Commit oder Rollback. Dafür wechselt der Interceptor von `IDbConnectionInterceptor` zu `IDbTransactionInterceptor` und setzt den Mandanten in `TransactionStarted`, im Befehl der neuen Transaktion. Das Beispiel bietet das als Option an:

Anwendung

```
services.AddDbContext<ShopContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp, t => t.UseTransactionPooling()));
```

Runtime/PostgreSqlTransactionSessionInterceptor.cs (gekürzt)

```
public override async ValueTask<DbTransaction> TransactionStartedAsync(
    DbConnection connection,
    TransactionEndEventData eventData,
    DbTransaction result,
    CancellationToken cancellationToken = default)
{
    await using var command = CreateSessionCommand(connection, result, eventData.Context);
    await command.ExecuteNonQueryAsync(cancellationToken);
```

```
        return result;
    }

    // SaveChanges immer in einer Transaktion, auch bei einem einzelnen Befehl
    private static void RequireTransaction(DbContextEventData eventData)
    {
        // ...
            context.Database.AutoTransactionBehavior = AutoTransactionBehavior.Always;
    }

    protected virtual DbCommand CreateSessionCommand(
        DbConnection connection, DbTransaction transaction, DbContext? context)
    {
        // ...
        var command = connection.CreateCommand();
        command.Transaction = transaction;
        command.CommandText = "SELECT set_config(@name, @value, true)";   // is_local = true
        // ...
    }
```

`UseAcmeTenancy(sp)` registriert dann diesen Interceptor statt des Connection-Interceptors. Drei Regeln gehören dazu:

- **Lesen in Transaktionen.** Abfragen außerhalb einer Transaktion haben keinen Mandanten und sehen keine Zeilen. Das ist sicher, aber die Anwendung muss lesende Arbeit, die Daten braucht, in eine Transaktion legen.
- **Speichern immer in einer Transaktion.** EF lässt die automatische Transaktion weg, wenn `SaveChanges` nur einen Befehl ausführt (`AutoTransactionBehavior.WhenNeeded`). Ein einzelner Insert liefe dann ohne Mandanten und scheiterte an der Policy. Der Interceptor ist deshalb zugleich ein `ISaveChangesInterceptor` und stellt `Always` ein.
- **Kein Session-Wert.** Ist die Option gesetzt und trotzdem ein `TenantConnectionInterceptor` registriert, verweigert dieser das Setzen mit einer `TenancyException`, statt einen Wert zu hinterlassen, der die Transaktion überdauert. Damit entfällt auch `ApplyTenantSession`: Ein Aufruf scheitert mit dem Hinweis, dass der Mandant pro Transaktion gesetzt wird und die Befehle in eine gehören.

`TransactionPoolingTests` prüft gegen PostgreSQL: Innerhalb der Transaktion ist der Wert gesetzt und RLS greift, nach Commit und nach Rollback ist er auf derselben Verbindung leer, ein einzelner Insert bekommt seine Transaktion, der Connection-Interceptor lehnt den Session-Wert ab, und `ApplyTenantSession` erklärt, warum es nicht verfügbar ist. PgBouncer selbst ist nicht Teil des Tests; geprüft wird die Eigenschaft, auf die es ankommt: dass nach der Transaktion kein Wert bleibt.

Performance von Policies

PostgreSQL fügt Policies wie zusätzliche `WHERE`-Bedingungen in jede Abfrage ein. `current_setting` ist `STABLE`. Das garantiert innerhalb eines Statements dasselbe Ergebnis für alle Zeilen und *erlaubt* dem Optimierer, mehrere Aufrufe zu einem zusammenzufassen (PostgreSQL: Function Volatility Categories); eine feste Zahl von Aufrufen garantiert es nicht. Wie oft die Funktion tatsächlich läuft, hängt vom Plan ab. Steht der Vergleich in einer `Index Cond`, wertet PostgreSQL den Vergleichswert einmal aus und liest nur die Zeilen des Mandanten. Steht er als `Filter` über einem Seq Scan, kann er pro Zeile ausgewertet werden, und dann wiegt der Scan schwerer als die Funktion. Teure Policies entstehen durch Unterabfragen (etwa Rechte aus einer anderen Tabelle). Zusätzlich hält

PostgreSQL Bedingungen aus der Abfrage, deren Funktionen nicht `LEAKPROOF` sind, hinter den Policy-Bedingungen zurück, was Optimierungen verhindern kann (PostgreSQL: Row Security Policies).

Aussagekräftig ist deshalb nur der Plan unter realen Bedingungen: `EXPLAIN (ANALYZE, BUFFERS)` als App-Rolle mit gesetztem Mandanten, mit realistischer Datenmenge und aktuellen Statistiken. Als Owner oder Superuser gemessen fehlen die Policies im Plan, weil Superuser und Rollen mit `BYPASSRLS` sie umgehen und der Owner ohne `FORCE` ebenso. Den Overhead zeigt der Vergleich derselben Abfrage mit und ohne Policy, nicht die Zahl der Funktionsaufrufe.

Legitimer mandantenübergreifender Zugriff

Früher oder später braucht jemand Daten über Mandanten hinweg: Berichte für den Betreiber, Wartungsjobs, Support. Die verlockende Abkürzung ist ein Schalter in der Anwendung, etwa ein zusätzliches Session-Setting `app.bypass`, das die Policy berücksichtigt. Das ist keine Sicherheitsgrenze: Was die Anwendungsrolle setzen kann, kann auch ein eingeschleustes SQL-Statement setzen. Die Trennung muss stattdessen über Datenbankrollen entstehen:

Bedarf	Lösung	Warum es sicher ist
Berichte, Analysen	Eigene Nur-Lese-Rolle mit `BYPASSRLS`, nur `SELECT`, eigenem Connection-String und einer Context-Konfiguration ohne Mandant	Die App-Rolle kann nicht zu dieser Rolle werden; ihre Zugangsdaten liegen woanders
Support liest ausgewählte Daten	Zusätzliche permissive Policy für eine Support-Rolle: `HasPolicy(...)` mit `For(Select)` und `To("shop_support")`	Beschränkt auf SELECT und auf Mitglieder der Rolle
Wartung pro Mandant	Hintergrundjob, der die Mandanten durchläuft und jeden einzeln setzt (Kapitel 40)	Läuft durch die normalen Policies

Reporting-Rolle (tests/...IntegrationTests/CrossTenantAccessTests.cs)

```
-- Angelegt von einem Administrator: BYPASSRLS erfordert einen Superuser
CREATE ROLE shop_reporting LOGIN PASSWORD '...' NOSUPERUSER BYPASSRLS;
GRANT USAGE ON SCHEMA public TO shop_reporting;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO shop_reporting;
```

Die Anwendungsrolle dagegen kann die Policies nicht abschalten. `SET row_security = off` deaktiviert RLS für eine Rolle ohne `BYPASSRLS` nicht; PostgreSQL lehnt stattdessen jede Abfrage ab, die die Policies betreffen würden. `CrossTenantAccessTests` prüft alle drei Punkte: Die Reporting-Rolle sieht alle Mandanten, sie kann nicht schreiben, und die App-Rolle kann sich nicht selbst helfen.

> **Mandantenübergreifenden Zugriff protokollieren**
>
> Jede Verbindung der Reporting-Rolle ist ein Zugriff auf alle Mandanten. Sie gehört protokolliert (`log_connections`, pgaudit), und die Rolle sollte so wenigen Personen und Diensten wie möglich zugewiesen werden. `CreateApplicationRole` des Plugins legt Rollen bewusst immer mit `NOBYPASSRLS` an: Bypass-Rollen gehören nicht in die Migrationen der Anwendung.

37 Mandantensichere Beziehungen

Versionsabhängigkeit: **stabil**

Welche Tabellenabbildungen das Plugin schützt und welche es beim Modellbau ablehnt, legt die Matrix auf Seite 51 fest; dieses Kapitel wiederholt sie nicht. Es geht um die zweite Achse: Beziehungen zwischen geschützten Tabellen. RLS regelt, welche *Zeilen* eine Sitzung sieht. Ob eine Zeile nur auf Zeilen desselben Mandanten verweist, regelt RLS nicht. Das ist *referenzielle Mandantenkonsistenz*, und sie entsteht im Modell.

Das Beispiel verwendet Kunde, Bestellung und Position: `Customer` → `Order` → `OrderItem`, alle drei `ITenantOwned`, dazu ein gemeinsames `Product` ohne Mandant.

Sichtbarkeit ist nicht Konsistenz

PostgreSQL dokumentiert ausdrücklich, dass Prüfungen von Fremdschlüsseln, eindeutigen und Primärschlüsseln RLS umgehen, damit die Datenintegrität erhalten bleibt, und warnt vor „covert channels“ durch diese Prüfungen (PostgreSQL: Row Security Policies). Die SQL-Server-Dokumentation sagt dazu nichts ausdrücklich; `SqlServerThreatModelTests` zeigt, dass die Fremdschlüsselprüfung dort ebenfalls Zeilen sieht, die das Filter-Prädikat verbirgt. Mit Fremdschlüsseln nur auf der ID, wie EF sie standardmäßig anlegt, folgt daraus:

Beobachtung	Ursache
Mandant A legt eine Position zu einer Bestellung von B an	Die FK-Prüfung findet die Bestellung, obwohl A sie nicht sieht.
A erfährt, ob eine fremde ID existiert	Unbekannte IDs scheitern am Fremdschlüssel, fremde nicht.
Die eigene Position verschwindet aus `Include(i => i.Order)`	Eine Pflichtnavigation wird ein `INNER JOIN`, und die Bestellung ist unsichtbar.
A löscht eine eigene Bestellung, eine Position von B verschwindet	Die kaskadierende Löschung ist eine referenzielle Aktion und umgeht RLS ebenfalls.
A erfährt, dass eine E-Mail-Adresse bei B existiert	Ein globaler Unique-Index meldet die Kollision.

`RelationshipTests` weist jede Zeile gegen PostgreSQL mit eingeschränkter Rolle nach. Keiner dieser Fälle ist ein Leseleck im engeren Sinn; A sieht keine fremde Zeile. Aber die Daten verlieren ihre Mandantengrenze, und die Fehlermeldungen beantworten Fragen, die A nicht stellen dürfte.

Zusammengesetzte Fremdschlüssel

Die Lösung ist, den Mandanten zum Teil jeder Referenz zu machen. Der Prinzipal bekommt einen alternativen Schlüssel aus Mandant und ID, die abhängige Seite einen Fremdschlüssel aus ihrer Mandantenspalte und der ID:

`tests/...IntegrationTests/RelationshipTests.cs › TenantSafeContext`

```
modelBuilder.Entity<Customer>(b =>
{
    b.HasAlternateKey(c => new { c.TenantId, c.Id });
```

```
    b.HasIndex(c => new { c.TenantId, c.Email }).IsUnique();   // eindeutig pro Mandant
});

modelBuilder.Entity<Order>(b =>
{
    b.HasAlternateKey(o => new { o.TenantId, o.Id });
    b.HasOne(o => o.Customer).WithMany(c => c.Orders)
        .HasForeignKey(o => new { o.TenantId, o.CustomerId })
        .HasPrincipalKey(c => new { c.TenantId, c.Id })
        .OnDelete(DeleteBehavior.Restrict);                  // Kunden nicht kaskadieren
});

modelBuilder.Entity<OrderItem>()
    .HasOne(i => i.Order).WithMany(o => o.Items)
    .HasForeignKey(i => new { i.TenantId, i.OrderId })
    .HasPrincipalKey(o => new { o.TenantId, o.Id })
    .OnDelete(DeleteBehavior.Cascade);                       // Positionen gehören dazu
```

Jetzt verweist eine Position von A auf das Paar (A, ID). Eine Bestellung von B erfüllt das nie, und der Fehler ist derselbe wie für eine unbekannte ID: Es gibt nichts mehr zu erfahren. Die Mandantenspalte ist zugleich Teil des Fremdschlüssels und Mandanten-Property. Der SaveChanges-Interceptor setzt sie bei neuen Positionen, und das Fixup über die Navigation setzt denselben Wert. Ein Index für die Policy ist nicht nötig, weil Schlüssel und Fremdschlüssel mit der Mandantenspalte beginnen; die Index-Convention erkennt das.

Der Preis ist ein zusätzlicher eindeutiger Index pro Prinzipal und eine breitere Fremdschlüsselspalte. Das Plugin legt diese Schlüssel nicht automatisch an. Eine Convention könnte es, müsste aber jede Beziehung zwischen geschützten Typen umschreiben und mit expliziter Konfiguration abgleichen. Das Beispiel überlässt die Entscheidung dem Modell der Anwendung.

Eindeutigkeit pro Mandant

Ein Unique-Index über eine fachliche Spalte gilt für die ganze Tabelle, also über alle Mandanten. Fast immer ist Eindeutigkeit pro Mandant gemeint: `(TenantId, Email)` statt `Email`. Wo Eindeutigkeit wirklich global sein muss, etwa bei Subdomains der Mandanten, gehört die Spalte in das Mandantenverzeichnis, das bewusst keine Mandantendaten enthält.

Gemeinsame Stammdaten

`Product` hat keinen Mandanten, keine Policy und ist für alle lesbar. Fremdschlüssel von Mandantendaten auf Stammdaten sind unproblematisch; der Test liest das Produkt über die Position. Die Gegenrichtung, Stammdaten mit Verweisen auf Mandantendaten, darf es nicht geben. Schreiben in Stammdaten sollte nicht die App-Rolle dürfen; `GrantTablePrivileges` gewährt Rechte auf alle Tabellen eines Schemas, eine feinere Vergabe ist Sache der Migrationen der Anwendung.

Join-Tabellen

Implizite Many-to-many-Tabellen lehnt das Plugin ab, wenn eine Seite mandantenbezogen ist. Eine Join-Entität mit eigener Mandanten-Property ist geschützt (Seite 51). Für Konsistenz bekommt sie dieselben zusammengesetzten Fremdschlüssel wie jede abhängige Tabelle: `(TenantId, LeftId)` und `(TenantId, RightId)` auf die alternativen Schlüssel beider Seiten. Dann kann eine Verknüpfung nur zwei Zeilen desselben Mandanten verbinden.

Löschregeln

Kaskadierende Löschungen laufen in der Datenbank als referenzielle Aktionen und umgehen RLS. Mit zusammengesetzten Fremdschlüsseln bleiben sie trotzdem im Mandanten, weil jede abhängige Zeile denselben Mandanten trägt. Ohne sie löscht eine Kaskade fremde Zeilen; der Test `Naive_cascade_deletes_rows_of_another_tenant` zeigt genau das. Für Wurzeln wie den Kunden ist `Restrict` meist richtig, für Teile eines Aggregats wie Positionen `Cascade`. EF kaskadiert zusätzlich im Change Tracker, aber nur für geladene Entitäten; maßgeblich ist die Regel in der Datenbank.

38 Mandantenfähigkeit nachrüsten und betreiben

Versionsabhängigkeit: **stabil**

Die bisherigen Kapitel gehen von einer neuen Datenbank aus. Meist gibt es aber schon Tabellen mit Daten, eine laufende Anwendung und Nutzer, die während der Umstellung weiterarbeiten. Dieses Kapitel zeigt den Weg von einer Tabelle ohne Mandantenspalte zu RLS und danach den Betrieb unter Retries und Verbindungsabbrüchen. Alle Schritte stehen ausführbar in `OperationsTests` gegen PostgreSQL. Den Abschluss bildet das Deployment: Migrationen in der Pipeline mit getrennten Rollen und Änderungen ohne Ausfallzeit.

Von der Tabelle ohne Mandant zu RLS

Wer eine bestehende Entität einfach `ITenantOwned` implementieren lässt, bekommt eine einzige Migration: Spalte mit dem leeren Mandanten als Default, RLS, Policy, Index. Danach sieht niemand mehr die alten Zeilen. Auch ein Backfill in derselben Migration hilft nicht sicher: Mit `FORCE ROW LEVEL SECURITY` gilt die Policy auch für die Migrationsrolle, und ohne gesetzten Mandanten findet ihr `UPDATE` keine Zeile. Der Weg führt deshalb über zwei Migrationen, nach dem Muster Erweitern, Befüllen, Absichern:

Nr.	Schritt	Vorgehen
1	Erweitern	Die Entität implementiert `ITenantOwned`, bleibt aber mit `HasNoRowLevelSecurity()` ausgenommen. Die Migration legt nur die Spalte an.
2	Befüllen	Als Eigentümer, in derselben oder einer eigenen Migration: Jede Zeile bekommt ihren Mandanten aus dem, was die Zuordnung kennt, etwa einer Zuordnungstabelle oder dem Mandanten des Eltern-Datensatzes.
3	Anwendung umstellen	Eine App-Version, deren Modell die Ausnahme nicht mehr hat: Der Interceptor setzt den Mandanten bei neuen Zeilen. Die Datenbank schützt noch nicht, das Modell prüft das nicht.
4	Lücke schließen	Zeilen, die zwischen Schritt 1 und 3 ohne Mandant entstanden sind, nachträglich zuordnen.
5	Absichern	Zweite Migration: Ausnahme entfernt, also `ENABLE`/`FORCE`, Policy und Index. Als Erstes ein Wächter, der abbricht, solange eine Zeile keinen Mandanten hat.
6	Verschärfen	Optional zusammengesetzte Schlüssel und Unique-Indizes pro Mandant (Kapitel 37).

Der Wächter ist ein eigener Schritt am Anfang von `Up`, vor den Operationen, die EF erzeugt:

Wächter (SQL)

```
DO $$ BEGIN
  IF EXISTS (SELECT 1 FROM "Tickets"
             WHERE "TenantId" = '00000000-0000-0000-0000-000000000000') THEN
    RAISE EXCEPTION 'Tickets without a tenant: backfill first';
  END IF;
END $$;
```

Migrations/…_EnableTicketIsolation.cs

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    // Vor allem anderen: ohne vollständige Zuordnung nicht absichern
    migrationBuilder.Sql(TicketGuardSql);      // der Wächter oben, als Konstante

    // ... von EF erzeugt: ENABLE/FORCE ROW LEVEL SECURITY, CREATE POLICY, CREATE INDEX
}
```

Die Migration läuft in einer Transaktion. Schlägt der Wächter an, bleibt die Tabelle ohne RLS, und kein halber Zustand entsteht; `Guard_stops_enabling_rls_while_rows_lack_a_tenant` prüft das. Nach vollständiger Zuordnung sieht jeder Mandant genau seine Zeilen.

App-Versionen, Down und Rollback

Während der Umstellung laufen oft zwei App-Versionen gleichzeitig, und jede trifft auf einen der beiden Datenbankzustände:

Datenbank	App ohne Mandant	App mit Mandant
Nach Migration 1 (Spalte, kein RLS)	Läuft; neue Zeilen ohne Mandant (Schritt 4)	Läuft, setzt den Mandanten; noch ohne Schutz
Nach Migration 2 (RLS)	Fail closed: liest nichts, Schreiben scheitert	Geschützt

Die alte App-Version darf also nicht mehr laufen, wenn Migration 2 eingespielt ist. Ein Rollback der Anwendung auf eine Version ohne Mandant setzt deshalb erst das Down von Migration 2 voraus. Das Down entfernt Policy und RLS; Spalte und Werte bleiben, die Tabelle ist aber wieder ungeschützt. Der Test prüft beides. Ein Down von Migration 1 löscht die Spalte und damit die Zuordnung; es gehört nur in die Entwicklung.

Policy-Änderungen folgen demselben Muster. Ändert sich der Setting-Name (`UseSettingName`), ersetzt die Migration die Policy in ihrer Transaktion. Zwischen Migration und neuer App-Version liest die Policy ein Setting, das die laufende App nicht setzt, und die Datenbank verweigert alles, bis beide zusammenpassen. Ein Übergang, bei dem der Interceptor beide Settings setzt, ist denkbar, aber nicht implementiert. Für SQL Server beschreibt Seite 183 denselben Fall.

Retries, Transaktionen und Verbindungsabbrüche

Das Plugin setzt den Mandanten, wenn EF eine Verbindung öffnet. Daraus ergibt sich das Verhalten unter den Resilienzmechanismen von EF:

- **Retries.** Eine Execution Strategy wie `EnableRetryOnFailure` wiederholt eine Operation nach einem transienten Fehler auf einer neu geöffneten Verbindung. Der Interceptor läuft dabei wieder, der Mandant ist gesetzt. Der Test beendet die Sitzung serverseitig mit `pg_terminate_backend`; die nächste Abfrage scheitert, wird wiederholt und liefert nur Zeilen des Mandanten.
- **Explizite Transaktionen.** Mit einer wiederholenden Strategie lehnt EF Befehle in einer selbst begonnenen Transaktion ab, weil ein Retry die ganze Transaktion wiederholen müsste. Die Transaktion gehört deshalb in `CreateExecutionStrategy().ExecuteAsync(…)`. Innerhalb öffnet EF die Verbindung einmal, setzt den Mandanten und hält sie bis zum Commit. Bricht die

Verbindung mitten in der Transaktion ab, wiederholt die Strategie den ganzen Delegaten auf einer neuen Verbindung; Seiteneffekte außerhalb der Datenbank müssen dann idempotent sein.

- **Neuinitialisierung der Session.** Npgsql setzt den Zustand einer Verbindung zurück, wenn sie in den Pool zurückgeht, solange `No Reset On Close` nicht aktiviert ist (Npgsql: Connection String Parameters). SQL Server leert den Session-Kontext beim Zurückgeben ebenfalls. Das Plugin verlässt sich auf keines von beidem, sondern setzt den Mandanten bei jedem Öffnen; nur Verbindungen, die nicht EF öffnet, brauchen `ApplyTenantSession` (Kapitel 18).

Für Transaction-Pooling setzt `UseTransactionPooling()` den Mandanten pro Transaktion (Seite 140). Nicht implementiert ist ein schreibgeschützter Session-Schlüssel (Abschnitt „Das Bedrohungsmodell“). `OperationsTests` enthält den Verbindungsabbruch und die Transaktion in der Strategie; die Fälle offener Verbindungen stehen in `ConnectionLifetimeTests`.

Deployment: Pipeline, Rollen und Zero-Downtime

Die Beispiele legen die Datenbank in der Entwicklung mit `DevelopmentDatabase` an. In Produktion gehört jede Schemaänderung in eine Pipeline, die zwei Rollen strikt trennt:

- **`shop_owner`** besitzt Schema, Tabellen und Policies und führt die Migrationen aus.
- **`shop_app`** ist die Rolle der Anwendung: nur `SELECT`, `INSERT`, `UPDATE`, `DELETE`, immer unter den Policies.

Die Anwendung ruft deshalb niemals `db.Database.Migrate()` beim Start auf. Dafür bräuchte ihre Rolle DDL-Rechte, und wer eine Tabelle ändern darf, darf auch `DISABLE ROW LEVEL SECURITY` ausführen oder die Policy löschen. `FORCE` bindet den Eigentümer an die Policy, schützt sie aber nicht vor ihm. Eine Lücke in der Anwendung, etwa eine SQL-Injection, wäre dann nicht mehr durch die Datenbank begrenzt. Hinzu kommt der Betrieb: Bei einem Rolling Deployment starten mehrere Instanzen gleichzeitig, und eine fehlgeschlagene Migration soll die Auslieferung stoppen, nicht die Instanzen in einer Neustartschleife halten.

Einmalig: die Rollen

Bevor die erste Pipeline läuft, legt die Datenbankadministration die Rollen an. Das ist keine Migration, denn Migrationen laufen bereits als Eigentümer. Das Skript liegt im Repository:

`docs/deployment/setup-roles.sql`

```
CREATE ROLE shop_owner LOGIN NOSUPERUSER NOBYPASSRLS PASSWORD :'owner_password';
CREATE ROLE shop_app LOGIN NOSUPERUSER NOBYPASSRLS PASSWORD :'app_password';
CREATE DATABASE shop OWNER shop_owner;

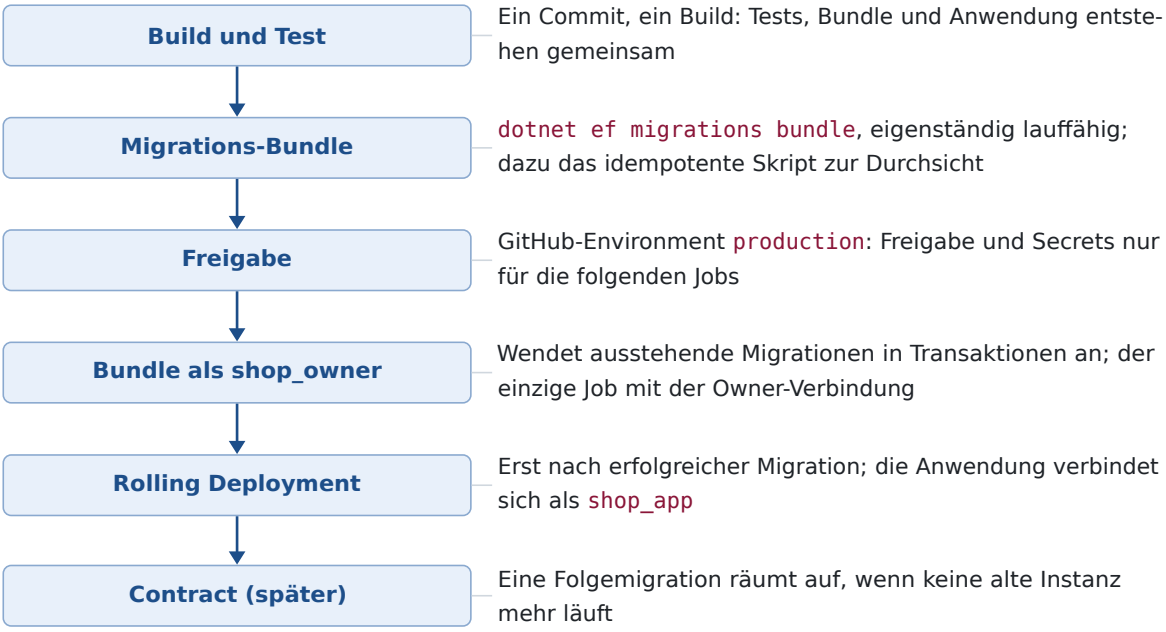
...
GRANT USAGE ON SCHEMA public TO shop_app;

ALTER DEFAULT PRIVILEGES FOR ROLE shop_owner IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO shop_app;
ALTER DEFAULT PRIVILEGES FOR ROLE shop_owner IN SCHEMA public
    GRANT USAGE, SELECT ON SEQUENCES TO shop_app;
```

Die Auslassung ist `\connect shop`: Ab dort läuft das Skript in der neuen Datenbank. Wichtig sind die `DEFAULT PRIVILEGES`: Jede Tabelle, die eine spätere Migration als `shop_owner` anlegt, ist für `shop_`

`app` sofort nutzbar. Ohne sie scheitert die neue Anwendungsversion mit `permission denied`, obwohl Migration und Policy stimmen. Die Operation `GrantTablePrivileges` des Plugins setzt ebenfalls Default Privileges, aber nur für die Rolle, die sie ausführt; das Skript legt sie unabhängig davon fest.

Die Pipeline



Die Vorlage `docs/deployment/migrations-deploy.yml` setzt das mit drei Jobs um. Sie ist im Repository nicht aktiv; eine Anwendung kopiert sie nach `.github/workflows` und ersetzt den Deploy-Schritt durch den ihrer Plattform:

docs/deployment/migrations-deploy.yml (gekürzt)

```
jobs:
  build:
    steps:
      ...
      - name: Migration bundle
        run: >
          dotnet ef migrations bundle --project "$APP_PROJECT" --context "$DB_CONTEXT"
          --configuration Release --self-contained -r linux-x64 -o artifacts/efbundle
      ...
      - run: dotnet publish "$APP_PROJECT" -c Release -o artifacts/app

  migrate:
    needs: build
    environment: production
    steps:
      ...
      - name: Apply pending migrations as shop_owner
        env:
          OWNER_CONNECTION: ${{ secrets.SHOP_OWNER_CONNECTION_STRING }}
        run: |
          chmod +x artifacts/efbundle           # artifacts lose the executable bit
          artifacts/efbundle --connection "$OWNER_CONNECTION"
```

```
  deploy:
    needs: migrate
    environment: production
    ...
```

Das Bundle enthält das Startprojekt mit dem Plugin; für dessen Optionen gilt, was Abschnitt „Migration Bundles“ in Kapitel 35 sagt. `--connection` überschreibt die Verbindung der Design-Time-Factory, und ein zweiter Lauf wendet nichts an. Der Owner-Verbindungsstring ist nur dem Job `migrate` bekannt; den der Anwendung erhält die Hosting-Plattform, nicht die Pipeline.

Der Consumer-Test (`eng/consumer-test/run.sh --nativeaot`) spielt genau diesen Ablauf gegen PostgreSQL in Docker durch: Er führt `setup-roles.sql` unverändert aus, baut das Bundle aus der erzeugten Migration, wendet es als `shop_owner` zweimal an (der zweite Lauf ändert nichts), prüft, dass `shop_app` die Tabelle nicht ändern darf (`must be owner`), und startet die Anwendung als `shop_app`, die nur die Zeilen ihres Mandanten sieht.

Zero-Downtime unter RLS: Erweitern und Zusammenziehen

Bei einem Rolling oder Blue-Green Deployment läuft die Migration vor der neuen Version, und eine Zeit lang arbeiten alte und neue Instanzen gegen dasselbe Schema. Jede Migration muss deshalb mit der Vorgängerversion der Anwendung funktionieren. Das Muster Expand and Contract teilt eine Änderung dafür in drei Phasen, wie schon die Umstellung in Abschnitt „Von der Tabelle ohne Mandant zu RLS“:

Phase	Schema	Unter RLS beachten
1 Erweitern	Neue Spalten nullable oder mit Default, neue Tabellen, neue Indizes; nichts wird entfernt oder umbenannt.	Neue Tabellen mit `ITenantOwned` bekommen ihre Policy sofort; die alte Version benutzt sie nicht. Muss eine Policy sich ändern, kommt eine zweite hinzu, die die neue Form erlaubt.
2 Ausrollen	Unverändert. Die neue Version schreibt in die neuen Spalten, alte Instanzen laufen aus.	Beide Versionen setzen denselben Mandanten; die Policies müssen die Zeilen beider Versionen zulassen.
3 Zusammenziehen	Folgemigration: alte Spalten löschen, `NOT NULL` setzen, temporäre Policies entfernen.	Erst wenn keine alte Instanz mehr läuft; vorher ein Wächter wie in Abschnitt „Von der Tabelle ohne Mandant zu RLS“.

Für Policies gilt eine Eigenheit von PostgreSQL: Mehrere `PERMISSIVE`-Policies einer Tabelle werden mit `OR` verknüpft. Eine zusätzliche Policy erweitert also, was sichtbar ist, und eignet sich als Übergang in Phase 1; ihr Entfernen in Phase 3 ist das Zusammenziehen. Jede dieser Policies muss für sich mandantentrennend sein, sonst öffnet der Übergang die Tabelle. Weil DDL in PostgreSQL transaktional ist, gibt es beim Ersetzen einer Policy innerhalb einer Migration keinen Moment ohne Policy.

Ein Rollback der Anwendung ist bis Phase 3 ohne Datenbankänderung möglich, weil das Schema die alte Version noch trägt. Danach braucht er das Down der Folgemigration, und gelöschte Spalten kommen damit nicht zurück. Ein Down gehört deshalb nicht in die Pipeline; ein Fehler nach Phase 3 wird mit einer neuen Migration vorwärts behoben.

39 Performance von Plugins

Versionsabhängigkeit: **stabil**

Plugin-Code läuft in drei Häufigkeiten. Wer weiß, welcher Code wie oft läuft, weiß, wo sich Optimierung lohnt:

Häufigkeit	Code	Folgerung
Einmal pro Provider	Conventions, Type-Mapping-Plugins, Singleton-Options	Lesbarkeit vor Mikro-Optimierung
Einmal pro Abfrageform	Translators, Query-Interceptors	Moderat; Ergebnis wird gecacht (Kapitel 13)
Bei jeder Ausführung	Interceptors für Commands, Verbindungen, SaveChanges, Materialisierung; Evaluatable-Filter, wenn der Funcletizer die Parameterwerte ausliest	Heißer Pfad: keine Reflection, keine unnötigen Allokationen

Daraus ergeben sich einige Regeln, die das Beispiel befolgt: `MethodInfo`- und `PropertyInfo`-Lookups liegen in statischen Feldern. Der SaveChanges-Interceptor legt seine Liste erst an, wenn er tatsächlich eine mandantenfähige Änderung findet. Der Connection-Interceptor kostet einen Roundtrip pro Öffnen, was dokumentiert ist. Solche Annahmen sollte man messen, nicht raten. Das Repository enthält ein BenchmarkDotNet-Projekt; der Benchmark für die SaveChanges-Logik läuft ohne Datenbank:

benchmarks/.../SaveChangesInterceptorBenchmark.cs (gekürzt)

```
[MemoryDiagnoser]
public class SaveChangesInterceptorBenchmark
{
    [Params(1, 100)]
    public int Entities { get; set; }

    // GlobalSetup: Context mit UseAcmeTenancy() (ohne Verbindung) und dem Interceptor

    [Benchmark(Baseline = true)]
    public int TrackOnly()                  // was EF vor dem Speichern ohnehin tut
    {
        AddOrders();
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }

    [Benchmark]
    public int TrackAndApplyTenant()        // dasselbe plus die SaveChanges-Logik des Plugins
    {
        AddOrders();
        _interceptor.ApplyTenant(_db);
        var count = _db.ChangeTracker.Entries().Count();
        _db.ChangeTracker.Clear();
        return count;
    }
```

```
}
```

Benchmark ausführen

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks \
    -- --filter '*SaveChanges*' --job short
```

Methode	Entitäten	Mittelwert	Verhältnis	Allokiert
TrackOnly	1	1,5 µs	1,00	1,8 KB
TrackAndApplyTenant	1	2,3 µs	1,54	2,7 KB
TrackOnly	100	123 µs	1,00	134 KB
TrackAndApplyTenant	100	170 µs	1,38	202 KB

Beispielergebnisse (Short Job, .NET 10, Rechner des Autors; die absoluten Werte schwanken, aussagekräftiger sind die Verhältnisse). Das Plugin fügt pro gespeicherter Entität etwa eine halbe Mikrosekunde hinzu, rund 40 bis 50 Prozent mehr als das Change Tracking allein. Das klingt nach viel, doch ein einziger Roundtrip zur Datenbank kostet Hunderte Mikrosekunden; der zusätzliche Roundtrip des Connection-Interceptors pro Öffnen (Kapitel 18) wiegt daher weit schwerer als die SaveChanges-Logik. Genau diese Art Erkenntnis liefert ein Benchmark: Er zeigt, wo sich Optimieren lohnt und wo nicht.

Lesen und Abfragekompilierung

Zwei weitere Benchmarks decken die anderen Häufigkeiten ab. `MaterializationInterceptorBenchmark` misst, was `VerifyMaterializedTenants()` beim Lesen kostet. Es braucht echte Zeilen aus einem `DbDataReader` und startet dafür PostgreSQL per Testcontainers (Docker). Die Verbindung bleibt offen, sodass der Roundtrip des Connection-Interceptors nicht mitgemessen wird; `AsNoTracking` hält das Change Tracking heraus. Eine dritte Variante mit einem leeren `IMaterializationInterceptor` trennt den Aufwand von EF vom Aufwand des Plugins:

benchmarks/.../MaterializationInterceptorBenchmark.cs (gekürzt)

```
[MemoryDiagnoser]
public class MaterializationInterceptorBenchmark
{
    [Params(10, 100, 1000)]
    public int Rows { get; set; }

    // GlobalSetup: Container, 1000 Zeilen eines Mandanten, Verbindungen offen

    [Benchmark(Baseline = true)]
    public Task<int> Read() => CountAsync(_plain);

    [Benchmark]
    public Task<int> ReadIntercepted() => CountAsync(_intercepted);

    [Benchmark]
    public Task<int> ReadAndVerify() => CountAsync(_verifying);

    private async Task<int> CountAsync(BenchmarkContext db)
        => (await db.Orders.AsNoTracking().OrderBy(o => o.Id).Take(Rows).ToListAsync()).Count;
```

```
    // Create: UseAcmeTenancy(sp, t => t.VerifyMaterializedTenants(verify))
}
```

Methode	Zeilen	Mittelwert	Verhältnis	Allokiert
`Read`	10	200 µs	1,00	12 KB
`ReadIntercepted`	10	210 µs	1,08	21 KB
`ReadAndVerify`	10	220 µs	1,12	22 KB
`Read`	100	278 µs	1,00	36 KB
`ReadIntercepted`	100	764 µs	2,77	121 KB
`ReadAndVerify`	100	741 µs	2,68	136 KB
`Read`	1000	846 µs	1,00	269 KB
`ReadIntercepted`	1000	3858 µs	4,70	1120 KB
`ReadAndVerify`	1000	4288 µs	5,22	1268 KB

Das Ergebnis überrascht: Den größten Teil kostet nicht die Prüfung, sondern dass überhaupt ein Materialisierungs-Interceptor registriert ist. EF erzeugt dann für jede Zeile die Daten, die `MaterializationInterceptionData` anbietet, und verlässt den schnellen, kompilierten Materialisierungspfad: bei 1000 Zeilen etwa 0,85 KB und 3 µs pro Zeile. Die Prüfung des Plugins (Provider aus dem Cache, Wert lesen, vergleichen) fügt rund 0,15 KB und 0,4 µs pro Zeile hinzu. Bei zehn Zeilen geht beides im Roundtrip unter, bei tausend Zeilen wird das Lesen auf dem Client etwa fünfmal so teuer.

`QueryCompilationBenchmark` braucht keine Datenbank: `ToQueryString` durchläuft die Pipeline bis zum SQL ohne Verbindung. `Compile` leert vor jedem Aufruf den Query-Cache von EF (den `IMemoryCache` des internen Service Providers), `CacheHit` führt dieselbe Abfrageform mit neuem Parameterwert aus. Verglichen werden EF Core allein, das Plugin und das Plugin mit Query-Filter:

Kontext	Kompilieren	Allokiert	Cache-Treffer	Allokiert
EF Core	2,64 ms	146 KB	18 µs	9,9 KB
Plugin	2,68 ms	147 KB	15 µs	9,9 KB
Plugin mit Query-Filter	2,78 ms	160 KB	15 µs	10,2 KB

Translator-Plugins, Evaluatable-Filter, eigener SQL-Generator und Nullability-Prozessor kosten beim Kompilieren nichts Messbares; der Query-Filter verlängert den Ausdrucksbaum und damit die Übersetzung um wenige Prozent. Bei einem Cache-Treffer verschwindet jeder Unterschied im Rauschen. Alle Werte stammen aus einem Short Job (.NET 10, Rechner des Autors); dessen Fehlerbalken sind breit, die Größenordnungen aber eindeutig:

Phase	Häufigkeit	Mehraufwand des Plugins
Kompilieren	Einmal pro Abfrageform	Kaum messbar, mit Query-Filter wenige Prozent
Cache-Treffer	Jede Abfrage	Nicht messbar
SaveChanges	Jedes Speichern	Etwa 0,5 µs pro Entität (40 bis 50 Prozent des Change Trackings)

Phase	Häufigkeit	Mehraufwand des Plugins
Materialisierung	Jede gelesene Zeile	Etwa 3,5 µs pro Zeile mit `VerifyMaterializedTenants()`, ohne: keiner

Wann die Prüfung beim Lesen lohnt

`VerifyMaterializedTenants()` ist Defense in Depth: Es schützt nicht vor einem Angriff, den RLS nicht abwehrt, sondern bemerkt, wenn RLS gar nicht greift, etwa weil die Anwendung mit dem Eigentümer, einem Superuser oder einer Rolle mit `BYPASSRLS` verbindet oder eine selbst geöffnete Verbindung ohne `ApplyTenantSession` arbeitet. Daraus ergibt sich die Empfehlung:

- **Einschalten**, wenn eine Fehlkonfiguration teurer ist als die Rechenzeit: Daten mit hohen Schutzanforderungen, viele Umgebungen mit eigener Konfiguration, kleine Ergebnismengen, wie sie typische API-Abfragen liefern. Dort geht der Aufwand im Roundtrip unter.
- **Ausschalten** für leselastige Pfade mit großen Ergebnismengen, etwa Exporte und Berichte. Dort trägt die Datenbank die Isolation allein, abgesichert durch die Rollentrennung (Seite 148) und Tests, die mit der eingeschränkten Rolle der Anwendung lesen.
- **Dazwischen**: in Tests und Staging immer einschalten, in Produktion nur für den Kontext der sensiblen Daten. Die Option gilt pro Kontextkonfiguration, nicht global.

`Alle Benchmarks, Probelauf`

```
dotnet run -c Release --project benchmarks/Acme.EntityFrameworkCore.Tenancy.Benchmarks \
    -- --filter '*' --job dry
```

> **Tipp**
>
> Für den Start großer Modelle ist das Compiled Model (Kapitel 24) der wirksamste Hebel. Conventions eines Plugins laufen dann gar nicht mehr zur Laufzeit; deshalb lohnt es sich, die Compiled-Model-Fähigkeit zu testen statt nur zu behaupten.

Second-Level-Caches

Ein Second-Level-Cache (etwa `EFCoreSecondLevelCacheInterceptor` oder Ergebnisse in Redis oder `IMemoryCache`) ist der naheliegende nächste Schritt, wenn Abfragen zu langsam sind. Mit Mandantenfähigkeit ist er auch der kürzeste Weg zu einem Datenleck. Der Cache erkennt ein Ergebnis am SQL-Text oder am normalisierten LINQ-Ausdruck, und bei Row Level Security steht der Mandant in keinem von beiden: Die Policy liest ihn aus der Session.

Schritt	Was passiert
Mandant A fragt ab	Die Datenbank filtert nach dem Session-Wert, der Cache speichert die Zeilen von A unter dem SQL-Text
Mandant B stellt dieselbe Abfrage	Gleicher SQL-Text, gleicher Schlüssel: Der Cache liefert die Zeilen von A. Die Datenbank wird nie gefragt, die Policies laufen nie

Die übrigen Schutzmechanismen des Plugins helfen hier auch nicht. Der SaveChanges-Interceptor sieht nur das Speichern. Die Prüfung beim Laden (Kapitel 17) läuft nur, wenn EF Entitäten aus Zeilen

baut; ein Cache, der Entitäten oder DTOs speichert, gibt sie ohne diesen Schritt heraus. Ein Filter mit dem Mandanten als Abfrageparameter (Cosmos DB, Testmodus) bringt den Mandanten nur dann in den Schlüssel, wenn der Cache Parameterwerte einbezieht, und das tut nicht jeder.

> **Der Mandant gehört in jeden Cache-Schlüssel**
>
> Ein Cache vor einer mandantenbezogenen Abfrage ist nur sicher, wenn sein Schlüssel den aktuellen Mandanten enthält, gesetzt von der Anwendung und nicht aus dem SQL abgeleitet. Ohne Mandant wird nichts gecacht. Die meisten Cache-Bibliotheken haben einen Haken für den Schlüssel oder ein Schlüsselpräfix; wo er fehlt, gehört der Cache nicht vor Mandantendaten. Ein Test mit zwei Mandanten und derselben Abfrage zeigt, ob der Haken wirkt.

Beispiel

```
// Überall, wo Abfrageergebnisse gecacht werden: Der Mandant gehört in jeden Schlüssel
public sealed class TenantCacheKeys
{
    private readonly ITenantProvider _tenants;

    public TenantCacheKeys(ITenantProvider tenants) => _tenants = tenants;

    public string For(string queryKey)
        => _tenants.CurrentTenantId is { } tenant
            ? $"tenant:{tenant}:{queryKey}"
            : throw new InvalidOperationException("No tenant: do not cache the result.");
}
```

40 Einsatz in ASP.NET Core

Versionsabhängigkeit: **stabil**

Das Beispiel `samples/ShopApi` zeigt den Weg vom Plugin in eine Web-API. Der Mandant kommt aus dem Claim `tenant_id` des angemeldeten Benutzers:

samples/ShopApi/HttpTenantProvider.cs

```
public sealed class HttpTenantProvider : ITenantProvider
{
    public const string ClaimType = "tenant_id";

    private readonly IHttpContextAccessor _httpContextAccessor;

    public HttpTenantProvider(IHttpContextAccessor httpContextAccessor)
    {
        ArgumentNullException.ThrowIfNull(httpContextAccessor);

        _httpContextAccessor = httpContextAccessor;
    }

    // Bei jedem Zugriff neu ermittelt: funktioniert auch als Singleton (Pooling)
    public TenantId? CurrentTenantId
    {
        get
        {
            var value = _httpContextAccessor.HttpContext?.User.FindFirst(ClaimType)?.Value;
            return Guid.TryParse(value, out var id) ? new TenantId(id) : null;
        }
    }
}
```

samples/ShopApi/Program.cs (Auszug)

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddAcmeTenancy<HttpTenantProvider>();
builder.Services.AddDbContext<ShopApiContext>((sp, options) => options
    .UseNpgsql(builder.Configuration.GetConnectionString("Shop"))
    .UseAcmeTenancy(sp));

// Kein Mandantenfilter im Code: RLS in der Datenbank sorgt für die Trennung
app.MapGet("/orders", async (ShopApiContext db) =>
    await db.Orders.OrderBy(o => o.Id).ToListAsync());
```

> **Woher der Mandant kommt**
>
> Der Mandant muss aus einer vertrauenswürdigen Quelle kommen: einem Claim aus einem signierten Token. Ein Header oder Query-Parameter, den der Client frei setzt, hebelt die gesamte Isolation aus. Das Beispiel enthält eine Header-Simulation nur für die Entwicklungsumgebung und kennzeichnet sie ausdrücklich.

Für `AddDbContextPool` registriert man den Provider als Singleton. Das ist möglich, weil `HttpTenantProvider` den Mandanten bei jedem Zugriff neu ermittelt; die Interceptors fragen ihn bei jeder Verbindung und jedem Speichern:

Variante mit Pooling

```
builder.Services.AddAcmeTenancy<HttpTenantProvider>(ServiceLifetime.Singleton);
builder.Services.AddDbContextPool<ShopApiContext>((sp, options) => options
    .UseNpgsql(connectionString)
    .UseAcmeTenancy(sp));   // sp: Root-Provider; Interceptors entstehen einmal
```

DbContext-Factories

`AddDbContextFactory` ist die übliche Wahl in Blazor Server und für parallele Arbeit. Anders als `AddDbContext` registriert es Factory und Options standardmäßig als **Singletons**. Der Options-Callback bekommt dann den Root-Provider, nicht den Scope der aktuellen Anfrage. Für einen Scoped-Tenant-Provider, der Zustand pro Scope hält, hat das Folgen:

Registrierung	Verhalten
Singleton-Factory, Scope-Validierung an (ASP.NET Core in Development)	Exception: „Cannot resolve scoped service 'ITenantProvider' from root provider.“
Singleton-Factory, Scope-Validierung aus (Standard in Produktion)	Kein Fehler. Die Interceptors bekommen eine Instanz aus dem Root-Container, die keine Anfrage je befüllt: kein Mandant, Speichern wirft, Abfragen sehen nichts
Factory als `Scoped` registriert	Jeder Scope bekommt eine eigene Factory mit eigenem Tenant-Provider: korrekt

Factory mit dem Mandanten der Anfrage

```
builder.Services.AddDbContextFactory<ShopApiContext>(
    (sp, options) => options.UseNpgsql(connectionString).UseAcmeTenancy(sp),
    ServiceLifetime.Scoped);         // Factory und Options pro Scope
```

Ein Provider, der den Mandanten bei jedem Zugriff neu ermittelt, etwa `HttpTenantProvider`, funktioniert in jeder Variante; das Problem betrifft Provider mit Zustand pro Scope. `AddPooledDbContextFactory` verhält sich immer wie Pooling: Die Options existieren einmal, der Tenant-Provider muss also ein solches Singleton sein. `DbContextFactoryTests` zeigt alle drei Varianten.

Außerhalb von HTTP-Anfragen

Hintergrunddienste, Message-Handler und geplante Jobs haben keine Anfrage und keinen Claim. Der Mandant kommt dann aus dem Job selbst, etwa aus der Nachricht oder aus einer Liste zu verarbeitender Mandanten. Der `AmbientTenantProvider` des Beispiels speichert ihn in einem `AsyncLocal`, das mit dem asynchronen Kontrollfluss mitwandert:

samples/ShopSample/AmbientTenantProvider.cs (gekürzt)

```
public sealed class AmbientTenantProvider : ITenantProvider
{
    private static readonly AsyncLocal<TenantId?> Current = new();

    public TenantId? CurrentTenantId => Current.Value;

    public static IDisposable Use(TenantId tenant)
    {
        var previous = Current.Value;
```

```
        Current.Value = tenant;
        return new Restore(previous);     // setzt beim Dispose den vorigen Mandanten zurück
    }

    // private sealed class Restore : IDisposable { ... Current.Value = _previous; }
}
```

Job-Muster (tests/.../BackgroundTenantTests.cs)

```
// Erst der Mandant, dann Scope und Context: die Interceptors entstehen im Scope
using (AmbientTenantProvider.Use(tenant))
{
    await using var scope = scopeFactory.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<ShopContext>();
    // ... Arbeit für diesen Mandanten ...
}
```

Weil der Provider das `AsyncLocal` bei jedem Zugriff liest, kann er als Singleton registriert werden. Parallele Jobs behalten ihren eigenen Mandanten, und nach dem `using`-Block bleibt kein Mandant zurück; `BackgroundTenantTests` prüft beides mit 20 parallelen Jobs. Eine Anwendung mit Web-Anfragen *und* Jobs kombiniert beide Quellen in einem Provider: zuerst der Claim, sonst der Ambient-Mandant.

Nachrichten. Bei einem Message Broker (RabbitMQ, Kafka, Azure Service Bus) oder einem Framework darauf (MassTransit, Wolverine) schreibt der Sender den aktuellen Mandanten in einen Header der Nachricht, und die Empfängerseite setzt ihn, bevor der Consumer läuft. Der Ort dafür ist der Consume-Filter oder die Middleware des Frameworks, damit kein einzelner Consumer es vergessen kann:

Consume-Filter (vereinfacht, unabhängig vom Framework)

```
// Wo der Broker eine Nachricht übergibt: Mandant aus dem Header, dann Scope und Context
public async Task HandleAsync(BrokerMessage message, CancellationToken cancellationToken)
{
    if (!message.Headers.TryGetValue("X-Tenant-Id", out var value)
        || !Guid.TryParse(value, out var id))
        throw new InvalidOperationException("Message without a tenant: rejected.");

    using (AmbientTenantProvider.Use(new TenantId(id)))
    {
        await using var scope = scopeFactory.CreateAsyncScope();
        var handler = scope.ServiceProvider.GetRequiredService<IOrderHandler>();
        await handler.HandleAsync(message, cancellationToken);   // nutzt ShopContext
    }
}
```

Zwei Regeln übertragen sich von den Jobs. Der Header wird gelesen, bevor der Scope entsteht, denn Context und Interceptors leben im Scope. Und eine Nachricht ohne gültigen Mandanten wird abgewiesen und landet in der Dead-Letter-Queue; sie wird nie für alle Mandanten verarbeitet. Ein Scoped-Tenant-Provider mit Setter funktioniert genauso, solange der Filter den Mandanten im Scope des Consumers setzt, bevor der Context aufgelöst wird.

> **Nie auf „alle Mandanten“ zurückfallen**
>
> Ohne Mandant muss ein Job scheitern oder nichts sehen, niemals alles. Das Plugin verhält sich so: Der Session-Wert bleibt leer, die Policies treffen keine Zeile, und Speichern wirft. Jobs, die wirklich alle

> Mandanten brauchen, nutzen die Reporting-Rolle aus Kapitel 36.

41 CI und API-Stabilität

Versionsabhängigkeit: **stabil**

Der Workflow

Der GitHub-Actions-Workflow im Repository setzt die Empfehlungen aus Kapitel 26 und 27 um. Ein Job baut und testet ohne und mit Datenbank, ein zweiter baut das Handbuch:

`.github/workflows/ci.yml (gekürzt)`

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: global.json }
      - run: dotnet build -c Release
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.IntegrationTests
          -c Release --no-build
      - run: dotnet test tests/Acme.EntityFrameworkCore.Tenancy.SqlServer.Tests
          -c Release --no-build

  handbook:                           # beide PDFs mit LuaLaTeX, im Ordner docs/handbook
    runs-on: ubuntu-latest
    container: texlive/texlive@sha256:540d…   # TeX Live 2026, per Digest fixiert
    steps:
      # ... checkout ...
      - run: python3 check_code.py       # Zeilenlänge, Auszüge gegen die Quellen
      - run: python3 check_structure.py  # beide Fassungen gleich, Verweise, Tabellen
      - run: latexmk EF-Core-Plugin-Handbuch.tex

  consumer:                           # die Pakete als Verbraucher prüfen
    steps:
      - run: eng/consumer-test/run.sh --nativeaot
```

Codeauszüge veralten leise, wenn sich der Code ändert. Deshalb bindet eine Kommentarzeile `% source: <Pfad>` direkt vor dem Listing einen Auszug an seine Quelldatei, und `check_code.py` vergleicht beide: Kommentare und Auslassungen (`...`) zählen nicht, sodass die deutsche Fassung Kommentare übersetzen und Auszüge kürzen darf; jede übrige Zeile muss in derselben Reihenfolge in der Quelle stehen, Zeilenumbrüche dürfen abweichen. Mit dem Zusatz `(strings)` bleiben auch übersetzte Meldungen außen vor. Ein Auszug, dessen Titel eine Datei des Repositorys nennt und der nicht gebunden ist, muss „vereinfacht“ heißen. `check_structure.py` prüft, dass beide Fassungen pro Kapitel dieselben Abschnitte, Listings, Bindungen, Tabellen und Hinweisboxen haben, dass jedes Verweisziel existiert und die Spaltenbreiten jeder Tabelle aufgehen.

Das Beispiel baut nur gegen stabile Versionen (.NET 10, EF Core 10). Ein zweiter Workflow, `compatibility.yml`, läuft wöchentlich und testet gegen die neuesten stabilen 10.0.x-Patches, denn die reguläre CI baut gegen den Anfang des Versionsbereichs. Einen Kanal für die nächste EF-Major-Version mit Previews und RCs beschreibt Kapitel 27 als Skizze; er braucht das passende

Vorab-SDK, weil die Vorabversionen von EF Core 11 nur `net11.0` anbieten. Das ist eine Eigenschaft dieser Version, keine allgemeine Regel: EF Core 9 zielte auf .NET 8.

Öffentliche API absichern

Ein Plugin hat zwei öffentliche APIs: die für Anwender (Fluent API, Options, Typen) und die für Erweiterer (virtuelle Methoden, Dependencies, öffentliche Dienste). Beide brechen leicht unbemerkt. Der `Microsoft.CodeAnalysis.PublicApiAnalyzers` führt die öffentliche API in zwei Textdateien (`PublicAPI.Shipped.txt`, `PublicAPI.Unshipped.txt`) und meldet jede Abweichung. Jede API-Änderung wird so im Pull Request als Diff sichtbar.

> **Einrichtung**
>
> Beim ersten Build meldet der Analyzer jede öffentliche API als RS0016, weil die Dateien leer sind. Der Code-Fix „Add all items ... to the public API“ füllt sie, auf der Kommandozeile mit `dotnet format analyzers --diagnostics RS0016 --severity warn`. Beim Release wandert der Inhalt von Unshipped nach Shipped; ab dann ist das Entfernen einer ausgelieferten API ein sichtbarer Breaking Change.

> **Warnungen reichen nicht**
>
> Solange RS0016 und RS0017 nur Warnungen sind, gehen sie in der Build-Ausgabe unter. Im Beispiel blieben die Dateien so bis Ausgabe 1.0 leer: 178 Warnungen bei einem vollständigen Build, keine bei inkrementellen, und niemand bemerkte es. `Directory.Build.props` macht beide deshalb zu Fehlern (`WarningsAsErrors`); eine neue öffentliche API muss jetzt in `PublicAPI.Unshipped.txt` eingetragen werden, sonst bricht der Build.

Teil X

Tiefe Erweiterungen

Eigene SQL-Ausdrücke, Aggregate, Laufzeit-Metadaten und Scaffolding: die Werkzeuge für sehr komplexe Plugins.

- Kapitel 42: Eigene SQL-Ausdrucksknoten
- Kapitel 43: Aggregat-Translators
- Kapitel 44: Fluent API auf Property-Ebene und Laufzeit-Annotations
- Kapitel 45: Scaffolding in der Praxis

42 Eigene SQL-Ausdrucksknoten

Versionsabhängigkeit: **volatil**

Kapitel 14 baut SQL aus vorhandenen Bausteinen: Funktionen, Konstanten, Casts. Das reicht für die meisten Plugins. Ein **eigener Ausdrucksknoten** (eine Ableitung von `SqlExpression`) wird nötig, wenn mindestens eins davon zutrifft:

Grund	Beispiel
Die Syntax ist kein Funktionsaufruf	Operatoren wie `@@`, `<->`, `IS DISTINCT FROM`, Schlüsselwörter wie `AT TIME ZONE`
Die Nullability-Semantik weicht ab	`IS DISTINCT FROM` ist nie NULL, auch wenn ein Operand NULL ist
Andere Teile sollen den Knoten wiedererkennen	Optimierungen, die nur für diesen Ausdruck gelten

Das Beispiel implementiert den null-sicheren Vergleich `EF.Functions.IsNotDistinctFrom(a, b)` und `IsDistinctFrom(a, b)` in einem eigenen kleinen Paket, `Acme.EntityFrameworkCore.Functions.PostgreSQL`, aktiviert mit `UseNpgsql(cs, o => o.UseAcmeFunctions())`. Npgsql bietet den Vergleich nicht als Funktion an; er erfüllt alle drei Gründe.

> **Ein Lehrbeispiel, bewusst ohne Mandantenbezug**
>
> Der Vergleich hat nichts mit Mandantenfähigkeit zu tun. Er liegt in einem eigenen Beispielpaket, weil ein eigener Knoten ein Plugin zwingt, zwei exklusive Provider-Dienste zu ersetzen: den SQL-Generator und den Nullability-Prozessor. Bis Ausgabe 1.1 tat das Mandanten-Plugin das allein für dieses Beispiel und ließ sich dadurch schwerer mit anderen Plugins kombinieren. Ein Mandanten-Plugin braucht keinen eigenen Knoten; Funktionskomposition (Kapitel 14) reicht. Die Tests betreiben beide Plugins nebeneinander, und das pgvector-Beispiel am Ende dieses Kapitels ergänzt ein drittes, das vom Generator des Funktionen-Plugins ableitet.

Anatomie eines Knotens

Mitglied	Zweck	Folge, wenn es fehlt
`VisitChildren` / `Update`	Visitors können Kinder ersetzen; unveränderte Kinder ergeben dieselbe Instanz	EF wirft beim ersten Visitor
`Equals` / `GetHashCode`	Strukturvergleich für den Query-Cache und Optimierungen	Cache-Fehltreffer, falsche Deduplizierung
`Print`	Lesbare Darstellung in DebugView und Fehlermeldungen	Unklare Fehler
`Quote`	C#-Code für vorkompilierte Abfragen und NativeAOT	Kein AOT-Support
SQL-Generator	Schreibt den Knoten als SQL	**Stillschweigend** fehlendes SQL
Nullability-Prozessor	Kennt die NULL-Semantik	Exception beim Übersetzen

Die beiden letzten Zeilen betreffen auch die Kombination mit anderen Plugins: Ersetzt ein zweites

Plugin den Generator oder den Prozessor, fehlt das SQL des Knotens still oder die Abfrage wirft. `CompositionMatrixTests` zeigt beides (Kapitel 6).

Query/Expressions/DistinctFromExpression.cs (gekürzt)

```
public class DistinctFromExpression : SqlExpression
{
    private static ConstructorInfo? _quotingConstructor;

    public DistinctFromExpression(SqlExpression left, SqlExpression right, bool negated,
        RelationalTypeMapping? typeMapping)
        : base(typeof(bool), typeMapping)
    {
        // ... ThrowIfNull ...
        Left = left;
        Right = right;
        IsNegated = negated;
    }

    public virtual SqlExpression Left { get; }
    public virtual SqlExpression Right { get; }
    public virtual bool IsNegated { get; }

    protected override Expression VisitChildren(ExpressionVisitor visitor)
        => Update((SqlExpression)visitor.Visit(Left), (SqlExpression)visitor.Visit(Right));

    public virtual DistinctFromExpression Update(SqlExpression left, SqlExpression right)
        => left == Left && right == Right
            ? this
            : new DistinctFromExpression(left, right, IsNegated, TypeMapping);

    public override Expression Quote()
        => New(
            _quotingConstructor ??= typeof(DistinctFromExpression).GetConstructor(
                [typeof(SqlExpression), typeof(SqlExpression), typeof(bool),
                 typeof(RelationalTypeMapping)])!,
            Left.Quote(), Right.Quote(), Constant(IsNegated),
            RelationalExpressionQuotingUtilities.QuoteTypeMapping(TypeMapping));

    protected override void Print(ExpressionPrinter expressionPrinter)
    {
        expressionPrinter.Visit(Left);
        expressionPrinter.Append(IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        expressionPrinter.Visit(Right);
    }

    public override bool Equals(object? obj)
        => obj is DistinctFromExpression other
           && (ReferenceEquals(this, other)
               || base.Equals(other) && Left.Equals(other.Left)
                  && Right.Equals(other.Right) && IsNegated == other.IsNegated);

    public override int GetHashCode()
        => HashCode.Combine(base.GetHashCode(), Left, Right, IsNegated);
}
```

Der Translator erzeugt den Knoten und gleicht dabei die Type Mappings beider Seiten an: Hat eine Seite ein Mapping (etwa eine Spalte), bekommt die andere dasselbe. So wird aus einer Konstante das

passende Literal:

Query/DistinctFromTranslator.cs (Auszug)

```
var definition = method.GetGenericMethodDefinition();
if (definition != IsNotDistinctFromMethod && definition != IsDistinctFromMethod)
    return null;

var left = arguments[1];                        // arguments[0] ist DbFunctions
var right = arguments[2];
var typeMapping = ExpressionExtensions.InferTypeMapping(left, right)
                  ?? _typeMappingSource.FindMapping(left.Type);

return new DistinctFromExpression(
    _sql.ApplyTypeMapping(left, typeMapping)!,
    _sql.ApplyTypeMapping(right, typeMapping)!,
    negated: definition == IsDistinctFromMethod,
    _typeMappingSource.FindMapping(typeof(bool)));
```

Den SQL-Generator erweitern

Der `QuerySqlGenerator` schreibt SQL per Visitor. Unbekannte Knoten gibt er an `Expression Visitor.VisitExtension` weiter, das nur die Kinder besucht. Ohne eigenen Override entsteht also kein Fehler, sondern **SQL ohne den Operator**. Das Funktionen-Plugin erbt vom Npgsql-Generator:

Query/Internal/FunctionsNpgsqlQuerySqlGenerator.cs (gekürzt)

```
public class FunctionsNpgsqlQuerySqlGenerator : NpgsqlQuerySqlGenerator
{
    public FunctionsNpgsqlQuerySqlGenerator(QuerySqlGeneratorDependencies dependencies,
        IRelationalTypeMappingSource typeMappingSource,
        bool reverseNullOrderingEnabled, Version postgresVersion)
        : base(dependencies, typeMappingSource, reverseNullOrderingEnabled, postgresVersion)
    {
    }

    protected override Expression VisitExtension(Expression extensionExpression)
        => extensionExpression switch
        {
            DistinctFromExpression e => VisitDistinctFrom(e),
            _ => base.VisitExtension(extensionExpression),
        };

    protected virtual Expression VisitDistinctFrom(DistinctFromExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append(expression.IsNegated ? " IS DISTINCT FROM " : " IS NOT DISTINCT FROM ");
        VisitOperand(expression.Right);
        return expression;
    }

    // Der Knoten als Operand anderer Operatoren: immer in Klammern
    protected override bool RequiresParentheses(SqlExpression outerExpression,
        SqlExpression innerExpression)
        => innerExpression is DistinctFromExpression
           || base.RequiresParentheses(outerExpression, innerExpression);
}
```

Klammern braucht es in zwei Richtungen. `VisitOperand` klammert zusammengesetzte Operanden *im* Knoten. Für den Knoten *als* Operand fragt EF `RequiresParentheses`, und das kennt nur seine eigenen Knoten. Ohne den Override schrieb das Plugin `a IS DISTINCT FROM 'x' <> a IS DISTINCT FROM 'y'`; PostgreSQL bindet `<>` stärker als `IS` und meldet einen Syntaxfehler. `DeepQueryTests` prüft das SQL, `AdvancedTests` die Abfrage gegen PostgreSQL.

Der Generator wird über `IQuerySqlGeneratorFactory` erzeugt, einen Singleton-Einzeldienst. Das Plugin ersetzt ihn durch eine eigene Factory mit denselben Abhängigkeiten wie die von Npgsql (`QuerySqlGeneratorDependencies`, Type-Mapping-Source, `INpgsqlSingletonOptions`).

Den Nullability-Prozessor erweitern

Nach der Übersetzung optimiert EF NULL-Vergleiche. Dafür muss es von jedem Knoten wissen, ob er NULL werden kann. Unbekannte Knoten landen in `VisitCustomSqlExpression`, dessen Basisimplementierung eine Exception wirft. Der Prozessor hängt an einer Kette aus drei Klassen, die alle ersetzt werden müssen:

`Query/Internal/FunctionsNpgsqlSqlNullabilityProcessor.cs (gekürzt)`

```
public class FunctionsNpgsqlSqlNullabilityProcessor : NpgsqlSqlNullabilityProcessor
{
    // Konstruktor: (…ProcessorDependencies, …ProcessorParameters) : base(...)

    protected override SqlExpression VisitCustomSqlExpression(
        SqlExpression sqlExpression, bool allowOptimizedExpansion, out bool nullable)
    {
        if (sqlExpression is DistinctFromExpression distinctFrom)
        {
            // Kinder besuchen (sie können optimiert werden), ihre Nullability ist egal
            var left = Visit(distinctFrom.Left, out _)!;
            var right = Visit(distinctFrom.Right, out _)!;

            nullable = false;                   // IS [NOT] DISTINCT FROM ist nie NULL
            return distinctFrom.Update(left, right);
        }

        return base.VisitCustomSqlExpression(
            sqlExpression, allowOptimizedExpansion, out nullable);
    }
}

public class FunctionsNpgsqlParameterBasedSqlProcessor : NpgsqlParameterBasedSqlProcessor
{
    protected override Expression ProcessSqlNullability(
        Expression selectExpression, ParametersCacheDecorator parametersDecorator)
        => new FunctionsNpgsqlSqlNullabilityProcessor(Dependencies, Parameters)
            .Process(selectExpression, parametersDecorator);
}

// + FunctionsNpgsqlParameterBasedSqlProcessorFactory
//   : IRelationalParameterBasedSqlProcessorFactory
```

Infrastructure/FunctionsOptionsExtension.cs › ApplyServices (Ergänzung)

```
// 4) Tiefe Erweiterung: eigener SQL-Ausdrucksknoten
services.Replace(ServiceDescriptor
    .Singleton<IQuerySqlGeneratorFactory, FunctionsNpgsqlQuerySqlGeneratorFactory>());
services.Replace(ServiceDescriptor
    .Scoped<IRelationalParameterBasedSqlProcessorFactory,
        FunctionsNpgsqlParameterBasedSqlProcessorFactory>());
```

Der Preis eigener Knoten

Eigene Knoten verlangen das Ersetzen zweier Einzeldienste und das Erben von internen Npgsql-Klassen. Zwei Plugins, die beide eigene Knoten einführen, lassen sich nur kombinieren, wenn eines vom anderen erbt. Deshalb gilt: Funktionskomposition zuerst; eigene Knoten nur, wenn einer der drei Gründe vorliegt. Und die Ersetzung gehört an prominente Stelle in die Dokumentation.

Experimentelle API

`Quote` und `RelationalExpressionQuotingUtilities` gehören zur experimentellen API für vorkompilierte Abfragen. Ihre Verwendung meldet der Compiler als EF9100; das Beispiel unterdrückt die Meldung projektweit und dokumentiert das in `Directory.Build.props`.

tests/.../DeepQueryTests.cs (Auszug)

```
[Fact]
public void IsDistinctFrom_in_projection_passes_nullability_processor()
{
    // Ohne eigenen Nullability-Prozessor würde EF hier eine Exception werfen
    using var ctx = Contexts.Create();

    var sql = ctx.Customers
        .Select(c => EF.Functions.IsDistinctFrom(c.Name, (string?)null))
        .ToQueryString();

    Assert.Contains("IS DISTINCT FROM", sql);
}
```

Praxisbeispiel: Operatoren für Vektorsuche und pgvector

Embeddings und Ähnlichkeitssuche bringen Operatoren in SQL, die keine Funktionen sind: pgvector schreibt die L2-Distanz als `<->`, die Kosinus-Distanz als `<=>` und das negative innere Produkt als `<#>`; die Volltextsuche verwendet `@@`. `SqlBinaryExpression` hilft hier nicht, denn ihr Operator ist ein `ExpressionType`, und für `<=>` gibt es keinen. `CustomOperatorTests` zeigt deshalb `EF.Functions.CosineDistance(a, b)` als eigenen Knoten `VectorDistanceExpression` mit dem Operator als Zeichenkette, mit Translator, Generator und Nullability-Prozessor. Es ist Testcode: Eine Anwendung nimmt `Pgvector.EntityFrameworkCore`, das den Typ `vector`, sein Type Mapping und diese Übersetzungen mitbringt. Npgsql selbst hat für solche Fälle `PgUnknownBinaryExpression`, einen Knoten mit beliebigem Operator und propagierender Nullability, der Generator und Prozessor nicht ersetzen muss. Er liegt aber in einem `Internal`-Namespace (EF1001).

Drei Dinge entscheiden darüber, ob ein solcher Operator richtiges SQL ergibt.

Type Mappings. Der Translator gibt beiden Operanden dasselbe Mapping, abgeleitet von der Seite, die eins hat, meist der Spalte. So geht der Parameter mit dem Speichertyp der Spalte an die

Datenbank:

tests/.../CustomOperatorTests.cs › Translator

```
if (method != CosineDistanceMethod)
    return null;

var left = arguments[1];                        // arguments[0] ist DbFunctions
var right = arguments[2];
var operandMapping = ExpressionExtensions.InferTypeMapping(left, right)
                     ?? _typeMappingSource.FindMapping(left.Type);

return new VectorDistanceExpression(
    _sql.ApplyTypeMapping(left, operandMapping)!,
    _sql.ApplyTypeMapping(right, operandMapping)!,
    "<=>",
    _typeMappingSource.FindMapping(typeof(double)));
```

Im Test ist die Spalte ein `float[]`, also `real[]`. pgvector definiert seine Operatoren aber nur für `vector`; gegen eine echte Datenbank braucht es deshalb ein Type Mapping mit diesem Speichertyp, wie es das Paket registriert. Das Ergebnis bekommt das Mapping von `double`, also `double precision`.

Rangfolge und Klammern. EF kennt die Rangfolge eines fremden Operators nicht. PostgreSQL ordnet selbst definierte Operatoren zwischen die arithmetischen und die Vergleichsoperatoren ein: `a <=> b * 2` heißt `a <=> (b * 2)`. Ohne Gegenmaßnahme schrieb der Generator für `Cosine Distance(...) * 2 + 1` genau das. Die Lösung sind dieselben zwei Richtungen wie oben: Operanden im Knoten klammern und den Knoten als Operand über `RequiresParentheses` immer klammern. `Print` klammert genauso, damit die DebugView zeigt, was die Datenbank liest.

tests/.../CustomOperatorTests.cs › Generator

```
public class VectorQuerySqlGenerator : FunctionsNpgsqlQuerySqlGenerator
{
    // ... Konstruktor, VisitExtension, VisitOperand

    protected virtual Expression VisitVectorDistance(VectorDistanceExpression expression)
    {
        VisitOperand(expression.Left);
        Sql.Append($" {expression.Operator} ");
        VisitOperand(expression.Right);
        return expression;
    }

    // Ohne Override: "a <=> b * 2.0", gelesen als a <=> (b * 2.0)
    protected override bool RequiresParentheses(SqlExpression outerExpression,
        SqlExpression innerExpression)
        => innerExpression is VectorDistanceExpression
           || base.RequiresParentheses(outerExpression, innerExpression);
}
```

Der Generator erbt vom Generator des Plugins, nicht von dem von Npgsql: Beide ersetzen denselben Einzeldienst, und nur so bleiben beide Knoten übersetzbar. Das ist der Preis aus dem Kasten oben, im Kleinen.

Nullability. Ein gewöhnlicher Operator ist NULL, sobald ein Operand NULL ist. Der Prozessor meldet das weiter:

tests/.../CustomOperatorTests.cs › Nullability

```
if (sqlExpression is VectorDistanceExpression distance)
{
    // NULL hinein, NULL heraus
    var left = Visit(distance.Left, out var leftNullable)!;
    var right = Visit(distance.Right, out var rightNullable)!;

    nullable = leftNullable || rightNullable;
    return distance.Update(left, right);
}
```

Die Meldung hat sichtbare Folgen. Für `!(CosineDistance(...) < 0.5)` schreibt EF bei einer Pflichtspalte `>= 0.5`, bei einer optionalen Spalte ein `CASE`, weil `NOT (NULL < 0.5)` in SQL NULL ist, in C# aber `true`. Ohne den Prozessor scheitert die Übersetzung. Die Tests prüfen das SQL aller Fälle:

tests/.../CustomOperatorTests.cs (Auszug)

```
var sql = ctx.Documents
    .Select(d => EF.Functions.CosineDistance(d.Embedding, query) * 2 + 1)
    .ToQueryString();

Assert.Contains("SELECT (d.\"Embedding\" <=> @query) * 2.0 + 1.0", sql);
```

43 Aggregat-Translators

Versionsabhängigkeit: **mittel**

Aggregate wie `Sum` oder `string_agg` fassen mehrere Zeilen zusammen, typischerweise nach `GroupBy`. Ein `IAggregateMethodCallTranslator` bekommt statt eines einzelnen Arguments eine `EnumerableExpression`, die beschreibt, *was* aggregiert wird:

Eigenschaft	Bedeutung	Herkunft in LINQ
`Selector`	Der aggregierte Ausdruck	`g.Select(o => o.Total > 0)`
`IsDistinct`	Nur verschiedene Werte	`g.Select(...).Distinct()`
`Predicate`	Filter vor dem Aggregat	`g.Where(o => o.Id > 10)`
`Orderings`	Reihenfolge (für reihenfolgeabhängige Aggregate)	`g.OrderBy(...)`

Das Funktionen-Beispielpaket aus Kapitel 42 ergänzt `EF.Functions.Every(values)` für PostgreSQLs `bool_and`; wie der Vergleich ist es ein Lehrbeispiel ohne Mandantenbezug:

`Query/EveryAggregateTranslator.cs (Auszug)`

```
public virtual SqlExpression? Translate(MethodInfo method, EnumerableExpression source,
    IReadOnlyList<SqlExpression> arguments,
    IDiagnosticsLogger<DbLoggerCategory.Query> logger)
{
    if (method != EveryMethod || source.Selector is not SqlExpression selector)
        return null;

    // Npgsqls Factory setzt DISTINCT, FILTER (WHERE ...) und ORDER BY im Aggregat um
    if (_sql is not NpgsqlSqlExpressionFactory npgsql)
        return null;

    var boolean = _typeMappingSource.FindMapping(typeof(bool))!;

    // bool_and liefert für eine leere Menge NULL, Enumerable.All liefert true
    return _sql.Coalesce(
        npgsql.AggregateFunction("bool_and", [selector], source,
            nullable: true, argumentsPropagateNullability: [false],
            typeof(bool), boolean),
        _sql.Constant(true, boolean),
        boolean);
}
```

`Erzeugtes SQL für g.Where(o => o.Id > 10).Select(o => o.Total > 0)`

```
SELECT COALESCE(bool_and(o."Total" > 0.0) FILTER (WHERE o."Id" > 10), TRUE)
FROM "Orders" AS o
GROUP BY o."TenantId"
```

> **Semantik vor Syntax**
>
> Ein Aggregat-Translator muss die Semantik der .NET-Seite treffen, nicht nur die Syntax der Datenbank. Typische Fallen: leere Mengen (NULL statt 0, true oder Leerstring), NULL-Werte in der Eingabe und

> Prädikate. Npgsql selbst gleicht etwa `string.Join` mit `COALESCE` an `string_agg` an; das Beispiel folgt diesem Muster.

Datenbanken ohne `FILTER`-Klausel (etwa SQL Server) emulieren Prädikate mit `CASE WHEN prädikat THEN wert END`. Ein provider-neutraler Aggregat-Translator muss beides kennen; deshalb liegt dieser im PostgreSQL-Paket.

44 Fluent API auf Property-Ebene und Laufzeit-Annotations

Versionsabhängigkeit: **volatil**

Property-Builder

Bisher setzt das Plugin voraus, dass die Mandanten-Property `TenantId` heißt und `ITenantOwned` implementiert ist. Mit einer Fluent-API-Methode auf `PropertyBuilder` geht es ohne beides, etwa bei bestehenden Modellen mit einer Property `MandantId` oder einer expliziten Interface-Implementierung:

Anwendung

```
modelBuilder.Entity<Contract>().Property(c => c.Mandant).IsTenantProperty();
```

Metadata/TenancyPropertyBuilderExtensions.cs (Kernpaket)

```
public static PropertyBuilder IsTenantProperty(
    this PropertyBuilder builder, bool isTenant = true)
{
    ArgumentNullException.ThrowIfNull(builder);

    if (isTenant)
        builder.Metadata.SetAnnotation(TenancyAnnotationNames.IsTenantProperty, true);
    else
        builder.Metadata.RemoveAnnotation(TenancyAnnotationNames.IsTenantProperty);

    return builder;
}

public static PropertyBuilder<TProperty> IsTenantProperty<TProperty>(
    this PropertyBuilder<TProperty> builder, bool isTenant = true)
    => (PropertyBuilder<TProperty>)IsTenantProperty((PropertyBuilder)builder, isTenant);

public static IConventionPropertyBuilder? IsTenantProperty(
    this IConventionPropertyBuilder builder,
    bool isTenant = true,
    bool fromDataAnnotation = false)
    => builder.HasAnnotation(TenancyAnnotationNames.IsTenantProperty,
           isTenant ? true : null, fromDataAnnotation) is null
        ? null
        : builder;
```

Die Methode liegt im Kernpaket, weil sie nur Metadaten setzt. Die Conventions beider Provider-Pakete suchen die Mandanten-Property über dieselbe Kernfunktion:

Metadata/TenancyEntityTypeExtensions.cs (Kernpaket, gekürzt, vereinfacht)

```
public static IReadOnlyProperty? FindTenantPropertyCandidate(
    this IReadOnlyEntityType entityType)
{
    var marked = entityType.GetProperties()
```

```
            .Where(p => p[TenancyAnnotationNames.IsTenantProperty] is true)
            .ToList();

        if (marked.Count > 1)
            throw new InvalidOperationException("... mehrere Mandanten-Properties ...");
        if (marked.Count == 1)
            return marked[0];

        return typeof(ITenantOwned).IsAssignableFrom(entityType.ClrType)
            ? entityType.FindProperty(nameof(ITenantOwned.TenantId))
            : null;
    }
```

Builder	Typische Plugin-Methoden	Hinweis
`PropertyBuilder` / `<T>`	Spaltenoptionen, Rollen von Properties	Hier: `IsTenantProperty`
`IndexBuilder` / `<T>`	Index-Methoden, Operator-Klassen, Filter	Annotation an `Metadata` (IMutableIndex)
`KeyBuilder`	Schlüsseloptionen (Clustering)	Wie oben
`ComplexTypePropertyBuilder`	Properties in Complex Types	Nur anbieten, wenn die Option dort Sinn ergibt (Kapitel 33)
`ModelBuilder`	Modellweite Optionen	Annotation am Modell

Laufzeit-Annotations

Der SaveChanges-Interceptor muss zur Laufzeit wissen, welche Entitäten isoliert sind und wie ihre Mandanten-Property heißt. Die Convention schreibt das in eine Annotation am Entity-Typ; der Interceptor liest sie aus dem *Laufzeitmodell*:

Convention und Kernpaket (Auszug)

```
// Convention (Design-Time-Modell)
entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, property.Name);

// Interceptor (Laufzeitmodell)
public static string? GetTenantPropertyName(this IReadOnlyEntityType entityType)
    => entityType.GetRootType()[TenancyAnnotationNames.TenantProperty] as string;
```

Zwei Fragen entscheiden, ob das funktioniert. Landet die Annotation im Laufzeitmodell? Ja: `RuntimeModelConvention` entfernt nur EFs eigene Design-Time-Annotations und übernimmt alle anderen. Landet sie im Compiled Model? Ja: Einfache Werte schreibt EF automatisch (Kapitel 24). Der eigene `ICSharpRuntimeAnnotationCodeGenerator` hat deshalb die umgekehrte Aufgabe: Er entfernt die RLS-Annotations, die nur Migrations brauchen:

Design/Internal/TenancyCSharpRuntimeAnnotationCodeGenerator.cs (gekürzt, vereinfacht)

```
public class TenancyCSharpRuntimeAnnotationCodeGenerator
    : NpgsqlCSharpRuntimeAnnotationCodeGenerator
{
    // Konstruktor: (CSharpRuntimeAnnotationCodeGeneratorDependencies,
    //               RelationalCSharpRuntimeAnnotationCodeGeneratorDependencies) : base(...)

    public override void Generate(IEntityType entityType,
```

```
        CSharpRuntimeAnnotationCodeGeneratorParameters parameters)
    {
        if (!parameters.IsRuntime)
        {
            foreach (var key in parameters.Annotations.Keys
                         .Where(k => k.StartsWith(RlsAnnotationNames.Prefix,
                                                  StringComparison.Ordinal))
                         .ToList())
            {
                parameters.Annotations.Remove(key);
            }
        }

        base.Generate(entityType, parameters);
    }
}
```

Wird das Compiled Model für NativeAOT erzeugt (`ForNativeAot`), schreibt EF zusätzlich das relationale Modell. Der Annotation-Provider hat die RLS-Annotations auf die Tabellen kopiert, sie tauchen dort also ein zweites Mal auf. Der Generator wendet deshalb denselben Filter in einer Überschreibung von `Generate(ITable, ...)` an.

Annotation	Design-Time	Laufzeit	Compiled Model
`Acme:Rls:*` (Policies)	ja (Migrations)	ja, ungenutzt	gefiltert
`Acme:Tenancy:TenantProperty`	ja	ja (Interceptor)	ja
`Acme:Tenancy:IsTenantProperty`	ja (Convention)	ja, ungenutzt	ja (klein)

Laufzeit-Annotation und Runtime-Annotation

„Laufzeit-Annotation“ heißt in diesem Buch: eine gewöhnliche Annotation, die zur Laufzeit gelesen wird. EF kennt zusätzlich eine eigene Runtime-Annotation-API (`AddRuntimeAnnotation`, `GetOrAddRuntimeAnnotationValue` auf `IAnnotatable`). Sie legt Werte erst am fertigen Modell ab, etwa abgeleitete Daten, die nicht bei jedem Zugriff neu berechnet werden sollen. Solche Werte erscheinen nie in Snapshot oder Migrations. Für Konfiguration, die ein Plugin beim Modellbau festlegt, ist die gewöhnliche Annotation der richtige Weg.

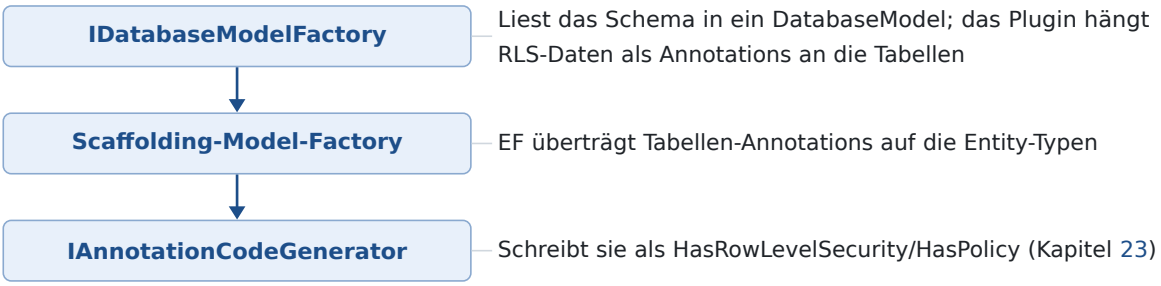
Tipp

Für jede Annotation sollte klar sein, wer sie wann liest. Nur zur Design-Zeit gelesene Annotations gehören nicht ins Compiled Model; zur Laufzeit gelesene müssen hinein und brauchen einen Test gegen das generierte Modell. Der Compiled-Model-Test des Beispiels prüft beides.

45 Scaffolding in der Praxis

Versionsabhängigkeit: **volatil**

Beim Scaffolding (`dotnet ef dbcontext scaffold`) liest EF das Schema und erzeugt Entitäten und Context. Ein Plugin, das eigene Schema-Objekte verwaltet, sollte sie dabei erkennen. Der Weg führt über drei Stationen, von denen das Plugin zwei bereits kennt:



Scaffolding/TenancyDatabaseModelFactory.cs (gekürzt)

```
public class TenancyDatabaseModelFactory : NpgsqlDatabaseModelFactory
{
    public TenancyDatabaseModelFactory(
        IDiagnosticsLogger<DbLoggerCategory.Scaffolding> logger)
        : base(logger)
    {
    }

    public override DatabaseModel Create(
        DbConnection dbConnection, DatabaseModelFactoryOptions options)
    {
        var model = base.Create(dbConnection, options);
        // ... Verbindung öffnen, falls nötig ...
        ReadRowLevelSecurity(dbConnection, model);    // pg_class.relrowsecurity / ...force...
        ReadPolicies(dbConnection, model);            // pg_policies
        return model;
    }

    protected virtual void ReadPolicies(DbConnection connection, DatabaseModel model)
    {
        // SELECT schemaname, tablename, policyname, permissive, roles::text[], cmd,
        //        qual, with_check FROM pg_catalog.pg_policies
        // ...
        var policy = new RlsPolicy(
            Name: reader.GetString(2),
            Command: Enum.Parse<PolicyCommand>(reader.GetString(5), ignoreCase: true),
            Permissive: reader.GetString(3) == "PERMISSIVE",
            Roles: roles is ["public"] ? null : roles,    // "public" = Default ohne TO
            Using: reader.IsDBNull(6) ? null : reader.GetString(6),
            WithCheck: reader.IsDBNull(7) ? null : reader.GetString(7));

        table[RlsAnnotationNames.PolicyPrefix + policy.Name] = policy.Serialize();
    }
}
```

Registriert wird die Factory wie der Code-Generator in den Design-Time-Services mit `RemoveAll`, aber mit `AddScoped`: EF und Npgsql registrieren die Database-Model-Factory als Scoped, und sie hängt vom ebenfalls Scoped Scaffolding-Logger ab. Das Datenformat ist dasselbe wie bei der Fluent API; deshalb funktionieren Snapshot, Migrations und Code-Generator ohne weitere Anpassung.

> **Normalisierte Ausdrücke**
>
> PostgreSQL speichert Policy-Ausdrücke in normalisierter Form, etwa `(("TenantId" = (NULLIF(current_setting('app.tenant_id'::text, true), ''::text))::uuid))`. Gescaffoldete Policies enthalten genau diesen Text. Das ist korrekt, sieht aber anders aus als die von der Convention erzeugte Form. Teams, die nach dem Scaffolding auf Migrations umsteigen, sollten die Policies einmal vereinheitlichen, sonst erzeugt die erste Migration ein DROP/CREATE mit gleichem Inhalt.

tests/...IntegrationTests/AdvancedTests.cs (Auszug)

```
[Fact]
public void Scaffolding_reads_rls_and_policies()
{
    using var ctx = ContextFor(null);
    var factory = new TenancyDatabaseModelFactory(
        ctx.GetService<IDiagnosticsLogger<DbLoggerCategory.Scaffolding>>());

    var model = factory.Create(
        _fixture.OwnerConnectionString, new DatabaseModelFactoryOptions());
    var orders = model.Tables.Single(t => t.Name == "Orders");

    Assert.Equal(true, orders[RlsAnnotationNames.Enabled]);
    Assert.Equal(true, orders[RlsAnnotationNames.Forced]);
}
```

Teil XI

Mehrere Provider

Welche Bausteine in einen gemeinsamen Kern gehören, welche in Provider-Pakete, wie ein zweiter Provider auf demselben Kern entsteht und was bei einer Datenbank ohne SQL trägt.

- Kapitel 46: Was in den Kern gehört
- Kapitel 47: Ein zweiter Provider: SQL Server
- Kapitel 48: Ohne SQL: Was bei Cosmos DB trägt

46 Was in den Kern gehört

Versionsabhängigkeit: **stabil**

Ein zweiter Provider ist der Härtetest für die Paketarchitektur. Er zeigt, welche Teile wirklich provider-unabhängig sind, und der erste Eindruck täuscht: Ein Baustein kann weder SQL-Syntax noch Provider-Typen kennen und trotzdem eine relationale Datenbank brauchen, weil er mit ADO.NET-Verbindungen oder mit Tabellen arbeitet. Das Beispiel hat deshalb drei Schichten, jede in einem eigenen Paket:

Baustein	Schicht	Begründung
`TenantId`, Konverter, JSON-Reader/Writer	Kern	Reine .NET-Typen und EF-Value-Converter
`ITenantOwned`, `ITenantProvider`, Attribute	Kern	Für Entitäts-Assemblies ohne Provider-Abhängigkeit
`IsTenantProperty()`, Kandidatensuche	Kern	Nur Metadaten
Laufzeit-Annotation, `GetTenantPropertyName`	Kern	Von den provider-neutralen Interceptors gelesen
SaveChanges-Interceptor	Kern	Arbeitet nur mit dem Change Tracker
Schutz von Massen-Updates (Query-Expression-Interceptor)	Kern	Prüft nur den LINQ-Ausdrucksbaum
Evaluatable-Filter mit Attribut	Kern	Erkennt Funktionsklassen aller Pakete
Prüfung beim Laden (Materialization-Interceptor)	Kern	Liest nur Modell, Context und Werte der Zeile
Value Generation, Logging, `AddAcmeTenancy`	Kern	Ohne Datenbankbezug
Query-Filter mit dem Mandanten als Parameter, Testmodus	Kern	Brauchen keine Session; genutzt von Cosmos DB und von Anwendungstests
`TenantSessionInterceptor` (abstrakt)	Relational	Arbeitet auf der `DbConnection`; nur der SQL-Befehl unterscheidet sich je Provider
`TenantTableMappingValidator`	Relational	Tabellen und Spalten: TPT, TPC, Entity Splitting
`Database.ApplyTenantSession()`	Relational	Nutzt den Session-Interceptor des Providers
Type Mapping, Translators, Migrations, RLS-Conventions	Provider	Store-Typen, SQL-Syntax, Schema-Objekte
Eigene SQL-Knoten, Aggregate	Provider	Abhängig von Syntax und Provider-Factory

Die Faustregel hat deshalb drei Stufen: **Was keine Datenbank braucht, gehört in den Kern. Was eine relationale Datenbank, aber keinen SQL-Dialekt braucht, gehört in die relationale Schicht. SQL-Syntax und Store-Typen gehören in den Provider.** Das Kernpaket referenziert nur `Microsoft.EntityFrameworkCore`, das Paket `Acme.EntityFrameworkCore.Tenancy.Relational` ergänzt `Microsoft.EntityFrameworkCore.Relational`, und beide Provider-Pakete bauen darauf auf. `PackageLayerTests` stellt sicher, dass kein relationaler Typ in den Kern zurückwandert. Dass sich das lohnt, zeigt Kapitel 48: Das Cosmos-Paket baut allein auf dem Kern auf.

Die relationale Schicht von Anfang an trennen

Wer später einen nicht-relationalen Provider wie Cosmos DB unterstützen könnte, legt die relationale Schicht von Anfang an in ein eigenes Paket. Typen nachträglich in eine andere Assembly zu verschieben bricht die Binärkompatibilität, und Entitäts-Assemblies, die nur den Kern referenziert haben, verlieren die transitive Abhängigkeit auf `Microsoft.EntityFrameworkCore.Relational`, auf die sie sich vielleicht für `ToTable` oder `HasColumnType` verlassen haben. Das Beispiel hat genau diesen Schritt in Ausgabe 1.2 gemacht.

Namenskonflikte zwischen Provider-Paketen

Zwei Provider-Pakete, die beide eine Extension-Method `UseAcmeTenancy` auf `DbContextOptionsBuilder` definieren, sind nicht gleichzeitig referenzierbar: Der Aufruf wäre mehrdeutig. Das PostgreSQL-Paket nutzt den Builder von EF, das SQL-Server-Paket den Builder des Providers (`UseSqlServer(cs, x => x.UseAcmeTenancy())`). Wer mehrere Provider-Pakete plant, sollte von Anfang an konsequent den Provider-Builder verwenden, wie das NetTopologySuite-Plugin.

47 Ein zweiter Provider: SQL Server

Versionsabhängigkeit: **mittel**

Das Paket `Acme.EntityFrameworkCore.Tenancy.SqlServer` setzt dieselbe Idee mit den Mitteln von SQL Server um. Die Unterschiede zeigen, was ein Plugin pro Datenbank tatsächlich neu lösen muss:

Aspekt	PostgreSQL	SQL Server
Mandant in der Session	`set_config(name, wert, false)`	`sp_set_session_context`
`CurrentTenant()` in SQL	`nullif(current_setting(...), '')::uuid`	`CAST(SESSION_CONTEXT(N'…') AS uniqueidentifier)`
Spaltentyp	`uuid`	`uniqueidentifier`
RLS-Mechanismus	`CREATE POLICY` pro Tabelle	Predicate-Funktion + `CREATE SECURITY POLICY`
Lesen / Schreiben	`USING` / `WITH CHECK`	`FILTER` / `BLOCK ... AFTER INSERT/UPDATE`
Tabelleneigentümer	Nur mit `FORCE` gebunden	Kein FORCE; Prädikate sind Teil der Abfrage
`DROP TABLE`, Umbenennen	Policies verschwinden mit bzw. folgen	Policy muss vorher weg und danach neu angelegt werden (SCHEMABINDING)
DDL in Batches	Transaktional, beliebig kombinierbar	`CREATE FUNCTION/SCHEMA` allein im Batch: `EXEC(N'...')`
Tabellenabbildungen	Gleicher Vertrag aus dem relationalen Paket (Kapitel 10, Seite 51)	
Bewusste Ausnahme	Attribut oder `HasNoRowLevelSecurity()`	Nur das Attribut

Aktivierung über den Provider-Builder

```
services.AddAcmeTenancy<HttpTenantProvider>();                    // Kernpaket
services.AddDbContext<ShopContext>((sp, o) => o
    .UseSqlServer(connectionString, sql => sql.UseAcmeTenancy(sp)));

public static SqlServerDbContextOptionsBuilder UseAcmeTenancy(
    this SqlServerDbContextOptionsBuilder sqlServerOptionsBuilder,
    string sessionKey = "TenantId",
    string policySchema = "acme")
{
    // Vom Provider-Builder zum eigentlichen DbContextOptionsBuilder
    var optionsBuilder =
        ((IRelationalDbContextOptionsBuilderInfrastructure)sqlServerOptionsBuilder)
        .OptionsBuilder;

    var extension = optionsBuilder.Options.FindExtension<SqlServerTenancyOptionsExtension>()
                    ?? new SqlServerTenancyOptionsExtension();
    // ... Settings validieren und setzen ...
    ((IDbContextOptionsBuilderInfrastructure)optionsBuilder)
        .AddOrUpdateExtension(extension.WithSettings(settings));

    return sqlServerOptionsBuilder;
```

```
}
```

Migrations: Security Policy statt Policy-Ausdruck

Die Convention vermerkt für SQL Server nur die Mandantenspalte; das SQL entsteht im Generator. Die Predicate-Funktion ist für alle Tabellen dieselbe und wird idempotent angelegt:

Migrations/Internal/SqlServerTenancyMigrationsSqlGenerator.cs (Auszug, vereinfacht)

```
protected virtual void CreatePolicy(string table, string? schema, string column,
    MigrationCommandListBuilder builder)
{
    var function = Delimit(PredicateFunctionName, Settings.PolicySchema);
    var target = Delimit(table, schema);
    var predicate = $"{function}({Delimit(column)})";

    // CREATE SCHEMA/FUNCTION müssen allein im Batch stehen -> EXEC(N'...')
    Statement(builder, $"IF SCHEMA_ID({Literal(Settings.PolicySchema)}) IS NULL "
        + $"EXEC({Literal($"CREATE SCHEMA {Delimit(Settings.PolicySchema)}")})");

    var functionSql =
        $"CREATE FUNCTION {function}(@TenantId uniqueidentifier) "
        + "RETURNS TABLE WITH SCHEMABINDING AS RETURN SELECT 1 AS [Allowed] "
        + $"WHERE @TenantId = CAST(SESSION_CONTEXT({Literal(Settings.SessionKey)}) "
        + "AS uniqueidentifier)";
    Statement(builder, $"IF OBJECT_ID({Literal(function)}, N'IF') IS NULL "
        + $"EXEC({Literal(functionSql)})");

    Statement(builder,
        $"CREATE SECURITY POLICY {PolicyName(table, schema ?? "dbo")} "
        + $"ADD FILTER PREDICATE {predicate} ON {target}, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER INSERT, "
        + $"ADD BLOCK PREDICATE {predicate} ON {target} AFTER UPDATE "
        + "WITH (STATE = ON)");
}

protected override void Generate(DropTableOperation operation, IModel? model,
    MigrationCommandListBuilder builder, bool terminate = true)
{
    // SCHEMABINDING: zuerst die Policy, gefunden über die Tabelle, die sie bindet
    DropPolicy(operation.Name, TableSchema(operation.Schema, model), builder);
    base.Generate(operation, model, builder, terminate);
}
```

`SCHEMABINDING` akzeptiert nur zweiteilige Namen. Eine Tabelle ohne explizites Schema qualifizieren die Aufrufer deshalb mit dem Default-Schema des Modells oder, falls keines gesetzt ist, mit `dbo`; sonst lehnt SQL Server die Security Policy ab.

> **Verschachteltes SQL**
>
> Das `EXEC(N'...')` enthält SQL in einem String-Literal, das wiederum Literale enthält. Die Verschachtelung entsteht ausschließlich über `GenerateSqlLiteral` des String-Mappings, das Anführungszeichen korrekt verdoppelt; Bezeichner laufen über `DelimitIdentifier`. Session-Schlüssel und Schema werden zusätzlich beim Konfigurieren validiert.

Schemaänderungen an geschützten Tabellen

`SCHEMABINDING` schützt die Predicate-Funktion, bindet aber auch die Tabelle. SQL Server lehnt `sp_rename` für Tabelle und Mandantenspalte, `ALTER SCHEMA ... TRANSFER`, `ALTER COLUMN` und `DROP COLUMN` ab, solange eine Security Policy die Tabelle bindet: „object participates in enforced dependencies“. Pro Operation lässt sich das nicht lösen. EF sortiert die Operationen einer Migration selbst, und beim Umbenennen der Mandantenspalte kommt die `RenameColumnOperation` *vor* der `AlterTableOperation`, die die geänderte Annotation trägt. Eine reine Tabellenumbenennung erzeugt gar keine `AlterTableOperation`.

Der Generator überschreibt deshalb den Einstieg, der die ganze Operationsliste bekommt. Er sucht alle Tabellen, deren gebundene Objekte sich ändern, verfolgt ihre Umbenennungen und umklammert die Liste. Vorne entfernt er die Policies der betroffenen Tabellen unter ihren alten Namen. Hinten legt er sie mit den endgültigen Tabellen-, Schema- und Spaltennamen aus dem Zielmodell neu an:

Generiertes SQL beim Umbenennen der Mandantenspalte (gekürzt)

```
DECLARE @policy nvarchar(400) = (SELECT TOP (1) ... FROM sys.security_policies AS p
    WHERE EXISTS (SELECT 1 FROM sys.security_predicates AS s WHERE s.object_id = p.object_id
        AND s.target_object_id = OBJECT_ID(N'[dbo].[Invoice]')
        AND s.predicate_definition LIKE N'%[[]fn[_]tenant[_]predicate]%'));
IF @policy IS NOT NULL EXEC(N'DROP SECURITY POLICY ' + @policy);
EXEC sp_rename N'[Invoice].[TenantId]', N'Mandant', 'COLUMN';
EXEC sp_rename N'[Invoice].[IX_Invoice_TenantId]', N'IX_Invoice_Mandant', 'INDEX';
IF SCHEMA_ID(N'acme') IS NULL EXEC(N'CREATE SCHEMA [acme]');
IF OBJECT_ID(N'[acme].[fn_tenant_predicate]', N'IF') IS NULL EXEC(N'CREATE FUNCTION ...');
CREATE SECURITY POLICY [acme].[tenant_dbo_Invoice]
    ADD FILTER PREDICATE [acme].[fn_tenant_predicate]([Mandant]) ON [dbo].[Invoice], ...
```

Drei Entscheidungen stecken darin:

- **Transaktion statt Ausschalten.** EF führt die Befehle einer Migration in einer Transaktion aus, und DDL ist in SQL Server transaktional. Scheitert ein Schritt, rollt SQL Server auch das Entfernen der Policy zurück. Eine dauerhaft entfernte oder mit `STATE = OFF` abgeschaltete Policy kann so nicht entstehen. Der Down-Weg ist dieselbe Umklammerung mit vertauschten Namen.
- **Entfernen über die Bindung, nicht über den Namen.** Gelöscht wird nur eine Policy, deren Prädikat die Predicate-Funktion des Plugins verwendet und die genau diese Tabelle bindet. Fremde Policies und Tabellen ohne RLS bleiben unberührt, und Policies unter einem früheren Namen werden trotzdem gefunden.
- **Schemaqualifizierte Namen.** Bis 1.0 hieß die Policy `tenant_Tabelle`, sodass gleichnamige Tabellen in zwei Schemas kollidierten. Neue Policies heißen `tenant_Schema_Tabelle`. Bestehende Installationen behalten den alten Namen, bis die Tabelle das nächste Mal angefasst wird; dann ersetzt der Generator die Policy. Die verbleibende Mehrdeutigkeit (Schema `a_b` mit Tabelle `c` gegen Schema `a` mit Tabelle `b_c`) und Namen über 128 Zeichen lehnt die Convention beim Modellbau ab.

Grenzen: Das Wiederherstellen braucht das Zielmodell. Das ist der Normalfall, weil `dotnet ef migrations add` zu jeder Migration eine Designer-Datei erzeugt. Fehlt es, oder kommt eine geänderte Tabelle darin nicht vor, bricht der Generator ab, statt eine Policy fallen zu lassen. Eigene schemagebundene Objekte der Anwendung (Views mit `SCHEMABINDING`, eigene Security Policies) behandelt das Plugin nicht; sie blockieren dieselben Operationen weiterhin. Datenoperationen, die EF zwischen die Schemaänderungen sortiert, laufen innerhalb der Transaktion ohne Policy, also

mit den Rechten der Migrationsrolle. `SqlServerMigrationTests` prüft mit von EF erzeugten Diffs gegen einen echten SQL Server: Anlegen, Spalte und Tabelle umbenennen, Schemawechsel, jeden Schritt zurück und Löschen, nach jedem Schritt Lese- und Schreibisolation über den App-Login. Dazu kommen gleichnamige Tabellen in zwei Schemas, eine Policy unter dem alten Namen und die fremde Policy einer Tabelle ohne RLS.

Session-Schlüssel und Policy-Schema ändern

`sessionKey` und `policySchema` sind Options, wirken aber an zwei Stellen: zur Laufzeit im Session-Interceptor, der den Schlüssel setzt, und in der Datenbank in der Predicate-Funktion, die ihn liest. Bis 1.0 las der Generator beide Werte aus den *aktuellen* Options und legte die Funktion nur an, wenn sie fehlte. Ein Wechsel von `TenantId` auf `NewTenant` erzeugte deshalb keine Migration. Der Interceptor setzte danach den neuen Schlüssel, die Funktion las weiter den alten, und kein Mandant sah mehr Zeilen. Schlimmer noch: Eine alte Migration hätte beim erneuten Ausführen, etwa für eine neue Testdatenbank, Objekte mit den neuen Werten erzeugt.

Seit 1.1 trennt das Paket die beiden Rollen. Die Convention schreibt Schlüssel und Schema als Modell-Annotationen `Acme:SqlServerRls:SessionKey` und `Acme:SqlServerRls:PolicySchema` ins Modell und damit in den Snapshot. Der Annotation-Provider gibt sie an das relationale Modell weiter, und der Differ erzeugt bei einer Änderung eine `AlterDatabaseOperation` mit altem und neuem Wert. Der Generator liest beide Werte aus dem Zielmodell der Migration, die er gerade erzeugt; die Options gelten nur noch für Modelle, die die Werte nicht festhalten. Damit bleibt jede Migration bei dem Stand, mit dem sie erzeugt wurde.

Fall	Verhalten der Migration
Unverändert	Keine Operation, keine Phantomänderung
Schlüssel oder Schema geändert, alter Wert im Snapshot	In einer Transaktion: Policies aller geschützten Tabellen entfernen, alte Funktion löschen, Funktion mit den neuen Werten anlegen, Policies neu anlegen. Der Down-Weg macht dasselbe rückwärts.
Erste Migration eines Contexts oder erste nach dem Upgrade von 1.0 (alter Wert unbekannt)	Nichts wird gelöscht. Eine vorhandene Funktion muss den Schlüssel lesen, den das Modell nennt, sonst bricht die Migration mit `THROW 50000` ab.
Down auf einen Snapshot vor 1.1	Funktion und Policies bleiben, wie sie sind.

Die dritte Zeile ist eine bewusste Grenze. Ohne festgehaltenen alten Wert lässt sich nicht beweisen, dass eine vorhandene Funktion dem Plugin gehört und nicht einem anderen Context. Eine bestehende Installation wechselt den Schlüssel deshalb in zwei Schritten: erst die erste Migration nach dem Upgrade mit unverändertem Schlüssel, dann der Wechsel in einer eigenen Migration.

Dieselbe Prüfung schützt mehrere Contexts in einer Datenbank. Die Funktion ist pro Policy-Schema eine, und jede Policy des Plugins prüft nach dem Anlegen der Funktion deren Definition. Konfiguriert ein zweiter Context einen anderen Schlüssel im selben Schema, scheitert seine Migration, statt seine Tabellen an eine Funktion zu binden, die den falschen Schlüssel liest. Contexts mit verschiedenen Schlüsseln brauchen verschiedene Policy-Schemas.

Laufzeit und Datenbank können trotzdem kurz auseinanderlaufen. Läuft eine neue App-Version mit neuem Schlüssel, bevor die Migration eingespielt ist, setzt der Interceptor einen Schlüssel, den die Funktion nicht liest. Die Folge ist kein Datenleck, sondern *fail closed*: keine Zeilen, und Schreibversuche scheitern am BLOCK-Prädikat. Die Migration gehört deshalb vor die neue App-Version. `SqlServerSessionSettingsTests` prüft ohne Datenbank die Annotationen, den Snapshot,

den leeren Diff, die erkannte Änderung und die historische Migration. Gegen einen echten SQL Server prüft es den Wechsel von Schlüssel und Schema mit Up und Down, das Upgrade einer 1.0-Installation und den zweiten Context.

Laufzeit: derselbe Ablauf, anderer Befehl

Runtime/SqlServerTenantSessionInterceptor.cs

```
public class SqlServerTenantSessionInterceptor : TenantSessionInterceptor
{
    public SqlServerTenantSessionInterceptor(ITenantProvider tenants)
        : base(tenants)
    {
    }

    protected override DbCommand CreateSessionCommand(
        DbConnection connection, DbContext? context, TenantId? tenant)
    {
        var key = context?.GetService<ISqlServerTenancySingletonOptions>()
                      .Settings.SessionKey
                  ?? SqlServerTenancySettings.Default.SessionKey;

        var command = connection.CreateCommand();
        command.CommandText = "EXEC sp_set_session_context @key = @key, @value = @value";
        AddParameter(command, "@key", key);
        AddParameter(command, "@value", tenant?.Value);    // NULL: keine Zeilen sichtbar
        return command;
    }
}
```

Bewusst nicht portiert

Das SQL-Server-Paket ist ein eigenständiger, aber schlankerer Provider. Fluent-API-Policies, der optionale Query-Filter, Snapshot-Code-Generator und Scaffolding fehlen dort, und der eigene SQL-Knoten und das Aggregat des Funktionen-Beispielpakets gibt es nur für PostgreSQL. Das ist eine Produktentscheidung, kein technisches Hindernis: Jedes dieser Kapitel lässt sich mit denselben Mustern auf SQL Server übertragen. Die Liste gehört in die Dokumentation des Pakets, damit Nutzer wissen, was sie erwarten können.

tests/...SqlServer.Tests/SqlServerTests.cs (Auszug)

```
[Fact]
public async Task Block_predicate_rejects_rows_of_other_tenant()
{
    await using var db = ContextFor(TenantId.New());

    var ex = await Assert.ThrowsAsync<SqlException>(() => db.Database.ExecuteSqlAsync(
        $"INSERT INTO [Orders] ([TenantId], [Total]) VALUES ({Guid.NewGuid()}, 1)"));

    Assert.Equal(33504, ex.Number);      // Verstoß gegen BLOCK PREDICATE
}
```

Die Tests laufen wie bei PostgreSQL in zwei Ebenen: ohne Datenbank für Modell, Abfrage- und Migrations-SQL und mit Testcontainers gegen einen echten SQL Server. Dort verbindet sich die Anwendung mit einem eigenen Login statt als `sa`, um die Produktionssituation nachzustellen.

48 Ohne SQL: Was bei Cosmos DB trägt

Versionsabhängigkeit: **mittel**

PostgreSQL und SQL Server teilen dieselbe Idee: Die Datenbank selbst verweigert fremde Zeilen, und das Plugin muss ihr nur den Mandanten mitteilen. Azure Cosmos DB hat weder SQL-DDL noch eine Session noch Row Level Security. Das Paket `Acme.EntityFrameworkCore.Tenancy.Cosmos` ist deshalb bewusst ein schlankes Kontrastbeispiel, kein vollwertiger Provider: Es zeigt, welche Teile des Plugins ohne SQL tragen und was verloren geht. Es referenziert nur das Kernpaket aus Kapitel 46, ohne die relationale Schicht.

Baustein	Im Cosmos-Paket	Anmerkung
`TenantId`, `ITenantOwned`, `AddAcmeTenancy`	Wiederverwendet	Unverändert aus dem Kern
SaveChanges-Interceptor	Wiederverwendet	Setzt und prüft den Mandanten vor dem Speichern
Prüfung beim Laden	Wiederverwendet	Die einzige Prüfung hinter dem Query-Filter
Value Generation für `TenantId`-Schlüssel	Wiederverwendet	Convention aus dem Kern
Type Mapping	Neu	`TenantId` als JSON-String
Partition-Key-Convention	Neu	Ersetzt die RLS-Convention
Query-Filter	Aus dem Kern	Mandant als Abfrageparameter statt als Session-Wert
Session-Interceptor, Migrations, SQL-Knoten	Nicht portiert	Es gibt keine Session und kein Schema

Der Mandant als Partition Key

In Cosmos DB entscheidet der Partition Key, wo ein Dokument liegt und welche Partition eine Abfrage liest. Der Mandant ist der natürliche Kandidat: Jede Abfrage der Anwendung betrifft genau einen Mandanten. Eine Convention setzt ihn für jeden mandantenbezogenen Wurzeltyp, mit derselben Kandidatensuche wie die RLS-Conventions. Sie läuft mit Convention-Quelle, ein explizites `HasPartitionKey` des Anwenders gewinnt also:

`Conventions/CosmosTenantPartitionKeyConvention.cs (Auszug)`

```
foreach (var entityType in modelBuilder.Metadata.GetEntityTypes())
{
    // Nur an der Wurzel einer Hierarchie; Owned Types stecken im Dokument des Besitzers
    if (entityType.BaseType is not null || entityType.IsOwned()
        || entityType.ClrType.IsDefined(typeof(NoTenantIsolationAttribute), inherit: true))
        continue;

    if (entityType.FindTenantPropertyCandidate() is not { } candidate)
        continue;

    entityType.Builder.HasAnnotation(TenancyAnnotationNames.TenantProperty, candidate.Name);

    string[] partitionKey = [candidate.Name];
```

```
    if (entityType.Builder.CanSetPartitionKey(partitionKey))
        entityType.Builder.HasPartitionKey(partitionKey);
}
```

Zwei Details unterscheiden sich von den relationalen Providern. Cosmos akzeptiert als Werte eines Partition Keys nur Strings, Zahlen und Booleans; der Konverter des Kerns auf `Guid` reicht nicht, deshalb bildet das Paket `TenantId` über ein `ITypeMappingSourcePlugin` auf einen String ab. Cosmos hat dafür keine eigene Plugin-Schnittstelle, seine Type-Mapping-Quelle fragt aber zuerst die Plugins des Kerns; das Mapping selbst baut auf `CosmosTypeMapping` aus einem `.Internal`-Namespace auf. Außerdem hat ein Container genau einen Partition Key: Daten ohne Mandant, etwa gemeinsame Stammdaten, brauchen einen eigenen Container (`ToContainer`).

Der Query-Filter auf den aktuellen Mandanten

Der relationale Filter vergleicht mit `EF.Functions.CurrentTenant()`, was in SQL zu `current_setting()` wird: Die Datenbank kennt den Mandanten aus der Session. Cosmos kennt nichts; der Mandant muss mit jeder Abfrage mitreisen. Eine Convention kann das nicht leisten, weil sie keinen Zugriff auf den Context hat. Der Filter entsteht deshalb in `OnModelCreating`, mit dem Context selbst. Die Erweiterung braucht weder SQL noch Cosmos und liegt im Kernpaket; auch der Testmodus für Anwendungen (Kapitel 26) nutzt sie:

Aufruf und Metadata/TenancyModelBuilderExtensions.cs (Auszug)

```
// OnModelCreating: modelBuilder.HasTenantQueryFilters(this);

var currentTenant = Expression.Call(
    typeof(TenancyModelBuilderExtensions).GetMethod(nameof(CurrentTenant))!,
    Expression.Constant(context));

// e => (TenantId?)EF.Property<TenantId>(e, "TenantId") == CurrentTenant(context)
var parameter = Expression.Parameter(entityType.ClrType, "e");
var body = Expression.Equal(
    Expression.Convert(
        Expression.Call(typeof(EF), nameof(EF.Property), [typeof(TenantId)],
            parameter, Expression.Constant(property.Name)),
        typeof(TenantId?)),
    currentTenant);

modelBuilder.Entity(entityType.ClrType)
    .HasQueryFilter(QueryFilterKey, Expression.Lambda(body, parameter));
```

EF behandelt eine Context-Instanz in einem Query-Filter besonders: Es ersetzt sie durch den Context, der die Abfrage ausführt, und wertet `CurrentTenant(context)` vor jeder Ausführung aus. Der Mandant wird so zum Abfrageparameter, und die kompilierte Abfrage bleibt für alle Mandanten im Cache. Weil der Filter den Partition Key mit diesem Parameter vergleicht, übernimmt EF Core ihn in die Anfrage: Die Abfrage geht an eine einzige Partition, statt sich über alle zu verteilen. Ein Test macht das über die Event-Daten sichtbar:

tests/...Cosmos.Tests/EmulatorTests.cs (Auszug)

```
await using var ctx = _fixture.CreateContext(
    new MutableTenantProvider { CurrentTenantId = tenant },
    configure: o => o.LogTo(
        (id, _) => id == CosmosEventId.ExecutingSqlQuery,
```

```
        e => partitions.Add(((CosmosQueryEventData)e).PartitionKeyValue.ToString())));

await ctx.Orders.Where(o => o.Total > 0).ToListAsync();

// EF übernimmt den Gleichheitsvergleich des Filters auf den Partition Key in die Anfrage
var partition = Assert.Single(partitions);
Assert.Contains(tenant.Value.ToString("D"), partition);
```

Keine Row Level Security: Die Anwendung ist die einzige Verteidigungslinie

Cosmos DB hat keine Row Level Security. Die Isolation ist genau so stark wie die Anwendungsschicht: der Query-Filter, der SaveChanges-Interceptor und, falls eingeschaltet, die Prüfung beim Laden (`UseAcmeTenancy(sp, verifyMaterializedTenants: true)`). Alles, was an EF vorbeigeht, geht auch an der Isolation vorbei: Abfragen über das Cosmos-SDK, Skripte und Werkzeuge mit dem Account-Schlüssel und `IgnoreQueryFilters`. Rohe Abfragen mit `FromSql` behalten den Filter, laufen aber über alle Partitionen. Das Bedrohungsmodell aus Kapitel 36 (Seite 139) gilt deshalb ohne seine datenbankseitige Hälfte. Wo Mandanten durch die Datenbank getrennt sein müssen, muss die Architektur das leisten: ein **Container pro Mandant** mit Zugriff über rollenbasierte Zugriffskontrolle oder ein **Account pro Mandant**. Beides kostet mehr und gibt den gemeinsamen Container auf, aber ein Leck braucht dann eine falsche Berechtigung, nicht nur eine falsche Abfrage.

Vergleich der drei Provider

Aspekt	PostgreSQL	SQL Server	Cosmos DB
Wer die Isolation erzwingt	Datenbank (RLS-Policies)	Datenbank (Security Policy)	Nur die Anwendung (Query-Filter)
Mandant in der Anfrage	Session-Setting	`SESSION_CONTEXT`	Abfrageparameter, Partition Key
Schema-Objekte	Policies über Migrations	Prädikatfunktion, Security Policy	Keine
Rohe Abfragen	Weiter durch RLS gefiltert	Weiter durch die Policy gefiltert	`FromSql` gefiltert, aber partitionsübergreifend; SDK ungefiltert
Genutzte Pakete	Kern, relational, Provider	Kern, relational, Provider	Kern, Provider
Interne API (EF1001)	10 Typen	2 Typen	1 Typ (`CosmosTypeMapping`)
Tests	Testcontainers PostgreSQL	Testcontainers SQL Server	Linux-Emulator (Preview)

Bewusst nicht portiert

Migrations und Scaffolding (Cosmos hat kein Schema), eigene SQL-Knoten und Aggregate, der Session-Interceptor und `Database.ApplyTenantSession()` (es gibt keine Session) sowie der Schutz von Massen-Updates (EF Core 10 hat für Cosmos kein `ExecuteUpdate`). Cosmos unterstützt außerdem nur asynchrone I/O: `SaveChanges()` und `ToList()` werfen, Anwendung und Tests nutzen also durchgehend die asynchronen Varianten.

Tests gegen den Emulator

Die Tests laufen gegen den Linux-Emulator von Cosmos DB (vNext), eine Preview. Der Image-Tag ist festgenagelt, die Verbindung nutzt den Gateway-Modus über den HTTP-Client des Containers, und das

> Testprojekt referenziert ein neueres Cosmos-SDK, als EF Core 10.0.0 mitbringt: Das SDK 3.51 kann nicht jede binäre Antwort des aktuellen Emulators lesen. Weil der Preview-Emulator gelegentlich hängt, blockiert der Cosmos-Job in der CI den Build noch nicht.

Teil XII

Anhänge

Nachschlagen: Erweiterungspunkte mit Lebensdauern, Dateien des Beispiels, Begriffe und ein Debugging-Werkzeugkasten.

- Anhang A: Erweiterungspunkte und Lebensdauern
- Anhang B: Dateien des Beispiel-Plugins
- Anhang C: Glossar
- Anhang D: Debugging-Werkzeugkasten
- Anhang E: Übungen mit Lösungen
- Anhang F: Quellen
- Lizenz

A Erweiterungspunkte und Lebensdauern

Dieser Anhang führt vom Mechanismus zu seinen Eigenschaften und zum Detailkapitel. Den umgekehrten Weg, vom Problem zum Mechanismus, geht die Entscheidungshilfe in Kapitel 4. Lebensdauer und Kardinalität stammen aus den Service-Tabellen von EF Core 10 (`EntityFrameworkServicesBuilder`, `EntityFrameworkRelationalServicesBuilder`, `EntityFrameworkRelationalDesignServicesBuilder`), die Angaben zur Provider-Implementierung aus Npgsql 10; `ServiceRegistrationTests` prüft die Registrierungen des Beispiels gegen dieselben Tabellen.

Kennzeichen	Bedeutung
Singleton, Scoped	Einmal pro internem Provider bzw. pro Context-Instanz, mit Pooling über mehrere Anfragen (Kapitel 8); „DT“: im Service Provider von `dotnet ef`
mehrfach, einzeln	Plugin-Interface, EF fragt alle Registrierungen; bzw. Einzeldienst mit genau einem Gewinner
öffentlich, Provider-Basis, pubternal	Wie in der Entscheidungshilfe: nur öffentliche API; Ersetzen heißt von der Provider-Implementierung erben; schon EFs eigener Typ ist intern
`TryAdd`, `Replace`, `ReplaceService`	Im `ApplyServices` der Extension ergänzen bzw. ersetzen; vom Anwender am Options-Builder ersetzt
†	Erhöhtes Upgrade-Risiko: Ersetzen oder Ableiten berührt interne API von EF oder Npgsql (EF1001, Upgrade-Playbook in Kapitel 27)
● ◐ ○	Bevorzugter Mechanismus; mit Bedacht einsetzen; letzte Option
exklusiv	Ersetzen schließt jedes andere Plugin aus, das denselben Dienst ersetzt (Kompatibilitätsmatrix in Kapitel 6)

Dienst und Aufgabe	Lebens-dauer, Anzahl	API	Registrie-rung	Bewertung	Kap.
Konfiguration und Dienste					
`ISingletonOptions` Konfiguration für Singleton-Dienste	Singleton, mehrfach	öffentlich	`TryAdd`, doppelt	● kombinierbar	6, 8
`IResettableService` Scoped-Zustand bei Pooling zurücksetzen	Scoped, mehrfach	öffentlich	zusätzlich registrieren	● kombinierbar	8
Modell					
`IConventionSetPlugin` Conventions und Validierung ergänzen	Scoped, mehrfach	öffentlich	`TryAdd`	● kombinierbar, Reihenfolge	10, 11
`IModelCacheKeyFactory` Modell pro Laufzeitkontext	Singleton, einzeln	öffentlich	`ReplaceService`	◐ **exklusiv**	34
`IModelCustomizer` Modellbau vor `OnModelCreating`	Singleton, einzeln	öffentlich	`Replace`	○ **exklusiv**; Conventions bevorzugen	4

Dienst und Aufgabe	Lebens-dauer, Anzahl	API	Registrie-rung	Bewertung	Kap.
`IModelValidator` Validierung des fertigen Modells	Singleton, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**; Finalized-Convention bevorzugen	11
Typen und Werte					
`IRelationalTypeMappingSourcePlugin` Eigene CLR-Typen als Spalten; Kern: `ITypeMappingSourcePlugin`	Singleton, mehrfach	öffentlich	`TryAdd`	● kombinierbar	12
`IRelationalTypeMappingSource` Alle Type Mappings des Providers	Singleton, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	12
`IValueGeneratorSelector` Wählt Value Generators	Scoped, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**; Factory per Convention bevorzugen	31
Abfragen					
`IEvaluatableExpressionFilterPlugin` Funktionen vor dem Funcletizer schützen	Singleton, mehrfach	öffentlich	`TryAdd`	● kombinierbar	15
`IMethodCallTranslatorPlugin`, `IMemberTranslatorPlugin` Methoden und Member nach SQL	Scoped, mehrfach	öffentlich	`TryAdd`	● kombinierbar; gleiche Methode: Reihenfolge	14
`IAggregateMethodCallTranslatorPlugin` Aggregatfunktionen	Scoped, mehrfach	öffentlich	`TryAdd`	● kombinierbar	43
`ISqlExpressionFactory` Baut SQL-Ausdrücke; in Translators injizieren	Scoped, einzeln	öffentlich	injizieren	● nur nutzen	14
`IMethodCallTranslatorProvider` u. ä. Translator-Infrastruktur, auch Member und Aggregate	Scoped, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**; Plug-ins bevorzugen	16
`IQueryTranslationPreprocessorFactory` LINQ-Baum vor der Übersetzung umbauen	Scoped, einzeln	öffentlich	`Replace`	◐ **exklusiv**; Query-Interceptor bevorzugen	13
`IQueryTranslationPostprocessorFactory` Übersetzten Baum nachbearbeiten	Scoped, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	13, 16

Dienst und Aufgabe	Lebens-dauer, Anzahl	API	Registrie-rung	Bewertung	Kap.
`IQuerySqlGenerator Factory` SQL-Text, eigene Knoten schreiben	Singleton, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	42, 6
`IRelationalParameter BasedSqlProcessor Factory` NULL-Semantik mit Parameterwerten	Scoped, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	42, 6
Laufzeit					
`IInterceptor`: Save-Changes, Connection, Command, Transaction, IdentityResolution Eingriffe bei Speichern und Ausführung	Scoped, mehrfach	öffentlich	`Add Interceptors` oder injiziert	● kombinierbar, Reihenfolge	18
`IMaterialization Interceptor` (ein `ISingletonInterceptor`) Materialisierung	Singleton, mehrfach	öffentlich	`TryAdd` als `ISingleton Interceptor` oder eine geteilte Instanz	● kombinierbar	17
`IQueryExpression Interceptor` (ebenfalls `ISingletonInterceptor`) LINQ-Baum vor der Kompilierung	Singleton per Interface, mehrfach	öffentlich	eine geteilte Instanz oder `TryAdd` als `IInterceptor`; als `ISingleton Interceptor` registriert ruft EF ihn nie	● kombinierbar	16, 30
`IModificationCommand Factory` Update-Pipeline: ein Befehl pro Zeile	Singleton, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	32
`ICommandBatchPreparer`, `IModificationCommand BatchFactory` Update-Pipeline: Reihenfolge und Batches	Scoped, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	32
`IUpdateSqlGenerator` INSERT, UPDATE, DELETE schreiben	Singleton, einzeln	Provider-Basis†	`Replace`	○ **exklusiv**	32

Dienst und Aufgabe	Lebens-dauer, Anzahl	API	Registrie-rung	Bewertung	Kap.
Migrations					
`IRelationalAnnotationProvider` Annotations auf Tabellen und Spalten übertragen	Singleton, einzeln	Provider-Basis†	`Replace`	◐ **exklusiv**; deklarativer Weg	21
`IMigrationsSqlGenerator` SQL für Migrations-Operationen	Scoped, einzeln	Provider-Basis†	Decorator oder `Replace`	◐ **exklusiv** beim Ersetzen, Decorator kombinierbar	21, 6
`IMigrationsModelDiffer` Vergleicht relationale Modelle	Scoped, einzeln	pubternal†	`Replace`	○ **exklusiv**; Annotations bevorzugen	22
`ISqlGenerationHelper` Bezeichner maskieren, Befehle beenden	Singleton, einzeln	öffentlich	injizieren	● nur nutzen	21
Design-Time (über `IDesignTimeServices`)					
`IAnnotationCodeGenerator` Annotations als Fluent API in Snapshot und Scaffolding	Singleton DT, einzeln	Provider-Basis†	`RemoveAll` + `Add`	◐ **exklusiv**	23
`ICSharpRuntimeAnnotationCodeGenerator` Annotations im Compiled Model filtern	Singleton DT, einzeln	pubternal†	`RemoveAll` + `Add`	◐ **exklusiv**	44, 24
`ICSharpMigrationOperationGenerator` Eigene Operationen als C# in Migrationen	Singleton DT, einzeln	öffentlich	`RemoveAll` + `Add`	○ **exklusiv**	22
`IDatabaseModelFactory` Schema für Scaffolding lesen	Scoped DT, einzeln	Provider-Basis†	`RemoveAll` + `AddScoped`	◐ **exklusiv**	45
`IProviderConfigurationCodeGenerator` Use-Aufruf im gescaffoldeten Context	Singleton DT, einzeln	Provider-Basis†	`RemoveAll` + `Add`	○ **exklusiv**	25

Erweiterungspunkte ohne eigenen Dienst

Viele Mechanismen wirken über Modell, Options oder Migrationen und haben keine Lebensdauer im internen Provider. Sie sind fast immer der erste Weg:

Erweiterungspunkt	Aufgabe	Aktivierung	Kap.
`IDbContextOptionsExtension`	Konfiguration, Dienste, Provider-Schlüssel	Use-Methode	5, 7

Erweiterungspunkt	Aufgabe	Aktivierung	Kap.
Annotations, Fluent API	Konfiguration am Modell	Erweiterungsmethoden	9, 44
Laufzeit-Annotation, Runtime-Annotation-API	Informationen für Laufzeitcode	Convention bzw. `GetOrAddRuntimeAnnotationValue`	44
`ValueConverter`, `JsonValueReaderWriter`	Werte umwandeln	Type Mapping oder `HasConversion`	12
`ValueGenerator` mit Factory	Werte für neue Entitäten	Convention per `HasValueGeneratorFactory`	31
Eigene `SqlExpression`	Syntax, die kein Funktionsaufruf ist	Translator plus Generator und Prozessor	42
Eigene `MigrationOperation`	Schritte außerhalb des Modells	`MigrationBuilder`-Erweiterung	22
`IDesignTimeServices`	Dienste für `dotnet ef`	`DesignTimeServicesReference`	23

B Dateien des Beispiel-Plugins

Datei	Inhalt	Kap.
Kernpaket		
`TenantId.cs`, `TenantIdConverter.cs`	Stark typisierte ID, Konverter	12
`TenantIdJsonReaderWriter.cs`, `TenantIdJsonConverter.cs`	JSON-Spalten in EF; JSON in Web-APIs	12, 33
`ITenantOwned.cs`, `ITenantProvider.cs`	Marker, Mandantenquelle	3
`NoTenantIsolationAttribute.cs`	Data Annotation	10
`TenancyDbFunctionsExtensions.cs`	`EF.Functions.CurrentTenant()`	14
`Diagnostics/*`	Event-IDs, Definitionen, Logger-Extensions	19
`Interceptors/TenantSaveChangesInterceptor.cs`	SaveChanges-Logik	18
`Interceptors/TenantMaterializationInterceptor.cs`	Prüfung beim Laden (optional)	17
`Metadata/*` (Kern)	Laufzeit-Annotation, IsTenantProperty, Kandidatensuche	44
`Query/TenancyEvaluatableExpressionFilterPlugin.cs`	Filter für alle Pakete	15, 46
`DependencyInjection/*`	AddAcmeTenancy	7, 46
`ValueGeneration/TenantIdValueGenerator.cs`, `…Factory.cs`	Wertgenerierung	31
`Metadata/TenancyModelBuilderExtensions.cs`	Query-Filter mit dem Mandanten als Parameter	48, 26
`Testing/*`	Testmodus für Anwendungen (SQLite, InMemory)	26
`Conventions/TenantIdKeyValueGenerationConvention.cs`	Generator für Schlüssel	31
Relationales Paket		
`Interceptors/TenantSessionInterceptor.cs`	Abstrakte Basis für Session-Befehle	18, 46
`Metadata/TenantTableMappingValidator.cs`	Unterstützte Tabellenabbildungen (TPT, TPC, Entity Splitting)	10, 46
`TenancyDatabaseFacadeExtensions.cs`	`Database.ApplyTenantSession()`	18
PostgreSQL-Paket		
`Infrastructure/TenancyOptionsExtension.cs`	Extension, Info, ApplyServices	5, 6
`Infrastructure/TenancySettings.cs`	Einstellungen als Record	5
`Infrastructure/ITenancySingletonOptions.cs`	Singleton-Options	6
`Infrastructure/TenancyOptionsBuilder.cs`	Options-Builder	7
`Infrastructure/Tenancy*Extensions.cs`	UseAcmeTenancy, AddAcmeTenancy	7
`Infrastructure/TenancyIdentifiers.cs`	Validierung von Bezeichnern	10, 22
`Storage/*`	Type Mapping und Plugin	12

Datei	Inhalt	Kap.
`Query/*`	Translators, Evaluatable-Filter	14, 15
`Metadata/*`	Annotations, RlsPolicy, PolicyCommand, Annotation-Provider	9, 21
`Builders/*`	Fluent API, PolicyBuilder	9
`Conventions/*`	Isolation, Attribut, Validierung, Plugin	10, 11
`Migrations/*`	SQL-Generator, eigene Operationen	21, 22
`Design/*`, `Properties/AssemblyInfo.cs`	Code-Generator, Design-Time	23
`Runtime/TenantConnectionInterceptor.cs`	Session-Setting setzen	18
`Design/Internal/TenancyCSharp Runtime…Generator.cs`	Compiled-Model-Filter	44
`Scaffolding/*`	RLS aus der Datenbank lesen	45
SQL-Server-Paket		
`Infrastructure/*`	Extension, Provider-Builder-Aktivierung	47
`Storage/*`, `Query/*`	uniqueidentifier, SESSION_CONTEXT	47
`Conventions/*`, `Metadata/*`	Isolation, Spalten-Annotation	47
`Migrations/*`	Predicate-Funktion, Security Policy	47
`Runtime/*`	sp_set_session_context	47
Cosmos-DB-Paket		
`Conventions/CosmosTenantPartitionKey Convention.cs`	Mandant als Partition Key	48
`Storage/*`	TenantId als String, Type-Mapping-Plugin	48
`Infrastructure/*`	Options-Extension, UseAcmeTenancy	48
Funktionen-Beispielpaket (ohne Mandanten-bezug)		
`Query/Expressions/*`, `Query/DistinctFrom Translator.cs`	Eigener SQL-Knoten	42
`Query/Internal/*`	SQL-Generator, Nullability-Prozessor, Factories	42
`Query/EveryAggregateTranslator.cs`	Aggregat-Translator	43
Weitere		
`samples/ShopSample`	Konsolen-Beispiel mit Design-Time-Factory	3, 35
`samples/ShopApi`	ASP.NET-Core-Beispiel (Minimal API)	40
`.github/workflows/compatibility.yml`	Wöchentlich gegen die neuesten stabilen 10.0.x-Patches	27
`.github/workflows/ci.yml`	Build, Tests, Handbuch	41
`PublicAPI.*.txt`	Public-API-Analyzer	41
`docs/handbook/*`	Quellen dieses Buchs (Deutsch und Englisch)	–
`tests/...Tests`	Unit-Tests ohne Datenbank	26
`tests/.../MinimalPluginTests.cs`	Das Mini-Plugin	2
`tests/.../CompositionTests.cs`	Decorator-Beispiel (Audit-Plugin) und Kompositionsmatrix zweier Plugins	6

Datei	Inhalt	Kap.
tests/.../FailureCaseTests.cs	Die „Was passiert, wenn …?“-Fälle des Buchs, falsche neben richtiger Variante	–
tests/.../ServiceRegistrationTests.cs	Lebensdauer und Kardinalität jeder Registrierung gegen EFs Service-Tabelle	27
tests/.../StateAndLifetimeTests.cs	Fehlerfälle zu Zustand und Lebensdauern, falsche neben richtiger Variante	8
tests/.../MaterializationTests.cs	Prüfung beim Laden: Registrierung, Logik, Fehlerfall pro Context (Unit), Leck, Grenzen und gemerkter Mandant (Integration)	17
tests/.../DesignTimeServicesTests.cs	Lebensdauern der ersetzten Design-Time-Dienste	4, 45
tests/.../DebuggingTests.cs	Debugging-Techniken	Anh. D
tests/.../SchemaPerTenantTests.cs	Modell-Cache-Schlüssel pro Schema	34
benchmarks/...	BenchmarkDotNet-Projekt für die SaveChanges-Logik	39
tests/...IntegrationTests	Testcontainers-Tests gegen PostgreSQL	18, 26, 30, 36, 45
tests/...SqlServer.Tests	Unit- und Testcontainers-Tests für SQL Server	47

C Glossar

Begriff	Erklärung
Annotation	Schlüssel-Wert-Metadatum an einem Modellelement.
ConfigurationSource	Herkunft eines Metadatums (Explicit, DataAnnotation, Convention); bestimmt, wer wen überschreiben darf.
Convention	Regel, die das Modell beim Bau automatisch anpasst.
Decorator	Dienst, der die zuvor registrierte Implementierung umhüllt und an sie delegiert; verträgt sich mit anderen Plugins (Kapitel 6).
Design-Time-Modell	Vollständiges Modell mit allen Annotations; Grundlage für Migrations. Gegenstück: schlankes Laufzeitmodell.
Differ	`IMigrationsModelDiffer`; vergleicht zwei relationale Modelle.
Funcletizer	Erste Query-Stufe; wertet Ausdrücke ohne Datenbezug in .NET aus.
Identity Resolution	Dieselbe Entität ergibt dieselbe Instanz: beim Tracking über den Change Tracker, sonst optional pro Abfrage.
Materializer	Teil des Shapers, der eine Entität erzeugt: Konstruktor, dann Properties und Felder.
Nullability-Prozessor	Query-Stufe, die NULL-Semantik auswertet und Vergleiche optimiert; muss eigene SQL-Knoten kennen.
Shaper	Kompilierter Ausdrucksbaum pro Abfrageform, der aus den Zeilen des `DbDataReader` das Ergebnis baut.
SqlExpression	Knoten im SQL-Ausdrucksbaum; eigene Knoten für Syntax, die keine Funktion ist.
Laufzeit-Annotation	Annotation, die zur Laufzeit gelesen wird und deshalb im Laufzeit- und Compiled Model erhalten bleiben muss.
Runtime-Annotation (EF-API)	Nicht dasselbe wie eine Laufzeit-Annotation: ein Wert, den Code erst am fertigen Modell ablegt (`GetOrAddRuntimeAnnotationValue`), etwa als Cache; er erscheint nie in Snapshot oder Migrations.
Interner Service Provider	EFs eigener DI-Container pro Konfiguration.
Pubternal	Öffentlich sichtbare, aber interne API ohne Kompatibilitätsgarantie (EF1001).
Relationales Modell	Tabellen, Spalten, Indizes, abgeleitet aus dem EF-Modell.
RLS	Row Level Security: Zeilenfilter, die PostgreSQL selbst erzwingt.
Snapshot	Generierter C#-Code mit dem Modellstand der letzten Migration.
Type Mapping	Verbindung zwischen CLR-Typ und Datenbanktyp inklusive Konvertierung, Vergleich und Literalen.

D Debugging-Werkzeugkasten

Wenn sich ein Plugin nicht wie erwartet verhält, lautet die Frage fast immer eine von sechs. Jede der folgenden Techniken wird in `DebuggingTests` geprüft.

Frage	Technik
Ist meine Annotation im Modell?	`ToDebugString(MetadataDebugStringOptions.LongDefault)` auf dem Design-Time-Modell
Welche Implementierung eines Einzeldienstes hat gewonnen?	`context.GetService<IMigrationsSqlGenerator>().GetType()`
Ist meine Convention registriert?	`GetService<IConventionSetBuilder>().CreateConventionSet()`, dann z. B. `ModelFinalizingConventions`
Wie sieht die Abfrage vor und nach der Übersetzung aus?	`query.Expression.Print()` und `query.ToQueryString()`
In welcher Stufe der Pipeline ändert sich etwas?	`LogTo` auf Debug mit `CoreEventId.QueryCompilationStarting` (Baum nach dem Funcletizer) und `QueryExecutionPlanned` (`SelectExpression` und Shaper), Kapitel 13
Protokolliert mein Plugin?	`LogTo(lines.Add, [eventId], LogLevel.Debug)`, beschränkt auf die Event-IDs des Plugins

Debugging-Snippets (geprüft in tests/.../DebuggingTests.cs)

```
using var ctx = CreateContext();

// 1. Modell: Annotations, Ergebnisse von Conventions, Type Mappings
var model = ctx.GetService<IDesignTimeModel>().Model;
Console.WriteLine(model.ToDebugString(MetadataDebugStringOptions.LongDefault));

// 2. Dienste: welcher Ersatz hat gewonnen?
Console.WriteLine(ctx.GetService<IMigrationsSqlGenerator>().GetType());

// 3. Conventions: ist meine in der Liste?
var conventions = ctx.GetService<IConventionSetBuilder>().CreateConventionSet();
Console.WriteLine(string.Join("\n", conventions.ModelFinalizingConventions));

// 4. Abfragen: LINQ-Baum vor, SQL nach der Übersetzung
var query = ctx.Orders.Where(o => o.TenantId == EF.Functions.CurrentTenant());
Console.WriteLine(query.Expression.Print());
Console.WriteLine(query.ToQueryString());
```

Weitere Werkzeuge

Werkzeug	Einsatz
Debugger mit Source Link, „Just My Code“ aus	In EF Core und Npgsql hineinsteppen; beide liefern Symbole und Source Link mit. Ein Breakpoint in der eigenen Convention zeigt, wann und wie oft sie läuft.
`DebugView` im Debugger	Modell, Entitätstypen, Query-Objekte und der Change Tracker zeigen dieselben Texte wie `ToDebugString`.

Werkzeug	Einsatz
`dotnet ef ... --verbose`	Zeigt, welche Assemblies und Design-Time-Dienste die Tools laden; die erste Anlaufstelle, wenn ein Design-Time-Dienst ignoriert zu werden scheint.
`EnableSensitiveDataLogging`, `EnableDetailedErrors`	Parameterwerte und ausführliche Fehler im Log; nur für die Entwicklung.
`ManyServiceProviders CreatedWarning`	EF bricht standardmäßig ab, wenn mehr als 20 interne Provider entstehen: fast immer ein Hash-Fehler (Kapitel 5) oder eine Singleton-Instanz pro Context (Kapitel 17).
Der Quellcode von EF Core auf GitHub	Die maßgebliche Dokumentation interner APIs; die Ordnerstruktur entspricht den Namespaces.

> **Im Test reproduzieren, nicht in der App**
>
> Fast jedes Plugin-Problem lässt sich ohne Datenbank reproduzieren: einen Context mit dem Plugin bauen und das Modell, die aufgelösten Dienste oder `ToQueryString()` ansehen. Ein solcher Test iteriert schneller als die Anwendung und bleibt danach als Regressionstest erhalten.

E Übungen mit Lösungen

Jede Übung stellt eine Frage, die sich aus dem Text beantworten lässt, und nennt einen Test des Beispiels, der die Antwort festhält. Wer die Lösung prüfen will, ändert das Beispiel wie in der Aufgabe beschrieben und sieht zu, welcher Test scheitert.

Übung 1: Convention gegen explizite Konfiguration

Ein Anwender konfiguriert für `Order` selbst eine Policy mit dem Namen der Plugin-Policy: `HasPolicy("tenant_isolation", p => p.Using("true"))`. Die Isolations-Convention setzt bei `ModelFinalizing` dieselbe Annotation. Welche Konfiguration gilt? Was ändert sich, wenn die Convention die Annotation mit `SetAnnotation` direkt am Entity-Typ setzt statt über den Builder?

> **Lösung**
>
> Die explizite Konfiguration gilt. Der Convention-Builder setzt Annotations mit `ConfigurationSource.Convention` und lässt eine explizite Konfiguration stehen; `HasAnnotation` gibt dann `null` zurück (Kapitel 10). `SetAnnotation` am Typ kennt keine Konfigurationsquelle und würde die Policy des Anwenders still überschreiben. Nachprüfen: `ConventionTests.Explicit_policy_with_same_name_wins_over_convention`.

Übung 2: Der Mandant im Cache-Schlüssel

Ein Plugin nimmt den aktuellen Mandanten in `GetServiceProviderHashCode` auf, damit jeder Mandant „sein“ Modell bekommt. Was passiert mit 1000 Mandanten?

> **Lösung**
>
> Jeder Mandant bekommt einen eigenen internen Service Provider mit eigenem Modell. Ab dem 21. Provider wirft EF standardmäßig die `ManyServiceProvidersCreatedWarning` als Fehler, und jeder Provider hält Modell, Query-Cache und Dienste im Speicher. Der Mandant ist Zustand pro Anfrage und gehört in den Interceptor (Kapitel 8); nur wenn sich das *Modell* wirklich unterscheidet, hilft ein eigener Modell-Cache-Schlüssel (Kapitel 34). Nachprüfen: `StateAndLifetimeTests.Tenant_in_the_service_provider_key_needs_a_provider_per_tenant`.

Übung 3: Nullability eines Translators

`CurrentTenant()` wird ohne Mandant in SQL zu `NULL`. Jemand deklariert den Ausdruck deshalb als nullable. Was liefert `o.TenantId != EF.Functions.CurrentTenant()` dann ohne Mandant, etwa unter der Reporting-Rolle ohne RLS?

> **Lösung**
>
> Für einen nullable Ausdruck ergänzt EF C#-Semantik: `<>` wird zu einem Vergleich mit `OR ... IS NULL`, und ohne Mandant ist die Bedingung für *alle* Zeilen wahr. Als nicht nullable bleibt es beim reinen `<>`, das mit `NULL` nie wahr ist: keine Zeilen, die sichere Richtung (Kapitel 14). Nachprüfen: `QueryTranslationTests.Inequality_with_CurrentTenant_never_matches_without_a_tenant`.

Übung 4: Ein Migrations-Diff unter SQL Server

Die Mandantenspalte wird mit `HasColumnName("Mandant")` umbenannt. In welcher Reihenfolge erzeugt EF die Operationen, und warum genügt es nicht, die Security Policy bei der `AlterTableOperation` neu anzulegen?

> **Lösung**
>
> EF erzeugt `RenameColumn`, `RenameIndex` und erst danach die `AlterTableOperation` mit der geänderten Annotation. Die schemagebundene Policy blockiert aber schon das `sp_rename`. Der Generator muss die ganze Operationsliste umklammern (Seite 182). Nachprüfen: `SqlServerMigrationOrderTests` ohne Datenbank, `SqlServerMigrationTests` gegen SQL Server.

Übung 5: Ein Update aus der API

Eine API nimmt eine Bestellung entgegen und speichert sie mit `Update`. Warum markiert EF die Mandanten-Property als geändert, und was verhindert, dass ein Angreifer die ID einer fremden Bestellung mit dem eigenen Mandanten schickt?

> **Lösung**
>
> `Update` einer nicht getrackten Entität markiert jede Property, Original- und aktueller Wert sind gleich. Das ist kein Mandantenwechsel; der Interceptor prüft nur den Wert (Seite 84). Gegen die fremde ID hilft er nicht, weil er ohne Datenbankzugriff nicht wissen kann, wem die Zeile gehört. Das `UPDATE` sieht die Zeile wegen der Policy nicht, ändert null Zeilen, und EF meldet einen Konkurrenzfehler. Nachprüfen: `DetachedUpdateTests` gegen PostgreSQL.

Ausführen lässt sich jede Prüfung mit einem Filter, zum Beispiel:

```Shell
dotnet test tests/Acme.EntityFrameworkCore.Tenancy.Tests --filter "Name~Inequality_with"
```

F Quellen

Die Auswahl beschränkt sich auf Primärquellen: die offizielle Dokumentation von EF Core, Npgsql, PostgreSQL, SQL Server und .NET sowie den Quellcode genau der Versionen, auf die sich das Buch bezieht. Implementierungsdetails sind deshalb an Tags gebunden (`v10.0.0`), nicht an Branches, die sich weiterbewegen. Alle Adressen waren bei Redaktionsschluss erreichbar.

Quelle	Kap.
EF Core: Releases und Erweiterungspunkte	
EF Core releases and planning `https://learn.microsoft.com/en-us/ef/core/what-is-new/`	27
Breaking changes in EF Core 10 `https://learn.microsoft.com/en-us/ef/core/what-is-new/ef-core-10.0/breaking-changes`	27
Writing a database provider `https://learn.microsoft.com/en-us/ef/core/providers/writing-a-provider`	6
Model bulk configuration (conventions) `https://learn.microsoft.com/en-us/ef/core/modeling/bulk-configuration`	10
Value conversions `https://learn.microsoft.com/en-us/ef/core/modeling/value-conversions`	12
Value comparers `https://learn.microsoft.com/en-us/ef/core/modeling/value-comparers`	24
Global query filters `https://learn.microsoft.com/en-us/ef/core/querying/filters`	10
Interceptors `https://learn.microsoft.com/en-us/ef/core/logging-events-diagnostics/interceptors`	18
Logging, events and diagnostics `https://learn.microsoft.com/en-us/ef/core/logging-events-diagnostics/`	19
EF Core: Modell, Migrationen, Betrieb	
Inheritance (TPH, TPT, TPC) `https://learn.microsoft.com/en-us/ef/core/modeling/inheritance`	10
Owned entity types `https://learn.microsoft.com/en-us/ef/core/modeling/owned-entities`	10
Table and entity splitting `https://learn.microsoft.com/en-us/ef/core/modeling/table-splitting`	10
Custom migrations operations `https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/operations`	22
Design-time services `https://learn.microsoft.com/en-us/ef/core/cli/services`	23
Connection resiliency `https://learn.microsoft.com/en-us/ef/core/miscellaneous/connection-resiliency`	38

Quelle	Kap.
Advanced performance topics (compiled models) `https://learn.microsoft.com/en-us/ef/core/performance/advanced-performance-topics`	24
NativeAOT and precompiled queries `https://learn.microsoft.com/en-us/ef/core/performance/nativeaot-and-precompiled-queries`	24
Quellcode zur verwendeten Version	
EntityFrameworkServicesBuilder (EF Core 10.0.0) `https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore/Infrastructure/EntityFrameworkServicesBuilder.cs`	A
EntityFrameworkRelationalServicesBuilder (EF Core 10.0.0) `https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore.Relational/Infrastructure/EntityFrameworkRelationalServicesBuilder.cs`	A
ConstructorBindingFactory (EF Core 10.0.0) `https://github.com/dotnet/efcore/blob/v10.0.0/src/EFCore/Metadata/Internal/ConstructorBindingFactory.cs`	17
Npgsql EF Core provider 10.0.0 `https://github.com/npgsql/efcore.pg/tree/v10.0.0`	6
Npgsql und PostgreSQL	
Npgsql Entity Framework Core provider `https://www.npgsql.org/efcore/`	3
Npgsql connection string parameters `https://www.npgsql.org/doc/connection-string-parameters.html`	38
PostgreSQL: Row Security Policies `https://www.postgresql.org/docs/17/ddl-rowsecurity.html`	36
PostgreSQL: CREATE POLICY `https://www.postgresql.org/docs/17/sql-createpolicy.html`	21
PostgreSQL: Function Volatility Categories `https://www.postgresql.org/docs/17/xfunc-volatility.html`	36
PostgreSQL: System Administration Functions (set_config) `https://www.postgresql.org/docs/17/functions-admin.html`	18
SQL Server	
SQL Server: Row-Level Security `https://learn.microsoft.com/en-us/sql/relational-databases/security/row-level-security`	47
CREATE SECURITY POLICY `https://learn.microsoft.com/en-us/sql/t-sql/statements/create-security-policy-transact-sql`	47
sp_set_session_context `https://learn.microsoft.com/en-us/sql/relational-databases/system-stored-procedures/sp-set-session-context-transact-sql`	47
.NET und NuGet	
Native AOT deployment `https://learn.microsoft.com/en-us/dotnet/core/deploying/native-aot/`	24

Quelle	Kap.
Package source mapping `https://learn.microsoft.com/en-us/nuget/consume-packages/package-source-mapping`	24
Auditing package dependencies `https://learn.microsoft.com/en-us/nuget/concepts/auditing-packages`	27
Public API analyzers `https://github.com/dotnet/roslyn-analyzers/blob/main/src/PublicApiAnalyzers/PublicApiAnalyzers.Help.md`	41

Lizenz

Dieses Handbuch und der Beispielcode im Repository stehen unter der MIT-Lizenz. Sie dürfen beides nutzen, kopieren, verändern und weitergeben, auch kommerziell, sofern der folgende Copyright- und Lizenzhinweis erhalten bleibt. Maßgeblich ist der englische Originalwortlaut:

Copyright (c) 2026 guenni81

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Die im PDF eingebetteten DejaVu-Schriften stehen unter einer eigenen freien Lizenz (Bitstream-Vera-Schriftlizenz; die DejaVu-Änderungen sind gemeinfrei); siehe `docs/handbook/fonts/LICENSE-DejaVu.txt` im Repository.

Genannte Produkt- und Firmennamen sind Marken ihrer jeweiligen Inhaber, etwa Entity Framework, .NET und SQL Server (Microsoft) sowie PostgreSQL (PostgreSQL Community Association). Sie werden nur beschreibend verwendet. Dieses Handbuch ist ein unabhängiges Projekt und steht in keiner Verbindung zu Microsoft, der PostgreSQL Global Development Group oder dem Npgsql-Projekt und wird von ihnen weder unterstützt noch empfohlen.